Чесноков С.Е.

ПРАКТИКУМ ПО ТЕХНОЛОГИЯМ СОЗДАНИЯ WEB-ПРИЛОЖЕНИЙ

Часть 2

ОГЛАВЛЕНИЕ

ПРАКТИЧЕСКАЯ РАБОТА 8. СОЗДАНИЕ ВЕБ-ИНТЕРФЕЙСА С ИСПОЛЬЗОВАНИЕМ BOOTSTRAP 7
Теоретические сведения 7
Задания для практической работы 10
Общие требования к вариантам и структуре страницы 11
Интерактивные компоненты 11
Требования к результату 12
Вариант 1. Сайт научной конференции 13
Вариант 2. Профиль специалиста 14
Вариант 3. Туристический путеводитель по городу 16
Вариант 4. Интернет-магазин одежды 17
Вариант 5. Сайт онлайн-курса 18
Вариант 6. Панель администратора 19
Вариант 7. Афиша кинотеатра 20
Вариант 8. Портфолио дизайнера 22
Вариант 9. Сервис заказа и доставки еды 23
Вариант 10. Учебная платформа 24
Вариант 11. Панель аналитики социальных сетей 25
Вариант 12. Интернет-магазин товаров повседневного спроса 26
Вариант 13. Сервис поиска туристических направлений 27
Вариант 14. Сервис поиска недвижимости 29
Вариант 15. Новостной портал 30
Контрольные вопросы 31
ПРАКТИЧЕСКАЯ РАБОТА 9. ИСПОЛЬЗОВАНИЕ JAVASCRIPT ДЛЯ РАБОТЫ С DOM 33
Задания к практической работе 33
Общая постановка задачи 33
Требования к интерфейсу приложения 34
Реализация логики приложения с помощью JavaScript 35
Работа с DOM и интерактивностью 35
Проверка пользовательского ввода 36
Визуальные эффекты 36
Требования к результату работы 36
Вариант 1. Интерактивный список покупок 37
Вариант 2. Планировщик учебного дня 37
Вариант 3. Галерея путешествий 38
Вариант 4. Трекер привычек 39
Вариант 5. Меню ресторана 40
Вариант 6. Расписание занятий 41
Вариант 7. Каталог компьютерной техники 42

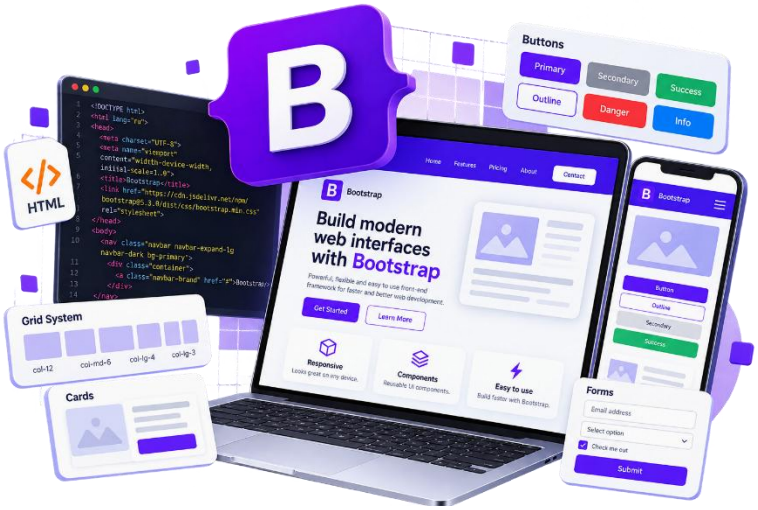
Вариант 8. Электронная библиотека ... 43
Вариант 9. Трекер расходов ... 44
Вариант 10. Конструктор пиццы ... 45
Вариант 11. Диспетчер задач проекта ... 46
Вариант 12. Бронирование переговорной комнаты ... 47
Вариант 13. Панель мониторинга погоды ... 48
Вариант 14. Интерактивная викторина ... 49
Вариант 15. Мини-магазин с корзиной ... 50
Контрольные вопросы ... 51
ПРАКТИЧЕСКАЯ РАБОТА 10. СОЗДАНИЕ И ИНТЕГРАЦИЯ REACT КОМПОНЕНТ ... 53
Задания к практической работе ... 53
Создание React-проекта ... 54
Работа с состоянием ... 55
Передача данных между компонентами ... 55
Обработка событий ... 56
Условный и списочный рендеринг ... 56
Использование useEffect ... 56
Стилизация React-приложения ... 57
Требования к результату работы ... 57
Вариант 1. Разработка компонентов для планировщика задач TaskFlow ... 57
Вариант 2. Разработка React-компонентов для приложения «StudyPlan — Планировщик учебного дня» ... 58
Вариант 3. Разработка компонентов для галереи путешествий TravelGallery ... 59
Вариант 4. азработка компонентов для системы учёта личных финансов FinanceTracker ... 60
Вариант 5. Разработка компонентов для интернет-магазина ShopHub ... 62
Вариант 6. Разработка компонентов для системы управления проектами TaskFlow ... 63
Вариант 7. Разработка компонентов для фитнес-приложения FitTrack ... 64
Вариант 8. Разработка компонентов для платформы изучения иностранных языков LinguaLearn ... 65
Вариант 9. Разработка компонентов для приложения управления личными финансами FinBuddy 66
Вариант 10. Разработка компонентов для платформы поиска и организации мероприятий EventHub ... 68
Вариант 11. Разработка компонентов для электронной библиотеки BookNest ... 69
Вариант 12. Разработка компонентов для системы управления задачами TaskFlow ... 70
Вариант 13. Разработка компонентов для системы управления задачами TaskFlow ... 71
Вариант 14. Разработка компонентов для планировщика питания MealPlanner ... 73
Вариант 15. Разработка компонентов для электронной библиотеки BookTracker ... 74
Контрольные вопросы ... 75
ПРАКТИЧЕСКАЯ РАБОТА 11. РАЗРАБОТКА BACKEND-ПРИЛОЖЕНИЯ НА NODE.JS С ИСПОЛЬЗОВАНИЕМ POSTGRESQL ... 77
Задания к практической работе ... 77
Структура проекта ... 78
Создание базы данных ... 79

Подключение Node.js к PostgreSQL 79
Создание REST API 80
Валидация данных и обработка ошибок 80
Требования к результату 81
Вариант 1. Список задач 81
Вариант 2. Планировщик учебного дня 83
Вариант 3. Галерея путешествий 84
Вариант 4. Трекер привычек 85
Вариант 5. Меню ресторана 87
Вариант 6. Расписание занятий 88
Вариант 7. Каталог компьютерной техники 89
Вариант 8. Электронная библиотека 90
Вариант 9. Трекер расходов 92
Вариант 10. Конструктор пиццы 93
Вариант 11. Диспетчер задач проекта 94
Вариант 12. Бронирование переговорных комнат 95
Вариант 13. Панель мониторинга погоды 96
Вариант 14. Интерактивная викторина 97
Вариант 15. Мини-магазин с корзиной 99
Контрольные вопросы 100
ПРАКТИЧЕСКАЯ РАБОТА 12. РАЗРАБОТКА BACKEND-ПРИЛОЖЕНИЯ НА PYTHON FLASK С ИСПОЛЬЗОВАНИЕМ POSTGRESQL 102
Задания к практической работе 102
Структура Flask-проекта 103
Создание Flask-приложения 104
Подключение PostgreSQL 104
Проектирование базы данных 105
Создание REST API 105
Работа с JSON 106
Валидация данных и обработка ошибок 106
Требования к результату 107
Вариант 1. Список задач 108
Вариант 2. Планировщик учебного дня 109
Вариант 3. Галерея путешествий 109
Вариант 4. Трекер привычек 110
Вариант 5. Меню ресторана 111
Вариант 6. Расписание занятий 112
Вариант 7. Каталог компьютерной техники 113
Вариант 8. Электронная библиотека 114
Вариант 9. Трекер расходов 115
Вариант 10. Конструктор пиццы 116

Вариант 11. Диспетчер задач проекта 117
Вариант 12. Бронирование переговорных комнат 118
Вариант 13. Панель мониторинга погоды 119
Вариант 14. Интерактивная викторина 120
Вариант 15. Мини-магазин с корзиной 121
Контрольные вопросы 123
ПРАКТИЧЕСКАЯ РАБОТА 13. РАЗРАБОТКА ПРОГРЕССИВНОГО WEB-ПРИЛОЖЕНИЯ (PWA) 125
Задания к практической работе 125
Структура PWA-проекта 126
Адаптивный интерфейс 127
Установка на мобильное устройство 127
Требования к результату 128
Вариант 1. Мобильный список покупок 128
Вариант 2. Планировщик учебного дня 129
Вариант 3. Галерея путешествий 130
Вариант 4. Трекер привычек 131
Вариант 5. Мобильное меню ресторана 132
Вариант 6. Мобильное расписание занятий 133
Вариант 7. Каталог компьютерной техники 134
Вариант 8. Электронная библиотека 135
Вариант 9. Трекер расходов 136
Вариант 10. Конструктор пиццы 137
Вариант 11. Диспетчер задач проекта 138
Вариант 12. Бронирование переговорных 139
Вариант 13. Панель мониторинга погоды 140
Вариант 14. Интерактивная викторина 141
Вариант 15. Мини-магазин с корзиной 142
Контрольные вопросы 143
ПРАКТИЧЕСКАЯ РАБОТА 14. РАЗРАБОТКА ПОЛНОСТЕКОВОГО WEB-ПРИЛОЖЕНИЯ (FULLSTACK) 145
Теоретическая часть 146
Задания к практической работе 149
Вариант 1. SafeVision — система контроля промышленной безопасности 154
Вариант 2. SafeVision — система контроля промышленной безопасности 161
Вариант 3. TrafficAI — система интеллектуального мониторинга и анализа дорожного движения 170
Вариант 4. AgroVision — система интеллектуальной диагностики заболеваний растений 180
Вариант 5. MediScan AI — система интеллектуального анализа медицинских изображений 190
Вариант 3. EnergyMind — система интеллектуального мониторинга энергопотребления 201
Вариант 7. EduVision — система интеллектуальной аналитики успеваемости студентов 214
Вариант 8. EcoCity — система интеллектуального управления отходами 228
Вариант 9. HealthHub AI — интеллектуальная система мониторинга здоровья 244

Вариант 10. SmartRetail AI — интеллектуальная аналитика розничной торговли 261
Вариант 11. SafeCity AI — интеллектуальная система общественной безопасности..... 279
Вариант 12. StudyMate AI — интеллектуальная образовательная платформа..... 297
Вариант 13. LogiTrack AI — интеллектуальная система управления транспортной логистикой 315
Вариант 14. EstatePro AI — интеллектуальная система управления недвижимостью..... 335
Вариант 15. RestoFlow AI — интеллектуальная система управления рестораном 359
БИБЛИОГРАФИЧЕСКИЙ СПИСОК 381

ПРАКТИЧЕСКАЯ РАБОТА 8. СОЗДАНИЕ ВЕБ-ИНТЕРФЕЙСА С ИСПОЛЬЗОВАНИЕМ BOOTSTRAP



Цель работы

- изучить назначение и основные возможности библиотеки Bootstrap;
- освоить подключение Bootstrap к HTML-документу;
- изучить систему сеток Bootstrap;
- научиться использовать готовые компоненты пользовательского интерфейса;
- освоить компоненты навигации, карточки, кнопки, формы, таблицы, уведомления, модальные окна и другие элементы;
- научиться комбинировать готовые компоненты для создания целостного интерфейса;
- освоить utility-классы Bootstrap для управления отступами, размерами, отображением, выравниванием и цветами;
- закрепить навыки работы с HTML и CSS, полученные в первой части практикума;
- подготовить интерфейс, который в дальнейшем может быть дополнен JavaScript, React и серверной частью;
- выполнить тестирование и публикацию разработанного проекта.

Теоретические сведения

Назначение Bootstrap

Bootstrap — библиотека готовых средств для разработки пользовательских интерфейсов веб-приложений.

Она предоставляет:

- готовые CSS-классы;
- систему сеток;

- компоненты интерфейса;
- классы для оформления элементов;
- JavaScript-компоненты;
- средства для быстрого построения типовых интерфейсов.

Использование Bootstrap позволяет сосредоточиться не на создании каждого элемента с нуля, а на построении структуры и логики интерфейса.

В рамках данной практической работы Bootstrap используется преимущественно как средство **создания пользовательского интерфейса**.

Подключение Bootstrap

Bootstrap можно подключить непосредственно к HTML-документу.

Например:

```
   <link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min
.css"
       rel="stylesheet">
```

Для интерактивных компонентов Bootstrap:

```
   <script
       src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/boot-
strap.bundle.min.js">
   </script>
```

После подключения становятся доступны классы и компоненты Bootstrap.

Система сеток Bootstrap

Одним из основных элементов Bootstrap является система сеток.

Базовая структура:

```
   <div class="container">
       <div class="row">
           <div class="col">
               Первый блок
           </div>

           <div class="col">
               Второй блок
```

```
            </div>
        </div>
    </div>
```

При разработке интерфейса необходимо использовать:

- container;
- container-fluid;
- row;
- col;
- col-sm-*;
- col-md-*;
- col-lg-*;
- col-xl-*.

Сетка должна использоваться для логического размещения элементов интерфейса.

Основные компоненты Bootstrap

В ходе работы необходимо познакомиться с компонентами:

Навигация

- navbar;
- nav;
- breadcrumb;
- pagination.

Информационные элементы

- alert;
- badge;
- progress;
- spinner.

Содержимое

- card;
- list-group;
- table;
- accordion;
- carousel.

Элементы управления

- button;
- dropdown;
- modal;

- collapse.

Формы

- form-control;
- form-select;
- form-check;
- input-group;
- floating-labels.

Utility-классы

Bootstrap предоставляет большое количество классов для быстрого оформления элементов.

Например:

```
<div class="d-flex justify-content-between align-items-center">
```

Для работы с отступами:

```
<div class="mt-4 mb-3 p-3">
```

Для выравнивания текста:

```
<div class="text-center">
```

Для управления цветами:

```
<div class="bg-light text-dark">
```

Для создания тени:

```
<div class="shadow">
```

В практической работе необходимо использовать utility-классы наряду с компонентами Bootstrap.

Задания для практической работы

Разработать адаптивный веб-интерфейс с использованием Bootstrap 5 по назначенному варианту. За основу принять графический концепт из практикума: сохранить назначение, общую композицию и ключевые блоки, допускается самостоятельный подбор текстов, фотографий и цветовых акцентов. Точное копирование изображения не требуется.

В этой работе основное внимание уделяется структуре HTML, сетке Bootstrap, готовым компонентам, удобству интерфейса и адаптивности. Работа с реальной базой данных, авторизацией, обработкой заказов и сложной логикой не требуется.

Варианты специально сделаны различными по типу интерфейса. При этом каждый вариант позволяет использовать разные возможности Bootstrap.

Общие требования к вариантам и структуре страницы

- Создать логически разделённую страницу с навигацией, основной областью содержимого и подвалом.
- Использовать Bootstrap Grid (container, row, col и адаптивные классы col-md-*, col-lg-*), не менее четырёх разных компонентов и не менее двух типов utility-классов.
- Содержательно заполнить карточки, списки и таблицы: использовать правдоподобные названия, изображения, даты, цены и другие данные в соответствии с тематикой.
- Проверить внешний вид на широком экране, планшете и смартфоне. На узких экранах меню, сетки карточек и боковые блоки должны адаптироваться.
- Разрешается добавлять собственный CSS для фирменных цветов, типографики и декоративных элементов. JavaScript допускается только для штатных компонентов Bootstrap; сложную бизнес-логику реализовывать не требуется.
- Подготовить файлы index.html, css/style.css, js/script.js (при необходимости), папку images и README.md с описанием проекта.

Рекомендуемая структура:

```
project/
│
├── index.html
│
├── css/
│   └── style.css
│
├── js/
│   └── script.js
│
├── images/
│   └── ...
│
└── README.md
```

Файл script.js на данной работе может не содержать функциональной логики. Он создаётся заранее, поскольку в следующей работе этот же проект будет расширен с использованием JavaScript.

Интерактивные компоненты

В данной работе допускается использование интерактивных компонентов Bootstrap:

- modal;

- dropdown;
- collapse;
- accordion;
- carousel;
- navbar.

Например, модальное окно:

```
<button
    class="btn btn-primary"
    data-bs-toggle="modal"
    data-bs-target="#infoModal">
    Подробнее
</button>
```

При этом сложная программная логика на JavaScript не требуется. Например, для магазина не требуется реализовывать реальное добавление товара в корзину. Для кинотеатра не требуется реализовывать настоящее бронирование. Для гостиницы не требуется реализовывать реальное бронирование. Это будет сделано на следующих этапах изучения web-разработки.

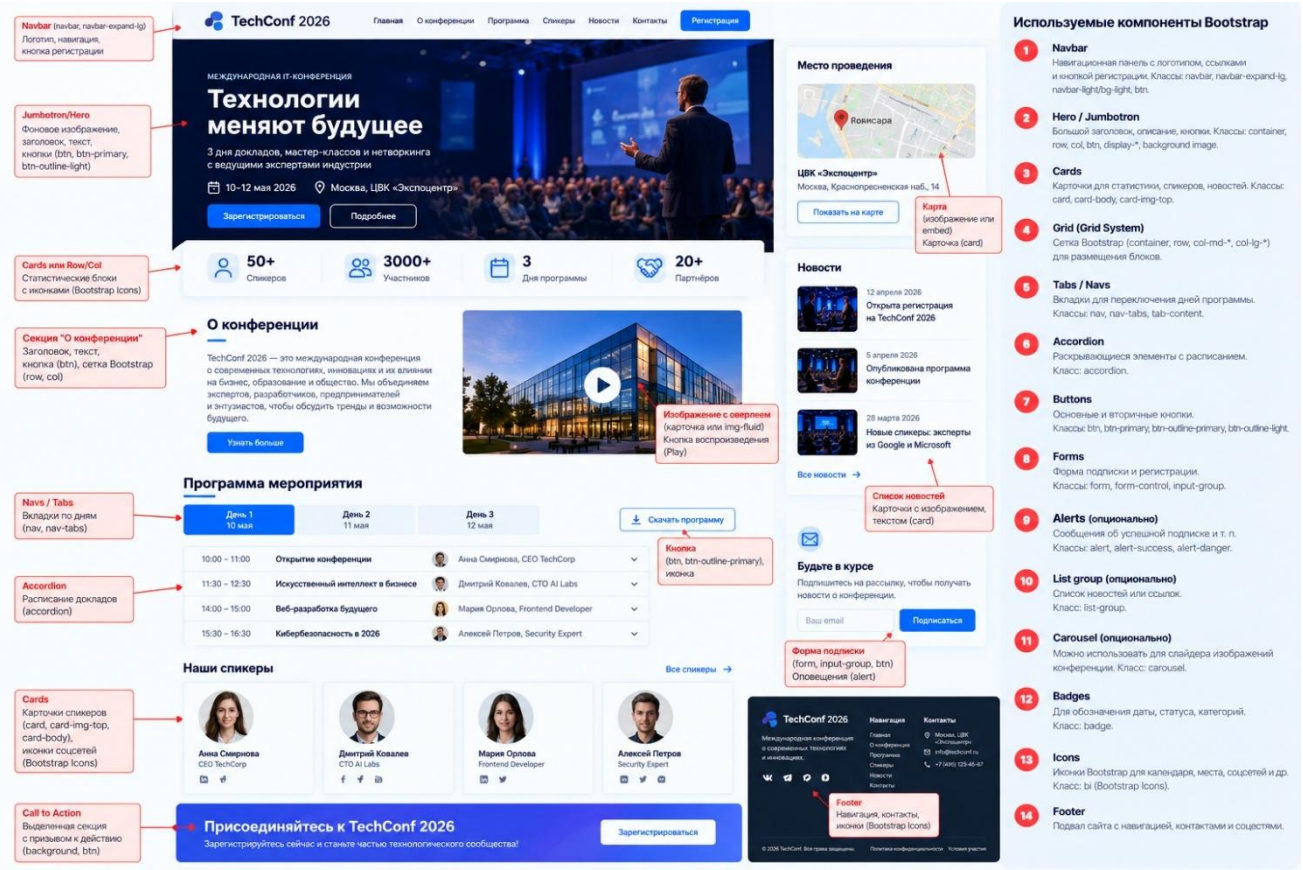
Требования к результату

Работа считается выполненной, если:

- Bootstrap подключён и используется;
- интерфейс соответствует выбранному варианту;
- используется система сеток Bootstrap;
- используется минимум 4 компонента Bootstrap;
- используются utility-классы;
- создана законченная структура страницы;
- присутствует навигация;
- присутствует основное содержимое;
- присутствует footer;
- интерфейс корректно отображается в браузере;
- интерактивные Bootstrap-компоненты работают;
- отсутствуют критические ошибки;
- проект загружен на GitHub;
- опубликована рабочая версия через GitHub Pages.

Вариант 1. Сайт научной конференции

Разработать главную страницу конференции с верхней навигацией, крупным приветственным блоком с названием, датой и местом проведения, а также кнопкой регистрации. Ниже разместить описание мероприятия, ключевые показатели, программу по дням и секциям, карточки докладчиков, новости и контакты. Предусмотреть форму регистрации и раскрывающиеся сведения о секциях.



Шапка и первый экран. Разместить логотип и пункты меню «О конференции», «Программа», «Спикеры», «Новости», «Контакты». На первом экране представить название, тематический слоган, дату, место, изображение мероприятия и основную кнопку «Зарегистрироваться».

Контент. Создать блок с описанием конференции и числовыми показателями: число участников, докладов и секций. Программу оформить по дням и временным интервалам; у каждого выступления указать тему, время и докладчика. Подготовить карточки спикеров с фотографиями, должностями и темами выступлений, блок новостей и информацию о площадке.

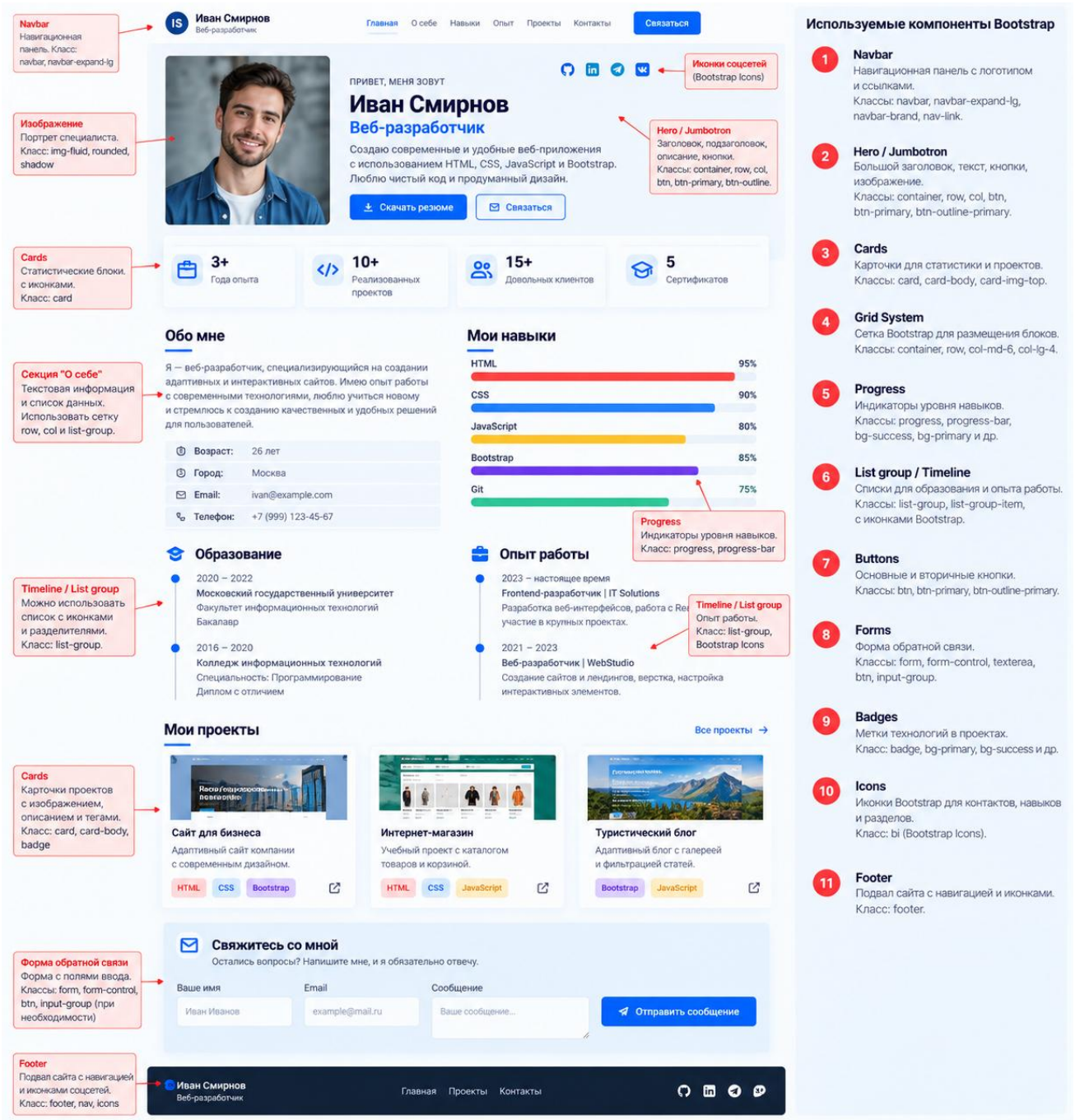
Управление. Разделить программу на дни с помощью nav, подробности секций раскрывать через accordion. Регистрационную форму снабдить полями имени, электронной почты, организации и выбора секции; условия участия можно вывести в modal.

Рекомендуемые компоненты Bootstrap. navbar, card, badge, accordion, form, modal, button.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 2. Профиль специалиста

Создать страницу профессионального профиля: шапку с фотографией, именем, специализацией, кратким представлением и кнопками связи. Основную часть разделить на блоки «Обо мне», навыки с индикаторами уровня, образование, опыт работы в виде временной шкалы и карточки выполненных проектов. Внизу разместить контактную форму и подвал.



Первый экран. Сформировать навигационную панель с разделами «Главная», «Обо мне», «Навыки», «Опыт», «Проекты», «Контакты». Рядом с крупной фотографией разместить имя специалиста, профессию, краткий профессиональный девиз и кнопки «Скачать резюме» и «Связаться».

Контент. Предусмотреть карточки достижений с числовыми показателями, развёрнутый рассказ о специализации, контактные сведения, блок образования и хронологию мест работы с датами и должностями. Для навыков показать названия технологий и условный уровень владения. Проекты оформить в виде карточек с превью, задачей и применёнными инструментами.

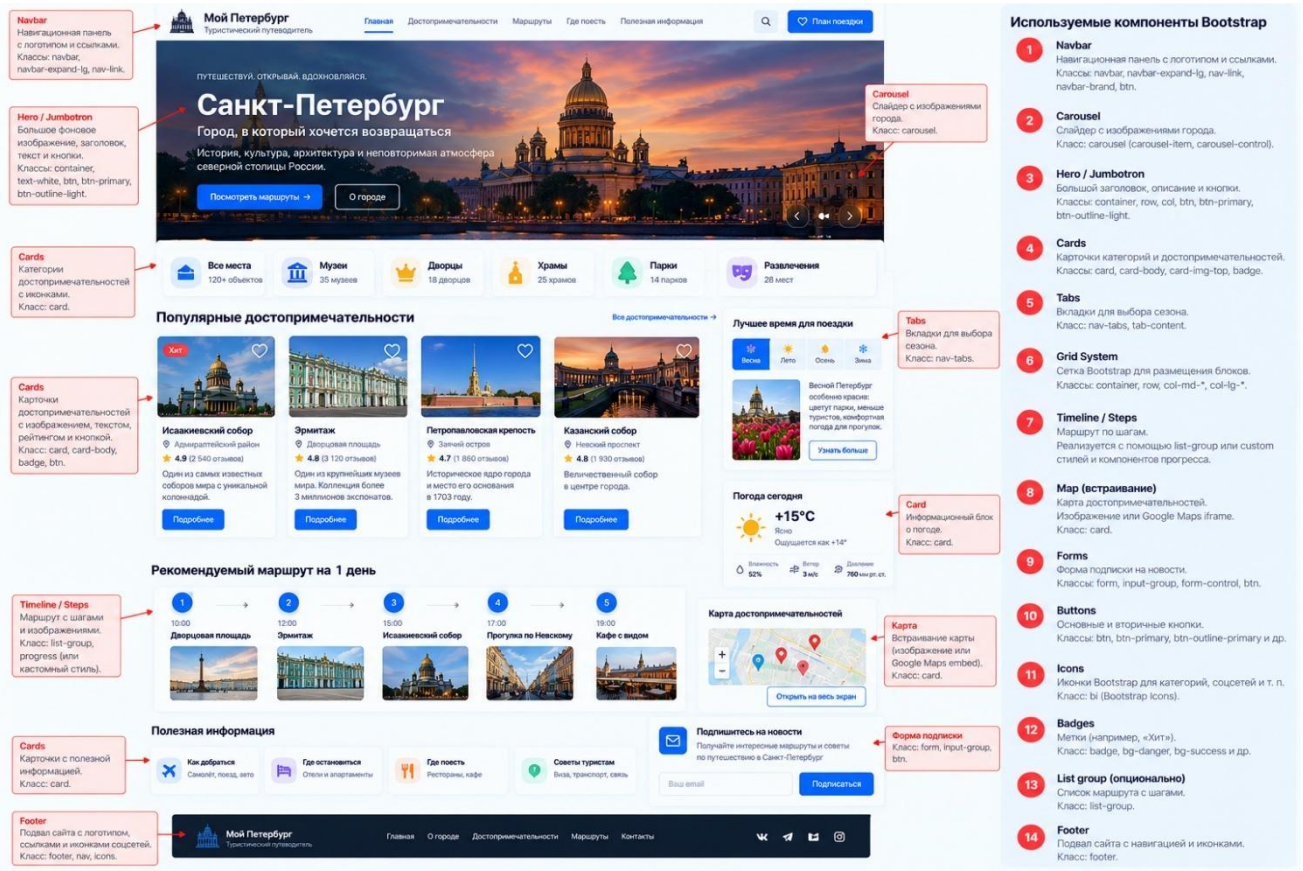
Управление. Оформить шкалы навыков через progress, технологии отметить badge, этапы карьеры представить как вертикальный список. В контактной форме предусмотреть имя, email и текст сообщения; отправка может быть демонстрационной.

Рекомендуемые компоненты Bootstrap. navbar, card, progress, badge, list-group, form, button.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 3. Туристический путеводитель по городу

Разработать путеводитель с большим фотобаннером города, кратким описанием и навигацией по разделам. Показать категории мест, карточки популярных достопримечательностей с фотографиями, оценками и описаниями, рекомендованный маршрут на один день, полезные советы и блок карты. Предусмотреть просмотр подробной информации о месте.



Первый экран. Предусмотреть логотип путеводителя, меню разделов, название города и широкое изображение узнаваемой городской панорамы. Разместить короткий текст о городе и кнопки перехода к достопримечательностям и маршруту.

Контент. Сформировать блок категорий («Музеи», «Архитектура», «Парки», «Развлечения»), сетку достопримечательностей с фото, названием, оценкой и краткой справкой. Добавить подборку рекомендаций, карточку погоды или полезных сведений, маршрут одного дня с последовательностью остановок и схематическую карту.

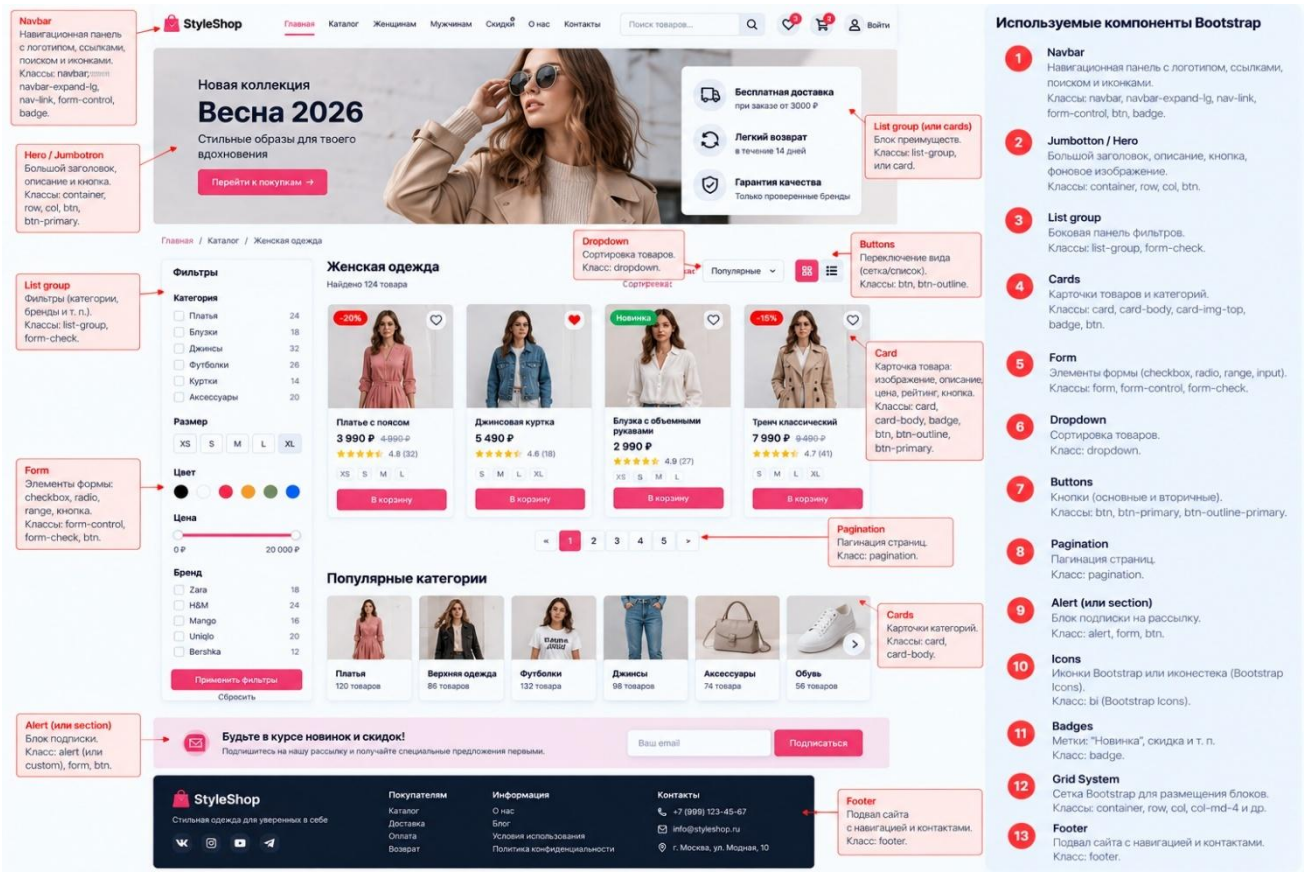
Управление. Для фотоподборки использовать carousel; карточки мест снабдить кнопками «Подробнее», открывающими modal. Маршрут показать в виде list-group, категории выделить badge, навигационный путь оформить через breadcrumb.

Рекомендуемые компоненты Bootstrap. navbar, carousel, card, badge, list-group, breadcrumb, modal.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 4. Интернет-магазин одежды

Разработать страницу каталога одежды с рекламным баннером, навигацией по категориям и строкой поиска. В центральной области показать сетку карточек товаров с фотографиями, ценой, размером, отметками скидок и наличия; сбоку расположить фильтры по категории, размеру, цвету и цене. Добавить блок популярных категорий, визуальную корзину и окно быстрого просмотра товара.



Первый экран. Создать верхнюю навигацию с логотипом магазина, категориями, поиском и значками профиля, избранного и корзины. Под ней разместить рекламный баннер сезонной коллекции с кнопкой перехода к каталогу.

Контент. Основную область организовать в две колонки: слева фильтры, справа каталог. В фильтрах предусмотреть категорию, размер, цвет, диапазон цены и наличие. В карточке товара показать фотографию, название, цену, старую цену при скидке, доступные размеры, рейтинг и статус. Добавить блок популярных категорий и пагинацию.

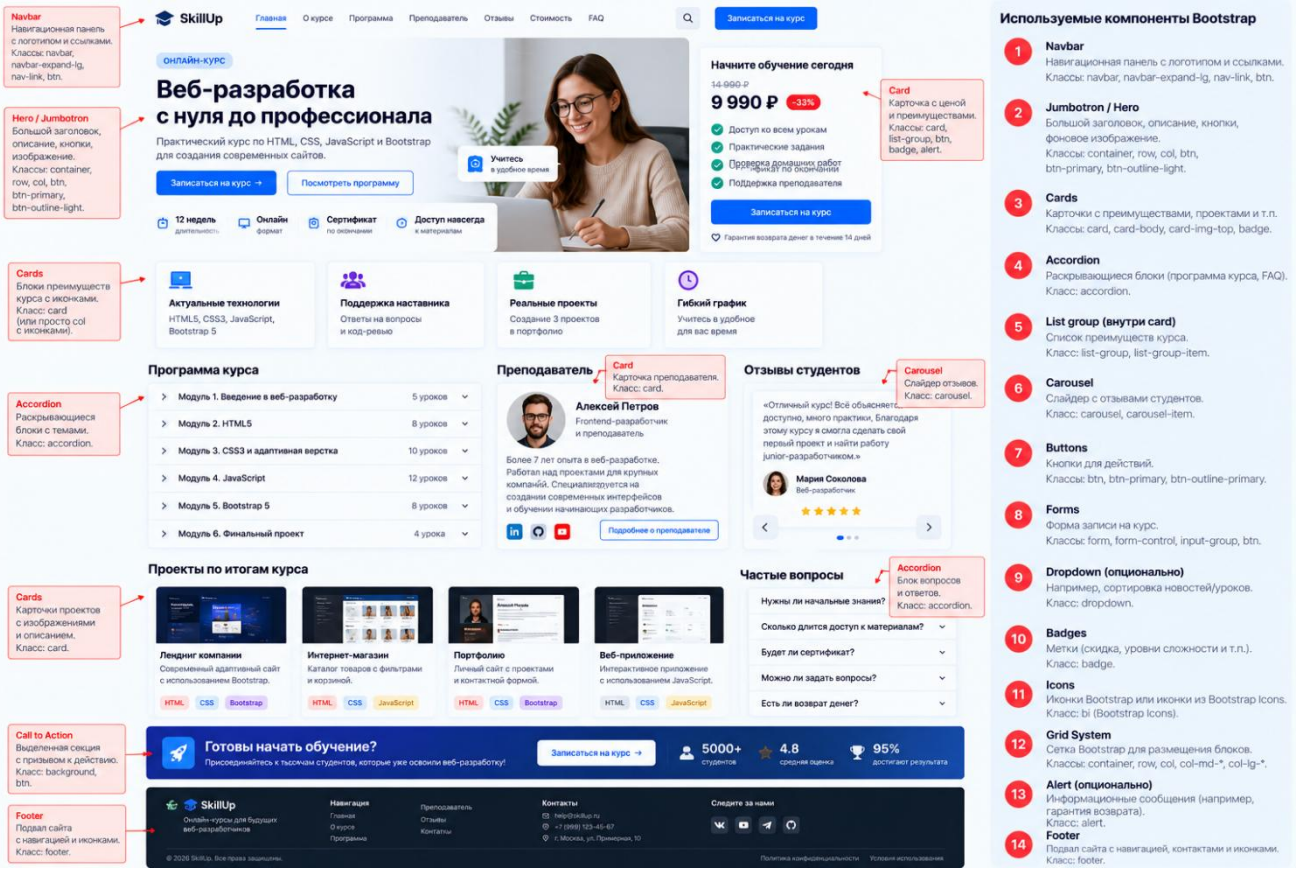
Управление. Предусмотреть выпадающую сортировку, переключатели и флажки фильтров, badge для скидок и новинок. Кнопка «Быстрый просмотр» открывает modal с увеличенным изображением и характеристиками; добавление в корзину допускается имитировать визуально.

Рекомендуемые компоненты Bootstrap. navbar, card, badge, dropdown, form-check, pagination, modal.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 5. Сайт онлайн-курса

Создать страницу образовательного курса с презентационным блоком, описанием, сведениями о преподавателе, продолжительности, уровне сложности и стоимости. Разместить преимущества обучения, программу по модулям с раскрывающимися темами, список видеоуроков, отзывы, индикатор учебного прогресса и кнопку записи.



Первый экран. Оформить шапку образовательной платформы с меню, поиском и кнопкой входа. В презентационном блоке курса указать название, тему, ожидаемый результат обучения, изображение преподавателя и заметную кнопку записи.

Контент. Рядом с описанием вывести информационную карточку: стоимость, длительность, формат и уровень. Ниже представить преимущества курса, программу по модулям, перечень уроков с длительностью, материалы для скачивания, биографию преподавателя, отзывы слушателей и условный процент прохождения.

Управление. Модули раскрывать через accordion, занятия перечислить в list-group, сложность обозначить badge, прогресс — progress. Кнопки записи и просмотра уроков должны иметь согласованные состояния и оформление.

Рекомендуемые компоненты Bootstrap. navbar, card, accordion, progress, badge, list-group, button.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 6. Панель администратора

Разработать информационную панель управления веб-сервисом: постоянное боковое меню, верхнюю панель с поиском и профилем, карточки ключевых показателей, графические блоки статистики, таблицу пользователей, список последних событий, уведомления и состояние системы. Визуально выделить основные показатели, статусы и быстрые действия; реальные расчёты и серверную обработку не реализовывать.

Каркас панели. Создать левую боковую навигацию с иконками и пунктами «Главная», «Пользователи», «Заказы», «Статистика», «Сообщения», «Настройки». В верхней полосе разместить поиск, уведомления и профиль администратора.

Контент. В начале рабочей области вывести карточки ключевых показателей с числами и динамикой. Ниже разместить области с условными графиками посещаемости и распределения заказов, таблицу последних пользователей или операций, список событий, блок состояния сервисов и быстрые действия.

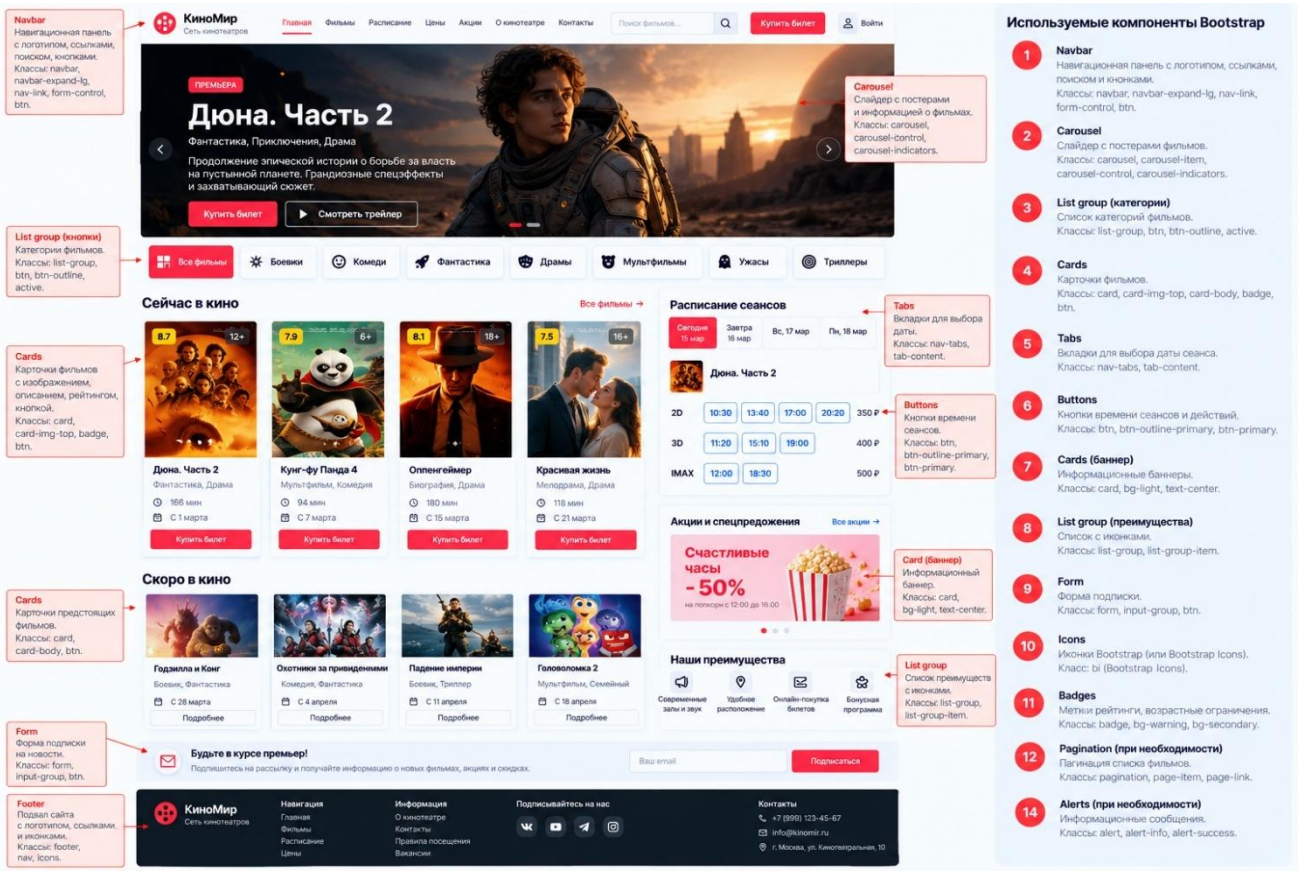
Управление. Для статусов использовать badge и alert, для меню профиля — dropdown, для заполненности ресурсов — progress. Данные и графики могут быть статическими; важно визуально отделить группы показателей и обеспечить читаемость таблицы.

Рекомендуемые компоненты Bootstrap. navbar, card, table, badge, alert, dropdown, progress.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 7. Афиша кинотеатра

Разработать страницу киноафиши с крупным баннером премьерного фильма, меню разделов и фильтрами по жанрам. Показать карточки фильмов с постерами, возрастным рейтингом, продолжительностью и кнопками просмотра; добавить расписание сеансов по датам и времени, стоимость билетов, блок ближайших премьер и окно подробного описания фильма.



Первый экран. Разместить логотип кинотеатра, меню «Афиша», «Сеансы», «Скоро», «Акции» и крупный баннер премьеры с названием фильма, жанром, возрастным ограничением и кнопкой просмотра расписания.

Контент. Создать раздел «Сейчас в кино» с сеткой постеров, названиями, жанрами, длительностью, возрастным рейтингом и кнопками «Сеансы». Отдельно оформить ближайшие премьеры, календарь дат, таблицу или сетку времени сеансов, диапазон цен и рекламный блок специальных предложений.

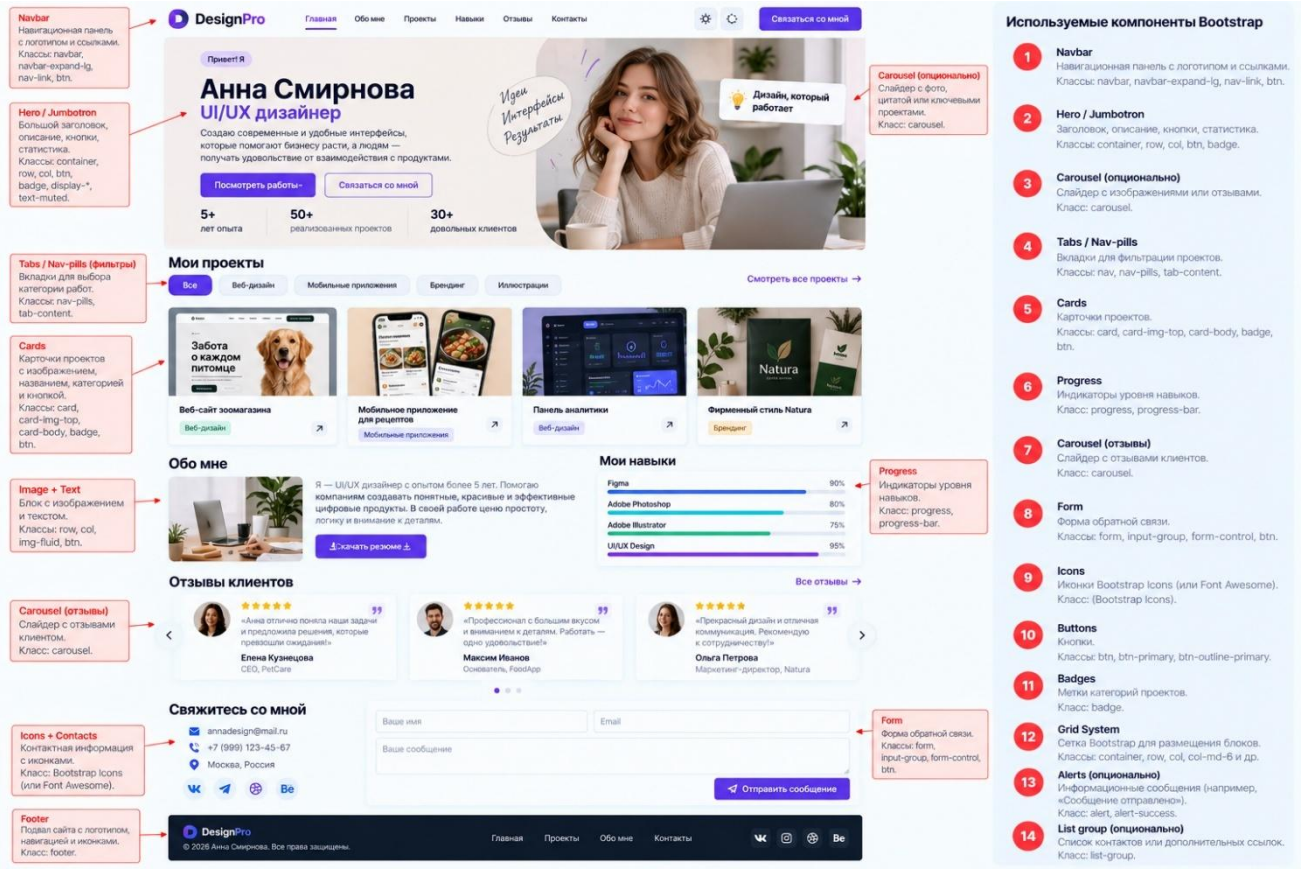
Управление. Фильтры жанров и дат выполнить с помощью nav, возрастные ограничения — badge. Подробности фильма показывать в modal, большие списки разбить pagination; покупка билетов на этом этапе не реализуется.

Рекомендуемые компоненты Bootstrap. navbar, carousel, card, badge, nav, table, modal, pagination.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 8. Портфолио дизайнера

Создать персональный сайт дизайнера с выразительным первым экраном, фотографией, специализацией и кнопкой связи. Разместить фильтруемую визуально галерею проектов по категориям, карточки работ с изображениями и описаниями, блок «Обо мне», шкалы навыков, отзывы клиентов и форму обратной связи. Для просмотра проекта предусмотреть модальное окно.



Первый экран. Создать шапку с логотипом или именем дизайнера, меню разделов и выразительный hero-блок с фотографией, специализацией, кратким описанием подхода и кнопкой связи.

Контент. Показать статистику проектов и опыта, галерею работ по категориям («Брендинг», «Web», «Интерфейсы», «Графика»), карточки проектов с превью и коротким описанием задачи. Добавить раздел «Обо мне», перечень инструментов, шкалы компетенций, отзывы заказчиков и контактные данные.

Управление. Категории представить вкладками или кнопками, технологии — badge, компетенции — progress. Просмотр отдельного кейса оформить в modal; отзывы допускается представить carousel. Контактная форма должна содержать тему обращения.

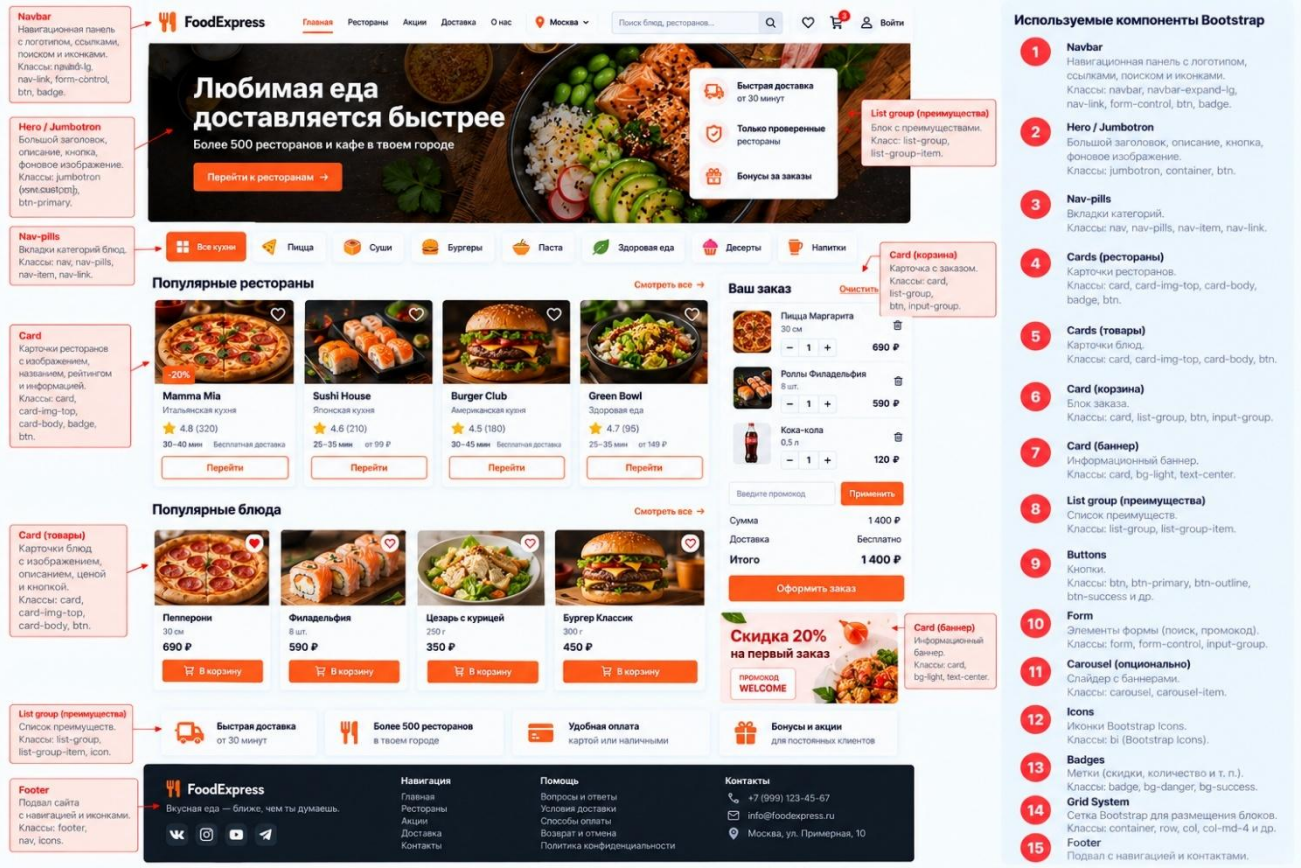
Рекомендуемые компоненты Bootstrap. navbar, card, carousel, badge, progress, modal, form.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным

оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 9. Сервис заказа и доставки еды

Разработать главную страницу сервиса доставки еды с аппетитным рекламным баннером, строкой поиска и категориями блюд. Показать карточки популярных блюд с изображениями, составом, рейтингом и ценой, разделы популярных предложений и ресторанов, компактный блок корзины с итоговой стоимостью и информацию об условиях доставки. Кнопки заказа на этой работе могут быть демонстрационными.



Первый экран. Создать шапку сервиса доставки с логотипом, выбором адреса, поиском блюд, значками избранного и корзины. На первом экране разместить крупный рекламный баннер с предложением доставки и призывом выбрать блюда.

Контент. Добавить горизонтальный список категорий кухни, карточки популярных блюд с фотографиями, составом, массой, ценой и оценкой. Отдельно представить популярные подборки, предложения дня, краткие условия доставки и компактный блок корзины с несколькими позициями, количеством и суммой.

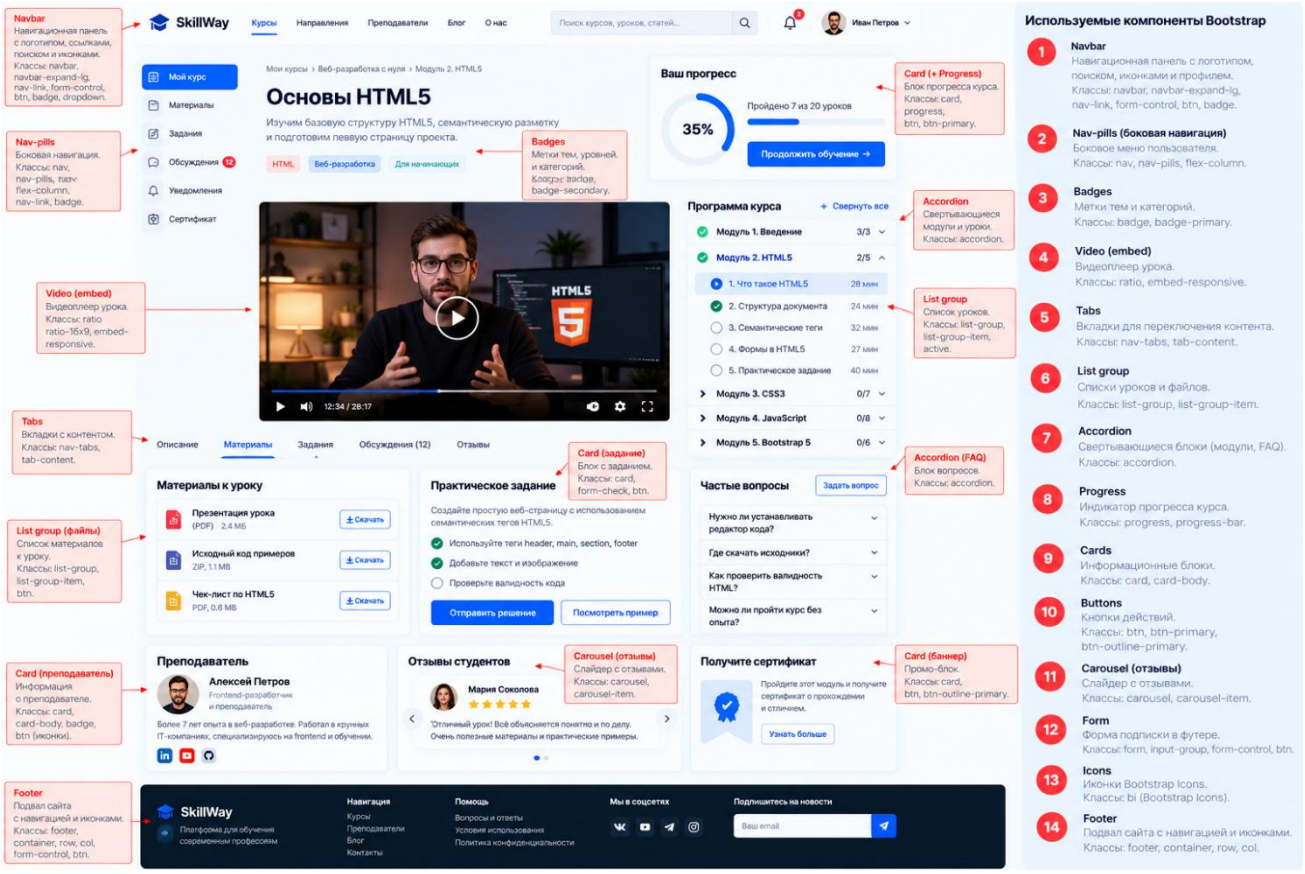
Управление. Категории оформить кнопками или nav, предложения — card и badge. В корзине использовать list-group и итоговый блок; кнопки «Добавить» и «Оформить заказ» могут оставаться демонстрационными.

Рекомендуемые компоненты Bootstrap. navbar, card, badge, list-group, button, form-control, modal.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 10. Учебная платформа

Разработать интерфейс страницы обучения: боковую навигацию по курсам, верхнюю панель, название изучаемого курса и основной блок видеозанятия. Рядом показать текущий прогресс, список уроков и модулей с отметками выполнения; ниже — вкладки с материалами, заданиями и обсуждениями, карточку преподавателя и рекомендации следующих занятий.



Каркас страницы. Построить учебный кабинет с боковым меню («Мои курсы», «Каталог», «Задания», «Достижения»), верхней панелью поиска и профиля. В центральной области вывести название курса и текущую тему.

Контент. Основной акцент сделать на большом блоке видеоплеера или его статической обложке. Справа разместить процент прохождения и список модулей с уроками, длительностью и отметками выполнения. Под видео добавить вкладки «Материалы», «Задания», «Обсуждение», описание урока, блок преподавателя и рекомендации.

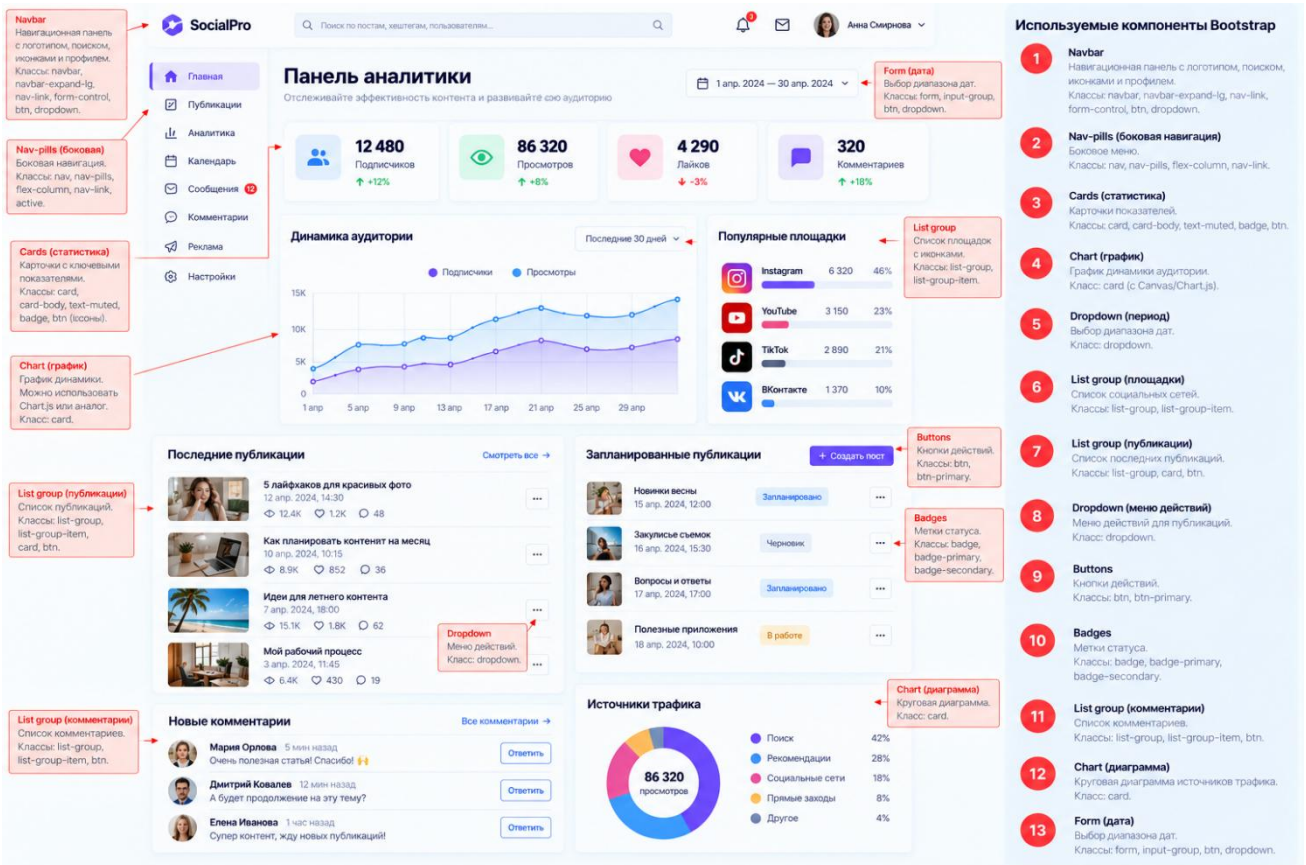
Управление. Список занятий оформить через list-group и accordion, текущий урок визуально выделить, выполнение показать progress и badge. Для переключения разделов использовать nav-tabs; воспроизведение реального видео необязательно.

Рекомендуемые компоненты Bootstrap. navbar, nav, list-group, progress, badge, accordion, card, tabs.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 11. Панель аналитики социальных сетей

Создать административную панель для анализа активности в социальных сетях: боковое меню, фильтр периода, карточки суммарных показателей, график изменения аудитории и список платформ с показателями охвата. Добавить таблицу популярных публикаций, календарь запланированных материалов, список комментариев и диаграмму распределения источников трафика. Статистику представить демонстрационными данными.



Каркас панели. Сформировать боковое меню аналитического сервиса, верхнюю панель с выбором периода, поиском и профилем. Заголовок рабочей области должен показывать, за какой период представлены данные.

Контент. В верхнем ряду разместить карточки подписчиков, охвата, вовлечённости и публикаций с процентным изменением. Ниже показать график динамики аудитории, сравнительный список социальных платформ, карточки популярных публикаций, календарь публикаций, последние комментарии и круговую диаграмму источников трафика.

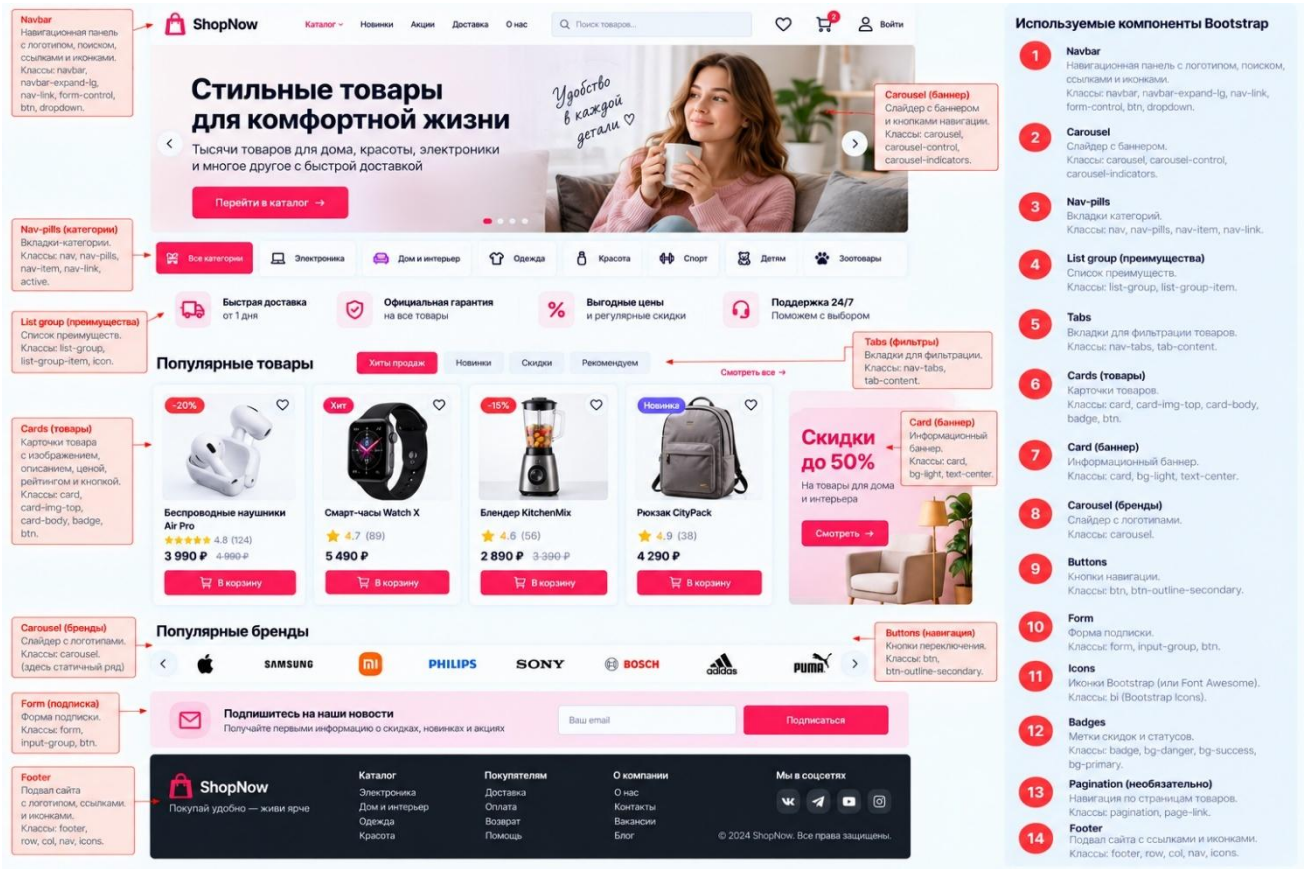
Управление. Использовать dropdown для периода и платформы, badge для изменения показателей и статусов публикаций, progress для сравнительных долей. Графики разрешается представить статическими изображениями или CSS-макетами без аналитической логики.

Рекомендуемые компоненты Bootstrap. navbar, card, badge, dropdown, table, list-group, progress.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 12. Интернет-магазин товаров повседневного спроса

Разработать главную страницу универсального интернет-магазина с поиском, категориями и большим промобаннером. Разместить преимущества сервиса, сетку карточек популярных товаров с фотографиями, ценами, рейтингами и акционными отметками, блок известных брендов, форму подписки и визуальные элементы избранного и корзины. Реальная обработка покупок не требуется.



Первый экран. Оформить универсальный магазин с логотипом, строкой поиска, меню категорий, иконками корзины и профиля. Под шапкой разместить широкий рекламный баннер с акционным предложением.

Контент. Добавить ряд преимуществ (доставка, гарантия, оплата, поддержка), блок категорий, сетку популярных товаров с фото, названием, рейтингом, ценой и скидкой. Ниже разместить рекламную карточку распродажи, логотипы популярных брендов, подписку на новости и информативный подвал.

Управление. Карточки должны содержать визуальные кнопки добавления в корзину и избранное. Для скидок и новинок использовать badge, для поиска — input-group, для подписки — form. Реальное оформление покупки не требуется.

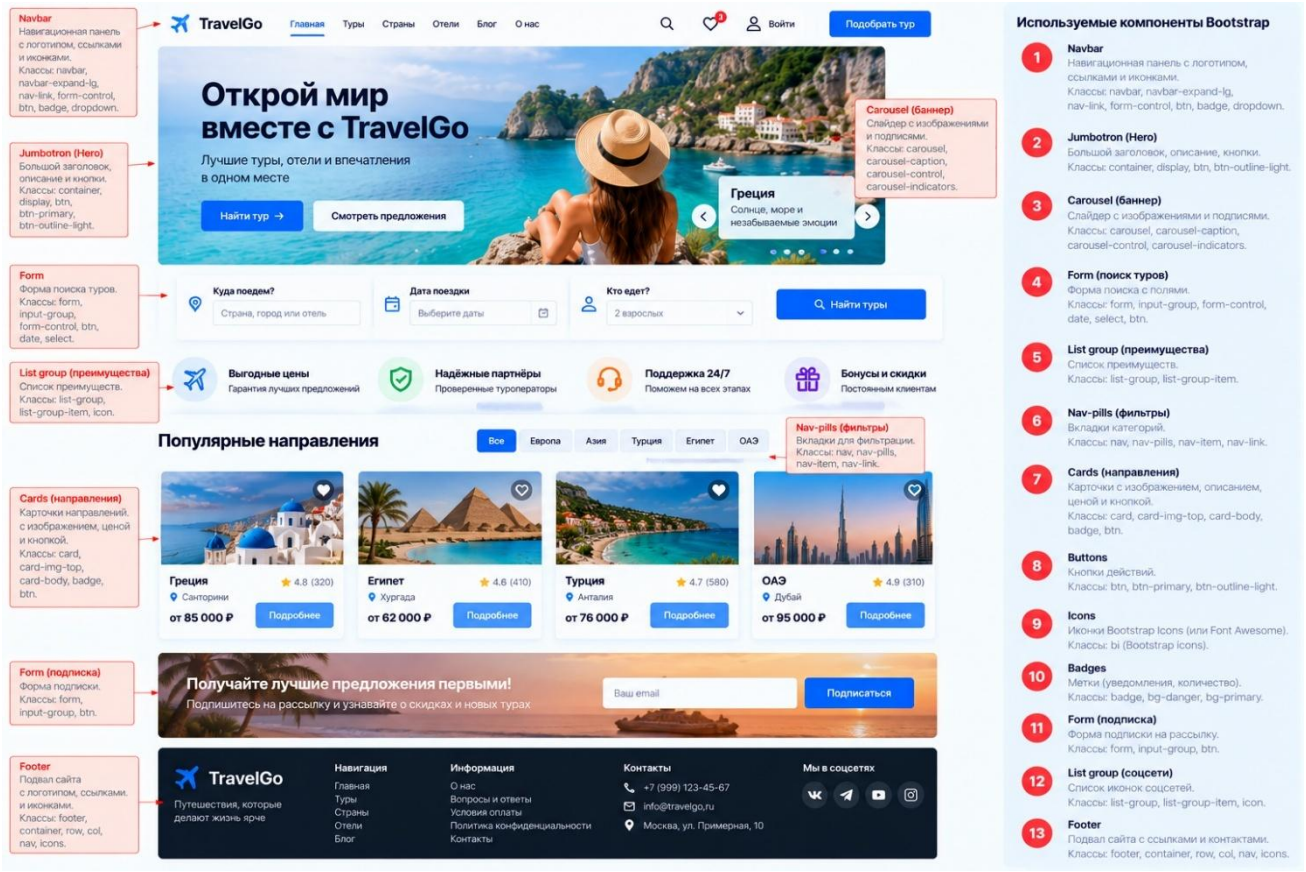
Рекомендуемые компоненты Bootstrap. navbar, carousel, card, badge, dropdown, form, button.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 13. Сервис поиска туристических направлений

Разработать стартовую страницу туристического сервиса с фотобаннером, предложением путешествий и формой поиска по направлению, датам и числу

путешественников. Ниже показать преимущества сервиса, популярные направления с изображениями и ценами, категории отдыха, специальное предложение и форму подписки. Предусмотреть кнопки просмотра подробностей и визуальные отметки избранного.



Первый экран. Создать навигацию туристического сервиса и большой фотобаннер с эмоциональным слоганом. На первом экране разместить форму поиска с направлением, датами поездки, количеством путешественников и кнопкой поиска.

Контент. Под формой показать преимущества сервиса, сетку популярных направлений с фотографиями, названием, ориентировочной ценой и рейтингом. Добавить категории отдыха, специальное предложение, рекомендации путешественникам и блок подписки на предложения.

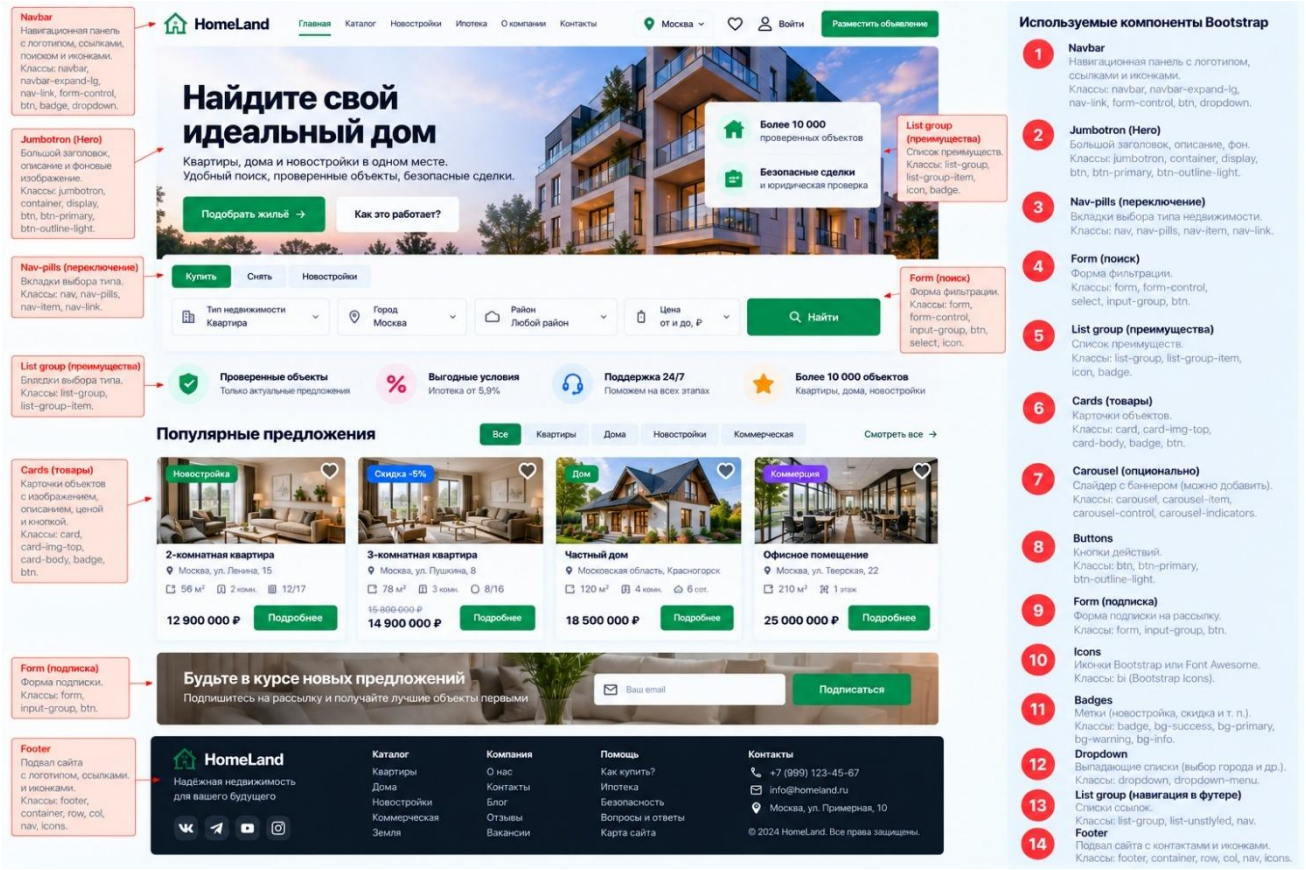
Управление. Для параметров поиска применить form-control, form-select и input-group, для направлений — card, для специальных предложений — badge. Кнопки поиска и просмотра направлений могут вести к демонстрационным секциям.

Рекомендуемые компоненты Bootstrap. navbar, carousel, card, badge, input-group, form-select, form, button.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 14. Сервис поиска недвижимости

Разработать страницу сервиса подбора недвижимости с презентационным баннером и формой поиска по типу объекта, расположению, стоимости и параметрам жилья. Ниже разместить преимущества сервиса, карточки популярных объектов с фотографиями, ценой, площадью и характеристиками, переключатели категорий недвижимости и блок подписки на новые предложения. Кнопки просмотра объекта и избранного оформить без обязательной серверной логики.



Первый экран. Создать шапку сервиса недвижимости с разделами «Купить», «Снять», «Новостройки», «О сервисе» и широким баннером с изображением современного жилого дома.

Контент. Под баннером разместить форму подбора с типом сделки, типом объекта, городом, диапазоном цены и числом комнат. Ниже представить преимущества сервиса и сетку объявлений с фото, ценой, адресом, площадью, количеством комнат и метками «Новое» или «Проверено». Добавить блок рекомендаций и подписку на новые объекты.

Управление. Типы сделок оформить через nav, поля поиска — form-select и input-group, карточки объявлений — card и badge. Кнопки просмотра объекта и избранного могут быть демонстрационными.

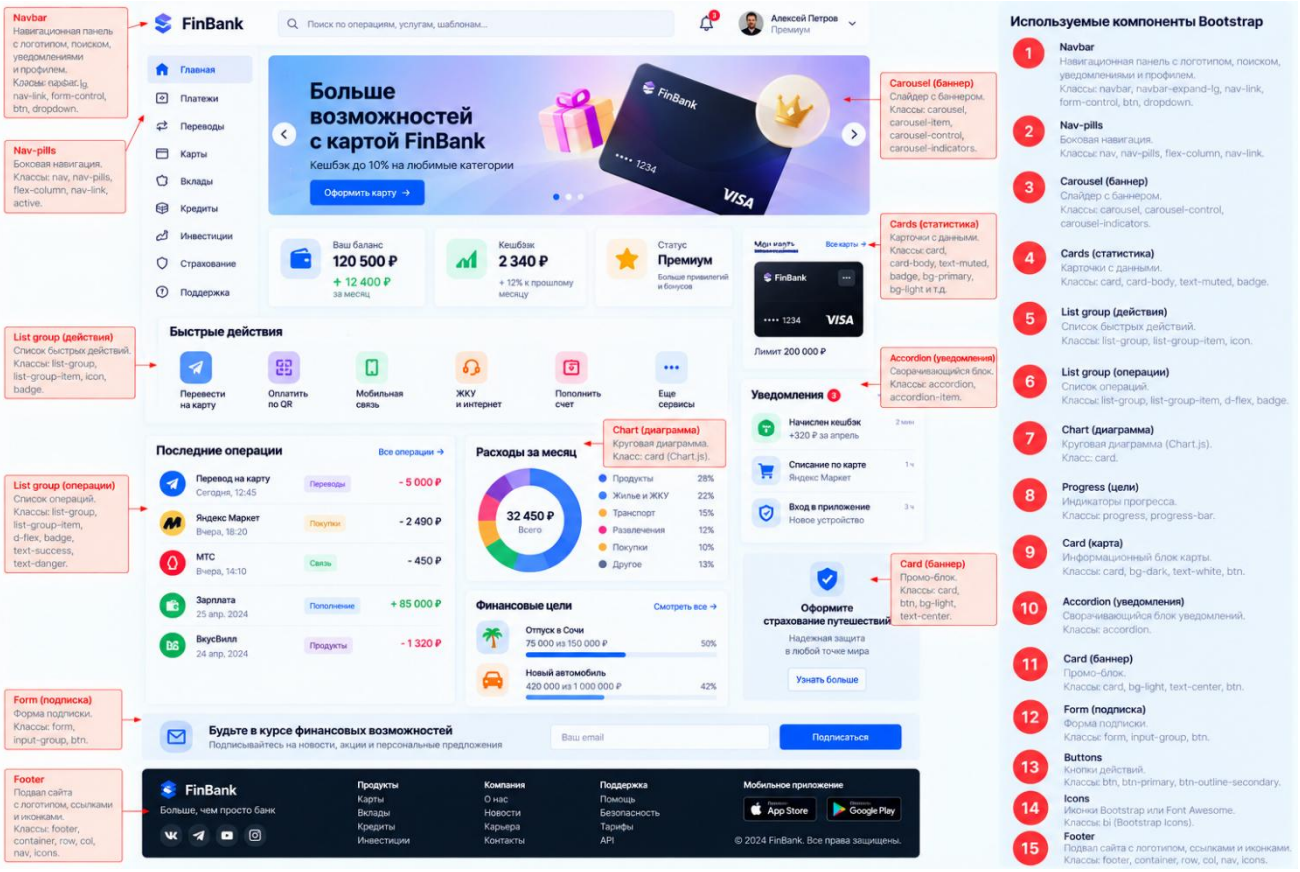
Рекомендуемые компоненты Bootstrap. navbar, card, badge, nav, form-select, input-group, form, button.

Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным

оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Вариант 15. Новостной портал

Разработать главную страницу новостного сайта с логотипом, меню рубрик и заметным блоком главной новости. Создать сетку карточек новостей с изображениями, заголовками, датами и категориями; дополнить её разделами популярных и последних материалов, блоком авторов, постраничной навигацией и формой подписки. Обеспечить чёткую визуальную иерархию редакционного контента.



Первый экран. Сформировать шапку новостного издания с названием, навигацией по рубрикам и поиском. Под ней разместить крупный материал дня с изображением, заголовком, лидом, рубрикой и датой.

Контент. Создать многоколоночную сетку публикаций, блок «Последние новости» с временем выхода, список самых читаемых материалов и подборку по рубрикам. Для карточек предусмотреть изображение, заголовок, краткий анонс, автора и дату. В нижней части разместить сведения о редакции и форму подписки.

Управление. Рубрики оформить nav или badge, подборку главных материалов — carousel, последние новости — list-group. Для перехода между страницами использовать pagination; реальная загрузка статей не требуется.

Рекомендуемые компоненты Bootstrap. navbar, card, badge, list-group, pagination, form, carousel.

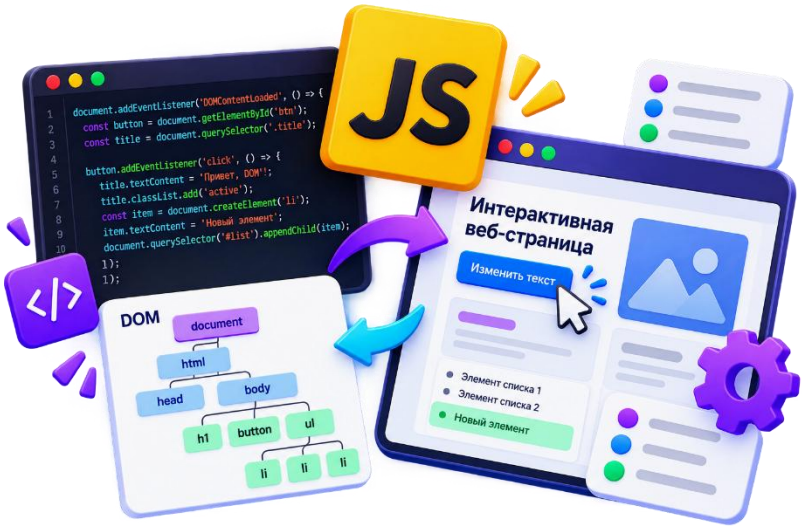
Ожидаемый результат. Целостная страница с реалистичными демонстрационными данными, ясной визуальной иерархией, адаптивной компоновкой и согласованным оформлением элементов. На мобильном экране блоки должны перестраиваться без горизонтальной прокрутки.

Контрольные вопросы

1. Что такое Bootstrap?
2. Для каких задач используется Bootstrap?
3. Какие преимущества предоставляет Bootstrap?
4. Как подключить Bootstrap к HTML-документу?
5. Что такое CDN?
6. Что такое система сеток Bootstrap?
7. Для чего используется container?
8. Чем container отличается от container-fluid?
9. Для чего используется row?
10. Для чего используются классы col-*?
11. Как создаётся кнопка Bootstrap?
12. Какие варианты оформления кнопок существуют?
13. Что такое card?
14. Для чего используется navbar?
15. Как создать форму с использованием Bootstrap?
16. Для чего используются form-control, form-select и form-check?
17. Что такое utility-классы?
18. Для чего используются классы m-*, p-*?
19. Для чего используется d-flex?
20. Что такое alert?
21. Для чего используется badge?
22. Что такое modal?
23. Как работает компонент dropdown?

24. Для чего используется accordion?
25. Для чего используется carousel?
26. Какие компоненты Bootstrap используют JavaScript?
27. Можно ли одновременно использовать Bootstrap и собственный CSS?
28. Почему не следует полностью заменять Bootstrap собственными стилями?
29. Как подготовить HTML-элемент для последующей работы с ним через JavaScript?
30. Как проверить корректность работы Bootstrap-компонентов?
31. Как проверить отображение страницы на различных размерах экрана?
32. Как опубликовать Bootstrap-проект на GitHub Pages?
33. Какие преимущества даёт использование готовых компонентов?
34. Какие недостатки может иметь использование большого количества готовых компонентов?
35. Чем создание интерфейса с Bootstrap отличается от разработки всех компонентов самостоятельно?

ПРАКТИЧЕСКАЯ РАБОТА 9. ИСПОЛЬЗОВАНИЕ JAVASCRIPT ДЛЯ РАБОТЫ С DOM



Цель работы:

- изучить основные конструкции языка JavaScript, типы данных, функции и объекты;
- получить навыки обработки событий пользователя;
- изучить принципы работы с DOM (Document Object Model);
- научиться получать доступ к элементам HTML-страницы и изменять их содержимое;
- научиться динамически добавлять и удалять элементы страницы;
- реализовать изменение классов и стилей элементов с помощью JavaScript;
- освоить работу с массивами и объектами для хранения данных приложения;
- реализовать простые визуальные эффекты и анимацию;
- разработать интерактивное клиентское веб-приложение без серверной части.

Задания к практической работе

Общая постановка задачи

Разработать интерактивное клиентское веб-приложение по назначенному варианту. Интерфейс должен реагировать на действия пользователя без перезагрузки страницы. Для хранения данных использовать массивы и объекты JavaScript; для изменения интерфейса — DOM и обработчики событий. Серверная часть и React не требуются.

Каждый вариант содержит требования к интерфейсу, действиям пользователя, проверкам и тестовому сценарию. Содержание соответствует тематике вариантов исходного практикума; дополнительные уточнения введены для однозначной реализации и проверки.

- Разделить проект на index.html, css/style.css и js/script.js; при необходимости создать папку с изображениями.
- Использовать querySelector/querySelectorAll либо другие методы поиска элементов, addEventListener, создание/удаление DOM-узлов, изменение textContent и CSS-классов.
- Обеспечить синхронизацию состояния JavaScript и интерфейса: любое добавление, изменение, удаление или фильтрация должны немедленно отражаться на странице.
- Применить функции, условия, массивы, объекты и методы массивов. Не подменять работу с DOM статическими заранее подготовленными экранами.
- Обработать пустые данные, некорректный ввод и действия, которые недопустимы в текущем состоянии; сообщения показывать в интерфейсе.
- Предусмотреть визуальную обратную связь для активных, выбранных, выполненных и ошибочных состояний.
- Проверить сценарии каждого варианта, консоль браузера и адаптивность страницы; разместить исходники в GitHub и опубликовать через GitHub Pages.

Примечание: сохранение состояния после перезагрузки через localStorage может быть выполнено дополнительно, но не является обязательным условием данной работы.

Требования к интерфейсу приложения

На основе выбранного варианта разработайте пользовательский интерфейс приложения.

Интерфейс должен:

- соответствовать назначению выбранного приложения;
- содержать все необходимые элементы управления;
- иметь понятную структуру;
- обеспечивать визуальное отображение текущего состояния данных;
- корректно отображаться на экране;
- обеспечивать удобное взаимодействие пользователя с приложением.

Необходимо предусмотреть различные состояния элементов интерфейса. Например:

- обычное состояние;
- выбранное состояние;
- активное состояние;
- выполненное состояние;
- состояние ошибки;
- недоступное состояние.

Для визуального отображения состояний следует использовать CSS-классы и их динамическое изменение с помощью JavaScript.

Реализация логики приложения с помощью JavaScript

Основная функциональность приложения должна быть реализована с использованием JavaScript.

В зависимости от выбранного варианта необходимо использовать:

- переменные и константы;
- условия if/else;
- функции;
- массивы;
- объекты;
- циклы;
- методы работы с массивами;
- обработчики событий;
- взаимодействие с DOM.

Для обработки действий пользователя рекомендуется использовать метод addEventListener().

Данные приложения могут храниться в массивах и объектах JavaScript. При изменении данных интерфейс страницы должен обновляться соответствующим образом.

Работа с DOM и интерактивностью

Для управления интерфейсом необходимо осуществить взаимодействие JavaScript с DOM.

В приложении должны быть реализованы операции, соответствующие выбранному варианту:

- получение элементов страницы;
- обработка событий пользователя;
- изменение содержимого элементов;
- изменение CSS-классов;
- добавление новых элементов;
- удаление элементов;
- изменение состояния существующих элементов;
- отображение и скрытие блоков;
- динамический пересчёт данных.

Например, при нажатии пользователем на кнопку или другой интерактивный элемент соответствующее изменение должно быть сразу отражено на странице.

В исходной практической работе аналогичный принцип применяется для интерактивного выбора мест: JavaScript обрабатывает действия пользователя и изменяет состояние элементов страницы в реальном времени. Практикум по технологиям создан…

Проверка пользовательского ввода

Если выбранный вариант предусматривает ввод данных пользователем, необходимо реализовать проверку введённых значений.

Например:

- обязательное поле не должно оставаться пустым;
- числовое значение должно находиться в допустимом диапазоне;
- нельзя создать дублирующую запись, если это противоречит логике приложения;
- нельзя выполнить операцию над отсутствующим элементом.

При обнаружении ошибки необходимо вывести понятное сообщение пользователю и предотвратить выполнение некорректной операции.

Визуальные эффекты

Для улучшения взаимодействия с приложением необходимо реализовать несколько простых визуальных эффектов.

В зависимости от варианта можно использовать:

- изменение цвета элемента;
- выделение выбранного элемента;
- появление или скрытие блока;
- плавное изменение состояния;
- анимацию добавления или удаления элемента;
- изменение размера или положения элемента;
- визуальное отображение прогресса.

JavaScript должен использоваться для управления состоянием элемента, а CSS — для его визуального оформления и анимации.

Требования к результату работы

В результате выполнения практической работы студент должен предоставить:

- исходный код HTML-страницы;
- CSS-файлы;
- JavaScript-файлы;
- реализованное интерактивное приложение;
- рабочую версию приложения, опубликованную через GitHub Pages;
- ссылку на опубликованную страницу;
- репозиторий с исходными файлами проекта.

Приложение должно демонстрировать практическое использование **JavaScript, событий и DOM**, а его интерфейс должен изменяться в зависимости от действий пользователя.

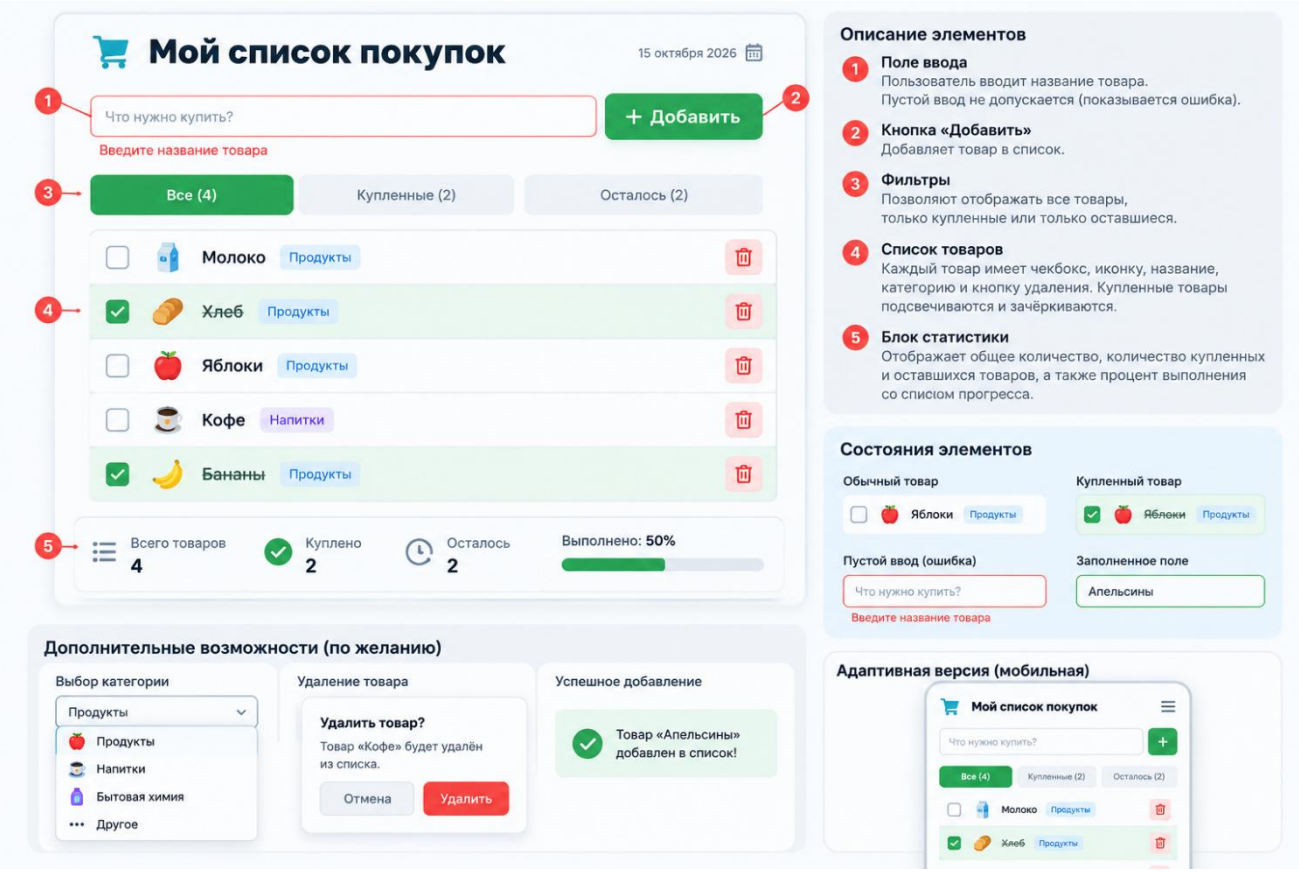
Вариант 1. Интерактивный список покупок

Интерфейс приложения. Форма ввода названия и категории, кнопка добавления, список с отметками, панель фильтров и счётчики.

Поведение и работа с DOM. Хранить товары как объекты с id, названием, категорией и признаком покупки. После добавления создавать строку списка, по отметке менять оформление, по удалению убирать запись; фильтры «Все», «Купить», «Куплено» должны обновлять видимые элементы.

Проверка данных и особые состояния. Не принимать пустое название; не допускать повторов названий без предупреждения. Счётчики общего, купленного и оставшегося пересчитывать после каждого действия.

Сценарий проверки. Добавить три товара, отметить один купленным, отфильтровать список, удалить запись и убедиться, что статистика обновилась.



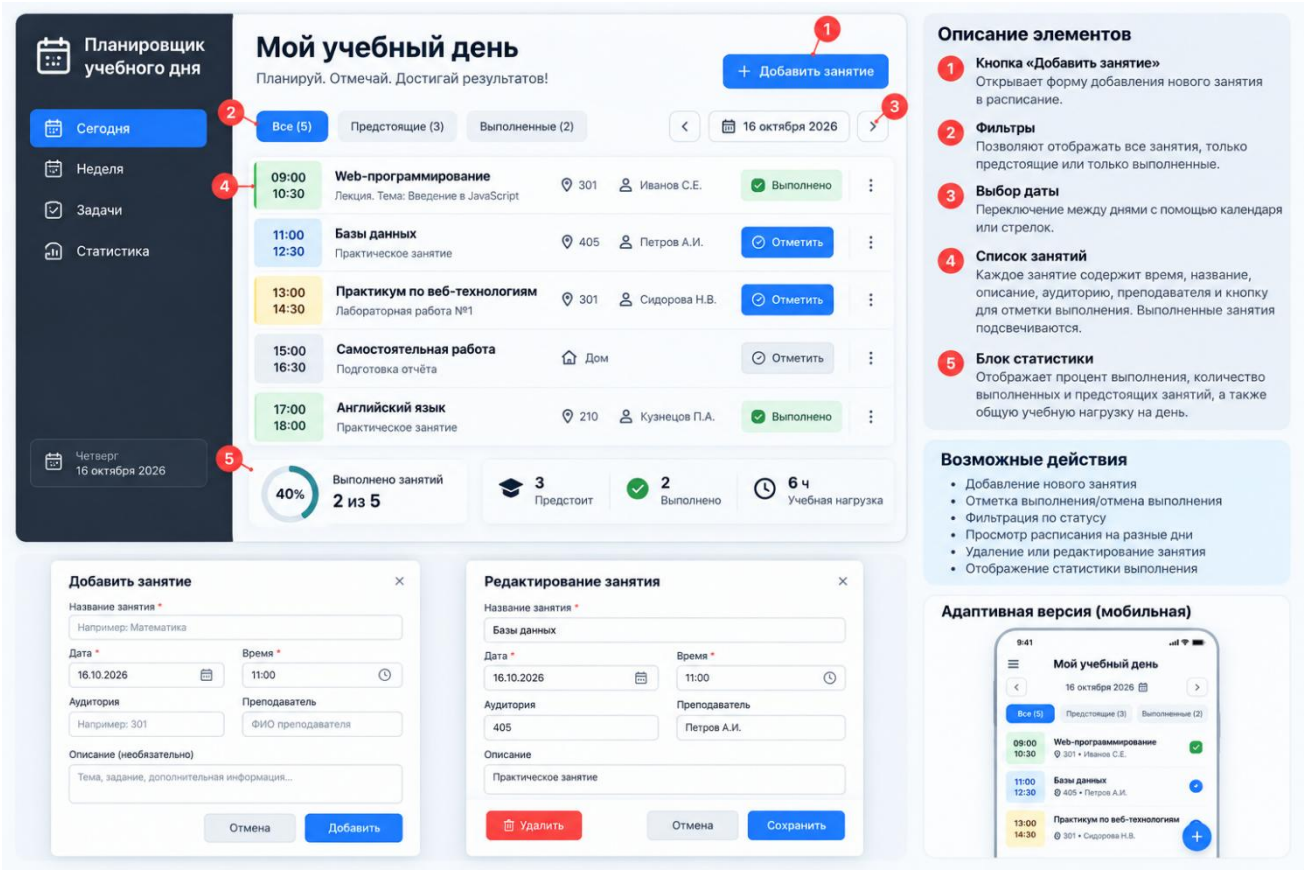
Вариант 2. Планировщик учебного дня

Интерфейс приложения. Переключатель даты, список занятий в хронологическом порядке, форма с названием, временем и примечанием, счётчики выполнения.

Поведение и работа с DOM. Представить занятия объектами с id, датой, временем, названием, примечанием и статусом. Реализовать создание, редактирование, удаление и отметку выполнения; при смене даты показывать только соответствующие записи и пересчитывать статистику.

Проверка данных и особые состояния. Проверять обязательное название и корректное время; при необходимости предупреждать о занятиях с одинаковыми датой и временем.

Сценарий проверки. Создать два занятия на разные даты, отредактировать одно, выполнить другое и проверить переключение дат.



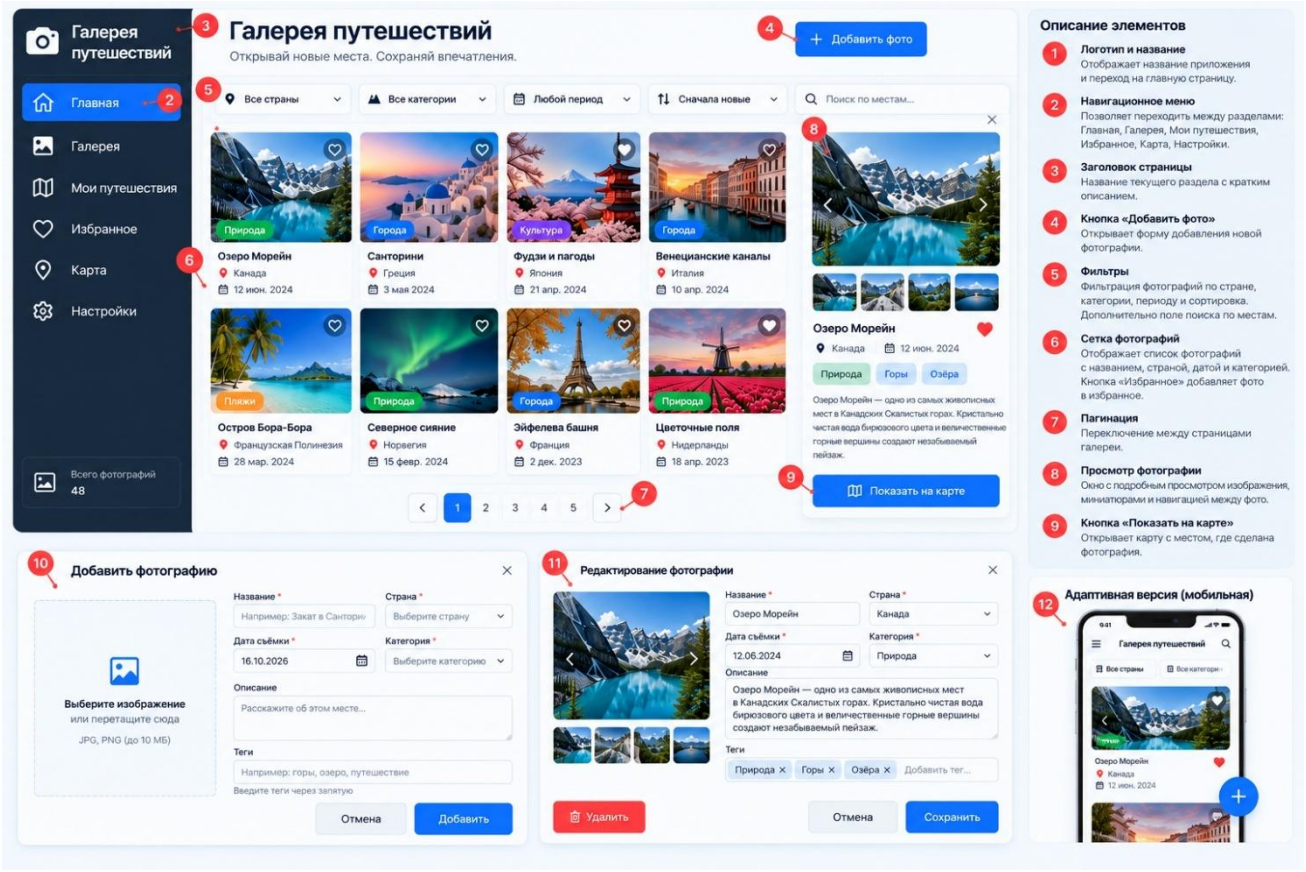
Вариант 3. Галерея путешествий

Интерфейс приложения. Сетка фотографий, фильтр направлений, полноэкранный или модальный просмотр, подпись и кнопка избранного.

Поведение и работа с DOM. Исходные фотографии хранить в массиве объектов с id, адресом изображения, названием, описанием и категорией. По нажатию открывать выбранное фото; кнопки «Назад» и «Вперёд» переключают записи в текущей выборке, избранное меняет значок и состояние карточки.

Проверка данных и особые состояния. Предусмотреть отсутствие фотографий в категории и корректное переключение на первой и последней фотографии.

Сценарий проверки. Выбрать категорию, открыть фотографию, пролистать несколько снимков и добавить один в избранное.



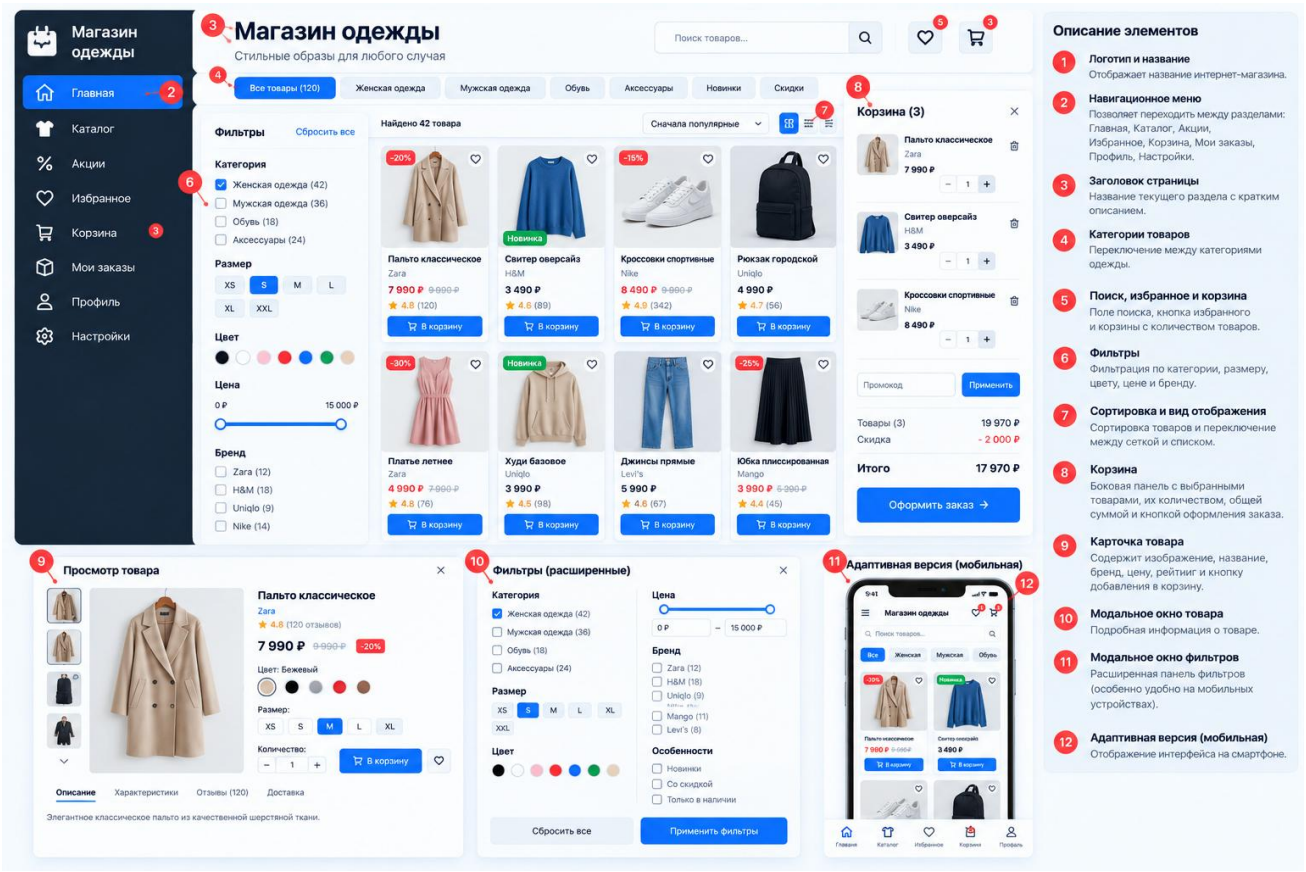
Вариант 4. Трекер привычек

Интерфейс приложения. Список привычек с отметками выполнения, поле добавления, сводка дня и индикатор общего прогресса.

Поведение и работа с DOM. Каждую привычку представить объектом с id, названием и признаком выполнения за выбранный день. Реализовать добавление, удаление, переключение состояния и динамический пересчёт доли выполненных привычек.

Проверка данных и особые состояния. Запретить пустые названия, корректно показывать 0 % при пустом списке и 100 % при выполнении всех пунктов.

Сценарий проверки. Создать три привычки, отметить две и проверить число выполненных и процент прогресса.



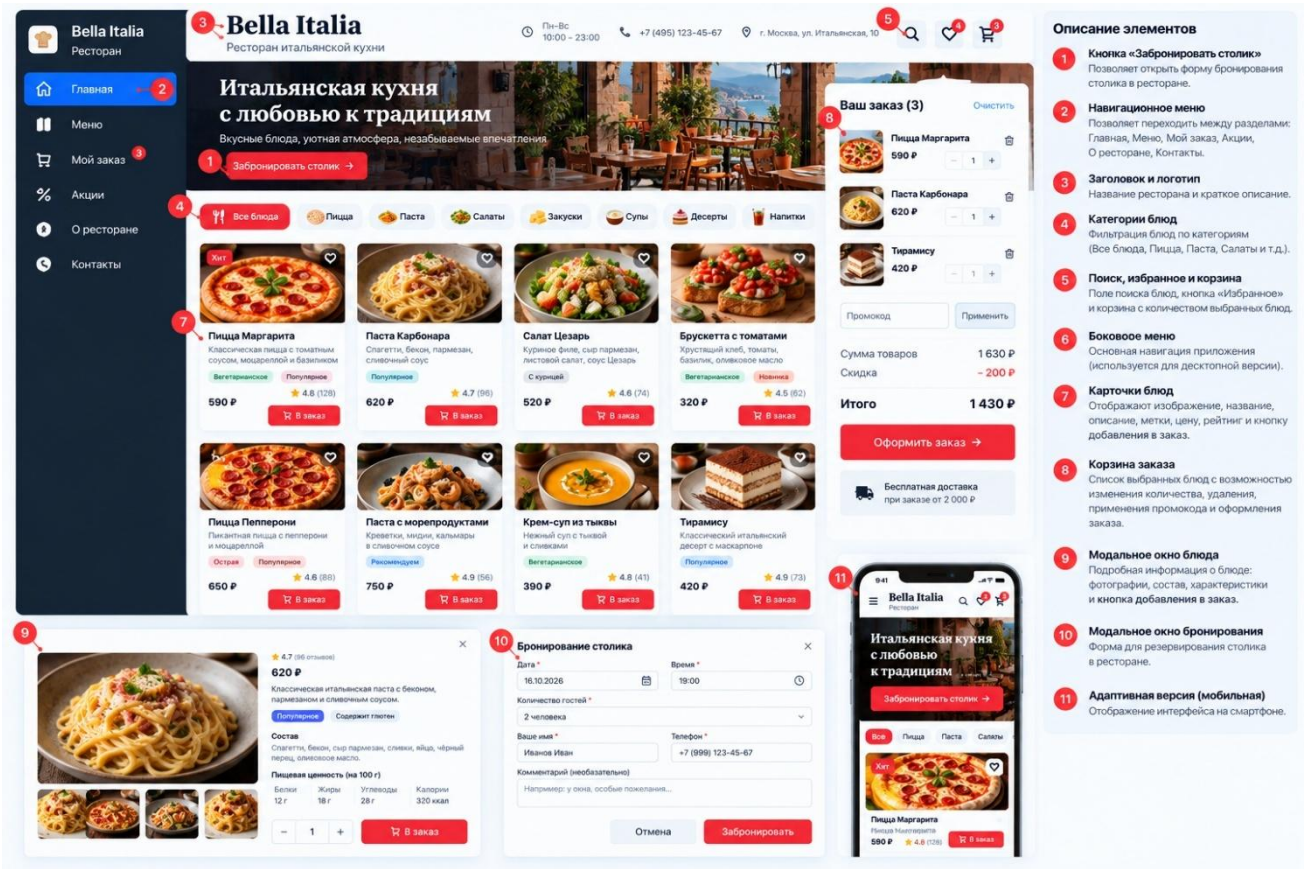
Вариант 5. Меню ресторана

Интерфейс приложения. Панель категорий, карточки блюд с фото и ценой, область заказа с количеством и общей суммой.

Поведение и работа с DOM. Данные блюд хранить как объекты с id, категорией, названием, описанием и ценой. При выборе категории перестраивать каталог; добавление блюда увеличивает количество в заказе, кнопки «+» и «−» изменяют его, удаление исключает позицию; итог рассчитывать из цен и количества.

Проверка данных и особые состояния. Не допускать отрицательного количества; пустой заказ сопровождать сообщением. Стоимость отображать в едином формате.

Сценарий проверки. Добавить блюда двух категорий, изменить количество, удалить позицию и проверить сумму.



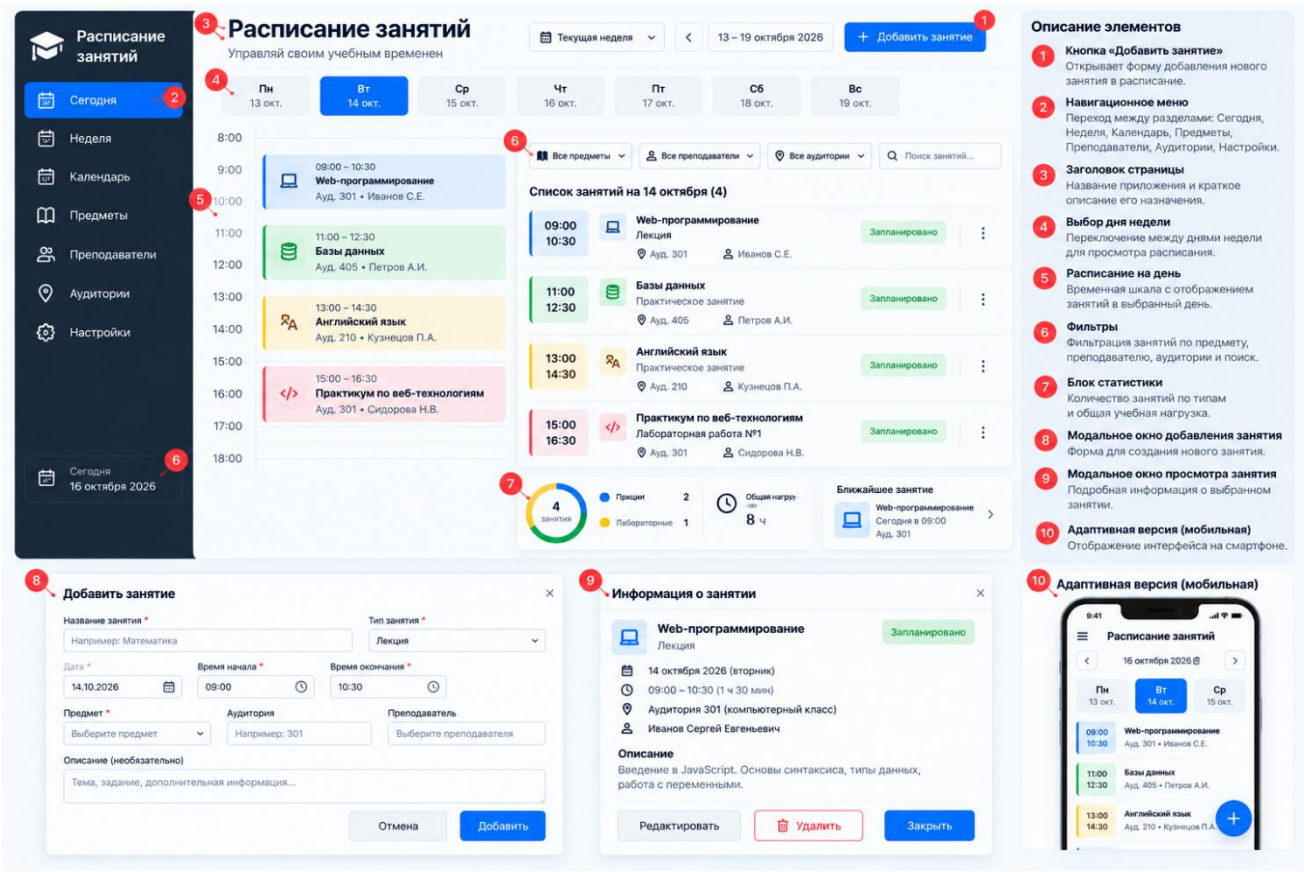
Вариант 6. Расписание занятий

Интерфейс приложения. Навигация по дням недели, карточки занятий, фильтр предметов и панель подробностей.

Поведение и работа с DOM. Хранить занятия с днём недели, временем, аудиторией, преподавателем и предметом. Переключение дня и фильтра предмета должно обновлять расписание; выбор занятия выделяет карточку и выводит детали.

Проверка данных и особые состояния. Для дня без занятий показывать заглушку; после смены дня сбрасывать некорректный выбор. Проверить порядок отображения по времени.

Сценарий проверки. Переключить два дня, отфильтровать предмет, открыть детали занятия.



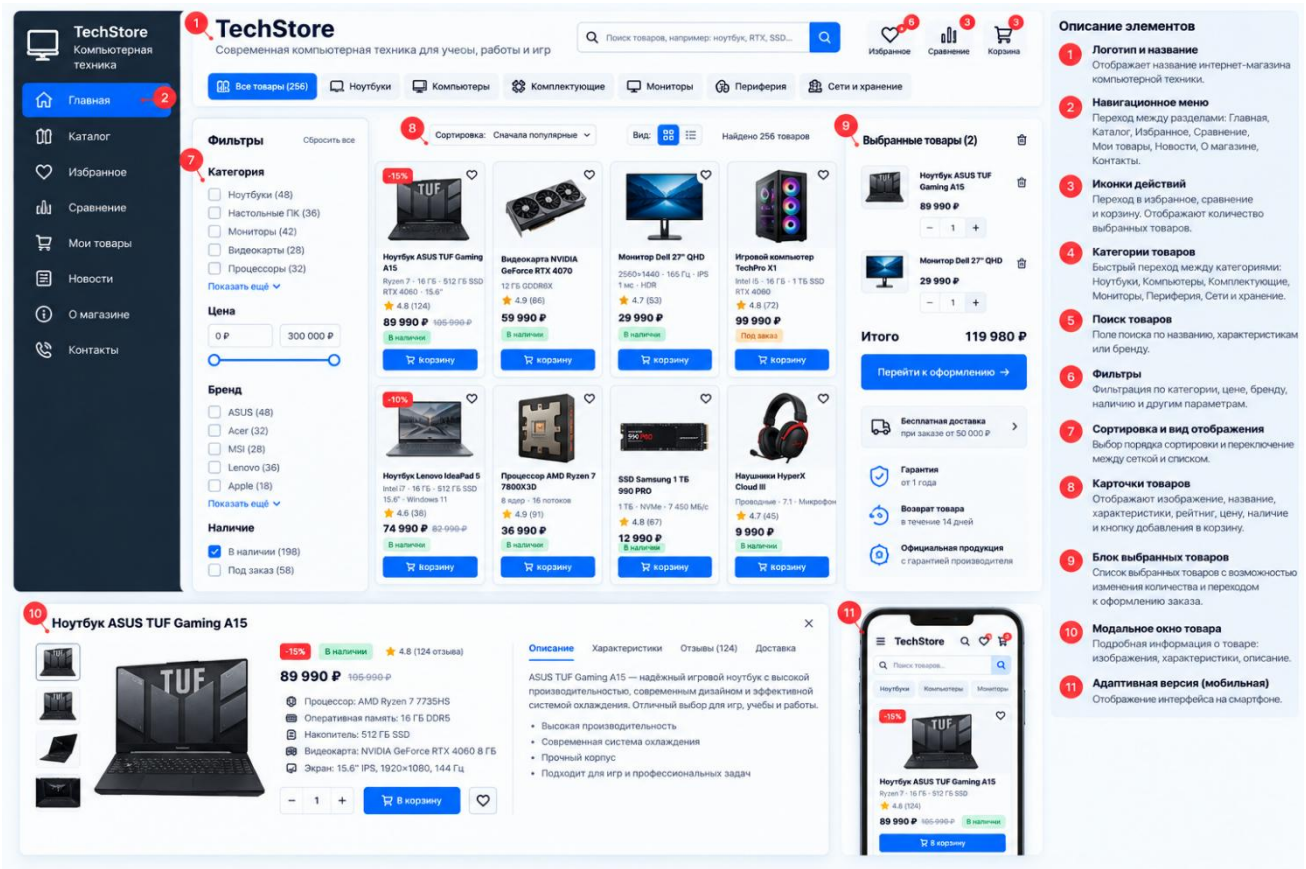
Вариант 7. Каталог компьютерной техники

Интерфейс приложения. Каталог карточек техники, фильтр категории, сортировка, подробная карточка и список выбранного.

Поведение и работа с DOM. Товары описать объектами с названием, категорией, ценой, характеристиками и изображением. При смене фильтра и сортировки перерисовывать каталог; подробности показывать по нажатию, выбранные товары добавлять и удалять без дублирования.

Проверка данных и особые состояния. Корректно обрабатывать пустую выдачу и отсутствие выбранных товаров; сортировать числовую цену как число.

Сценарий проверки. Отсортировать по цене, выбрать два товара, удалить один и проверить список.



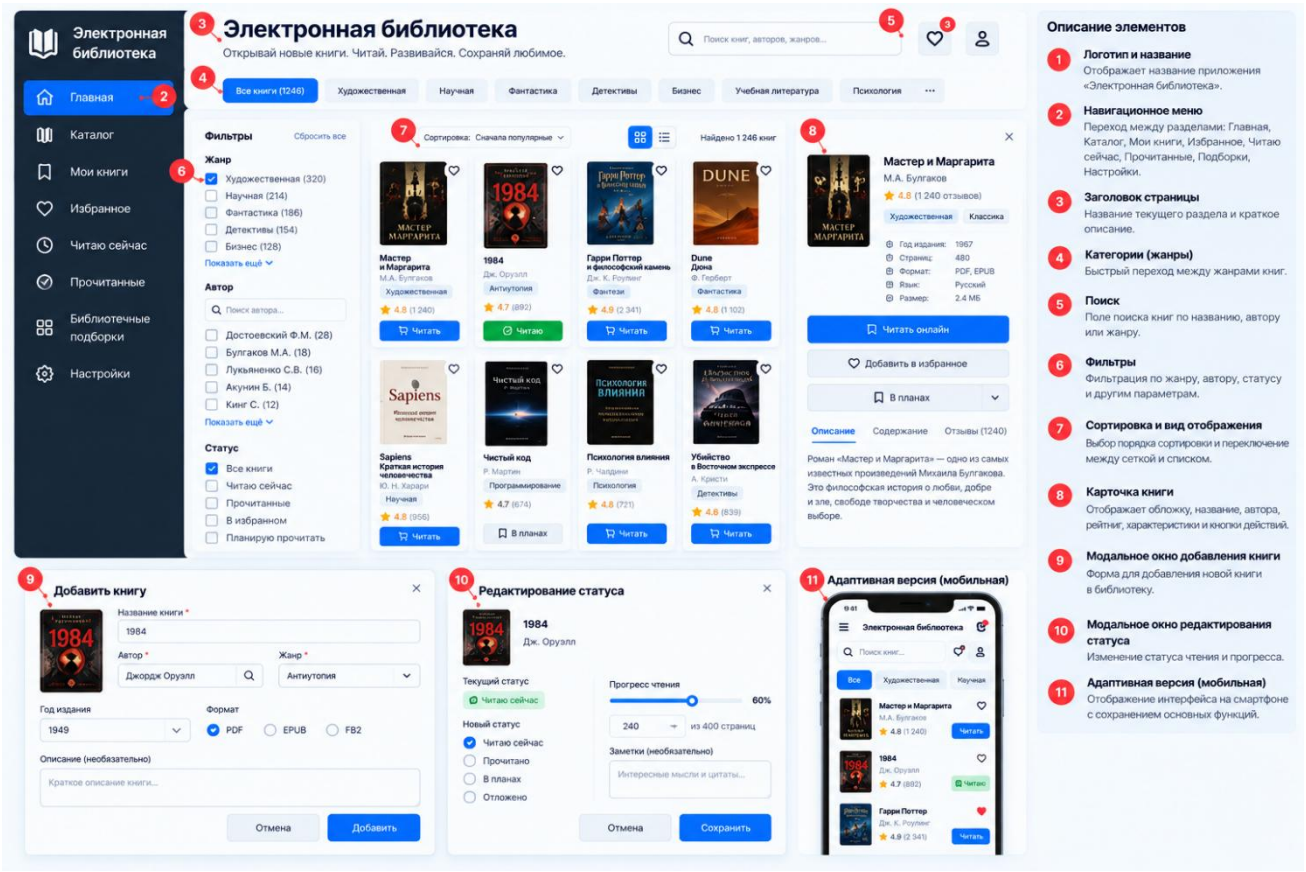
Вариант 8. Электронная библиотека

Интерфейс приложения. Поисковая строка, выбор жанра, сетка книг, карточка подробностей и раздел избранного.

Поведение и работа с DOM. Книги хранить с id, автором, названием, жанром, аннотацией и статусом чтения. Реализовать поиск по названию без учёта регистра, фильтр жанра, смену статуса «Не прочитана / Читаю / Прочитана» и добавление в избранное.

Проверка данных и особые состояния. При пустом результате показывать сообщение; статус и избранное должны сохраняться в состоянии приложения при фильтрации.

Сценарий проверки. Найти книгу, сменить статус, добавить в избранное и проверить поиск.



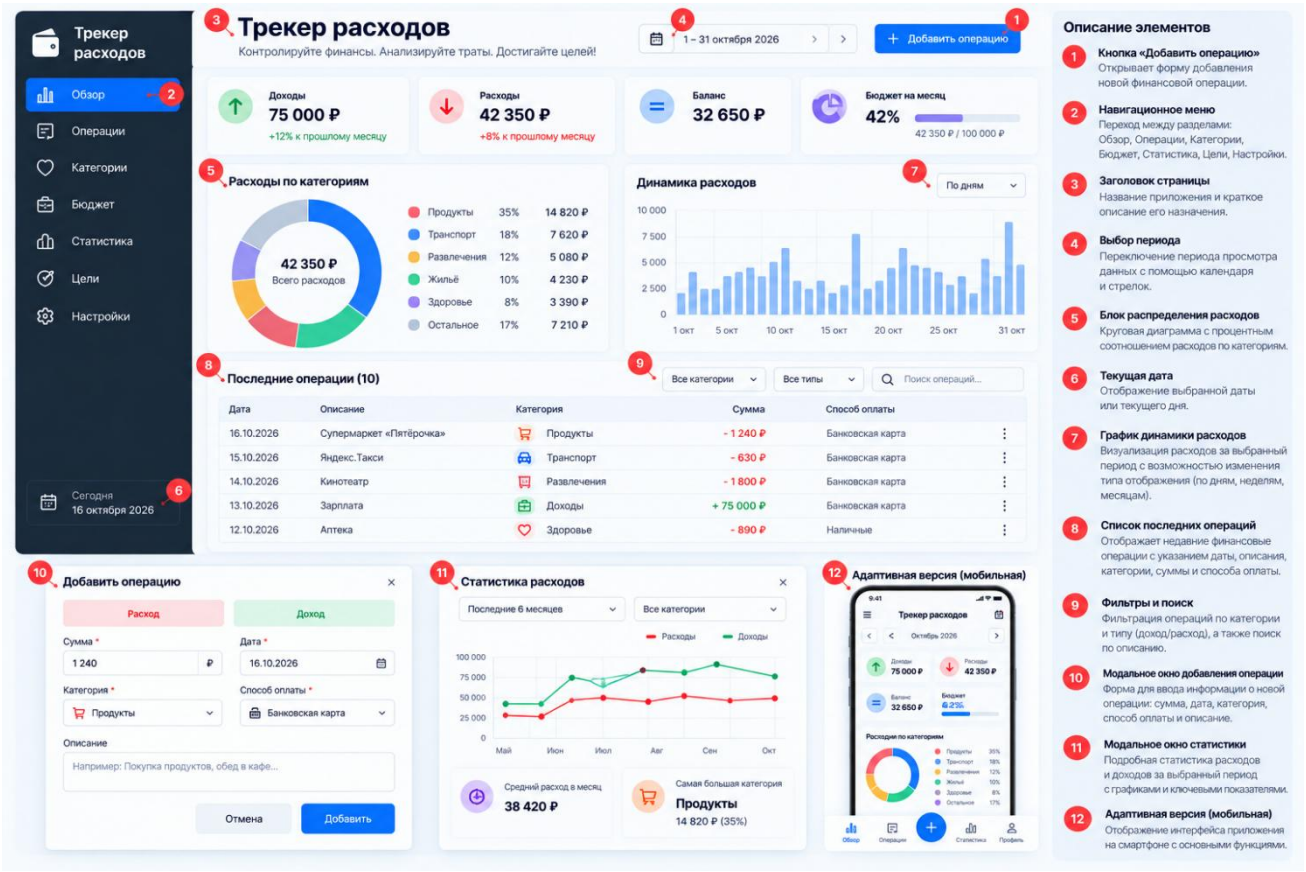
Вариант 9. Трекер расходов

Интерфейс приложения. Форма расхода с суммой, категорией, датой и описанием, список операций, фильтр и итоговые показатели.

Поведение и работа с DOM. Расходы хранить как объекты с id, суммой, датой, категорией и комментарием. Добавление и удаление должны обновлять список, общую сумму и суммы по категориям; фильтр отображает выбранную группу операций.

Проверка данных и особые состояния. Проверять положительную числовую сумму и обязательную категорию; пустой список сопровождать подсказкой. Ясно обозначить, относится ли итог к общим или отфильтрованным данным.

Сценарий проверки. Добавить три расхода в двух категориях, отфильтровать и удалить один; проверить итоги.



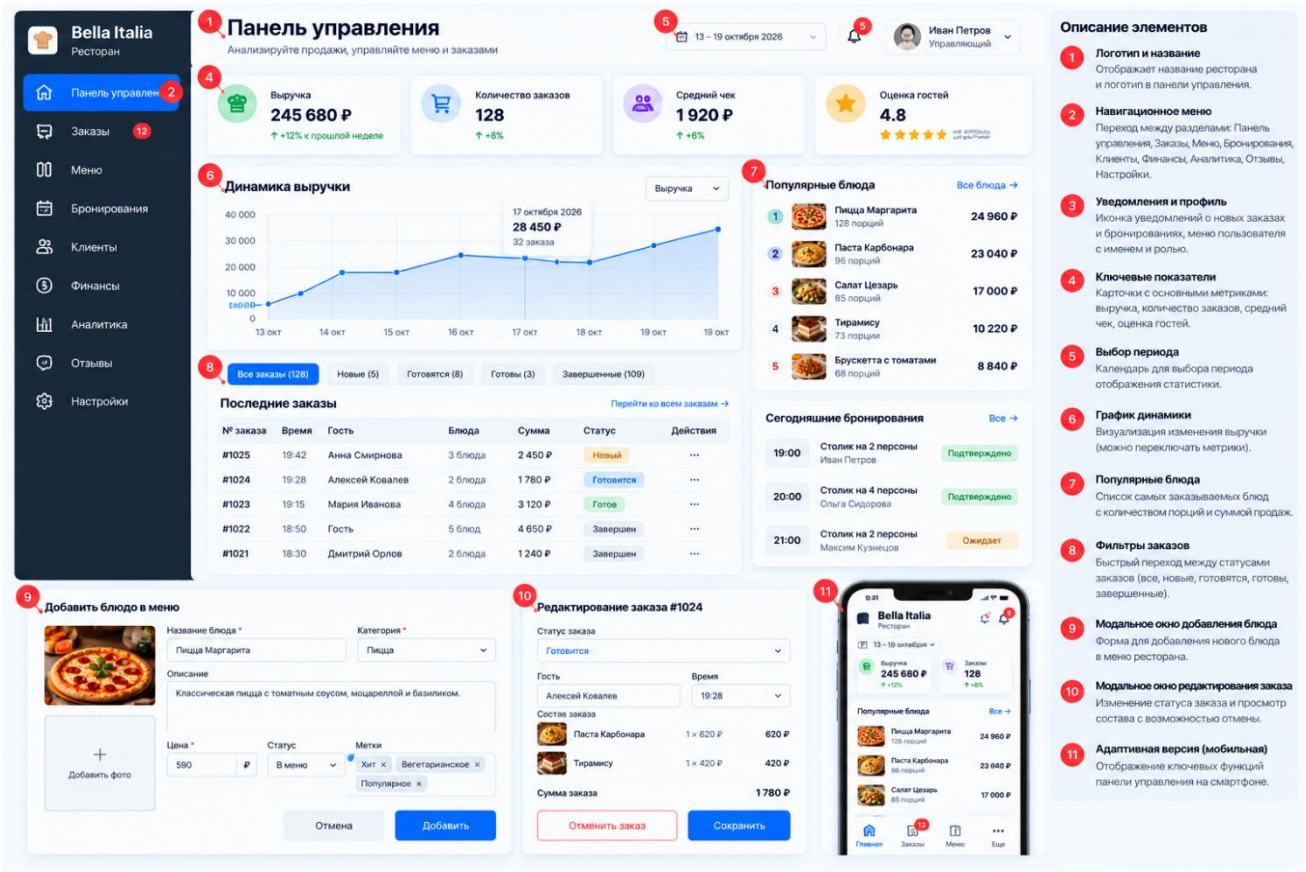
Вариант 10. Конструктор пиццы

Интерфейс приложения. Панель выбора размера и основы, список ингредиентов с ценами, визуальная сводка состава и итоговая стоимость.

Поведение и работа с DOM. Создать справочники размеров, основ и ингредиентов. При выборе размера, основы и переключении ингредиентов обновлять описание пиццы и стоимость как сумму базовой цены и выбранных добавок.

Проверка данных и особые состояния. Ограничить выбор одной основы и одного размера; не учитывать ингредиент дважды; корректно отображать цену без дополнительных ингредиентов.

Сценарий проверки. Изменить размер, выбрать основу, добавить и удалить два ингредиента, проверить перерасчёт.



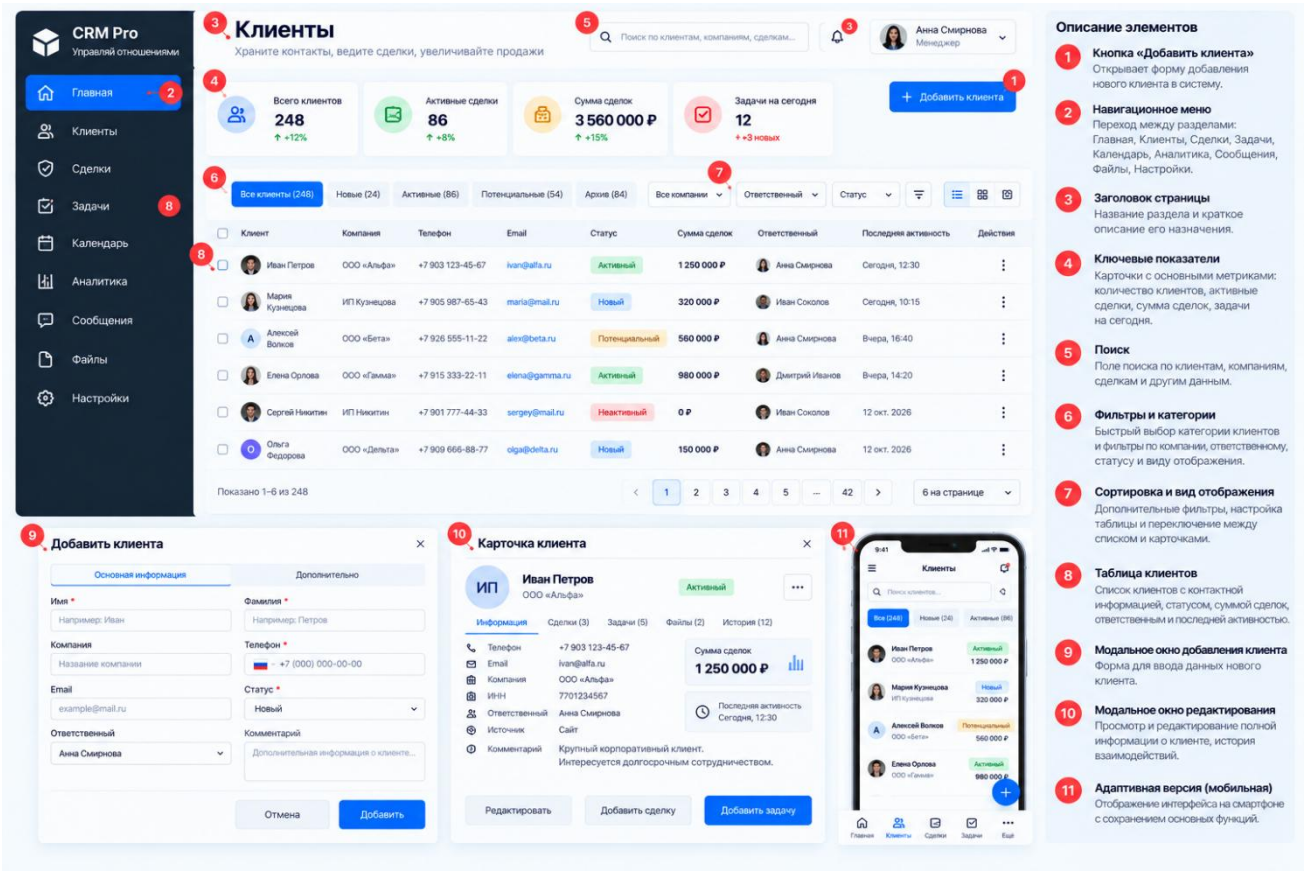
Вариант 11. Диспетчер задач проекта

Интерфейс приложения. Форма задачи, список или колонки статусов, фильтры по приоритету и состоянию, общий прогресс.

Поведение и работа с DOM. Задачи хранить с id, названием, описанием, приоритетом и статусом. Реализовать создание, редактирование, удаление, переход между состояниями «Новая», «В работе», «Готово», фильтрацию и динамический процент завершения.

Проверка данных и особые состояния. Не принимать пустое название, показывать пустые состояния и пересчитывать прогресс после изменения статуса и удаления.

Сценарий проверки. Создать задачи разных приоритетов, изменить статус, отредактировать и удалить одну запись.



Вариант 12. Бронирование переговорной комнаты

Интерфейс приложения. Выбор даты, сетка временных слотов, форма бронирования с именем и целью встречи, список записей.

Поведение и работа с DOM. Бронирования хранить с id, датой, временем начала и окончания, ответственным и темой. При смене даты показывать занятые интервалы; при создании проверять пересечения, добавлять запись и блокировать занятые слоты; отмена освобождает время.

Проверка данных и особые состояния. Проверять обязательные поля, допустимость времени и отсутствие пересечений; при конфликте выводить понятное сообщение.

Сценарий проверки. Создать бронирование, попытаться занять тот же интервал, затем отменить запись и проверить освобождение слота.

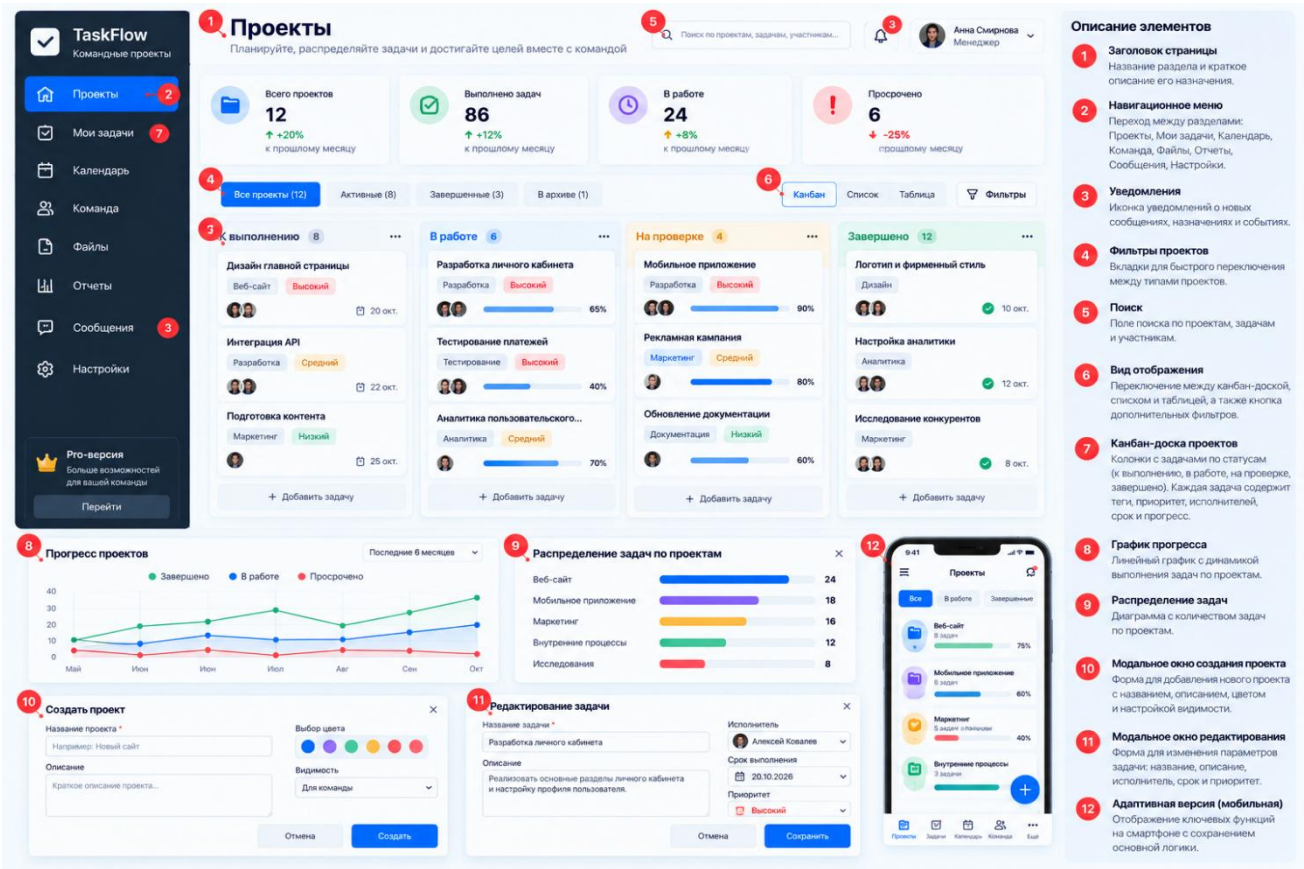
Вариант 13. Панель мониторинга погоды

Интерфейс приложения. Выбор города, крупный показатель температуры, карточки влажности, ветра и давления, переключатель периода.

Поведение и работа с DOM. Подготовить массив погодных объектов для нескольких городов и периодов. При смене города или периода обновлять показатели, подписи, иконку и оформление блока в зависимости от погоды.

Проверка данных и особые состояния. Не требовать внешнего API; значения должны браться из подготовленных данных. Предусмотреть сообщение при отсутствии данных для выбранного периода.

Сценарий проверки. Переключить два города и два периода, убедиться в обновлении всех показателей и темы.



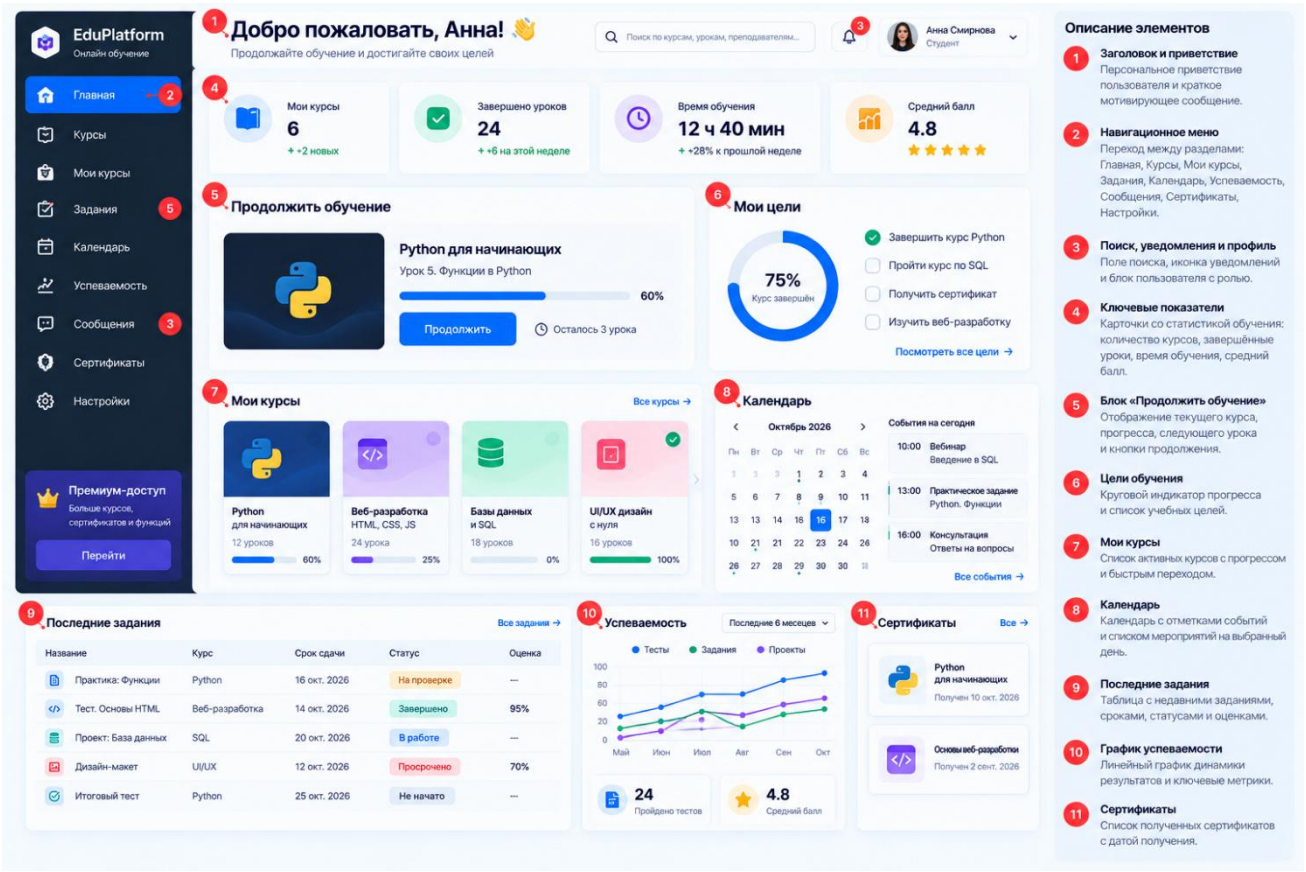
Вариант 14. Интерактивная викторина

Интерфейс приложения. Экран вопроса с вариантами ответов, индикатор номера, кнопка перехода и итоговый экран.

Поведение и работа с DOM. Хранить вопросы как объекты с текстом, вариантами и индексом правильного ответа. По выбору фиксировать ответ, показывать обратную связь, блокировать повторный выбор и разрешать переход дальше; после последнего вопроса вывести результат и кнопку перезапуска.

Проверка данных и особые состояния. Не разрешать переход без ответа (либо явно объяснить пропуск); не начислять баллы дважды; корректно сбрасывать состояние при повторном прохождении.

Сценарий проверки. Пройти викторину с правильными и ошибочными ответами, проверить счёт и перезапуск.



Вариант 15. Мини-магазин с корзиной

Интерфейс приложения. Каталог товаров с фильтрами, карточки с кнопкой покупки, панель корзины, управление количеством и итог.

Поведение и работа с DOM. Хранить товары и позиции корзины в массивах объектов. Фильтровать каталог, показывать подробности, добавлять товар в корзину, увеличивать и уменьшать количество, удалять позиции и динамически пересчитывать сумму и число единиц товара.

Проверка данных и особые состояния. Не допускать отрицательного количества и повторных независимых строк одного товара; пустую корзину отображать отдельно.

Сценарий проверки. Добавить два товара, увеличить количество первого, удалить второй и проверить итог и счётчик корзины.

Контрольные вопросы

1. Что такое JavaScript и какую роль он играет в веб-разработке?
2. Как подключить JavaScript к HTML-документу?
3. Что такое обработчик событий и как его использовать в JavaScript?
4. Как реализовать обработку события клика на элементе HTML?
5. Что такое DOM и как JavaScript взаимодействует с ним?
6. Как получить доступ к элементам HTML с помощью JavaScript?
7. Что такое методы getElementById, getElementsByClassName и querySelector?
8. Как изменить текст или содержимое элемента на странице с помощью JavaScript?
9. Что такое массивы в JavaScript и как они могут быть использованы для хранения данных о местах?
10. Как добавить и удалить элементы из массива в JavaScript?
11. Как использовать условия (if/else) для изменения поведения скрипта в зависимости от состояния места?
12. Что такое функции в JavaScript и как их использовать для организации кода?
13. Как можно передавать параметры в функции и возвращать значения?
14. Что такое циклы (for, while) и как их использовать для перебора массивов?
15. Как использовать метод addEventListener для добавления обработчиков событий?
16. Что такое JSON и как его использовать для хранения и передачи данных?
17. Как реализовать визуальные изменения (например, цвет) при выборе места?
18. Как создать простую форму для ввода данных и обработать ее с помощью JavaScript?

19. Что такое AJAX и как он может быть использован для динамического обновления контента на странице?
20. Как отладить JavaScript-код с помощью консоли браузера?

ПРАКТИЧЕСКАЯ РАБОТА 10. СОЗДАНИЕ И ИНТЕГРАЦИЯ REACT КОМПОНЕНТ

Цель работы:

- изучить основные принципы библиотеки React;
- познакомиться с компонентным подходом к разработке пользовательских интерфейсов;
- научиться создавать функциональные React-компоненты;
- изучить передачу данных между компонентами с помощью props;
- освоить управление состоянием компонента с использованием useState;
- познакомиться с использованием useEffect для выполнения побочных эффектов;
- научиться обрабатывать события пользователя в React;
- реализовать динамическое изменение интерфейса в зависимости от состояния приложения;
- освоить передачу данных между компонентами;
- применить CSS для оформления React-компонентов;
- выполнить тестирование созданного компонента;
- интегрировать компонент в веб-страницу;
- опубликовать готовый проект на GitHub Pages.

Задания к практической работе

1. Общая постановка задачи

Необходимо разработать **интерактивное React-приложение или самостоятельный React-компонент** в соответствии с выбранным вариантом.

Приложение должно представлять собой законченный пользовательский интерфейс, построенный с использованием компонентного подхода React.

В отличие от предыдущей практической работы, где интерактивность реализовывалась непосредственно через работу с DOM, в данной работе необходимо организовать интерфейс в виде **React-компонентов**, состояние которых управляется средствами React.

Создаваемое приложение должно:

- состоять из одного или нескольких React-компонентов;
- использовать состояние для хранения изменяемых данных;
- реагировать на действия пользователя;
- динамически изменять содержимое интерфейса;
- передавать данные между компонентами;
- использовать CSS для оформления;
- корректно обрабатывать основные пользовательские сценарии.

В исходной методичке для React также предусмотрено создание компонента, обработка состояния с помощью хуков, стилизация, тестирование и последующая публикация проекта.

Задание выполняется **по вариантам**. Каждый вариант представляет отдельную предметную область и содержит индивидуальный набор данных, компонентов и функциональных возможностей.

Создание React-проекта

Создайте проект React с использованием рекомендуемого инструментария. Структура проекта должна обеспечивать разделение компонентов, стилей и вспомогательных файлов. Например:

```
project/
│
├── src/
│   ├── components/
│   │   ├── Component1.jsx
│   │   ├── Component2.jsx
│   │   └── ...
│   │
│   ├── App.jsx
│   ├── main.jsx
│   └── ...
│
├── public/
│
├── package.json
└── ...
```

Конкретная структура каталогов может изменяться в зависимости от выбранного варианта.

Разбейте интерфейс приложения на логически самостоятельные компоненты. Каждый компонент должен отвечать за определённую часть интерфейса или функциональности. Например, для интернет-магазина могут быть созданы:

```
App
 ├── Header
 ├── FilterPanel
 ├── ProductList
 │     └── ProductCard
 └── Cart
       └── CartItem
```

Компоненты должны быть переиспользуемыми и получать необходимые данные через props.

Работа с состоянием

Для хранения изменяемых данных приложения необходимо использовать механизм состояния React. В обязательной части работы необходимо применить **useState**.

Состояние может использоваться для хранения:

- выбранного элемента;
- списка объектов;
- введённого пользователем значения;
- текущего фильтра;
- текущего шага;
- количества элементов;
- состояния отображения;
- результатов действий пользователя.

При изменении состояния интерфейс React-компонента должен автоматически обновляться.

Передача данных между компонентами

Необходимо реализовать передачу данных между компонентами с использованием **props**. В зависимости от выбранного варианта необходимо продемонстрировать:

- передачу данных от родительского компонента дочернему;
- передачу обработчиков событий;
- изменение состояния родительского компонента из дочернего через переданную функцию.

Например:

```
App
 ↓ данные
ProductList
```

```
 ↓ данные
ProductCard
и:
App
 ↑ событие
ProductCard
```

Обработка событий

Необходимо реализовать обработку пользовательских событий средствами React. Приложение должно реагировать как минимум на несколько действий пользователя:

- нажатие кнопки;
- изменение поля ввода;
- выбор элемента;
- изменение состояния;
- отправку формы.

При выполнении действия состояние компонента должно изменяться, а интерфейс — обновляться.

Условный и списочный рендеринг

В приложении необходимо использовать:

- условный рендеринг;
- рендеринг массивов;
- map() для вывода коллекций;
- уникальный key для элементов списка.

Например, состояние приложения может определять, какой интерфейс отображается пользователю:

- пустой список;
- заполненный список;
- сообщение об отсутствии результатов;
- сообщение об ошибке;
- итоговый экран.

Использование useEffect

В работе необходимо продемонстрировать использование хука **useEffect**. В зависимости от варианта его можно использовать для:

- выполнения действия при загрузке компонента;
- реакции на изменение состояния;
- обновления дополнительных данных;
- имитации загрузки данных;

- синхронизации нескольких элементов интерфейса.

Использование useEffect должно быть связано с логикой приложения, а не добавляться формально.

Стилизация React-приложения

Разработайте CSS-оформление приложения. Интерфейс должен:

- иметь единую визуальную структуру;
- обеспечивать понятное расположение элементов;
- визуально отображать разные состояния компонентов;
- содержать состояния hover, active, disabled, если они применимы;
- корректно отображать списки, карточки, формы и кнопки.

Допускается использовать обычные CSS-файлы либо другой предусмотренный проектом способ стилизации.

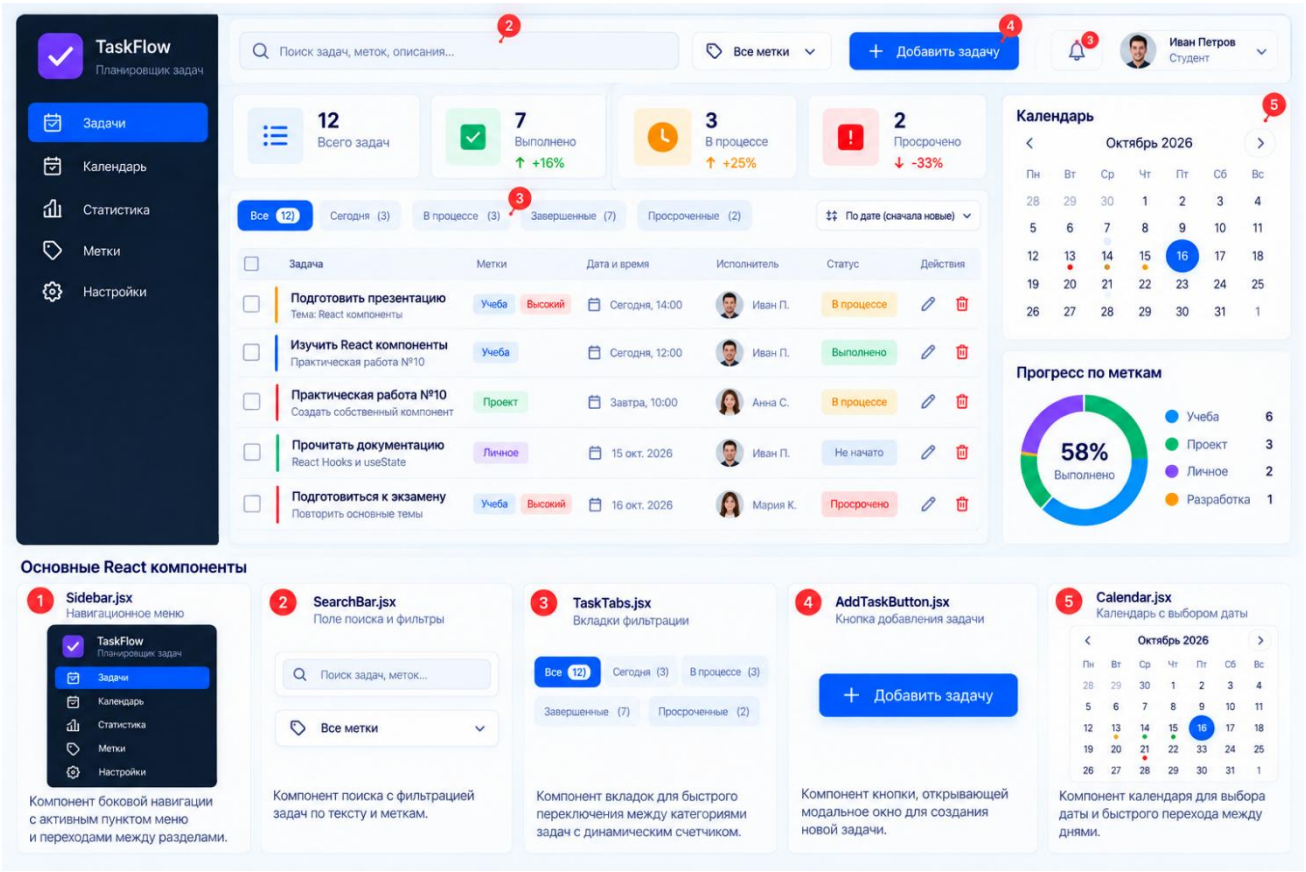
Требования к результату работы

В результате выполнения практической работы студент должен получить **работающее React-приложение**, в котором:

- интерфейс построен из React-компонентов;
- используется props;
- используется useState;
- используется useEffect;
- реализована обработка пользовательских событий;
- реализован условный и списочный рендеринг;
- данные хранятся и изменяются в состоянии компонентов;
- компоненты взаимодействуют между собой;
- интерфейс динамически обновляется при изменении состояния;
- приложение имеет CSS-оформление;
- проект опубликован на GitHub.

Вариант 1. Разработка компонентов для планировщика задач TaskFlow

Разработать набор переиспользуемых React-компонентов для веб-приложения TaskFlow, предназначенного для управления задачами. Реализовать отображение задач, поиск, фильтрацию, навигацию и выбор даты в соответствии с представленным графическим концептом.



Необходимые компоненты:

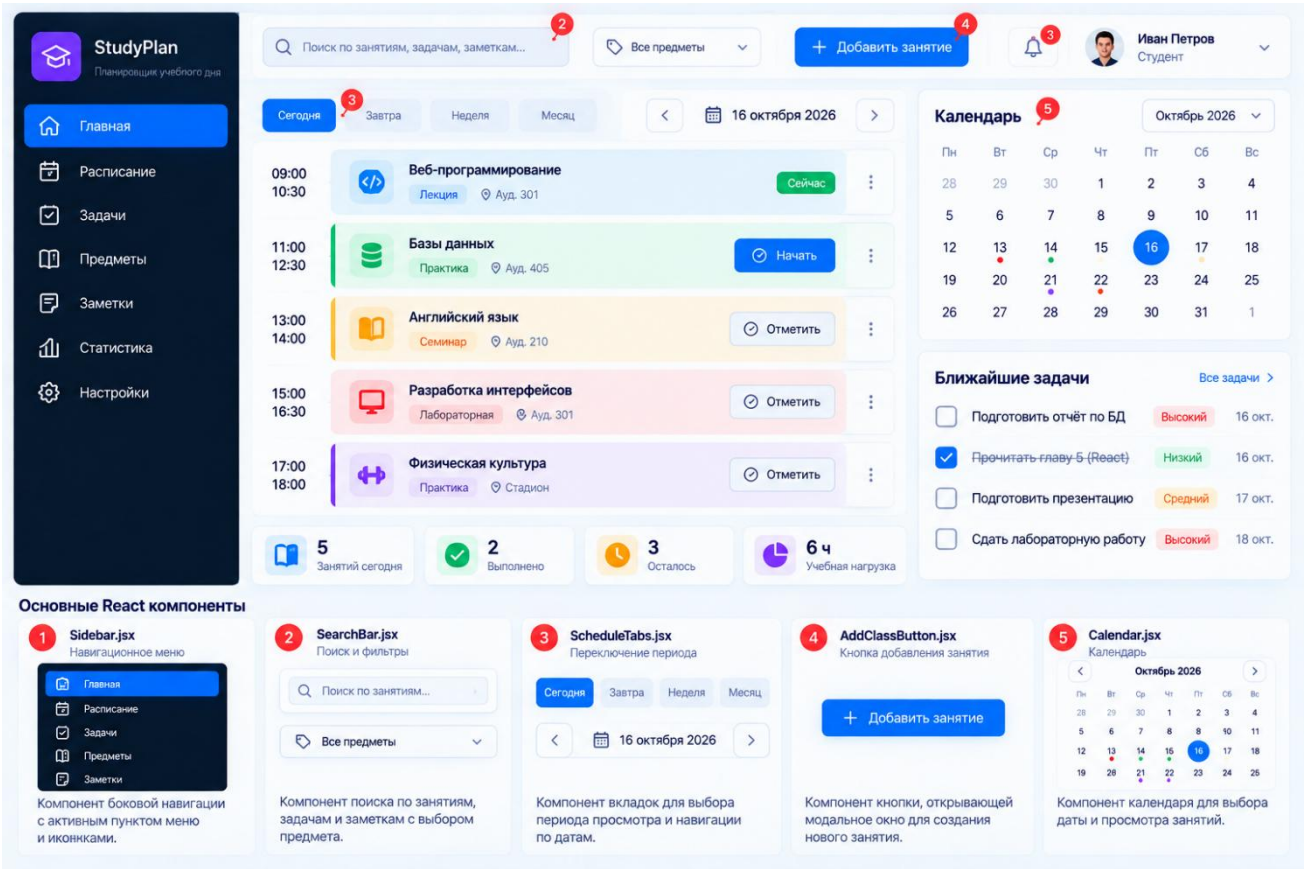
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск задач по названию и фильтрация по меткам.
3. TaskTabs.jsx — переключение между категориями задач с отображением количества задач.
4. AddTaskButton.jsx — кнопка открытия модального окна для добавления новой задачи.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра задач.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление интерфейса и визуальное отображение активных состояний.

Рабочая страница TaskFlow, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 2. Разработка React-компонентов для приложения «StudyPlan — Планировщик учебного дня»

Разработать набор переиспользуемых React-компонентов для веб-приложения StudyPlan, предназначенного для планирования учебных занятий и управления расписанием. Реализовать навигацию, поиск, переключение периодов, добавление занятий и выбор даты в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск занятий по названию и фильтрация по учебным предметам.
3. ScheduleTabs.jsx — переключение между периодами «Сегодня», «Завтра», «Неделя», «Месяц» с обновлением расписания.
4. AddClassButton.jsx — кнопка открытия модального окна для добавления нового занятия.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра занятий выбранного дня.

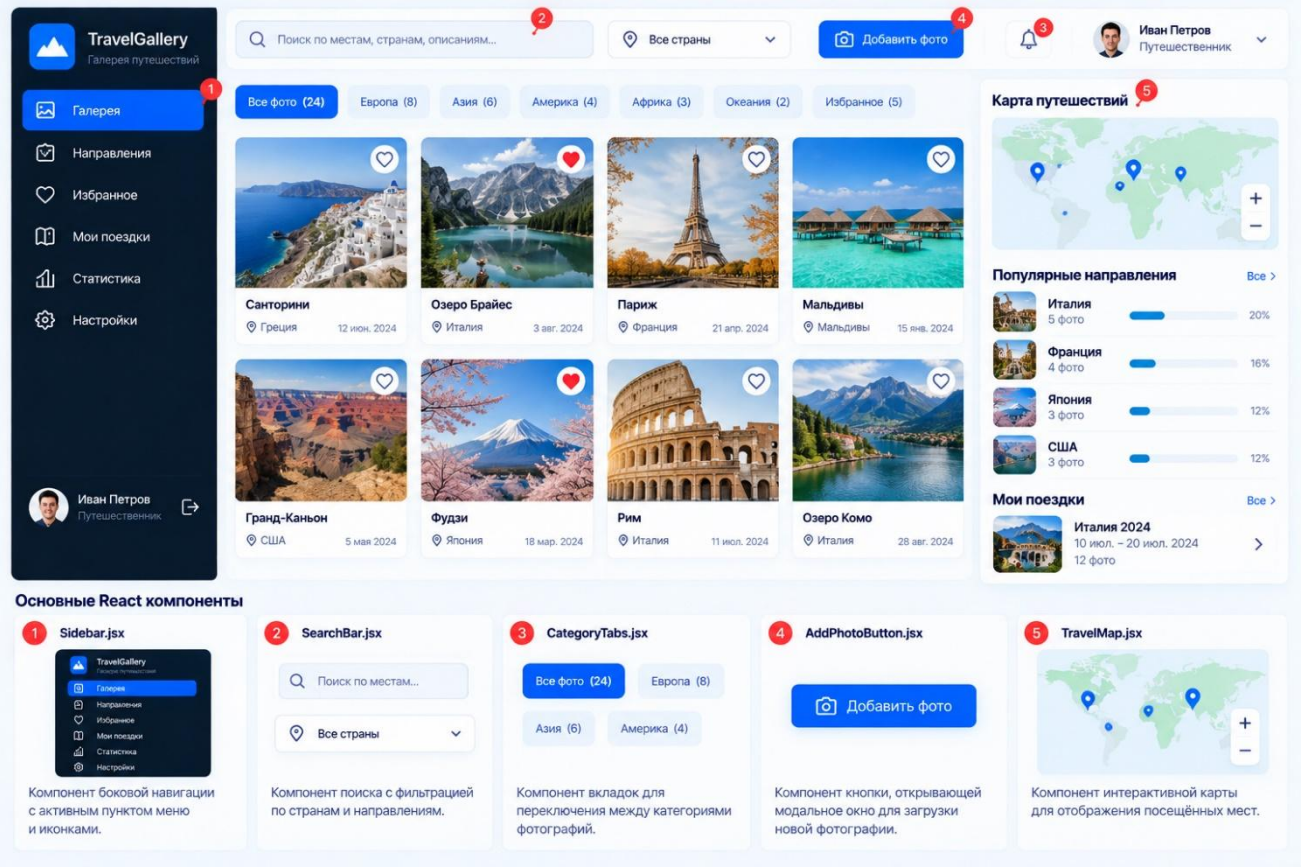
Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление расписания и визуальное отображение активных состояний.

Рабочая страница StudyPlan, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 3. Разработка компонентов для галереи путешествий TravelGallery

Разработать набор переиспользуемых React-компонентов для веб-приложения TravelGallery, предназначенного для просмотра и организации фотографий путешествий. Реализовать навигацию, поиск, фильтрацию фотографий по категориям,

добавление новых изображений и отображение посещённых мест на карте в соответствии с представленным графическим концептом.



Необходимые компоненты:

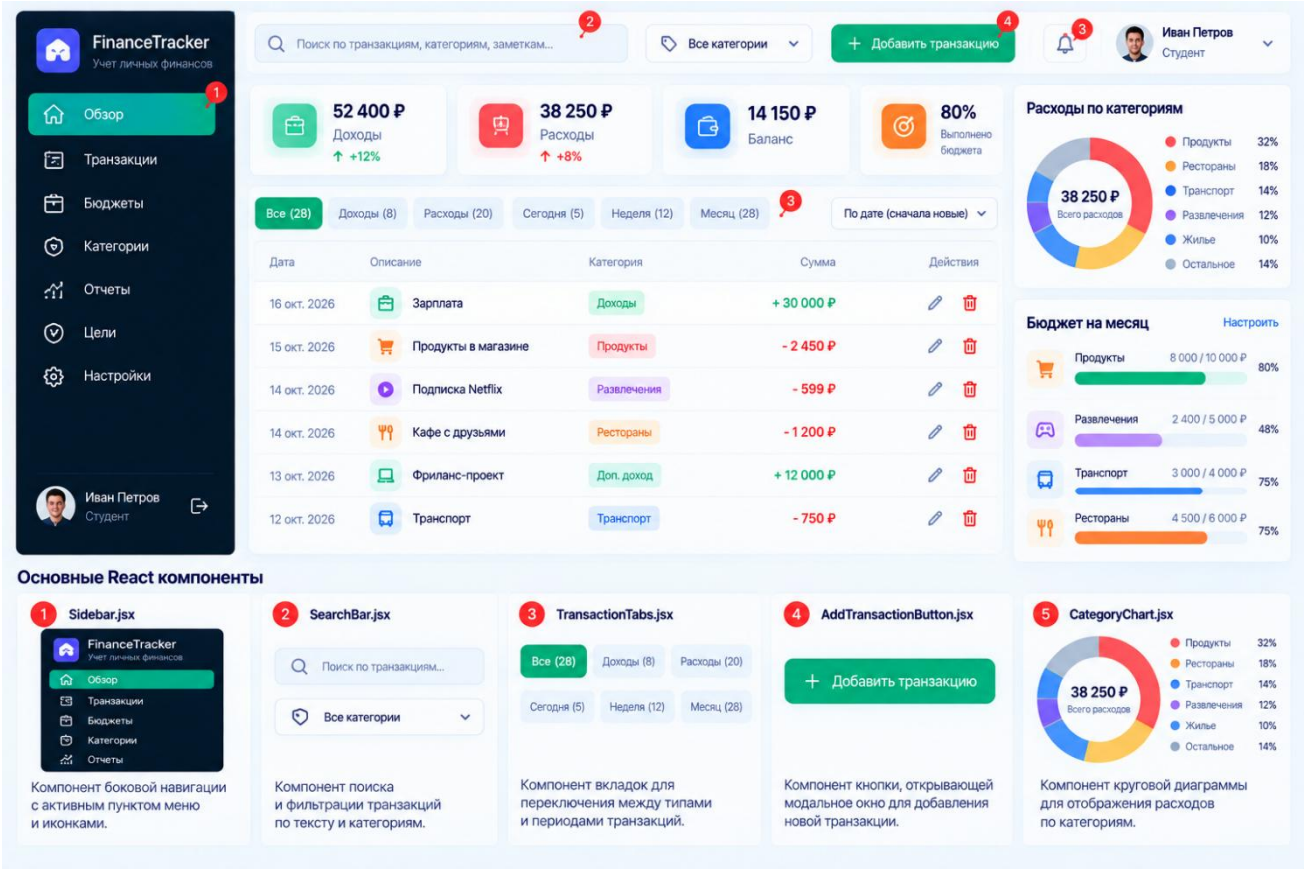
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск фотографий по названию места и фильтрация по странам и направлениям.
3. CategoryTabs.jsx — переключение между категориями фотографий с отображением их количества.
4. AddPhotoButton.jsx — кнопка открытия модального окна для добавления новой фотографии с указанием названия, страны и даты.
5. TravelMap.jsx — интерактивная карта с маркерами посещённых мест и возможностью просмотра информации о них.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление галереи, фильтрацию фотографий и визуальное отображение активных состояний.

Рабочая страница TravelGallery, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 4. азработка компонентов для системы учёта личных финансов FinanceTracker

Разработать набор переиспользуемых React-компонентов для веб-приложения FinanceTracker, предназначенного для учёта личных доходов и расходов. Реализовать навигацию, поиск, фильтрацию транзакций, добавление финансовых операций и визуализацию расходов по категориям в соответствии с представленным графическим концептом.



Необходимые компоненты:

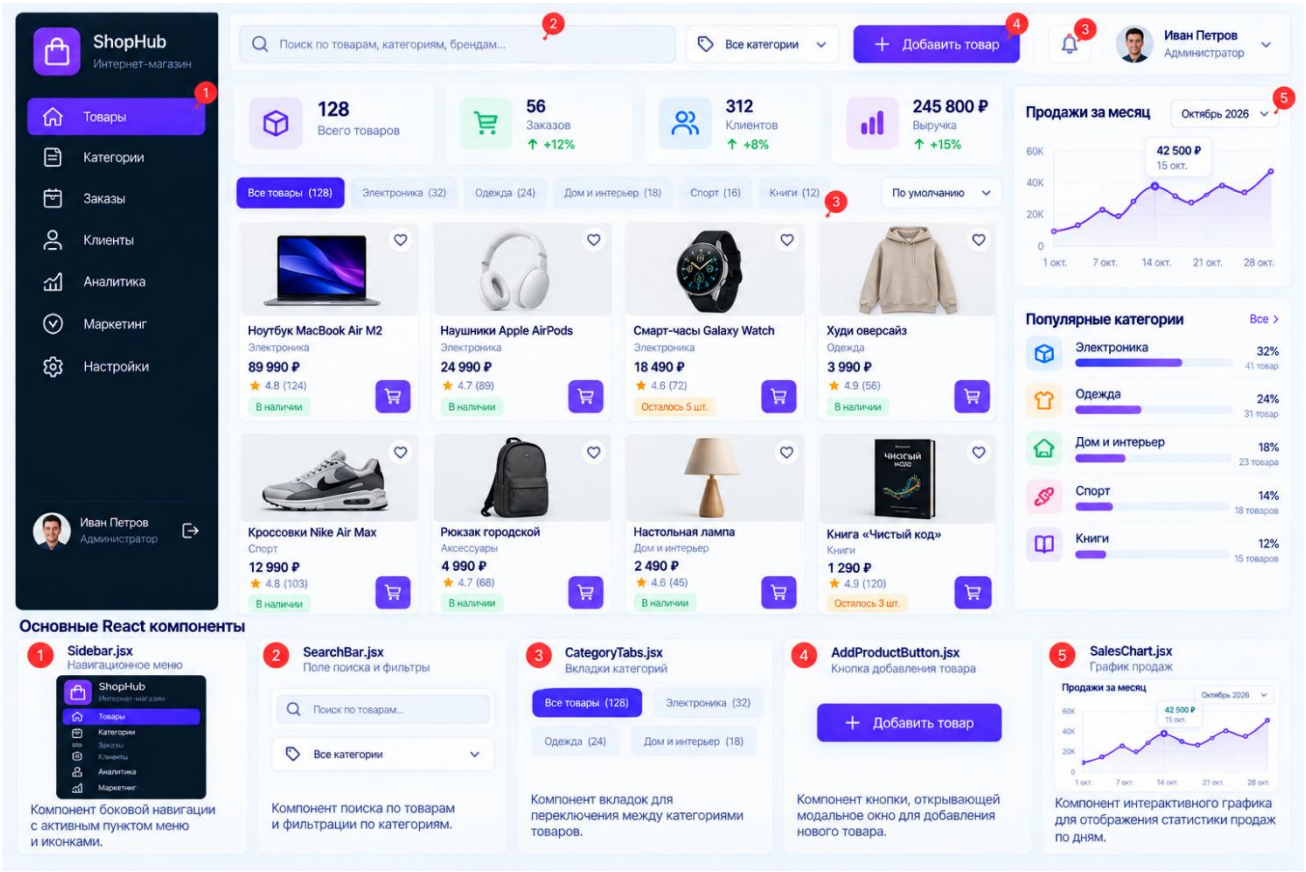
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск транзакций по описанию и фильтрация по категориям.
3. TransactionTabs.jsx — переключение между типами операций «Все», «Доходы», «Расходы» и периодами «Сегодня», «Неделя», «Месяц» с обновлением списка транзакций.
4. AddTransactionButton.jsx — кнопка открытия модального окна для добавления новой финансовой операции с указанием суммы, категории, даты и описания.
5. CategoryChart.jsx — интерактивная круговая диаграмма для отображения распределения расходов по категориям.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка транзакций, фильтрацию финансовых операций, пересчёт расходов по категориям и визуальное отображение активных состояний.

Рабочая страница FinanceTracker, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 5. Разработка компонентов для интернет-магазина ShopHub

Разработать набор переиспользуемых React-компонентов для веб-приложения ShopHub, предназначенного для управления каталогом товаров интернет-магазина. Реализовать навигацию, поиск, фильтрацию товаров по категориям, добавление новых товаров и визуализацию статистики продаж в соответствии с представленным графическим концептом.



Необходимые компоненты:

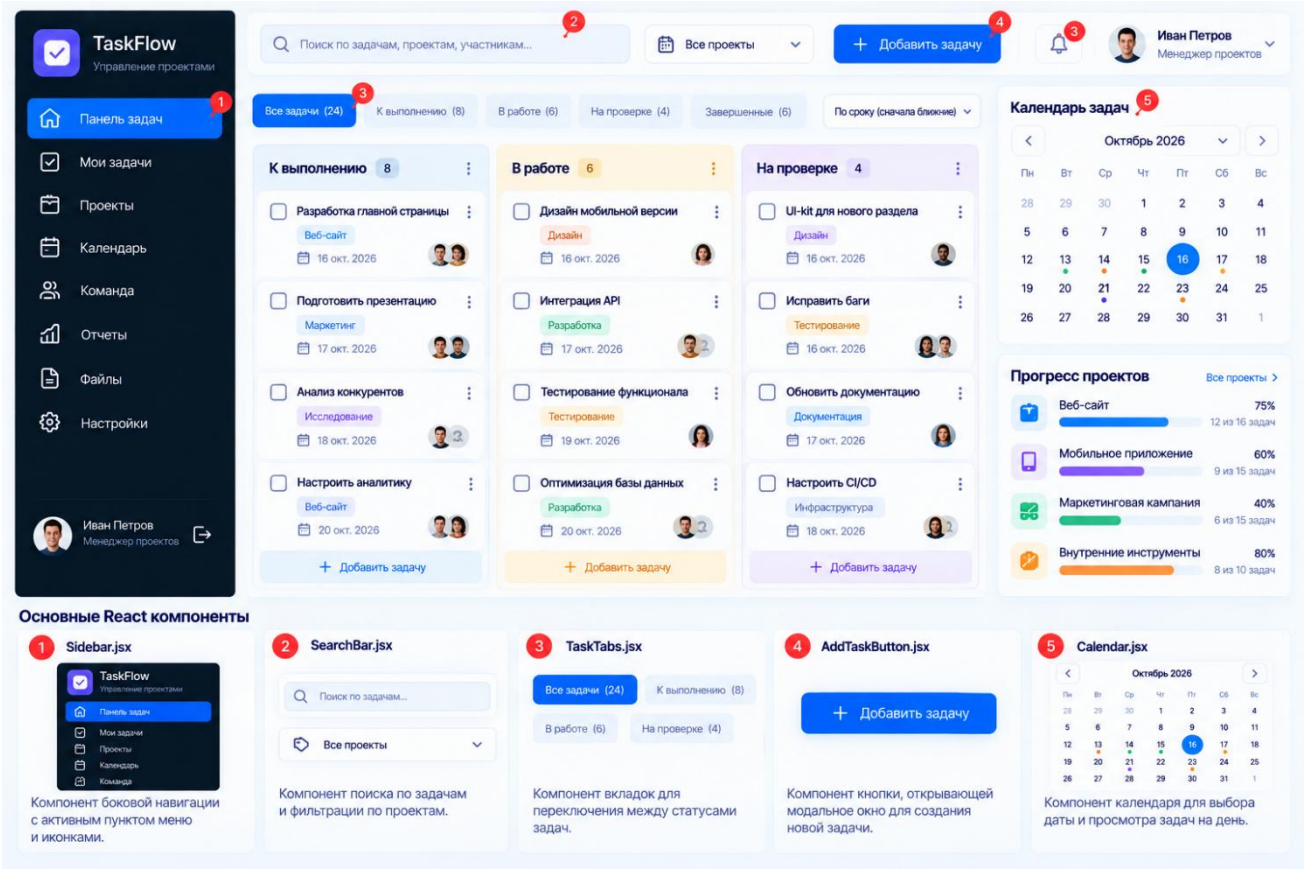
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск товаров по названию, бренду и фильтрация по категориям.
3. CategoryTabs.jsx — переключение между категориями товаров с отображением их количества и обновлением каталога.
4. AddProductButton.jsx — кнопка открытия модального окна для добавления нового товара с указанием названия, категории, цены, количества и изображения.
5. SalesChart.jsx — интерактивный график для отображения динамики продаж по дням с возможностью выбора периода.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление каталога товаров, поиск и фильтрацию, добавление новых товаров, переключение периодов статистики и визуальное отображение активных состояний.

Рабочая страница ShopHub, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 6. Разработка компонентов для системы управления проектами TaskFlow

Разработать набор переиспользуемых React-компонентов для веб-приложения TaskFlow, предназначенного для управления проектами и задачами. Реализовать навигацию, поиск, фильтрацию задач по проектам, переключение статусов, добавление новых задач и просмотр задач в календаре в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск задач по названию и фильтрация по проектам.
3. TaskTabs.jsx — переключение между статусами задач «Все задачи», «К выполнению», «В работе», «На проверке», «Завершённые» с отображением их количества.

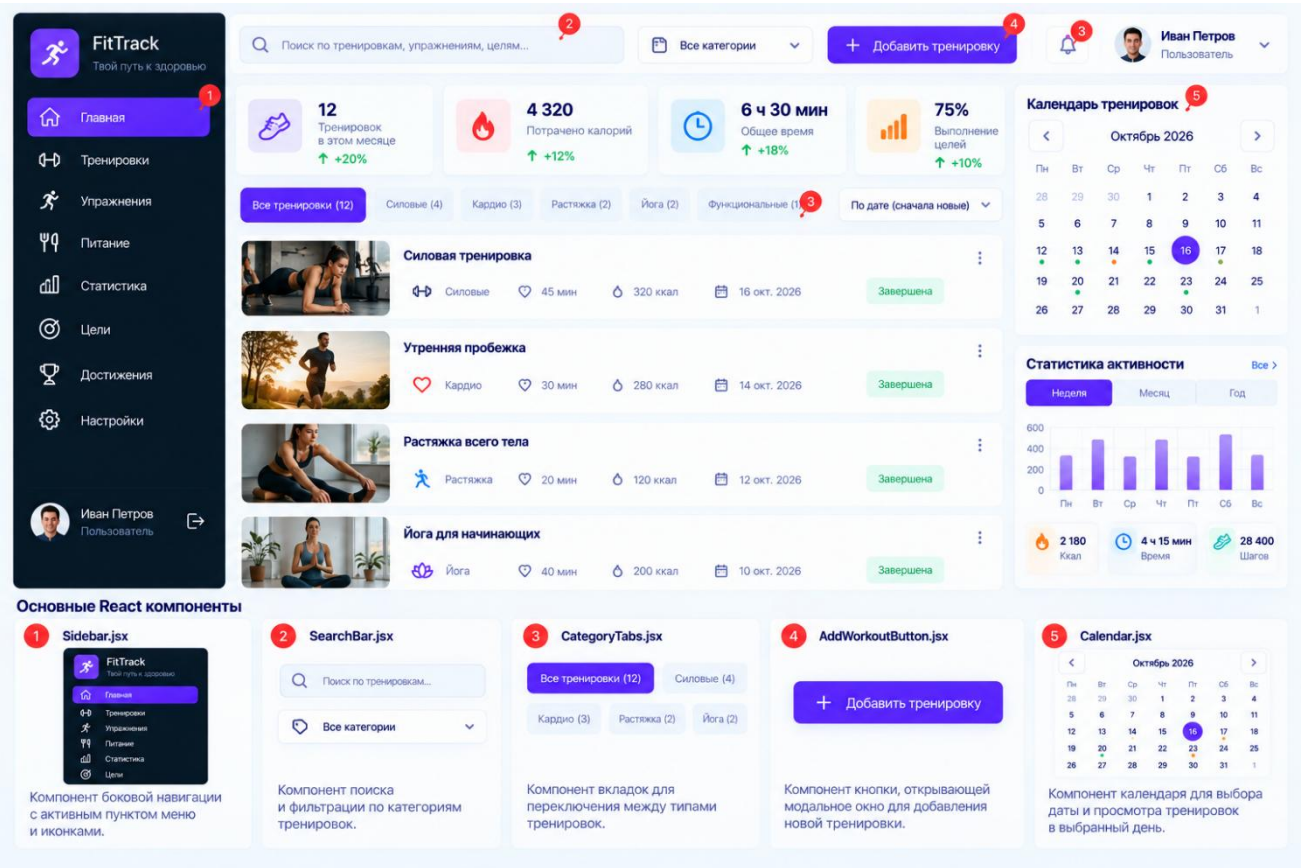
4. AddTaskButton.jsx — кнопка открытия модального окна для добавления новой задачи с указанием названия, проекта, срока выполнения и исполнителя.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра задач с соответствующими сроками выполнения.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка задач, поиск и фильтрацию, добавление новых задач, переключение статусов и визуальное отображение активных состояний.

Рабочая страница TaskFlow, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 7. Разработка компонентов для фитнес-приложения FitTrack

Разработать набор переиспользуемых React-компонентов для веб-приложения FitTrack, предназначенного для планирования тренировок и отслеживания физической активности. Реализовать навигацию, поиск, фильтрацию тренировок по категориям, добавление новых тренировок и просмотр расписания в календаре в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.

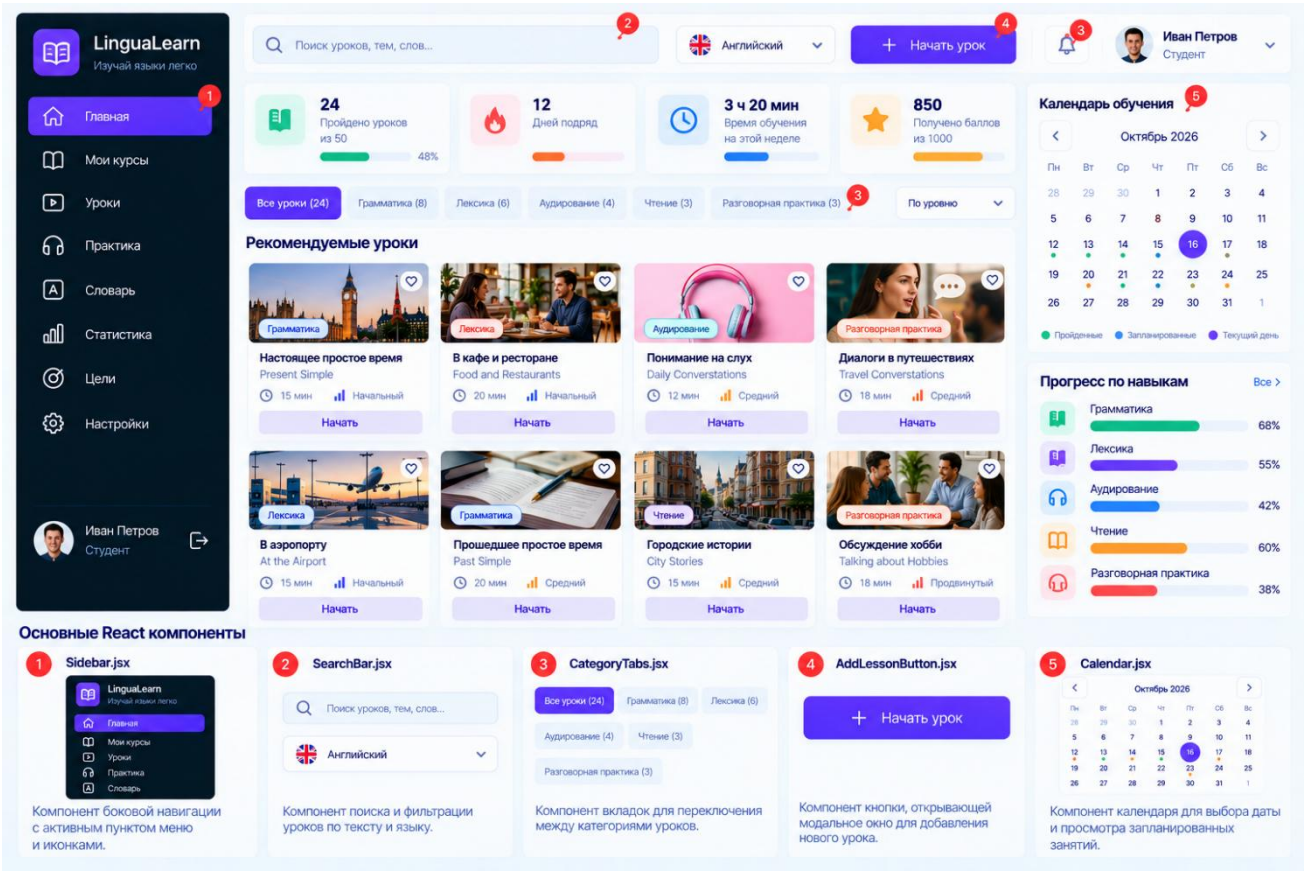
2. SearchBar.jsx — поиск тренировок по названию и фильтрация по категориям.
3. CategoryTabs.jsx — переключение между типами тренировок «Все тренировки», «Силовые», «Кардио», «Растяжка», «Йога», «Функциональные» с отображением их количества.
4. AddWorkoutButton.jsx — кнопка открытия модального окна для добавления новой тренировки с указанием названия, категории, даты, продолжительности и количества потраченных калорий.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра тренировок, запланированных на выбранный день.

Требования к реализации: Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка тренировок, поиск и фильтрацию, добавление новых тренировок, выбор даты и визуальное отображение активных состояний.

Результат: Рабочая страница FitTrack, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 8. Разработка компонентов для платформы изучения иностранных языков LinguaLearn

Разработать набор переиспользуемых React-компонентов для веб-приложения LinguaLearn, предназначенного для изучения иностранных языков и планирования учебных занятий. Реализовать навигацию, поиск уроков, фильтрацию по категориям, добавление новых занятий и просмотр расписания в календаре в соответствии с представленным графическим концептом.



Необходимые компоненты:

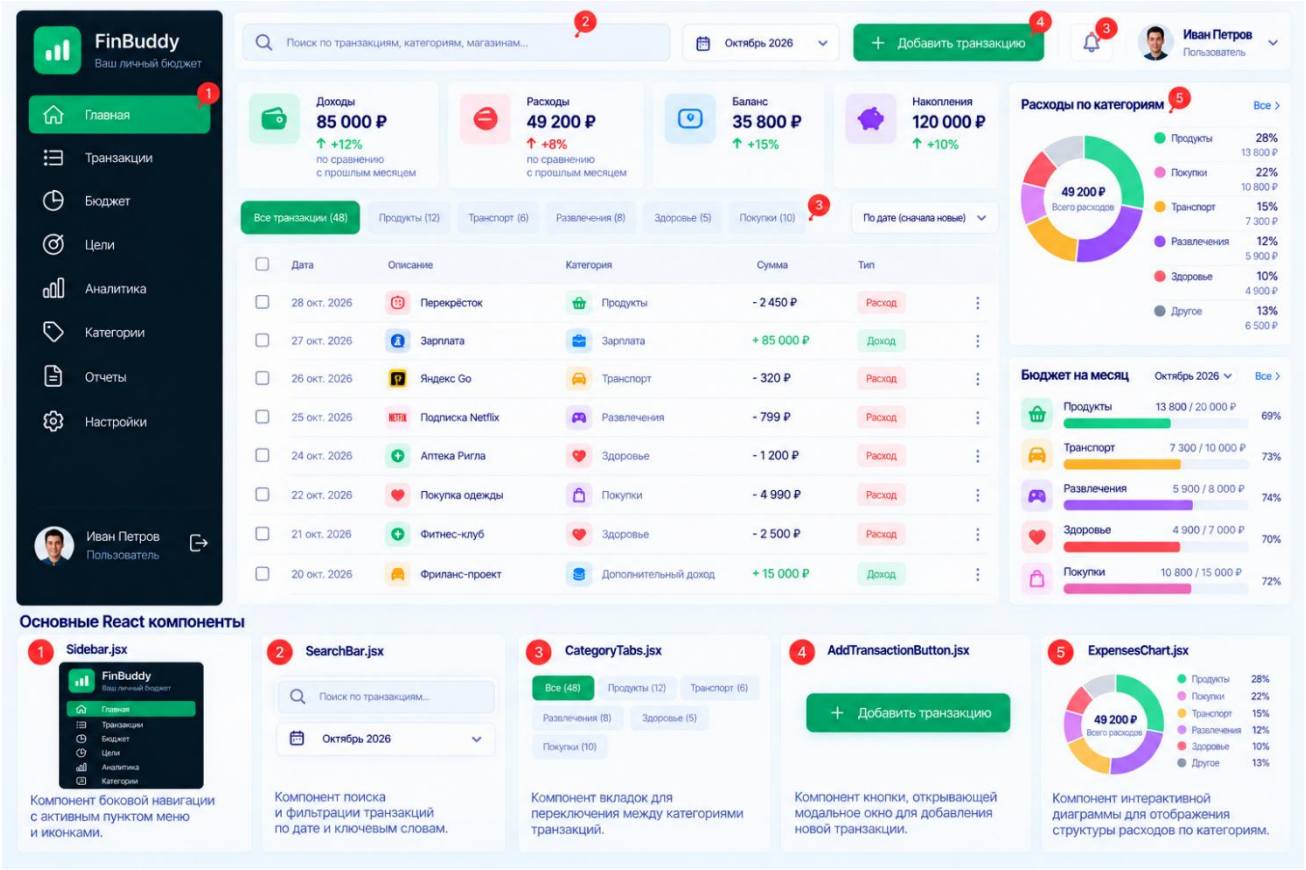
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск уроков по названию и фильтрация по выбранному языку.
3. CategoryTabs.jsx — переключение между категориями уроков «Все уроки», «Грамматика», «Лексика», «Аудирование», «Чтение», «Разговорная практика» с отображением их количества.
4. AddLessonButton.jsx — кнопка открытия модального окна для добавления нового занятия с указанием названия, категории, даты и времени проведения.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра занятий, запланированных на выбранный день.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка уроков, поиск и фильтрацию, добавление новых занятий, выбор даты и визуальное отображение активных состояний.

Рабочая страница LinguaLearn, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 9. Разработка компонентов для приложения управления личными финансами FinBuddy

Разработать набор переиспользуемых React-компонентов для веб-приложения FinBuddy, предназначенного для управления личными финансами, учета доходов и расходов. Реализовать навигацию, поиск, фильтрацию транзакций по категориям, добавление новых операций и визуализацию структуры расходов в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск транзакций по названию, категории или магазину с возможностью фильтрации по дате.
3. CategoryTabs.jsx — переключение между категориями транзакций «Все транзакции», «Продукты», «Транспорт», «Развлечения», «Здоровье», «Покупки» с отображением их количества.
4. AddTransactionButton.jsx — кнопка открытия модального окна для добавления новой финансовой операции с указанием типа (доход или расход), названия, категории, суммы и даты.
5. ExpensesChart.jsx — интерактивная круговая диаграмма для отображения распределения расходов по категориям с указанием сумм и процентных долей.

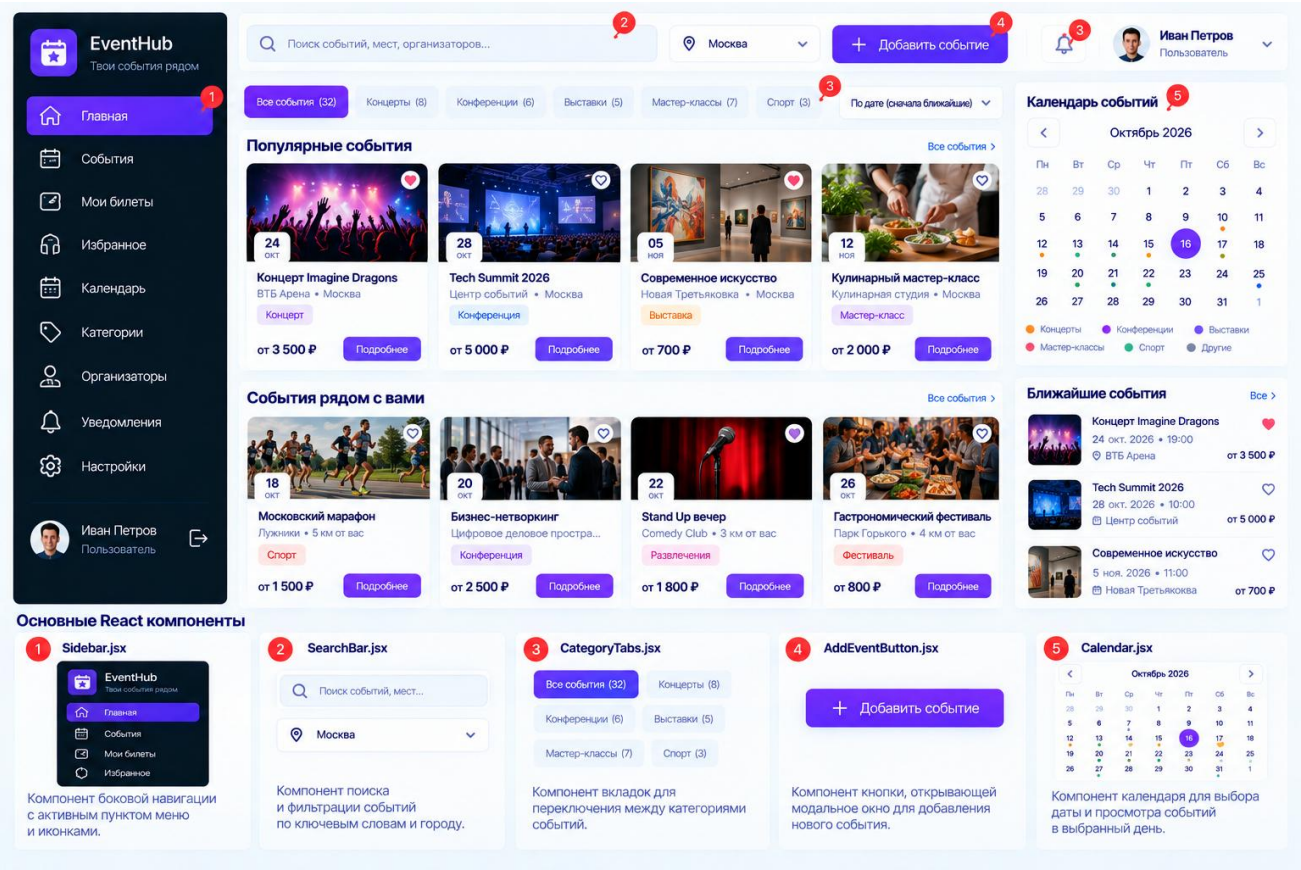
Требования к реализации: Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка транзакций, поиск и фильтрацию, добавление новых

финансовых операций, автоматический пересчет расходов по категориям и визуальное отображение активных состояний.

Результат: Рабочая страница FinBuddy, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 10. Разработка компонентов для платформы поиска и организации мероприятий EventHub

Разработать набор переиспользуемых React-компонентов для веб-приложения EventHub, предназначенного для поиска, просмотра и организации мероприятий. Реализовать навигацию, поиск событий, фильтрацию по категориям, добавление новых мероприятий и просмотр событий в календаре в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск мероприятий по названию, месту проведения или организатору с возможностью фильтрации по городу.
3. CategoryTabs.jsx — переключение между категориями мероприятий «Все события», «Концерты», «Конференции», «Выставки», «Мастер-классы», «Спорт» с отображением их количества.

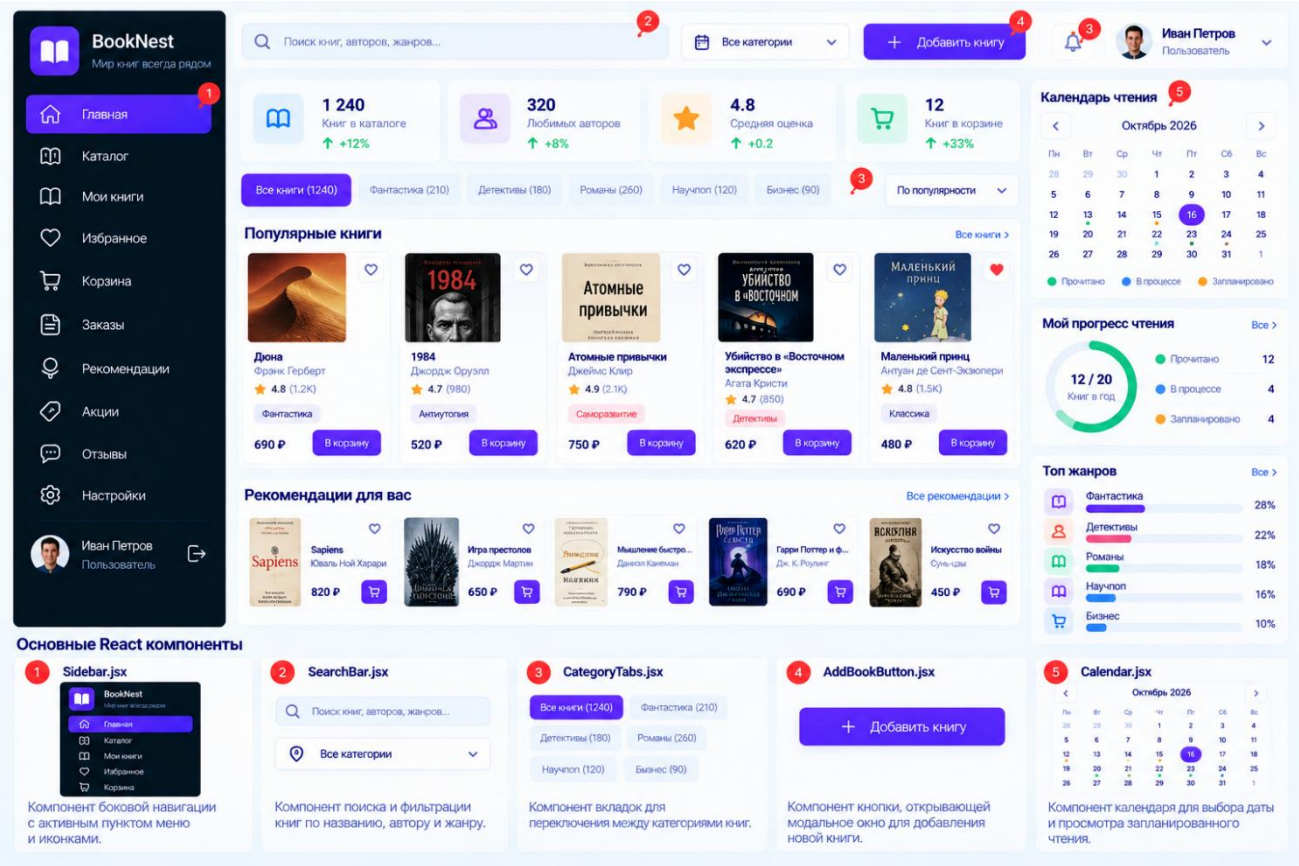
4. AddEventButton.jsx — кнопка открытия модального окна для добавления нового мероприятия с указанием названия, категории, даты, времени, места проведения и стоимости билета.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра мероприятий, запланированных на выбранный день.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка мероприятий, поиск и фильтрацию, добавление новых событий, выбор даты и визуальное отображение активных состояний.

Рабочая страница EventHub, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 11. Разработка компонентов для электронной библиотеки BookNest

Разработать набор переиспользуемых React-компонентов для веб-приложения BookNest, предназначенного для поиска, просмотра и управления каталогом книг, а также планирования чтения. Реализовать навигацию, поиск книг, фильтрацию по жанрам, добавление новых книг и просмотр плана чтения в календаре в соответствии с представленным графическим концептом.



Необходимые компоненты:

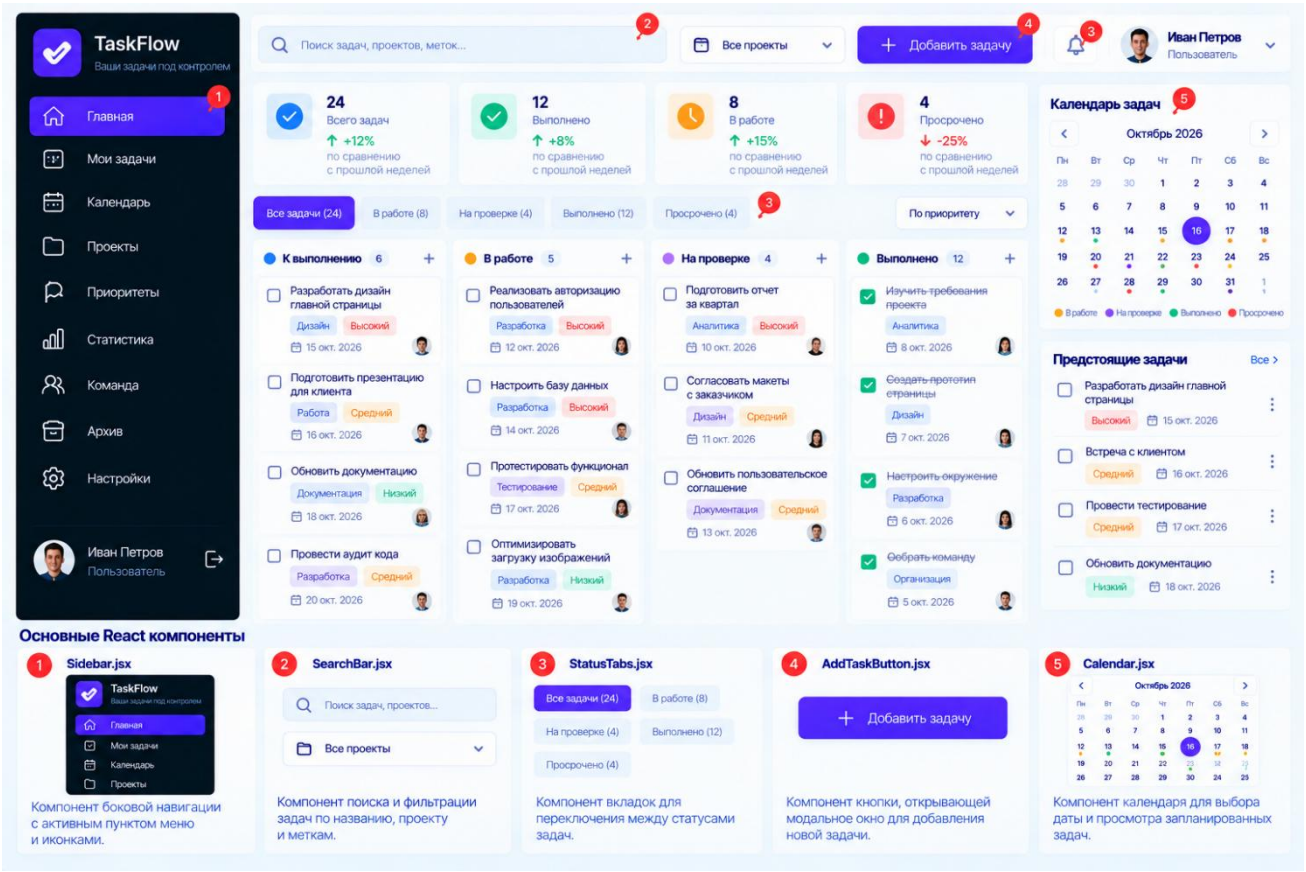
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск книг по названию, автору или жанру с возможностью фильтрации по категориям.
3. CategoryTabs.jsx — переключение между жанрами книг «Все книги», «Фантастика», «Детективы», «Романы», «Научпоп», «Бизнес» с отображением их количества.
4. AddBookButton.jsx — кнопка открытия модального окна для добавления новой книги с указанием названия, автора, жанра, стоимости и обложки.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра запланированных сеансов чтения с отображением статусов «Запланировано», «В процессе», «Прочитано».

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление каталога книг, поиск и фильтрацию по жанрам, добавление новых книг, выбор даты, отображение плана чтения и визуальное выделение активных состояний.

Рабочая страница BookNest, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 12. Разработка компонентов для системы управления задачами TaskFlow

Разработать набор переиспользуемых React-компонентов для веб-приложения TaskFlow, предназначенного для планирования, организации и контроля выполнения задач. Реализовать навигацию, поиск, фильтрацию задач по проектам, переключение между статусами, добавление новых задач и просмотр сроков выполнения в календаре в соответствии с представленным графическим концептом.



Необходимые компоненты:

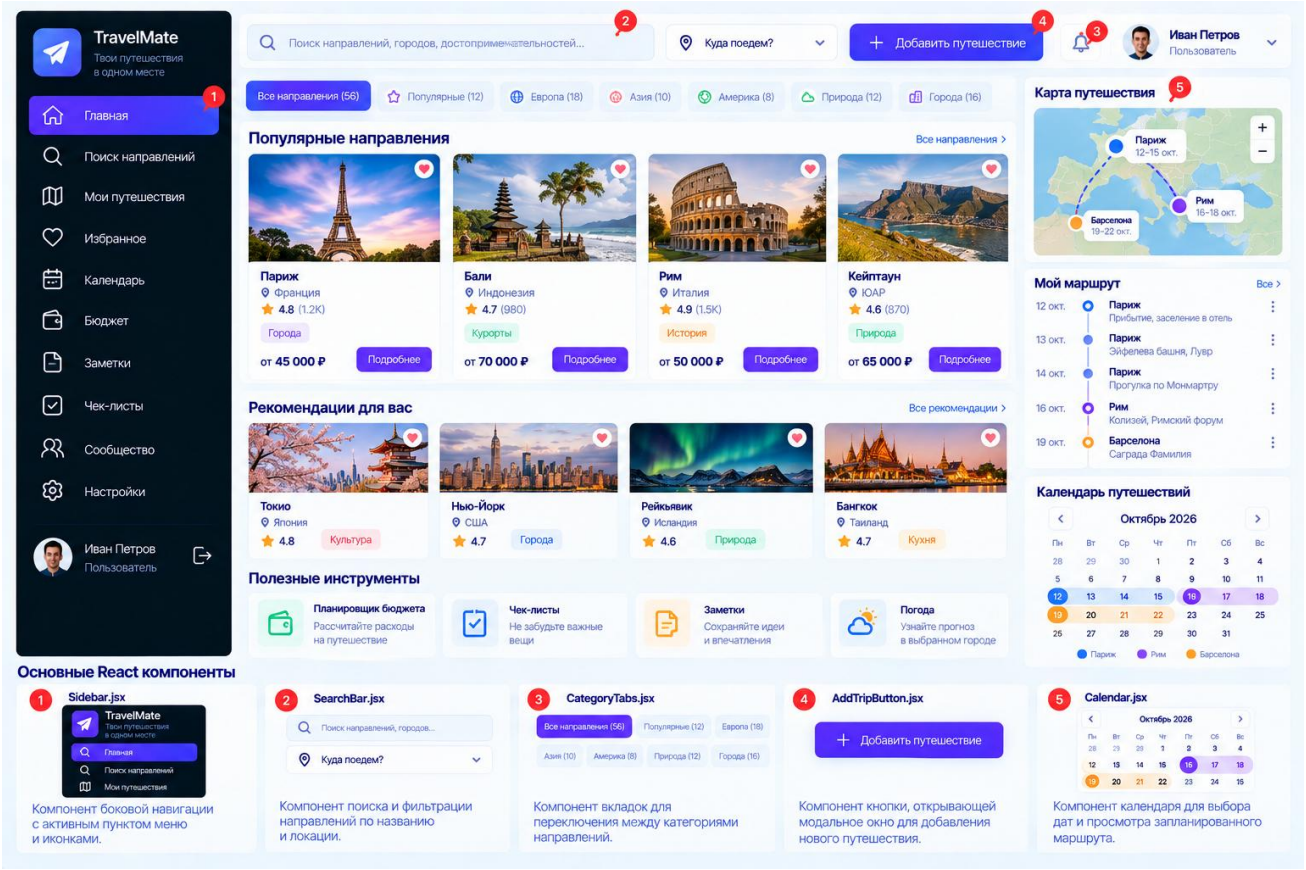
1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск задач по названию, проекту или меткам с возможностью фильтрации по выбранному проекту.
3. StatusTabs.jsx — переключение между статусами задач «Все задачи», «В работе», «На проверке», «Выполнено», «Просрочено» с отображением их количества.
4. AddTaskButton.jsx — кнопка открытия модального окна для добавления новой задачи с указанием названия, проекта, описания, приоритета, исполнителя и срока выполнения.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра задач, запланированных на выбранный день, с цветовым обозначением их статусов.

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка задач, поиск и фильтрацию, добавление новых задач, переключение статусов, выбор даты и визуальное отображение активных состояний.

Рабочая страница TaskFlow, объединяющая разработанные компоненты в единый интерактивный интерфейс.

Вариант 13. Разработка компонентов для системы управления задачами TaskFlow

Разработать набор переиспользуемых React-компонентов для веб-приложения TravelMate, предназначенного для планирования путешествий, поиска туристических направлений и управления запланированными поездками. Реализовать навигацию, поиск направлений, фильтрацию по категориям, добавление новых путешествий и просмотр календаря поездок в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск туристических направлений по названию города или страны с динамическим обновлением результатов.
3. CategoryTabs.jsx — переключение между категориями «Все направления», «Европа», «Азия», «Природа», «Города» с отображением соответствующих предложений.
4. AddTripButton.jsx — кнопка открытия модального окна для добавления нового путешествия с указанием названия поездки, страны, города, дат начала и окончания, бюджета и описания.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра запланированных путешествий с цветовым обозначением статусов «Запланировано», «В процессе», «Завершено».

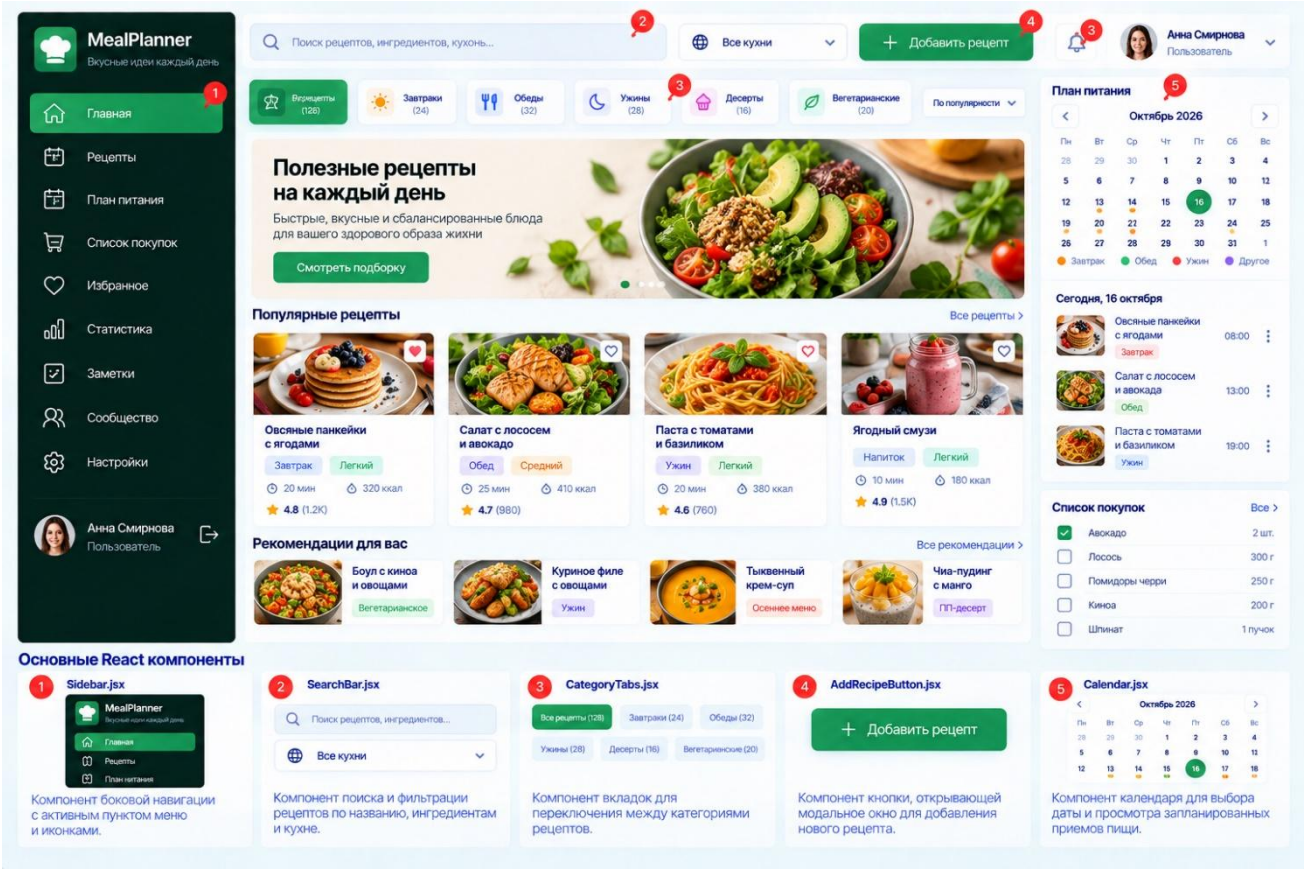
Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка путешествий, поиск и фильтрацию направлений, добавление новых поездок, выбор

даты, отображение запланированных путешествий и визуальное выделение активных состояний.

Рабочая страница TravelMate, объединяющая разработанные компоненты в единый интерактивный Desktop-интерфейс для планирования и управления путешествиями.

Вариант 14. Разработка компонентов для планировщика питания MealPlanner

Разработать набор переиспользуемых React-компонентов для веб-приложения MealPlanner, предназначенного для поиска кулинарных рецептов, организации питания и планирования ежедневного меню. Реализовать навигацию, поиск рецептов, фильтрацию по категориям, добавление новых рецептов и просмотр календаря питания в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск рецептов по названию блюда или ингредиентам с возможностью фильтрации по типу кухни.
3. CategoryTabs.jsx — переключение между категориями «Все рецепты», «Завтраки», «Обеды», «Ужины», «Десерты», «Вегетарианские» с отображением количества рецептов в каждой категории.

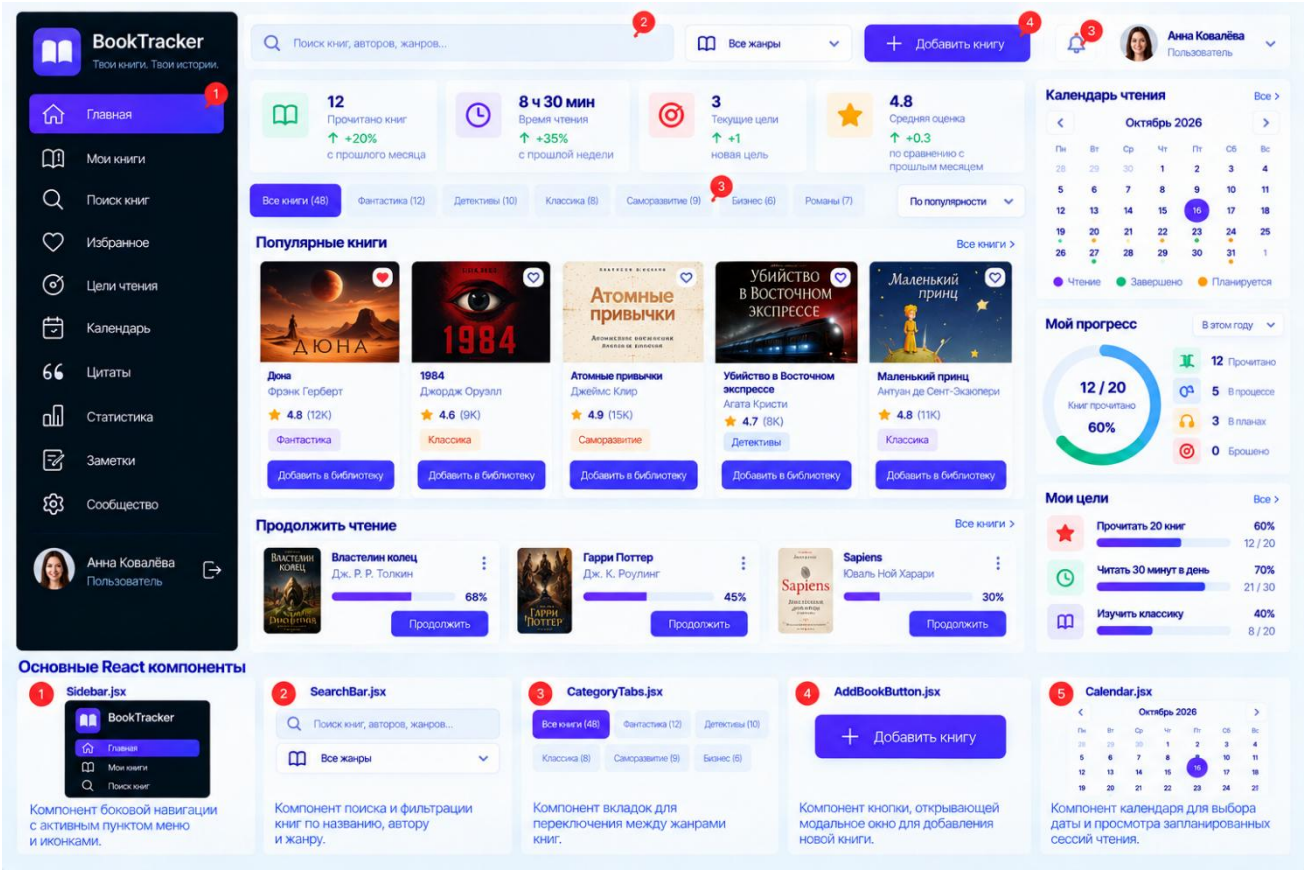
4. AddRecipeButton.jsx — кнопка открытия модального окна для добавления нового рецепта с указанием названия блюда, категории, ингредиентов, времени приготовления, калорийности, уровня сложности и описания процесса приготовления.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра запланированных приёмов пищи с цветовым обозначением категорий «Завтрак», «Обед», «Ужин», «Другое».

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка рецептов, поиск и фильтрацию по категориям и типам кухни, добавление новых рецептов, выбор даты, отображение запланированных приёмов пищи и визуальное выделение активных состояний.

Рабочая страница MealPlanner, объединяющая разработанные компоненты в единый интерактивный Desktop-интерфейс для поиска рецептов и планирования питания.

Вариант 15. Разработка компонентов для электронной библиотеки BookTracker

Разработать набор переиспользуемых React-компонентов для веб-приложения BookTracker, предназначенного для поиска книг, формирования личной библиотеки, планирования чтения и отслеживания читательской активности. Реализовать навигацию, поиск книг, фильтрацию по жанрам, добавление новых книг и просмотр календаря чтения в соответствии с представленным графическим концептом.



Необходимые компоненты:

1. Sidebar.jsx — боковое навигационное меню с выделением активного раздела.
2. SearchBar.jsx — поиск книг по названию, автору или ключевым словам с возможностью фильтрации по литературному жанру.
3. CategoryTabs.jsx — переключение между категориями «Все книги», «Фантастика», «Детективы», «Классика», «Саморазвитие», «Бизнес», «Романы» с отображением количества книг в каждой категории.
4. AddBookButton.jsx — кнопка открытия модального окна для добавления новой книги с указанием названия, автора, жанра, количества страниц, статуса чтения и краткого описания.
5. Calendar.jsx — интерактивный календарь для выбора даты и просмотра запланированных сеансов чтения с цветовым обозначением статусов «Запланировано», «Чтение», «Завершено».

Использовать функциональные React-компоненты, props, хуки useState и обработчики событий. Обеспечить взаимодействие компонентов, динамическое обновление списка книг, поиск и фильтрацию по жанрам, добавление новых книг, выбор даты, отображение запланированных сеансов чтения и визуальное выделение активных состояний.

Рабочая страница BookTracker, объединяющая разработанные компоненты в единый интерактивный Desktop-интерфейс для управления личной библиотекой и планирования чтения.

Контрольные вопросы

1. Что такое React и в чем его основные преимущества?
2. Объясните концепцию виртуального DOM и как она улучшает производительность.
3. Что такое компоненты в React и как они различаются на функциональные и классовые?
4. Как работает жизненный цикл компонентов в React? Назовите основные методы жизненного цикла.
5. Что такое хуки в React и как они используются? Приведите примеры использования useState и useEffect.
6. Как передаются данные между компонентами в React? Объясните, что такое пропсы.
7. Что такое контекст в React и когда его следует использовать?
8. Как реализовать маршрутизацию в приложении React? Какова роль библиотеки React Router?
9. Что такое Redux и как он используется для управления состоянием в приложениях React?
10. Как можно оптимизировать производительность приложения на React?
11. Что такое Node.js и в чем его основные преимущества по сравнению с традиционными серверными технологиями?
12. Объясните, как работает событийно-ориентированная архитектура в Node.js.
13. Что такое npm и как он используется в проектах Node.js?

14. Как создать простой сервер на Node.js? Приведите пример кода.
15. Что такое middleware в Express.js и как его использовать?
16. Как обрабатывать асинхронные операции в Node.js? Объясните разницу между колбэками, промисами и async/await.
17. Что такое RESTful API и как его реализовать с использованием Node.js и Express?
18. Как подключить базу данных к приложению на Node.js? Приведите примеры для MongoDB и PostgreSQL.
19. Что такое CORS и как его настроить в приложении Node.js?
20. Как реализовать обработку ошибок в приложении на Node.js? Приведите пример.

ПРАКТИЧЕСКАЯ РАБОТА 11. РАЗРАБОТКА BACKEND-ПРИЛОЖЕНИЯ НА NODE.JS С ИСПОЛЬЗОВАНИЕМ POSTGRESQL

Цель работы

- создание серверного приложения на Node.js;
- организацию backend-проекта;
- работу с HTTP-запросами;
- создание REST API;
- обработку запросов GET, POST, PUT, DELETE;
- создание маршрутов и обработчиков;
- подключение PostgreSQL;
- создание таблиц и связей между ними;
- выполнение SQL-запросов из Node.js;
- получение, добавление, изменение и удаление данных;
- обработку ошибок;
- валидацию входных данных;
- разделение приложения на модули;
- работу с переменными окружения;
- тестирование API;
- интеграцию backend с клиентским приложением;
- публикацию проекта.

Задания к практической работе

Необходимо разработать серверное web-приложение на Node.js, предоставляющее REST API для выбранной предметной области.

Приложение должно взаимодействовать с базой данных PostgreSQL и обеспечивать выполнение основных операций над данными.

В зависимости от выбранного варианта необходимо:

1. спроектировать структуру базы данных;
2. создать необходимые таблицы;
3. определить поля и типы данных;
4. при необходимости создать связи между таблицами;
5. создать Node.js-приложение;
6. настроить подключение к PostgreSQL;
7. реализовать REST API;
8. реализовать операции добавления, получения, изменения и удаления данных;
9. добавить валидацию входных данных;
10. реализовать обработку ошибок;
11. протестировать API;
12. подключить API к ранее разработанному React-интерфейсу либо подготовить API для последующего подключения;
13. оформить проект и разместить его в GitHub.

Структура проекта

Для выполнения работы рекомендуется использовать модульную структуру:

```
project/
│
├── src/
│   ├── controllers/
│   │   ├── usersController.js
│   │   └── productsController.js
│   │
│   ├── routes/
│   │   ├── usersRoutes.js
│   │   └── productsRoutes.js
│   │
│   ├── db/
│   │   └── database.js
│   │
│   ├── middleware/
│   │   └── errorHandler.js
│   │
│   ├── app.js
│   └── server.js
│
├── sql/
│   ├── schema.sql
│   └── seed.sql
│
```

```
├── .env
├── .gitignore
├── package.json
└── README.md
```

Структура может изменяться в зависимости от выбранной архитектуры приложения.

Создание базы данных

Для выбранного варианта необходимо спроектировать базу данных PostgreSQL. Необходимо:

- создать базу данных;
- определить таблицы;
- определить первичные ключи;
- определить внешние ключи;
- выбрать типы данных;
- определить обязательные и необязательные поля;
- определить связи между таблицами.

Например, для каталога товаров:

```
categories
    │
    └──── products
```

Таблица products может содержать:

- id
- name
- description
- price
- category_id
- created_at

При наличии нескольких связанных сущностей необходимо использовать соответствующие внешние ключи.

Подключение Node.js к PostgreSQL

Необходимо создать серверное приложение на Node.js и настроить подключение к PostgreSQL. Параметры подключения не должны непосредственно записываться в исходный код. Для хранения конфигурации необходимо использовать переменные окружения, например:

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASSWORD
PORT
```

Файл .env не должен загружаться в публичный репозиторий.

Создание REST API

Для каждой основной сущности необходимо реализовать API. Например:

```
GET    /api/products
GET    /api/products/:id
POST   /api/products
PUT    /api/products/:id
DELETE /api/products/:id
```

Таким образом, необходимо реализовать основные CRUD-операции:

```
Create — создание;
Read — получение;
Update — изменение;
Delete — удаление.
```

Валидация данных и обработка ошибок

Сервер должен проверять поступающие данные. Например, при добавлении товара необходимо проверить:

- указано ли название;
- указана ли цена;
- является ли цена числом;
- не является ли цена отрицательной;
- существует ли указанная категория.

Некорректные запросы должны возвращать соответствующий HTTP-статус и сообщение об ошибке. Необходимо предусмотреть обработку типичных ошибок:

- некорректные данные;
- объект с указанным ID отсутствует;
- нарушение ограничений базы данных;
- ошибка подключения к БД;

- попытка выполнить недопустимую операцию;
- ошибка сервера.

Ответ API должен иметь понятный формат. Например:

```
{
  "error": "Товар не найден"
}
```

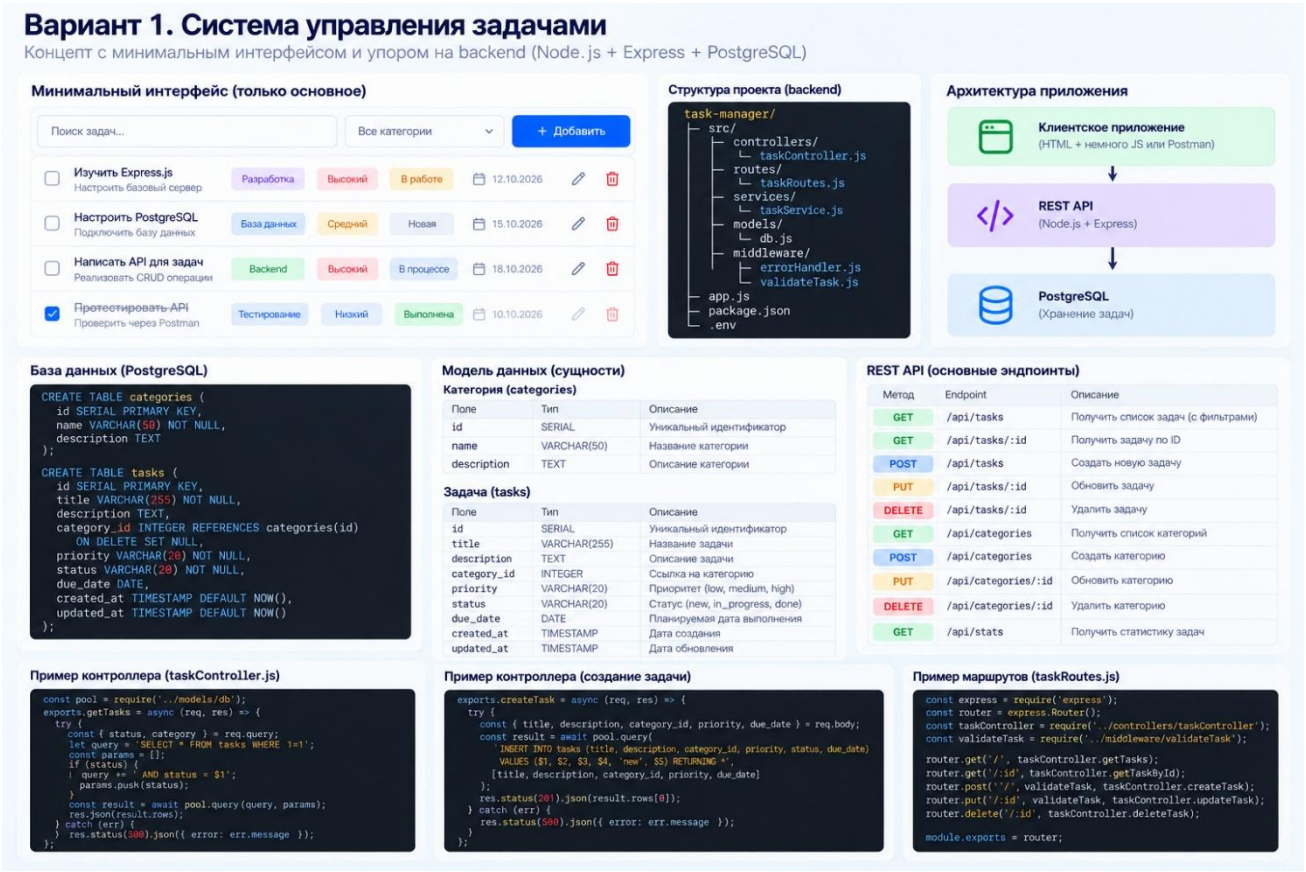
Требования к результату

В результате выполнения практической работы необходимо получить работающее backend-приложение, которое:

- разработано на Node.js;
- использует PostgreSQL;
- имеет спроектированную базу данных;
- содержит несколько связанных сущностей;
- предоставляет REST API;
- реализует CRUD-операции;
- использует SQL-запросы;
- выполняет валидацию данных;
- обрабатывает ошибки;
- использует переменные окружения;
- имеет структурированный исходный код;
- протестировано;
- при возможности подключено к React-интерфейсу;
- размещено в GitHub.

Вариант 1. Список задач

Разработать серверное приложение для управления персональными задачами, позволяющее пользователю планировать свою деятельность, распределять задачи по категориям и приоритетам, контролировать сроки выполнения и отслеживать текущий прогресс. Приложение должно обеспечивать централизованное хранение информации о задачах в базе данных PostgreSQL и предоставлять REST API для взаимодействия с клиентским интерфейсом.



Основные сущности:

- Задачи — название, описание, дата создания, срок выполнения, приоритет и текущий статус.
- Категории — тематические группы для классификации задач.
- Статусы — состояния задачи: «Новая», «В работе», «Выполнена».

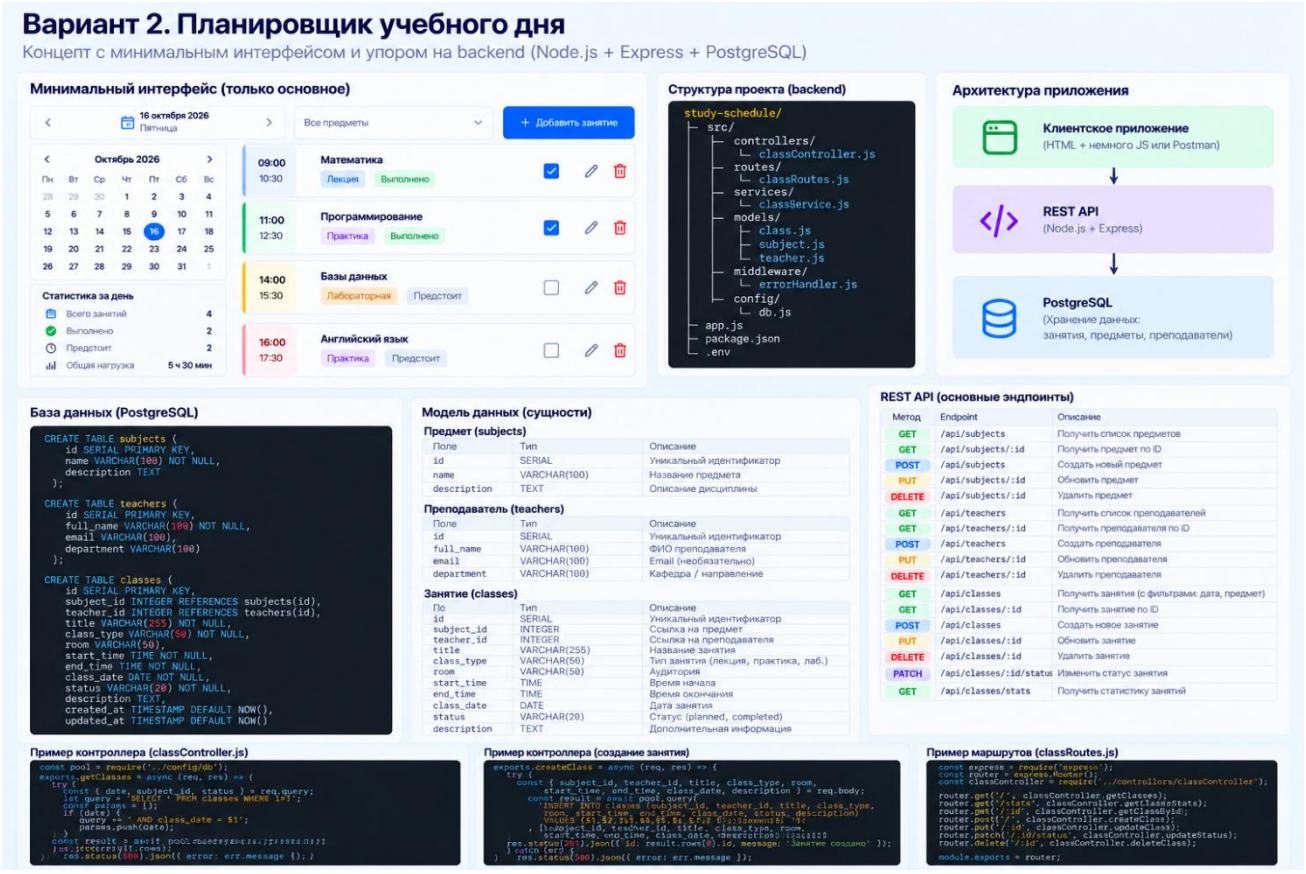
Необходимо реализовать:

1. Получение полного списка задач с возможностью сортировки по дате создания, сроку выполнения и приоритету.
2. Создание новой задачи с указанием названия, описания, категории, приоритета и планируемой даты завершения.
3. Редактирование существующих задач и удаление ненужных записей.
4. Изменение статуса задачи, включая отметку о выполнении и возможность возврата в активное состояние.
5. Поиск задач по названию и фильтрацию по категориям, статусам и приоритетам.
6. Получение статистики: общее количество задач, количество выполненных, активных и просроченных задач.
7. Проверку корректности вводимых данных и обработку ошибок при обращении к базе данных.

Работоспособное REST API для управления задачами, обеспечивающее хранение данных, выполнение CRUD-операций, фильтрацию и получение статистики выполнения.

Вариант 2. Планировщик учебного дня

Разработать backend-приложение для организации персонального учебного расписания студента. Система должна позволять планировать занятия на определённые даты, сохранять сведения о дисциплинах и преподавателях, контролировать выполнение учебных мероприятий и анализировать ежедневную учебную нагрузку. Все данные должны храниться в PostgreSQL и предоставляться клиентскому приложению через REST API.



Основные сущности:

- Занятия — название, дата, время начала и окончания, описание, аудитория и статус выполнения.
- Предметы — учебные дисциплины, к которым относятся занятия.
- Преподаватели — сведения о преподавателях, проводящих занятия.

Необходимо реализовать:

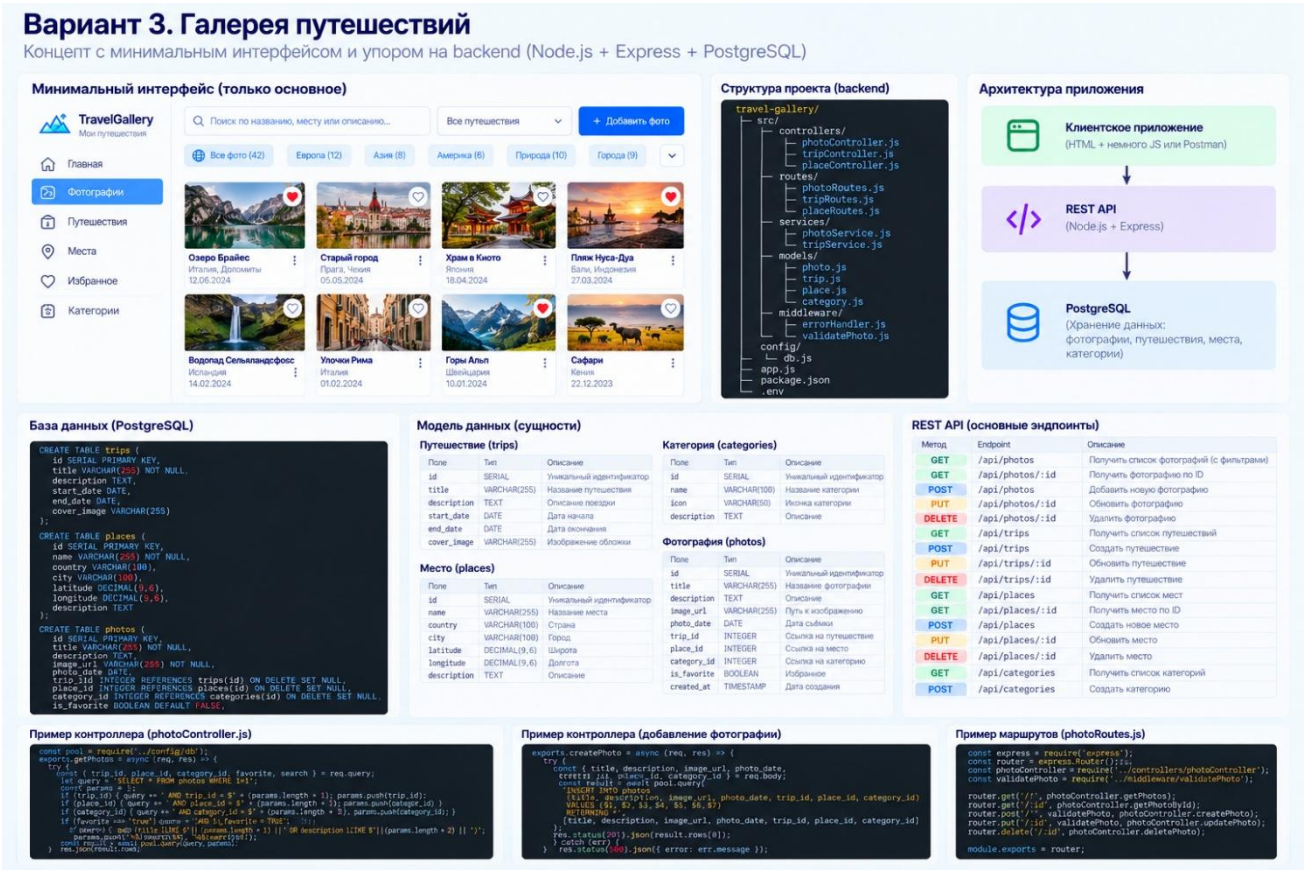
1. Получение расписания занятий на выбранную дату с сортировкой по времени начала.

2. Добавление занятия с указанием дисциплины, преподавателя, аудитории, времени и описания.
3. Редактирование и удаление запланированных занятий.
4. Изменение статуса занятия: «Предстоит» или «Выполнено».
5. Фильтрацию занятий по статусу и выбранной дате.
6. Переключение между датами с получением соответствующего расписания.
7. Расчёт ежедневной статистики: количество занятий, число выполненных и предстоящих, общая продолжительность учебной нагрузки.
8. Проверку корректности временных интервалов и обязательных полей.

REST API персонального учебного планировщика, обеспечивающее управление расписанием, контроль выполнения занятий и формирование ежедневной статистики.

Вариант 3. Галерея путешествий

Разработать серверное приложение для создания и управления персональной галереей фотографий из путешествий. Система должна обеспечивать хранение информации о поездках, посещённых местах и фотографиях, предоставлять возможности поиска и группировки изображений, а также формировать тематические коллекции. Информация о фотографиях и путешествиях должна храниться в PostgreSQL.



Основные сущности:

- Фотографии — название, описание, путь к изображению, дата съёмки и отметка избранного.
- Путешествия — название поездки, даты начала и окончания, описание.
- Места — страна, город или достопримечательность, связанные с фотографией.
- Категории — тематические группы изображений.

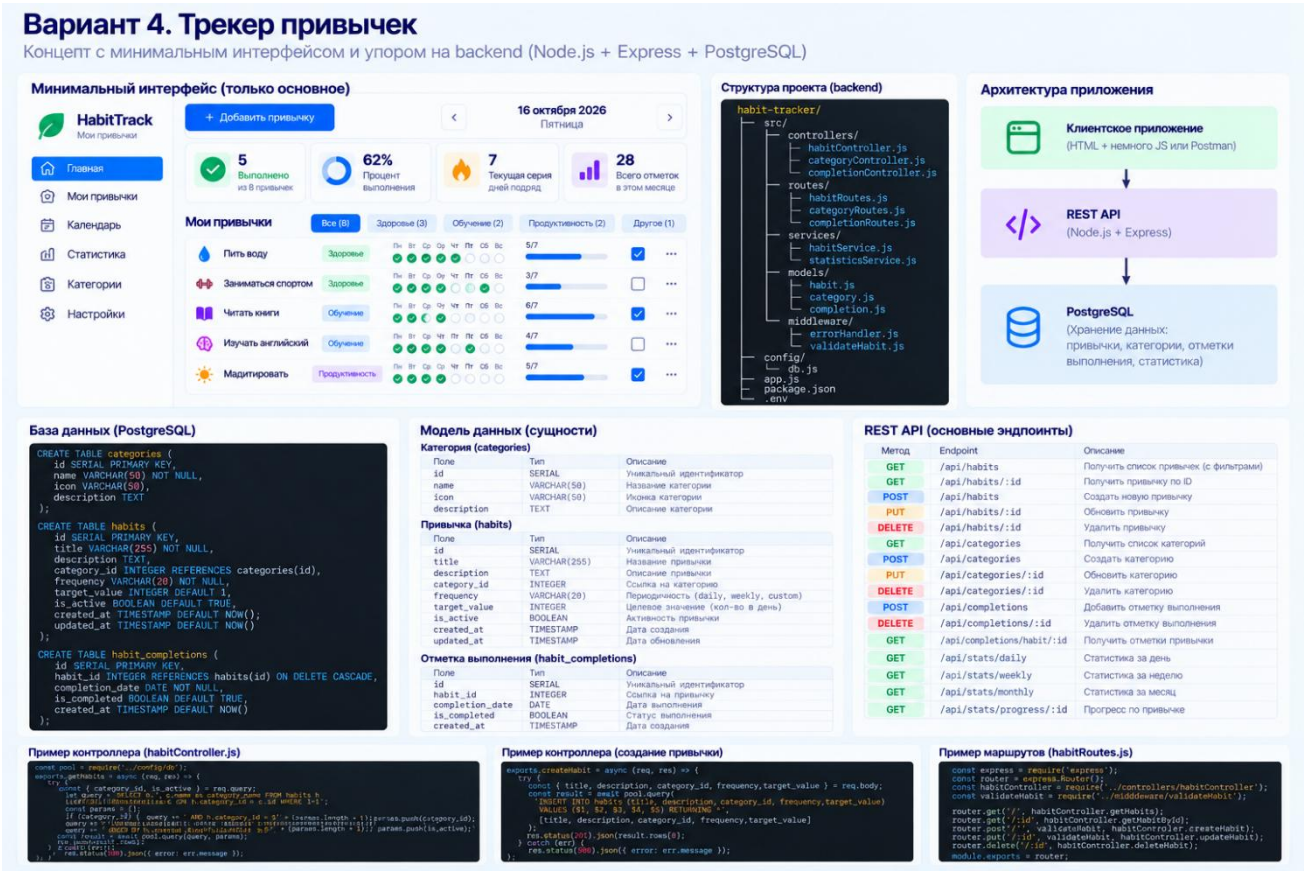
Необходимо реализовать:

1. Получение списка фотографий с основной информацией и ссылками на изображения.
2. Добавление фотографии с указанием названия, описания, места съёмки и принадлежности к путешествию.
3. Редактирование информации о фотографиях и удаление записей.
4. Создание и изменение путешествий и связанных с ними коллекций фотографий.
5. Поиск изображений по названию, стране, городу или описанию.
6. Фильтрацию фотографий по путешествиям, местам и категориям.
7. Добавление фотографий в избранное и получение отдельного списка избранных изображений.
8. Получение статистики по количеству фотографий, путешествий и посещённых мест.

REST API для организации фотогалереи путешествий с возможностью управления коллекциями, поиска изображений и хранения информации о поездках.

Вариант 4. Трекер привычек

Разработать backend-приложение для формирования полезных привычек и контроля регулярности их выполнения. Система должна позволять пользователю создавать привычки, устанавливать периодичность выполнения, ежедневно отмечать результаты и анализировать личный прогресс. Данные о привычках и истории выполнения необходимо хранить в PostgreSQL.



Основные сущности:

- Привычки — название, описание, дата создания, периодичность и целевое значение.
- Категории — направления привычек: здоровье, спорт, обучение, продуктивность и другие.
- Отметки выполнения — дата, привычка и состояние выполнения.

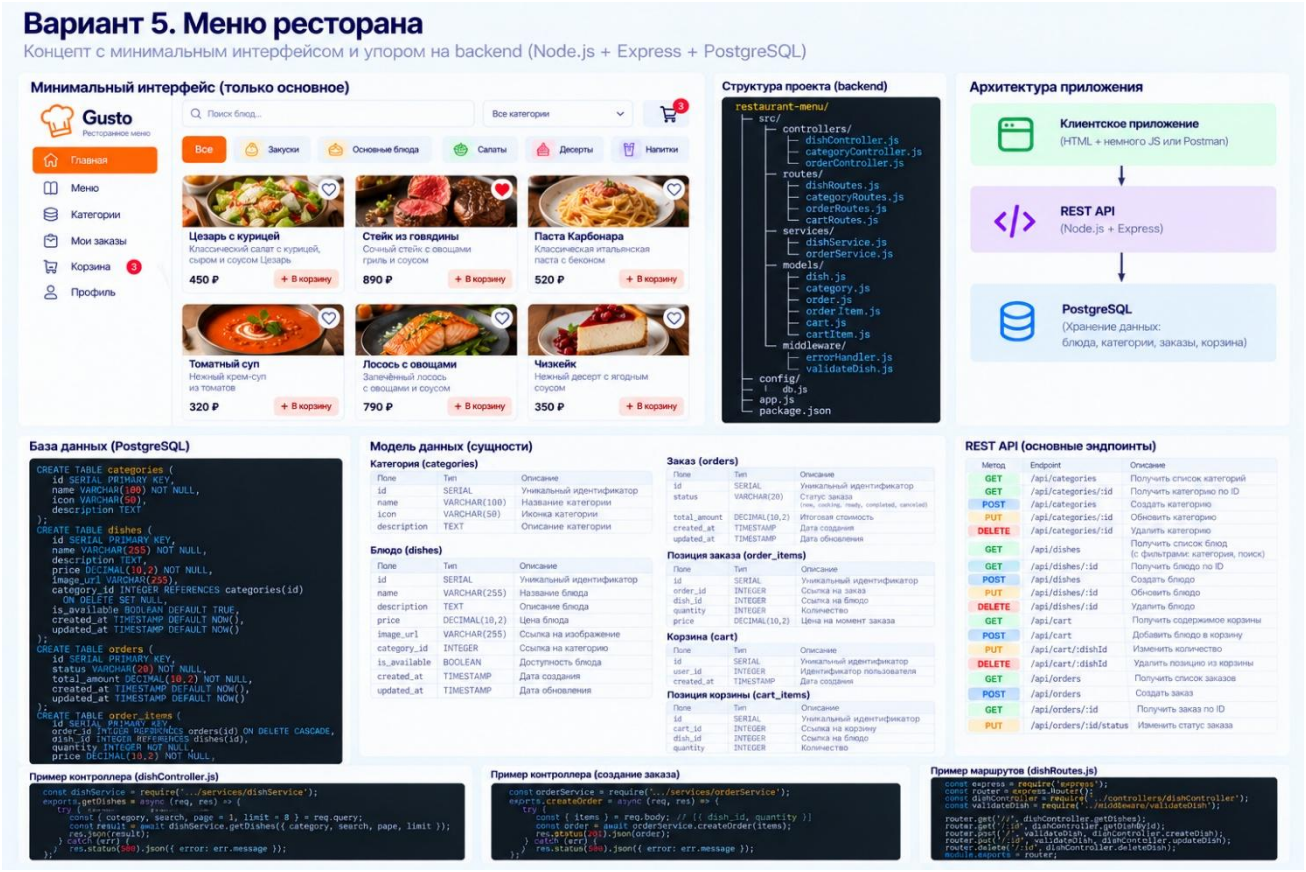
Необходимо реализовать:

1. Создание привычки с указанием названия, категории, описания и периодичности выполнения.
2. Получение списка активных привычек и подробной информации о каждой.
3. Редактирование параметров привычки и удаление записей.
4. Регистрацию выполнения привычки за выбранную дату с возможностью отмены отметки.
5. Получение календаря выполнения привычек за определённый период.
6. Расчёт текущей серии последовательных выполнений и максимальной достигнутой серии.
7. Формирование ежедневной и недельной статистики выполнения.
8. Расчёт общего процента выполнения привычек за выбранный период.

REST API трекера привычек, позволяющее регистрировать ежедневную активность, хранить историю и рассчитывать показатели личного прогресса.

Вариант 5. Меню ресторана

Разработать серверную часть электронного меню ресторана, обеспечивающую хранение информации о блюдах, их категориях, составе и стоимости, а также формирование заказов. Приложение должно предоставлять REST API для просмотра меню, управления ассортиментом и обработки заказов. Все сведения о блюдах и заказах должны храниться в PostgreSQL.



Основные сущности:

- Блюда — название, описание, цена, изображение и доступность.
- Категории — группы блюд: закуски, основные блюда, напитки, десерты.
- Заказы — дата создания, текущий статус и итоговая стоимость.
- Позиции заказа — выбранные блюда, количество и цена на момент заказа.

Необходимо реализовать:

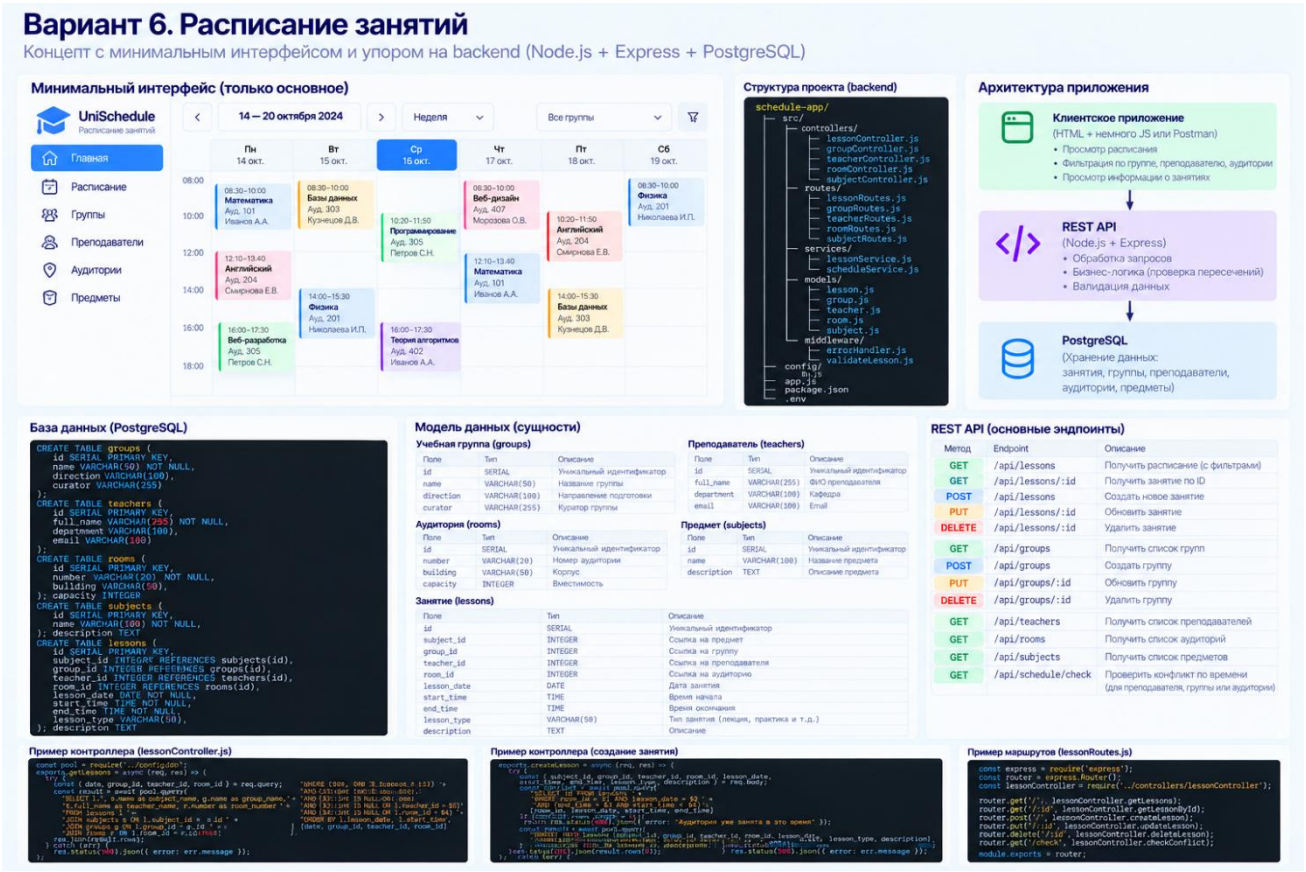
1. Получение полного меню ресторана с группировкой блюд по категориям.
2. Поиск блюд по названию и фильтрацию по категориям.
3. Получение подробной информации о блюде, включая описание, цену и доступность.
4. Добавление новых блюд, изменение их характеристик и удаление из меню.
5. Создание заказа с возможностью выбора нескольких блюд.
6. Добавление и удаление позиций заказа, изменение количества порций.

7. Автоматический расчёт общей стоимости заказа на стороне сервера.
8. Получение информации о заказе и изменение его статуса: «Новый», «Готовится», «Готов», «Завершён».

Ожидаемый результат: REST API электронного меню ресторана с поддержкой управления блюдами, формирования заказов и автоматического расчёта стоимости.

Вариант 6. Расписание занятий

Разработать backend-приложение для управления расписанием учебных занятий образовательной организации. Система должна обеспечивать хранение информации об учебных группах, дисциплинах, преподавателях, аудиториях и занятиях, а также предоставлять расписание с различными параметрами фильтрации. Данные должны храниться в PostgreSQL.



Основные сущности:

- Занятия — дисциплина, дата, время, тип занятия, группа, преподаватель и аудитория.
- Учебные группы — название группы и направление подготовки.
- Преподаватели — ФИО и дополнительная информация.
- Аудитории — номер, корпус и вместимость.
- Предметы — название учебной дисциплины.

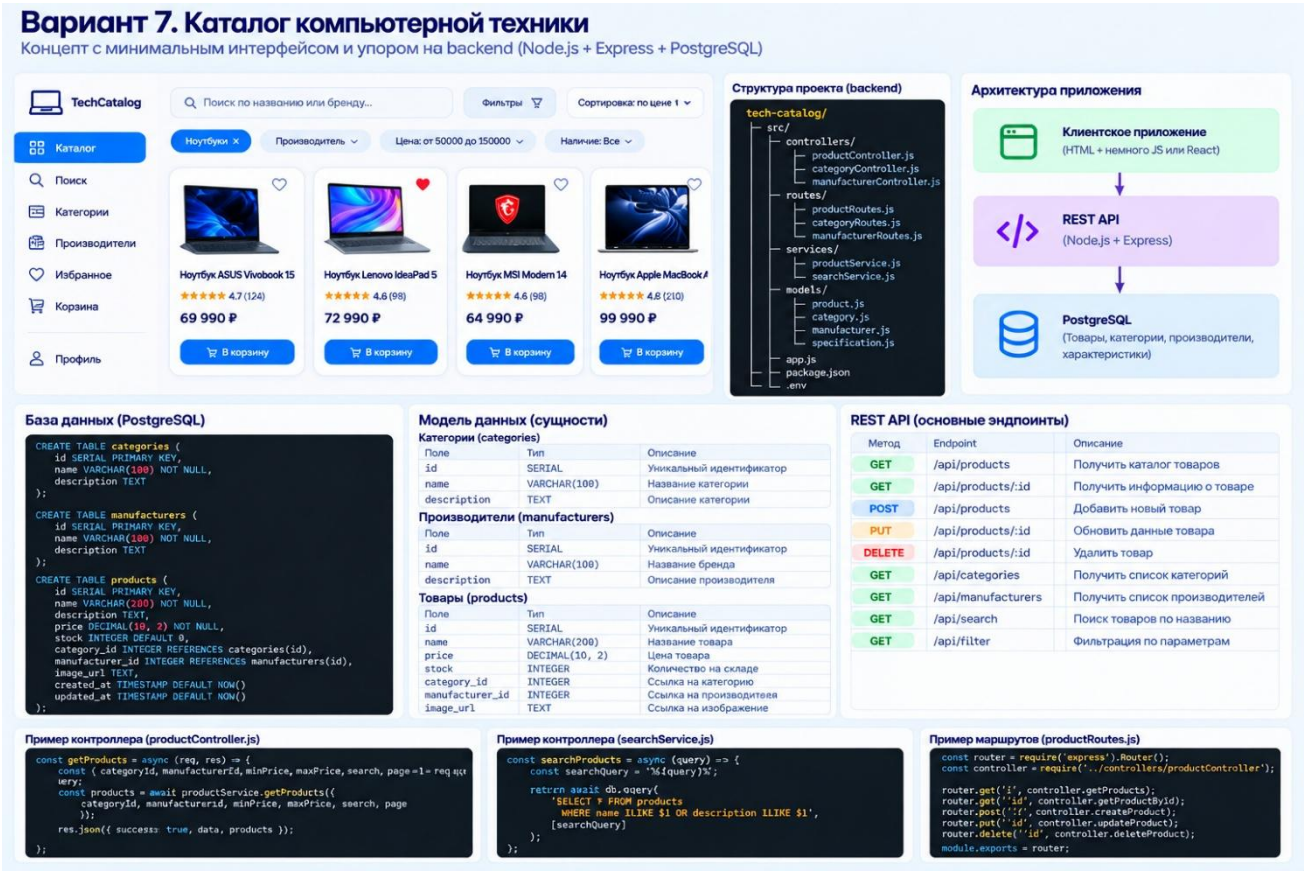
Необходимо реализовать:

1. Получение общего расписания занятий на выбранный день или неделю.
2. Получение расписания конкретной учебной группы.
3. Получение расписания преподавателя и отдельной аудитории.
4. Создание нового занятия с указанием дисциплины, группы, преподавателя, аудитории и времени.
5. Редактирование и удаление существующих занятий.
6. Поиск занятий по дисциплине, преподавателю или аудитории.
7. Проверку отсутствия пересечений занятий по времени для преподавателей, групп и аудиторий.
8. Получение информации о количестве занятий и распределении учебной нагрузки.

REST API университетского расписания, позволяющее управлять занятиями, выполнять поиск и предотвращать конфликты при планировании.

Вариант 7. Каталог компьютерной техники

Разработать backend-приложение для управления электронным каталогом компьютерной техники. Система должна обеспечивать хранение характеристик устройств, информации о производителях, категориях, ценах и наличии товаров. REST API должно поддерживать поиск, сортировку и фильтрацию каталога по различным параметрам.



Основные сущности:

- Товары — название, описание, цена, изображение, количество на складе.
- Категории — ноутбуки, компьютеры, мониторы, комплектующие и периферия.
- Производители — название бренда и описание.
- Характеристики — параметры оборудования и их значения.

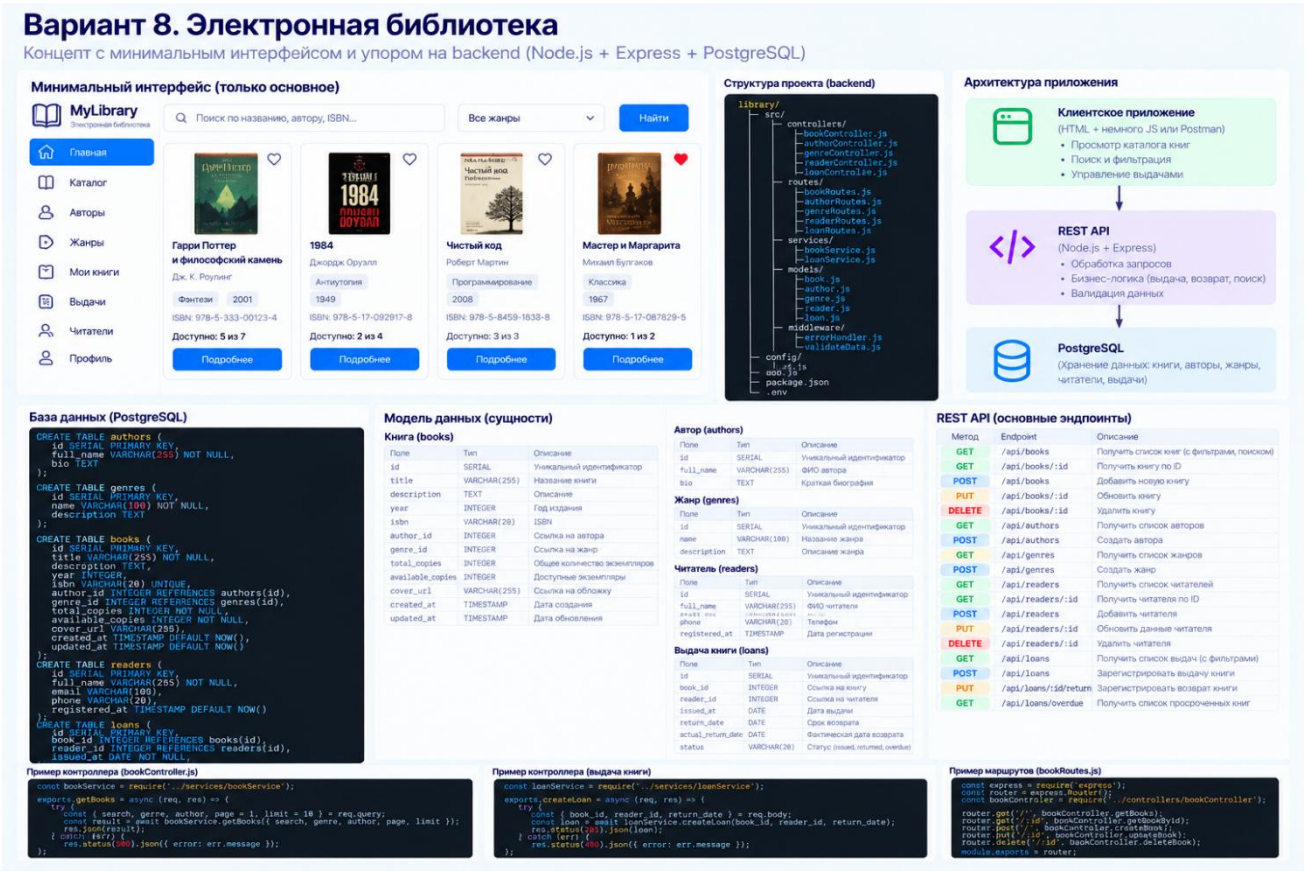
Необходимо реализовать:

1. Получение списка товаров с основной информацией, ценами и изображениями.
2. Получение подробной карточки товара с техническими характеристиками.
3. Добавление новых товаров в каталог.
4. Редактирование описания, цены, характеристик и количества товаров.
5. Удаление товаров из каталога.
6. Поиск по названию и производителю.
7. Фильтрацию по категории, бренду, наличию и диапазону стоимости.
8. Сортировку по цене и названию, а также постраничную загрузку каталога.

REST API каталога компьютерной техники, обеспечивающее управление ассортиментом, поиск и фильтрацию товаров по заданным характеристикам.

Вариант 8. Электронная библиотека

Разработать серверное приложение для организации электронной библиотеки, позволяющее хранить сведения о книгах, авторах и литературных жанрах, выполнять поиск по каталогу и контролировать выдачу книг читателям. Данные должны храниться в PostgreSQL и предоставляться через REST API.



Основные сущности:

- Книги — название, описание, год издания, ISBN, количество доступных экземпляров.
- Авторы — ФИО и краткая биографическая информация.
- Жанры — литературные направления.
- Читатели — сведения о пользователях библиотеки.
- Выдачи книг — дата выдачи, срок возврата и фактическая дата возврата.

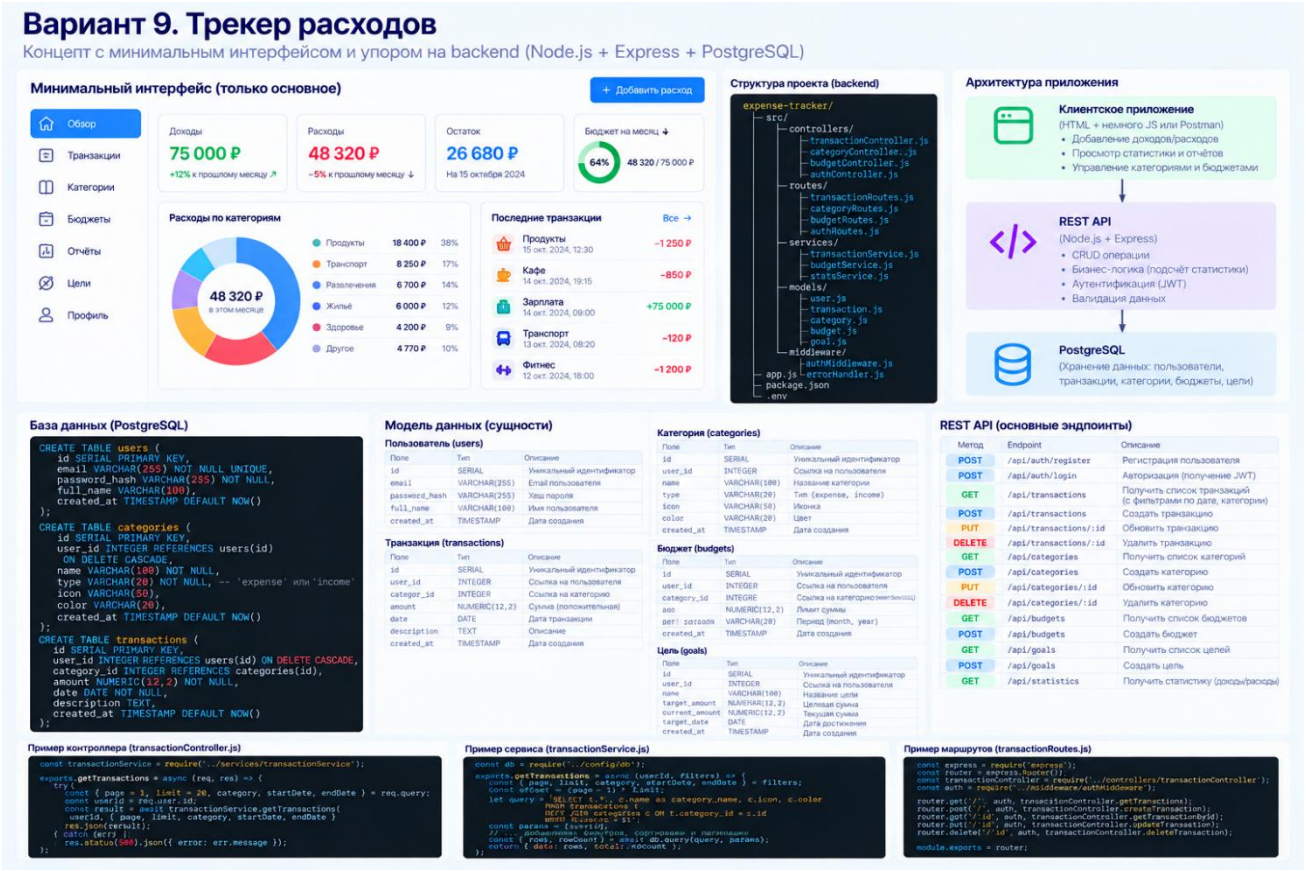
Необходимо реализовать:

1. Получение каталога книг с возможностью постраничного просмотра.
2. Поиск книг по названию, автору или ISBN.
3. Фильтрацию по жанрам и году издания.
4. Получение подробной информации о книге и её доступности.
5. Добавление, редактирование и удаление книг.
6. Регистрацию выдачи книги читателю с указанием даты возврата.
7. Регистрацию возврата книги и автоматическое изменение количества доступных экземпляров.
8. Получение истории выдач и списка книг, срок возврата которых истёк.

REST API электронной библиотеки, обеспечивающее управление книжным каталогом, читателями и процессом выдачи литературы.

Вариант 9. Трекер расходов

Разработать backend-приложение для учёта личных финансовых расходов. Система должна позволять регистрировать финансовые операции, распределять расходы по категориям, анализировать затраты за выбранные периоды и формировать данные для статистических диаграмм. Информация должна храниться в PostgreSQL.



Основные сущности:

- Расходы — сумма, дата, описание, категория.
- Категории — питание, транспорт, покупки, развлечения и другие.
- Пользователи — владельцы финансовых записей.

Необходимо реализовать:

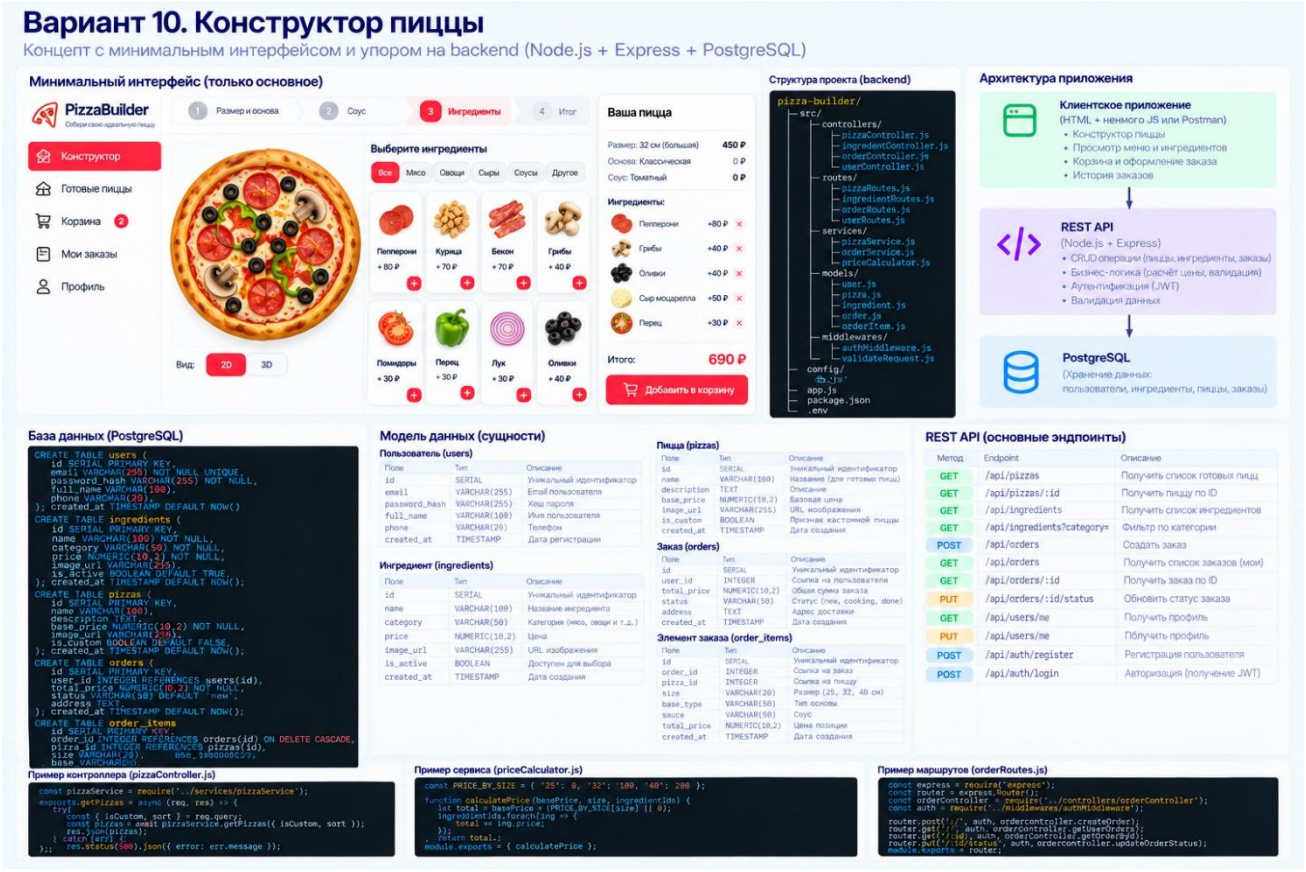
1. Добавление расхода с указанием суммы, категории, даты и комментария.
2. Получение списка расходов в хронологическом порядке.
3. Редактирование и удаление финансовых операций.
4. Поиск расходов по описанию и фильтрацию по категориям.
5. Получение расходов за выбранный день, неделю, месяц или произвольный период.
6. Расчёт общей суммы расходов за выбранный период.
7. Группировку расходов по категориям с вычислением суммы и процентной доли каждой категории.

8. Получение статистических данных для построения графиков динамики расходов.

Ожидаемый результат: REST API для учёта личных расходов, обеспечивающее хранение операций, фильтрацию и формирование финансовой аналитики.

Вариант 10. Конструктор пиццы

Разработать серверную часть интерактивного конструктора пиццы, позволяющего пользователю самостоятельно формировать заказ, выбирая размер, основу и дополнительные ингредиенты. Приложение должно хранить информацию о доступных компонентах, рассчитывать итоговую стоимость заказа и обеспечивать управление сохранёнными заказами.



Основные сущности:

- Пиццы — название, описание и базовая стоимость.
- Размеры — диаметр и коэффициент или стоимость размера.
- Основы — доступные виды теста.
- Ингредиенты — название, категория и дополнительная стоимость.
- Заказы — дата, состав, итоговая стоимость и статус.

Необходимо реализовать:

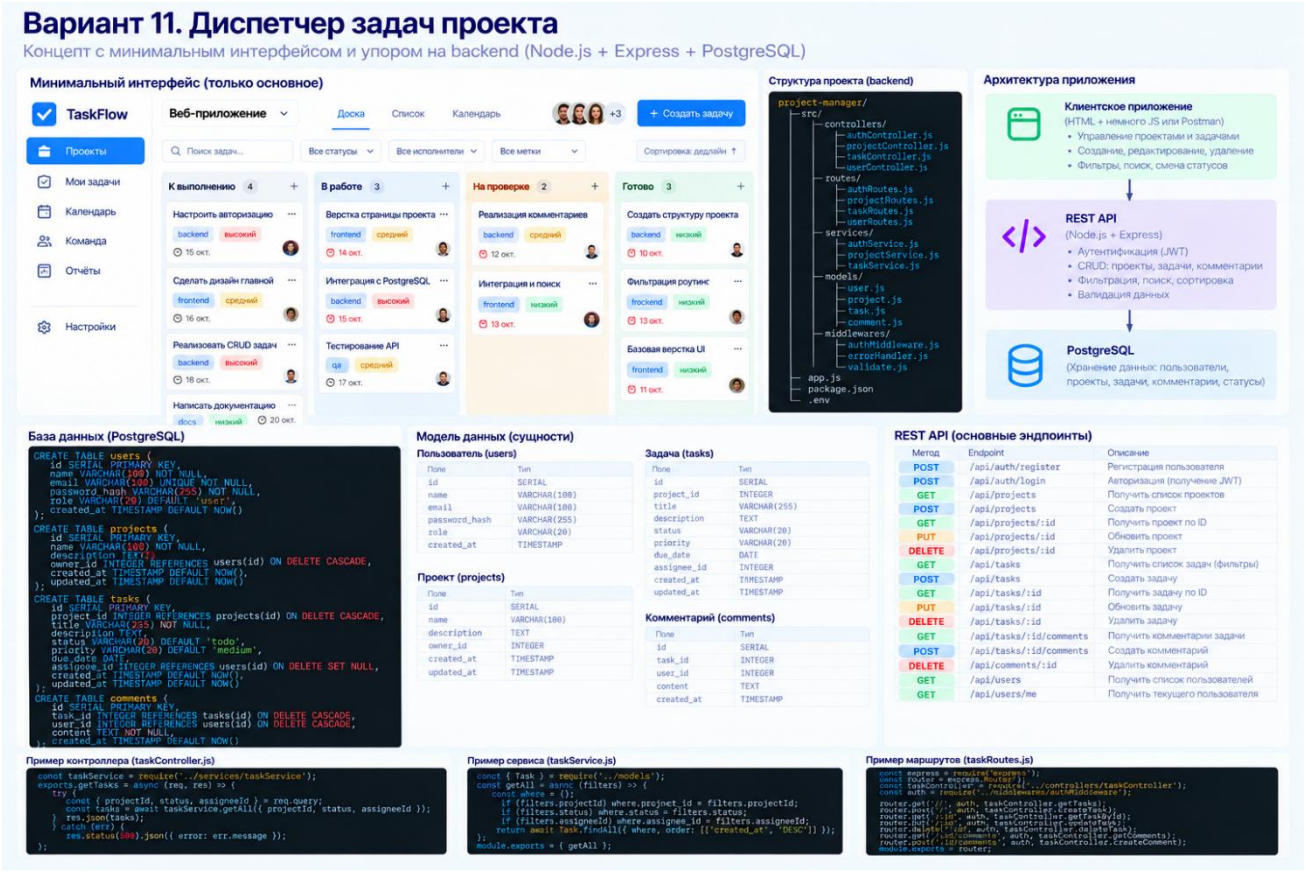
1. Получение списка доступных размеров, основ и ингредиентов.

2. Получение информации о базовых вариантах пиццы.
3. Создание индивидуальной конфигурации пиццы.
4. Добавление и удаление ингредиентов из выбранной конфигурации.
5. Автоматический пересчёт стоимости с учётом размера, основы и дополнительных ингредиентов.
6. Создание и сохранение заказа в базе данных.
7. Получение подробной информации о составе и стоимости заказа.
8. Редактирование или отмену заказа с повторным расчётом итоговой суммы.

REST API конструктора пиццы с возможностью индивидуальной настройки состава и автоматического расчёта стоимости заказа на сервере.

Вариант 11. Диспетчер задач проекта

Разработать backend-приложение для управления проектами и распределения задач между участниками команды. Система должна обеспечивать создание проектов, назначение исполнителей, контроль сроков, отслеживание статусов задач и формирование сводной информации о ходе выполнения проекта.



Основные сущности:

- Проекты — название, описание, дата начала, срок завершения.
- Задачи — название, описание, статус, приоритет, срок выполнения.
- Пользователи — участники команды.

- Статусы — этапы выполнения задач.

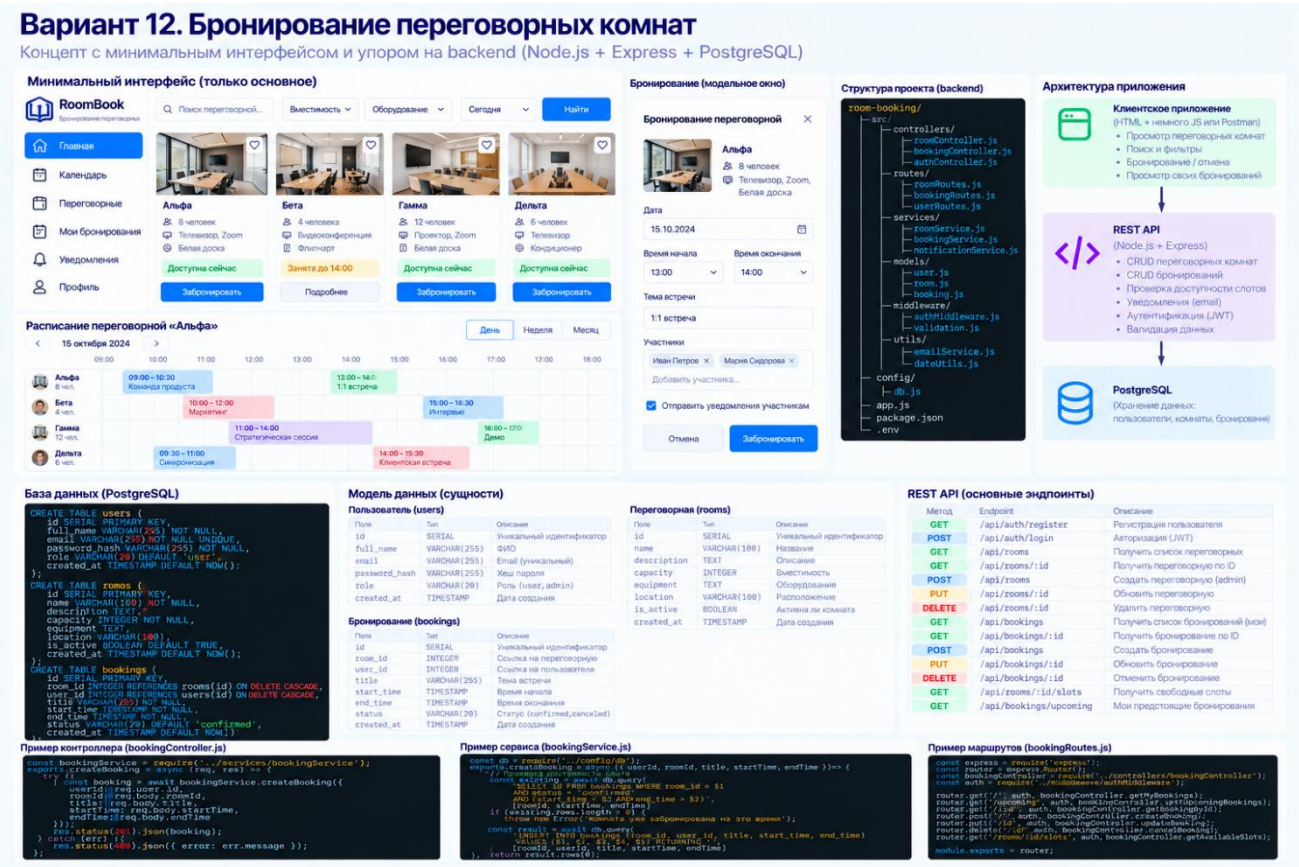
Необходимо реализовать:

1. Создание, редактирование и удаление проектов.
2. Получение списка проектов и подробной информации о выбранном проекте.
3. Добавление задач в проект с указанием приоритета и срока выполнения.
4. Назначение исполнителя для каждой задачи.
5. Изменение статуса задачи: «Новая», «В работе», «На проверке», «Завершена».
6. Редактирование и удаление задач.
7. Фильтрацию задач по проекту, исполнителю, статусу и приоритету.
8. Расчёт статистики проекта: общее количество задач, число завершённых и просроченных, процент выполнения.

REST API для управления командными проектами, распределения задач и отслеживания прогресса их выполнения.

Вариант 12. Бронирование переговорных комнат

Разработать серверное приложение для организации бронирования переговорных комнат в офисе или образовательной организации. Система должна позволять просматривать доступные помещения, выбирать дату и временной интервал, создавать бронирования и контролировать занятость комнат. Особое внимание необходимо уделить предотвращению пересечений бронирований.



Основные сущности:

- Комнаты — название, расположение, вместимость и оборудование.
- Пользователи — сотрудники, создающие бронирования.
- Бронирования — комната, пользователь, дата, время начала и окончания, статус.

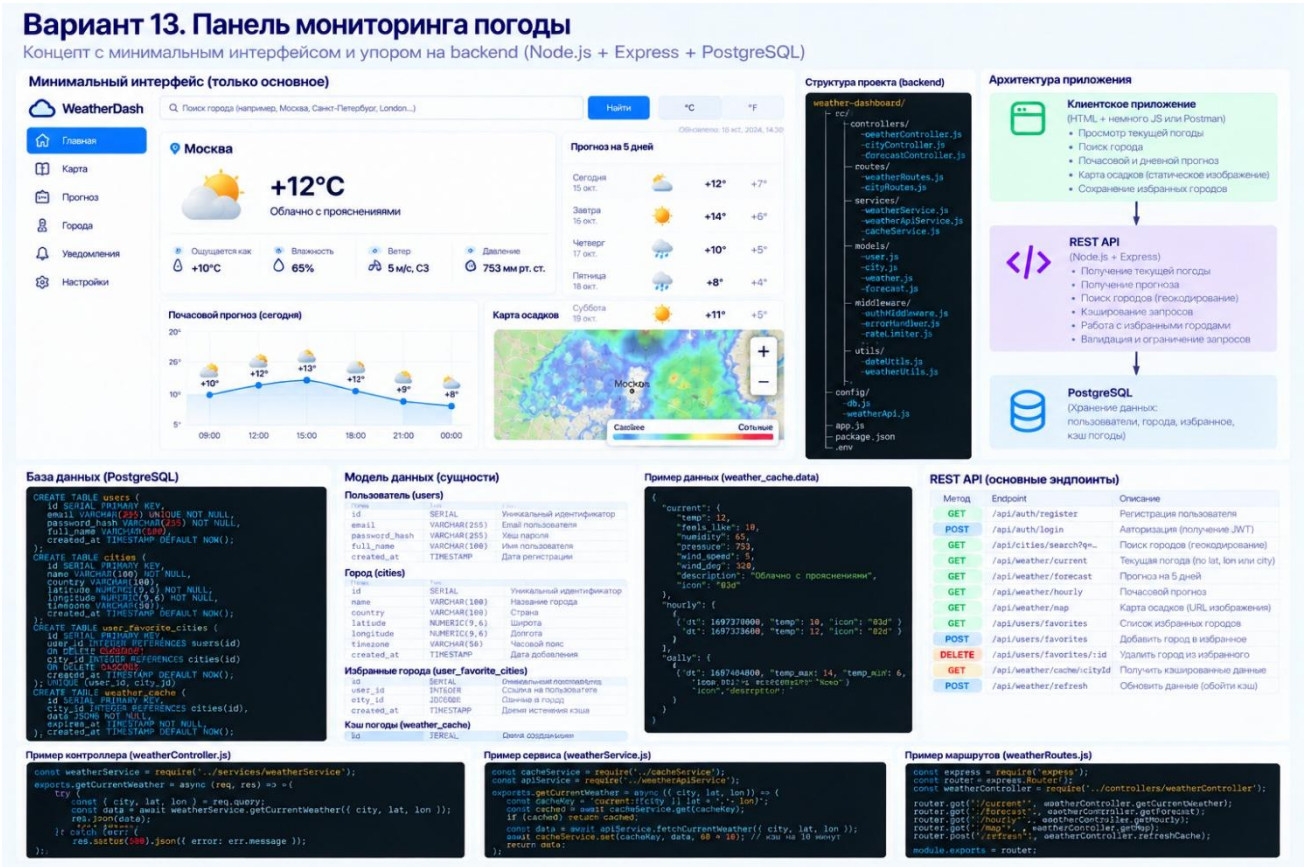
Необходимо реализовать:

1. Получение списка переговорных комнат с указанием вместимости и оборудования.
2. Получение расписания бронирований выбранной комнаты.
3. Поиск свободных комнат на указанную дату и временной интервал.
4. Создание бронирования с указанием помещения, времени и организатора.
5. Редактирование даты и времени существующего бронирования.
6. Отмену бронирования с обновлением статуса.
7. Получение списка активных и завершённых бронирований пользователя.
8. Серверную проверку отсутствия временных пересечений для одной комнаты при создании и изменении бронирований.

Ожидаемый результат: REST API системы бронирования переговорных комнат с проверкой доступности помещений и защитой от конфликтующих бронирований.

Вариант 13. Панель мониторинга погоды

Разработать backend-приложение для хранения, обработки и предоставления метеорологической информации по выбранным городам. Система должна обеспечивать просмотр текущих погодных условий, истории наблюдений и прогноза на ближайшие дни. Для выполнения учебной работы допускается использовать заранее подготовленные данные без обращения к внешним погодным сервисам.



Основные сущности:

- Города — название, страна, географические координаты.
- Погодные показатели — температура, влажность, давление, скорость ветра, погодные условия и время наблюдения.
- Прогнозы — город, дата прогноза, ожидаемые погодные показатели.

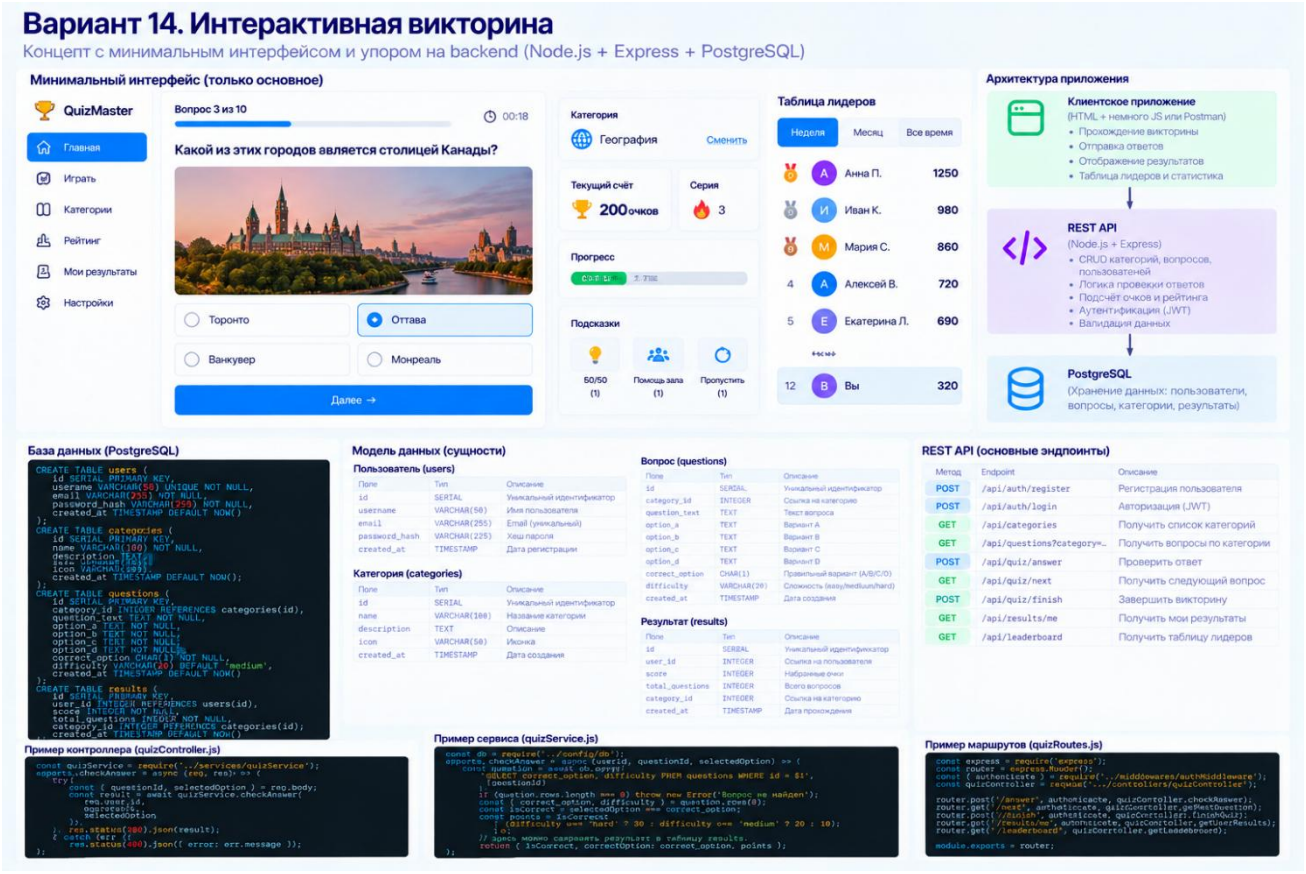
Необходимо реализовать:

1. Получение списка доступных городов.
2. Добавление нового города и редактирование его информации.
3. Удаление города с учётом связанных погодных записей.
4. Сохранение метеорологических показателей с указанием времени наблюдения.
5. Получение актуальных погодных данных для выбранного города.
6. Получение прогноза погоды на несколько дней.
7. Получение истории изменения температуры и других показателей за выбранный период.
8. Расчёт минимальной, максимальной и средней температуры за определённый промежуток времени.

REST API панели мониторинга погоды, предоставляющее текущие показатели, прогнозы и исторические данные для построения графиков.

Вариант 14. Интерактивная викторина

Разработать backend-приложение для организации интерактивных викторин и тестирования знаний пользователей. Система должна обеспечивать создание викторин, хранение вопросов и вариантов ответов, проведение тестирования, автоматическую проверку результатов и формирование статистики прохождения.



Основные сущности:

- Викторины — название, описание, категория и настройки прохождения.
- Вопросы — текст вопроса, принадлежность к викторине и количество баллов.
- Варианты ответов — текст ответа и признак правильности.
- Результаты — пользователь, викторина, дата прохождения, количество набранных баллов.

Необходимо реализовать:

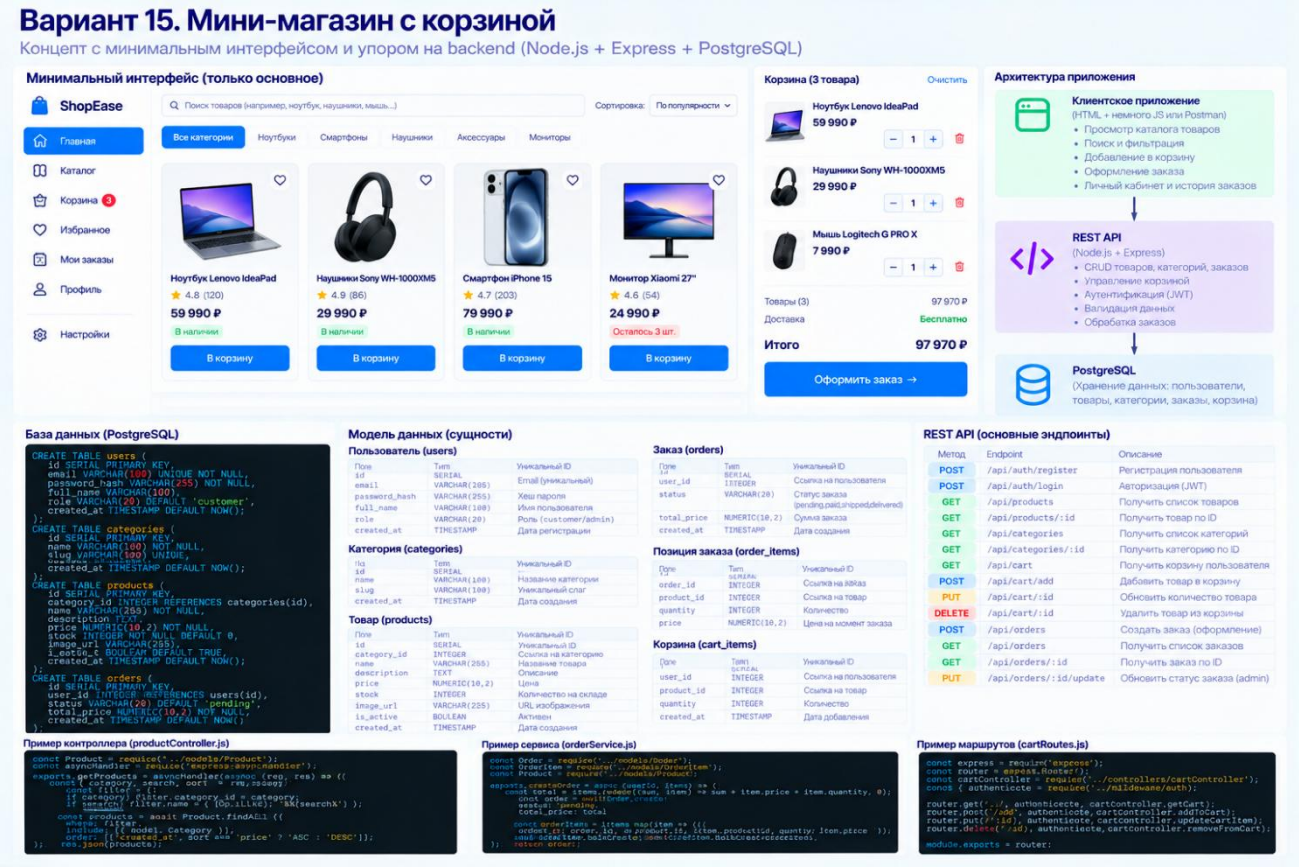
1. Создание новой викторины с указанием названия, описания и категории.
2. Получение списка доступных викторин.
3. Добавление, редактирование и удаление вопросов.
4. Создание вариантов ответов и указание правильного ответа.
5. Получение вопросов выбранной викторины без передачи признаков правильности ответов.
6. Приём выбранных пользователем ответов и их проверку на стороне сервера.
7. Расчёт количества правильных ответов, набранных баллов и процента успешности.

8. Сохранение результатов прохождения и получение истории результатов пользователя.

Ожидаемый результат: REST API интерактивной викторины с автоматической проверкой ответов, подсчётом баллов и хранением результатов тестирования.

Вариант 15. Мини-магазин с корзиной

Разработать серверное приложение небольшого интернет-магазина, обеспечивающее управление каталогом товаров, формирование покупательской корзины и оформление заказов. Система должна предоставлять REST API для поиска товаров, управления содержимым корзины, расчёта стоимости покупки и сохранения истории заказов.



Основные сущности:

- Товары — название, описание, цена, изображение и количество на складе.
- Категории — группы товаров.
- Пользователи — покупатели магазина.
- Корзины — текущие наборы выбранных товаров.
- Позиции корзины — товар и его количество.
- Заказы — дата оформления, статус и общая стоимость.
- Позиции заказа — состав оформленного заказа.

Необходимо реализовать:

1. Получение каталога товаров с ценами, изображениями и информацией о наличии.
2. Поиск товаров по названию и фильтрацию по категориям.
3. Получение подробной информации о выбранном товаре.
4. Добавление, редактирование и удаление товаров в каталоге.
5. Добавление товара в корзину и изменение количества выбранных единиц.
6. Удаление отдельных позиций и очистку корзины.
7. Автоматический пересчёт стоимости корзины на стороне сервера.
8. Создание заказа на основе содержимого корзины с проверкой наличия товаров.
9. Сохранение состава и итоговой стоимости заказа в PostgreSQL.
10. Получение истории заказов и просмотр подробной информации о каждом заказе.

Ожидаемый результат: REST API мини-магазина, обеспечивающее управление каталогом, корзиной и заказами, включая проверку наличия товаров и серверный расчёт стоимости.

Контрольные вопросы

1. Что такое Node.js?
2. Для чего Node.js используется при разработке web-приложений?
3. Что такое backend?
4. Что такое REST API?
5. Что такое endpoint?
6. Чем отличаются HTTP-методы GET, POST, PUT и DELETE?
7. Что такое CRUD?
8. Что такое PostgreSQL?
9. Что такое таблица базы данных?
10. Что такое первичный ключ?
11. Что такое внешний ключ?
12. Какие типы связей между таблицами существуют?
13. Что такое SQL?
14. Для чего используется SELECT?
15. Для чего используются INSERT, UPDATE и DELETE?
16. Зачем Node.js-приложению подключение к базе данных?
17. Что такое middleware?

18. Для чего используется маршрутизация?
19. Что такое HTTP-код 200?
20. Что означает HTTP-код 404?
21. Что означает HTTP-код 400?
22. Что означает HTTP-код 500?
23. Для чего нужна валидация данных на сервере?
24. Зачем используются переменные окружения?
25. Почему пароль от базы данных нельзя хранить в исходном коде?
26. Как React взаимодействует с Node.js API?
27. Как Node.js взаимодействует с PostgreSQL?
28. Что такое SQL-запрос с параметрами?
29. Для чего нужна обработка ошибок?
30. В чем отличие frontend от backend?

ПРАКТИЧЕСКАЯ РАБОТА 12. РАЗРАБОТКА BACKEND-ПРИЛОЖЕНИЯ НА PYTHON FLASK С ИСПОЛЬЗОВАНИЕМ POSTGRESQL

Цель работы

- создание web-приложения на Python;
- работу с Flask;
- организацию серверного приложения;
- создание маршрутов;
- обработку HTTP-запросов;
- разработку REST API;
- работу с HTTP-методами GET, POST, PUT, DELETE;
- подключение PostgreSQL;
- проектирование структуры базы данных;
- выполнение SQL-запросов из Python;
- работу с несколькими связанными таблицами;
- передачу данных в формате JSON;
- валидацию входных данных;
- обработку ошибок;
- использование переменных окружения;
- разделение приложения на модули;
- тестирование REST API;
- интеграцию Flask API с React-приложением.

Задания к практической работе

Необходимо разработать backend-приложение на Python Flask, которое предоставляет REST API для выбранной предметной области и взаимодействует с базой данных PostgreSQL.

В зависимости от выбранного варианта необходимо:

1. спроектировать базу данных;
2. создать таблицы и связи между ними;
3. создать Flask-приложение;
4. настроить маршруты;
5. реализовать REST API;
6. подключить PostgreSQL;
7. реализовать CRUD-операции;
8. обеспечить передачу данных в формате JSON;
9. добавить валидацию входных данных;
10. реализовать обработку ошибок;
11. протестировать API;
12. при необходимости подключить API к React-интерфейсу;
13. разместить исходный код проекта в GitHub.

Структура Flask-проекта

Для работы рекомендуется использовать модульную структуру:

```
project/
│
├── app/
│   ├── __init__.py
│   │
│   ├── routes/
│   │   ├── products.py
│   │   ├── categories.py
│   │   └── users.py
│   │
│   ├── models/
│   │   ├── product.py
│   │   └── category.py
│   │
│   ├── services/
│   │   └── product_service.py
│   │
│   ├── db/
│   │   └── database.py
│   │
│   └── ...
│
├── sql/
│   ├── schema.sql
```

```
│   └── seed.sql
│
├── tests/
│   └── ...
│
├── .env
├── .gitignore
├── requirements.txt
├── run.py
└── README.md
```

Для небольших вариантов допускается использовать более простую структуру:

```
project/
│
├── app.py
├── database.py
├── routes.py
├── schema.sql
├── requirements.txt
├── .env
└── README.md
```

Создание Flask-приложения

Необходимо создать Flask-приложение и настроить запуск локального сервера. Приложение должно:

- принимать HTTP-запросы;
- определять маршрут;
- обрабатывать параметры запроса;
- обращаться к базе данных;
- формировать JSON-ответ;
- возвращать HTTP-статус.

Пример маршрута:

```
@app.route("/api/products", methods=["GET"])def get_products():    ...
```

Подключение PostgreSQL

Необходимо создать базу данных PostgreSQL и подключить ее к Flask-приложению. Параметры подключения должны храниться в переменных окружения:

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
```

```
DB_PASSWORD
```

Секретные данные не должны находиться непосредственно в исходном коде. Для управления зависимостями необходимо создать файл: requirements.txt. В нем должны быть указаны используемые Python-библиотеки.

Проектирование базы данных

Для выбранного варианта необходимо определить:

- таблицы;
- поля;
- типы данных;
- первичные ключи;
- внешние ключи;
- ограничения;
- связи между таблицами.

Например:

```
categories
     │
     └──── products
```

SQL-структура базы данных должна быть вынесена в отдельный файл: sql/schema.sql

Начальные тестовые данные рекомендуется вынести в: sql/seed.sql

Создание REST API

Для основных сущностей необходимо реализовать REST API. Например, для товаров:

```
GET    /api/products
GET    /api/products/<id>
POST   /api/products
PUT    /api/products/<id>
DELETE /api/products/<id>
```

API должно использовать соответствующие HTTP-методы.

Основные операции:

```
GET — получение;
POST — создание;
PUT — изменение;
DELETE — удаление.
```

Работа с JSON

Flask-приложение должно принимать данные от клиента в формате JSON и возвращать результаты также в JSON.

Пример запроса:

```
{
    "name": "Ноутбук",
    "price": 899.99,
    "category_id": 2
}
```

Пример ответа:

```
{
    "id": 15,
    "name": "Ноутбук",
    "price": 899.99,
    "category_id": 2
}
```

Валидация данных и обработка ошибок

Сервер должен проверять входящие данные. Например:

- Название не должно быть пустым
- Цена должна быть числом
- Цена не может быть отрицательной
- ID категории должен существовать

При некорректных данных API должно вернуть соответствующий HTTP-статус и описание ошибки.

Необходимо предусмотреть обработку:

- некорректных данных;
- отсутствующего объекта;
- неверного ID;
- нарушения ограничений БД;
- ошибки подключения к PostgreSQL;
- некорректного JSON;
- внутренних ошибок сервера.

Например:

```
{
    "error": "Товар не найден"
}
```

Требования к результату

В результате выполнения работы необходимо получить работающее Flask-приложение, которое:

- разработано на Python;
- использует Flask;
- использует PostgreSQL;
- имеет спроектированную базу данных;
- содержит не менее 3 таблиц;
- содержит связи между таблицами;
- предоставляет REST API;
- реализует CRUD-операции;
- принимает и возвращает JSON;
- выполняет валидацию;
- обрабатывает ошибки;
- использует переменные окружения;
- имеет структурированный исходный код;
- содержит requirements.txt;
- протестировано;
- имеет README;
- размещено в GitHub.

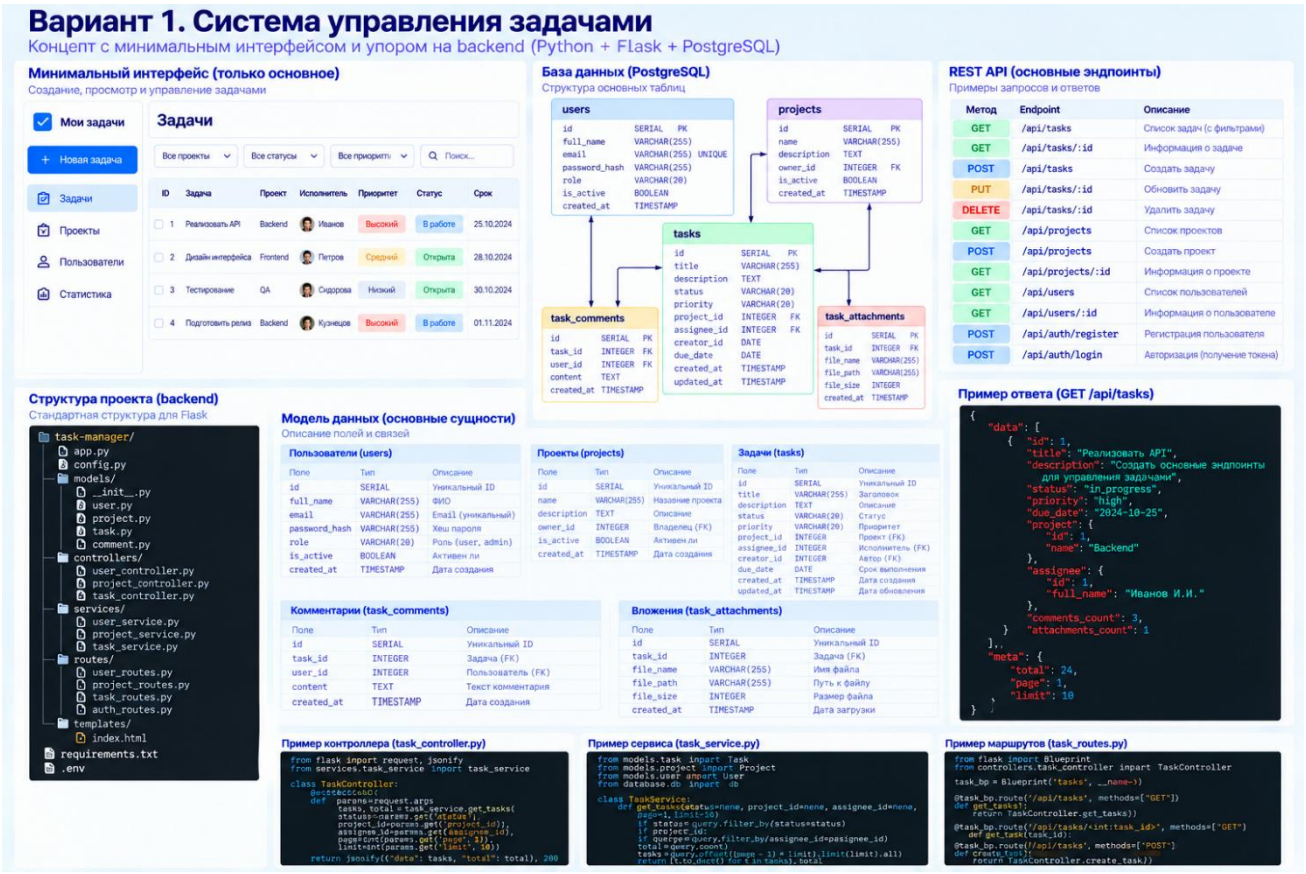
Минимальные требования для каждого варианта. Для унификации заданий:

- не менее 3 таблиц;
- не менее 5 endpoints;
- CRUD для основной сущности;
- не менее одной связи между таблицами;
- валидация данных;
- обработка ошибок;

- работа с PostgreSQL;
- JSON API;
- README с инструкцией запуска.

Вариант 1. Список задач

Разработать Flask API для приложения управления задачами.



Сущности:

- задачи;
- категории;
- статусы.

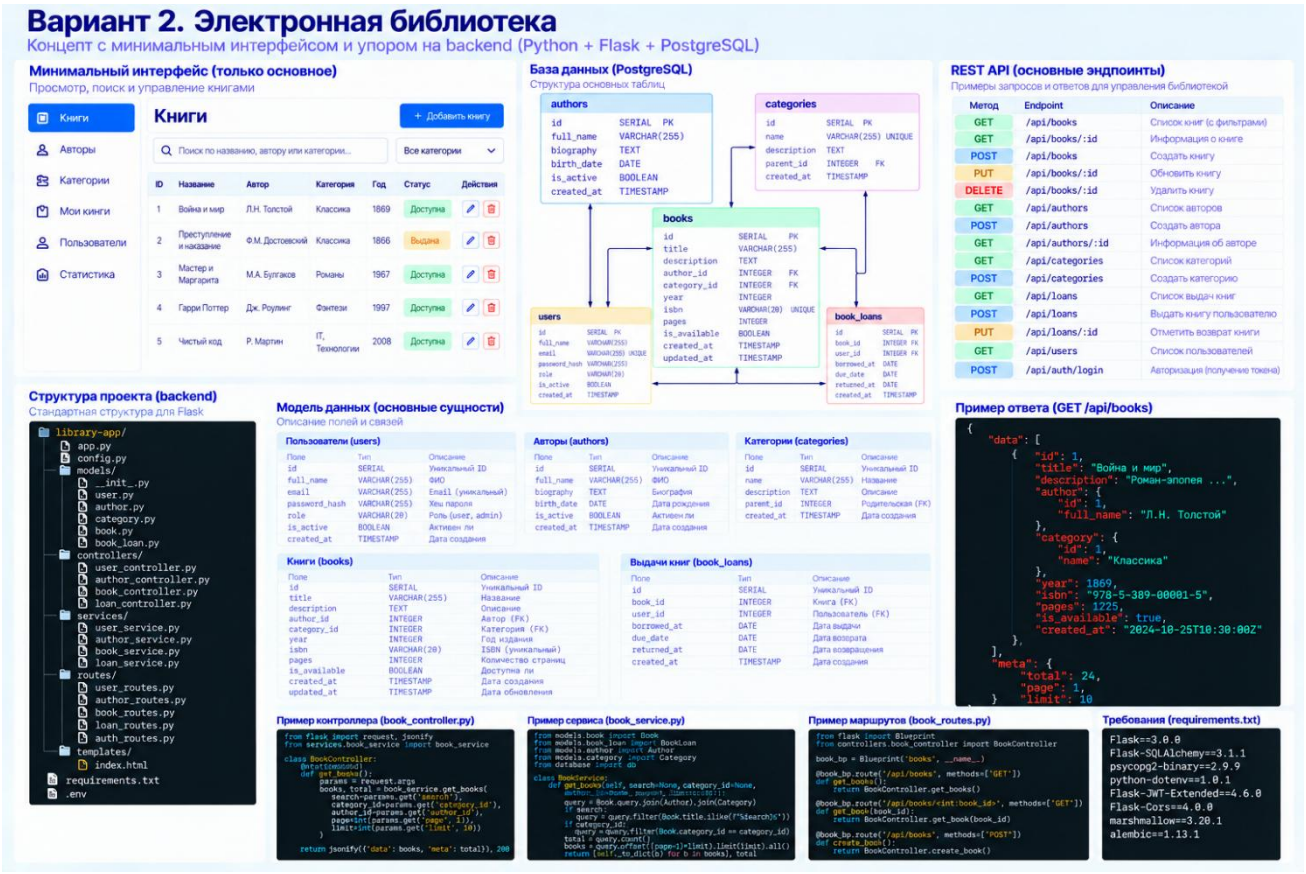
Необходимо реализовать:

- получение списка задач;
- получение задачи по ID;
- создание задачи;
- изменение задачи;
- удаление задачи;
- фильтрацию по статусу;

- получение списка категорий.

Вариант 2. Планировщик учебного дня

Разработать API для хранения учебного расписания.



Сущности:

- занятия;
- предметы;
- преподаватели.

Необходимо реализовать:

- получение расписания;
- добавление занятия;
- изменение занятия;
- удаление занятия;
- получение расписания на определенный день;
- получение информации о предмете.

Вариант 3. Галерея путешествий

Разработать серверную часть приложения для хранения фотографий путешествий.

Сущности:

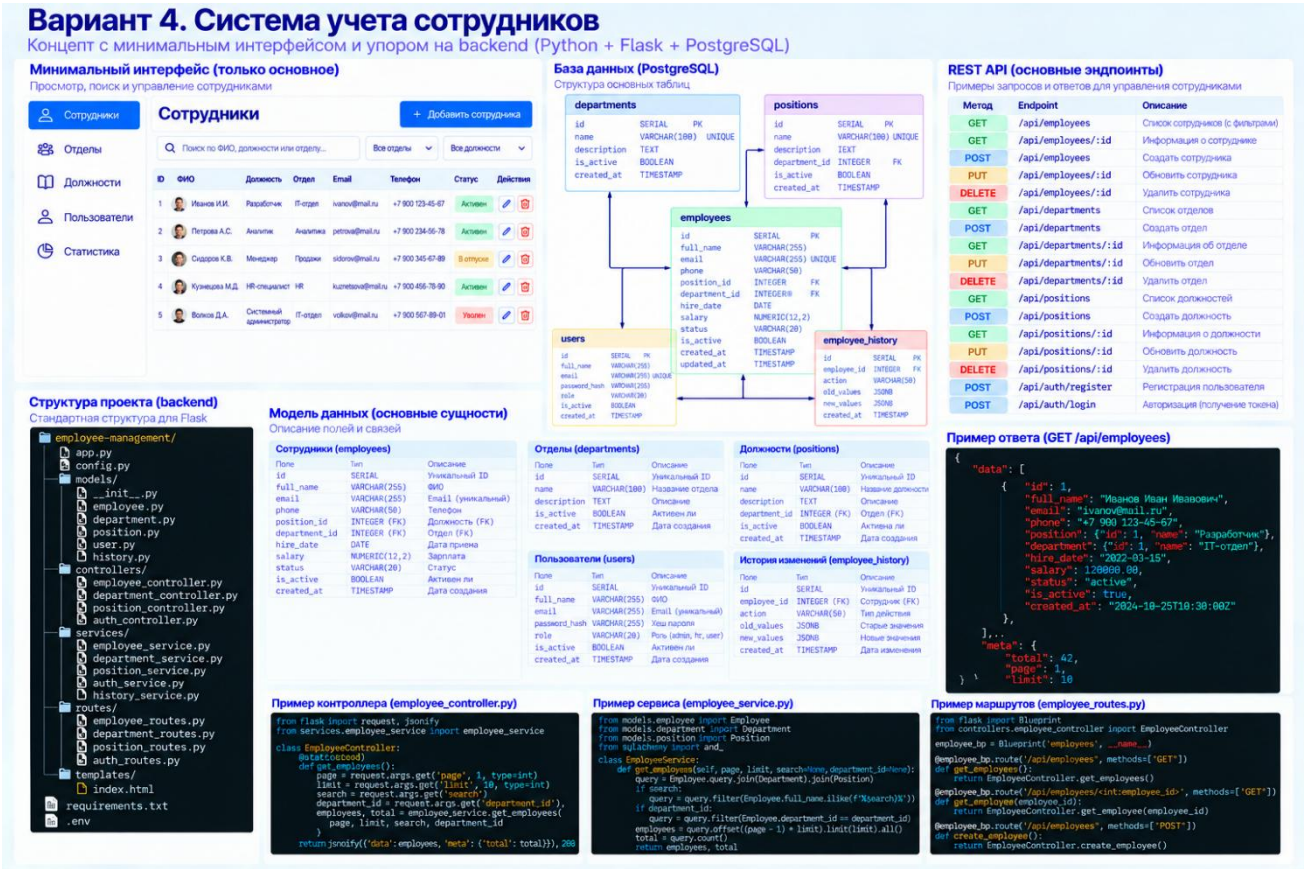
- фотографии;
- путешествия;
- места.

Необходимо реализовать:

- добавление фотографии;
- получение списка фотографий;
- получение фотографии по ID;
- изменение информации;
- удаление фотографии;
- получение фотографий конкретного путешествия;
- поиск по месту.

Вариант 4. Трекер привычек

Разработать Flask API для контроля полезных привычек.



Сущности:

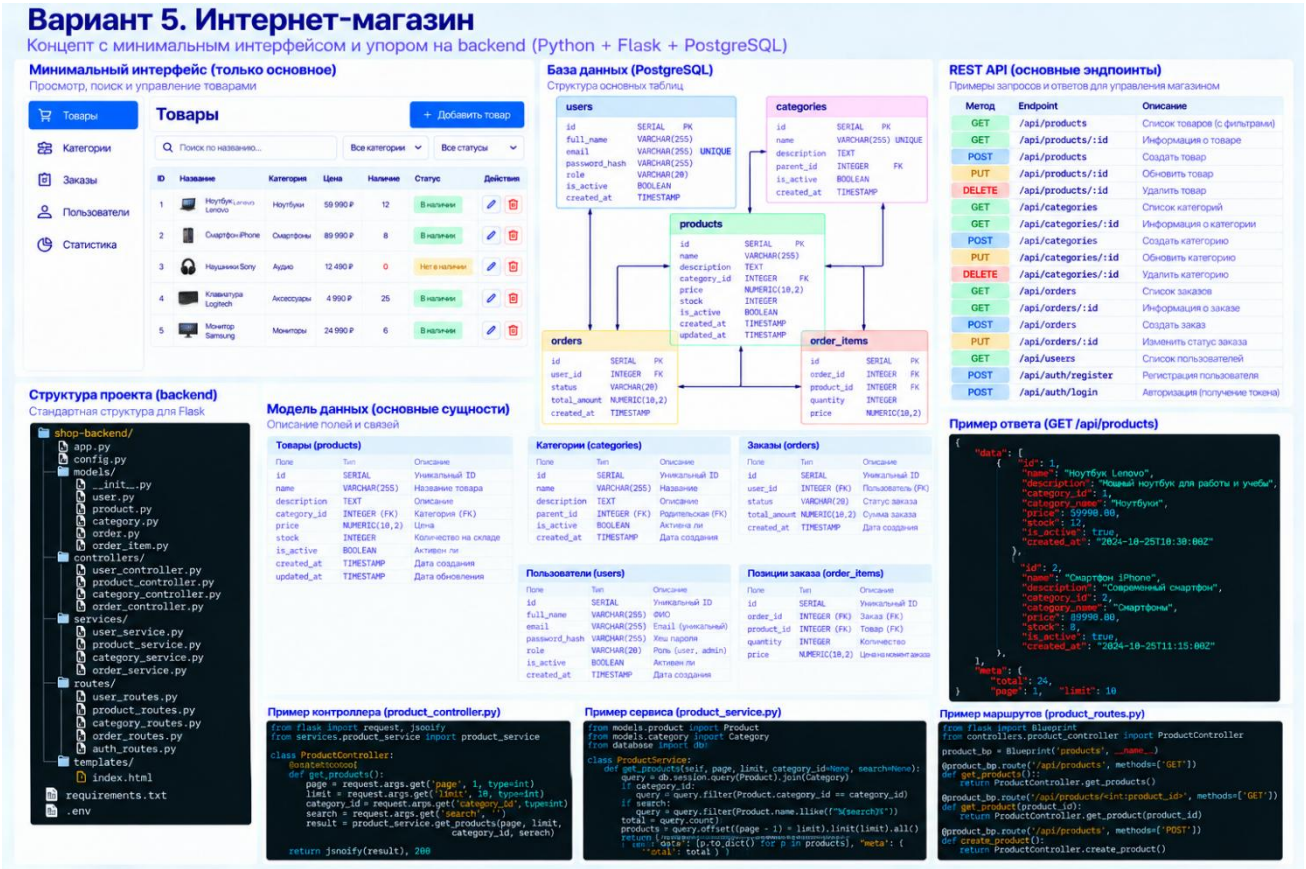
- привычки;
- категории;
- отметки выполнения.

Необходимо реализовать:

- создание привычки;
- получение списка привычек;
- изменение привычки;
- удаление привычки;
- отметку выполнения;
- получение истории выполнения;
- расчет прогресса.

Вариант 5. Меню ресторана

Разработать backend для электронного меню ресторана.



Сущности:

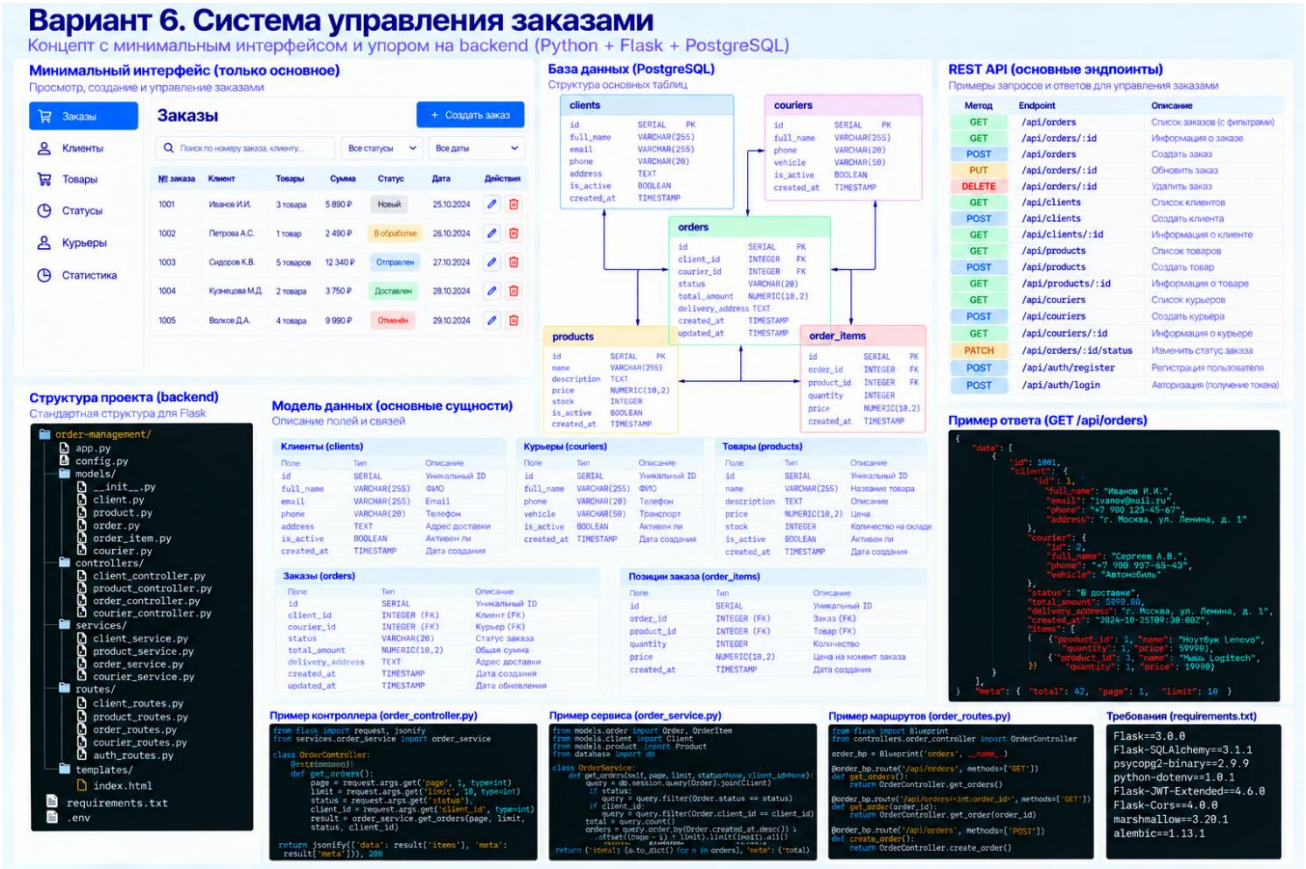
- блюда;
- категории;
- заказы.

Необходимо реализовать:

- получение меню;
- получение блюд категории;
- добавление блюда;
- изменение блюда;
- удаление блюда;
- создание заказа;
- получение состава заказа.

Вариант 6. Расписание занятий

Разработать API университетского расписания.



Сущности:

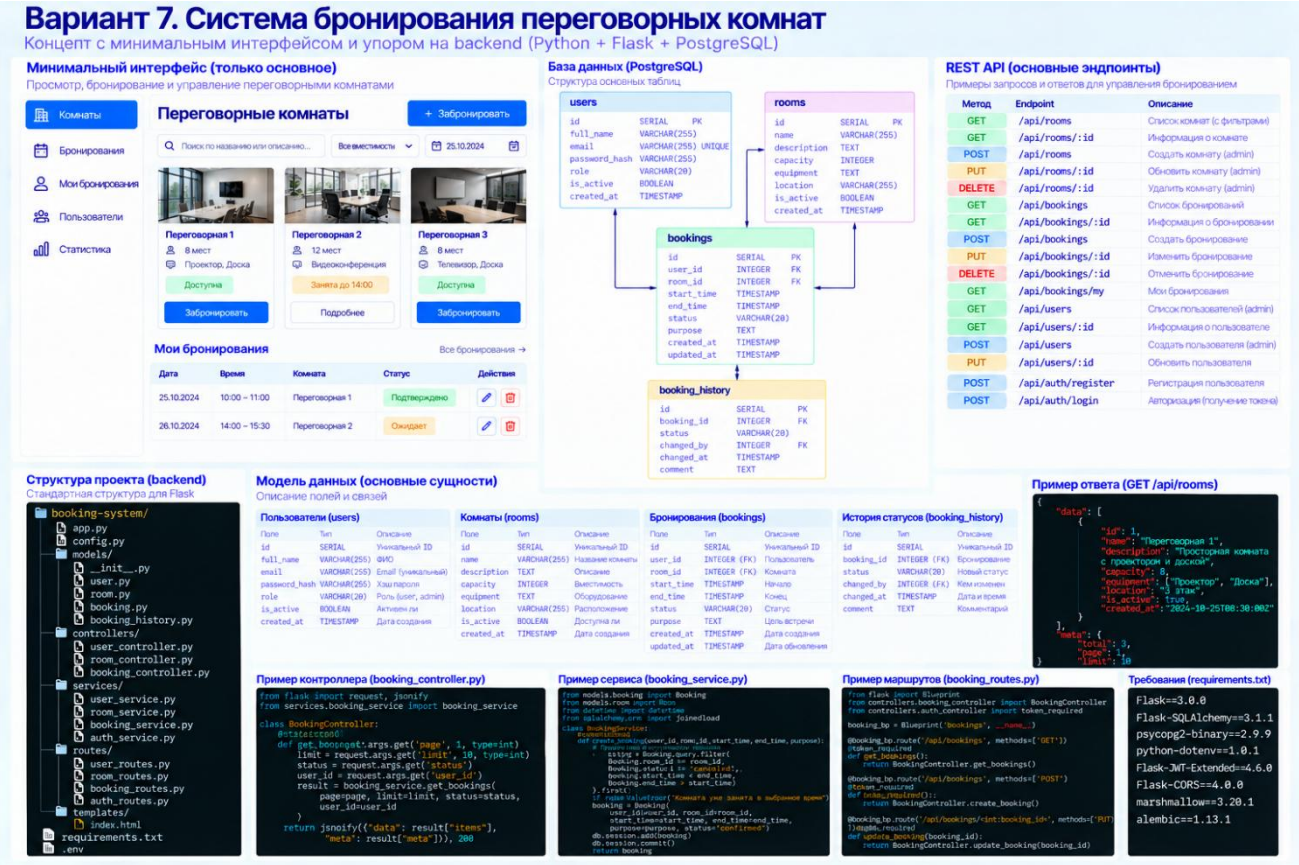
- группы;
- преподаватели;
- аудитории;
- занятия.

Необходимо реализовать:

- получение расписания;
- добавление занятия;
- изменение занятия;
- удаление занятия;
- получение расписания группы;
- получение расписания преподавателя;
- получение расписания аудитории.

Вариант 7. Каталог компьютерной техники

Разработать backend интернет-каталога компьютерной техники.



Сущности:

- товары;
- категории;
- производители.

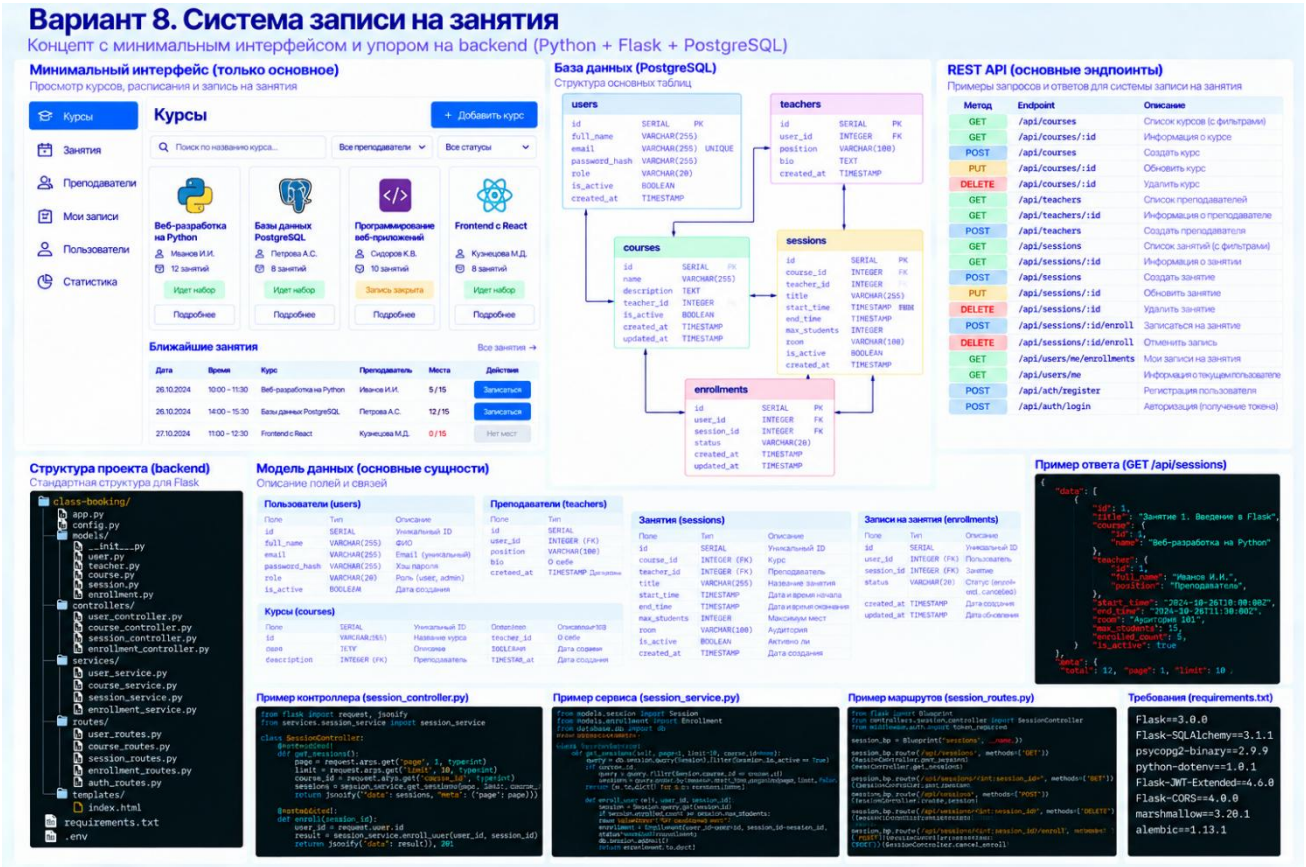
Необходимо реализовать:

- получение каталога;
- получение товара;
- добавление товара;
- изменение товара;
- удаление товара;
- поиск;
- фильтрацию по категории;
- фильтрацию по производителю.

Дополнительно можно реализовать фильтрацию по диапазону цены.

Вариант 8. Электронная библиотека

Разработать Flask API электронной библиотеки.



Сущности:

- книги;
- авторы;
- жанры;
- читатели.

Необходимо реализовать:

- получение списка книг;
- получение книги по ID;
- добавление книги;
- изменение книги;
- удаление книги;
- поиск;
- фильтрацию по жанру;
- получение книг автора.

Дополнительно реализовать выдачу и возврат книг.

Вариант 9. Трекер расходов

Разработать backend для учета личных расходов.

Сущности:

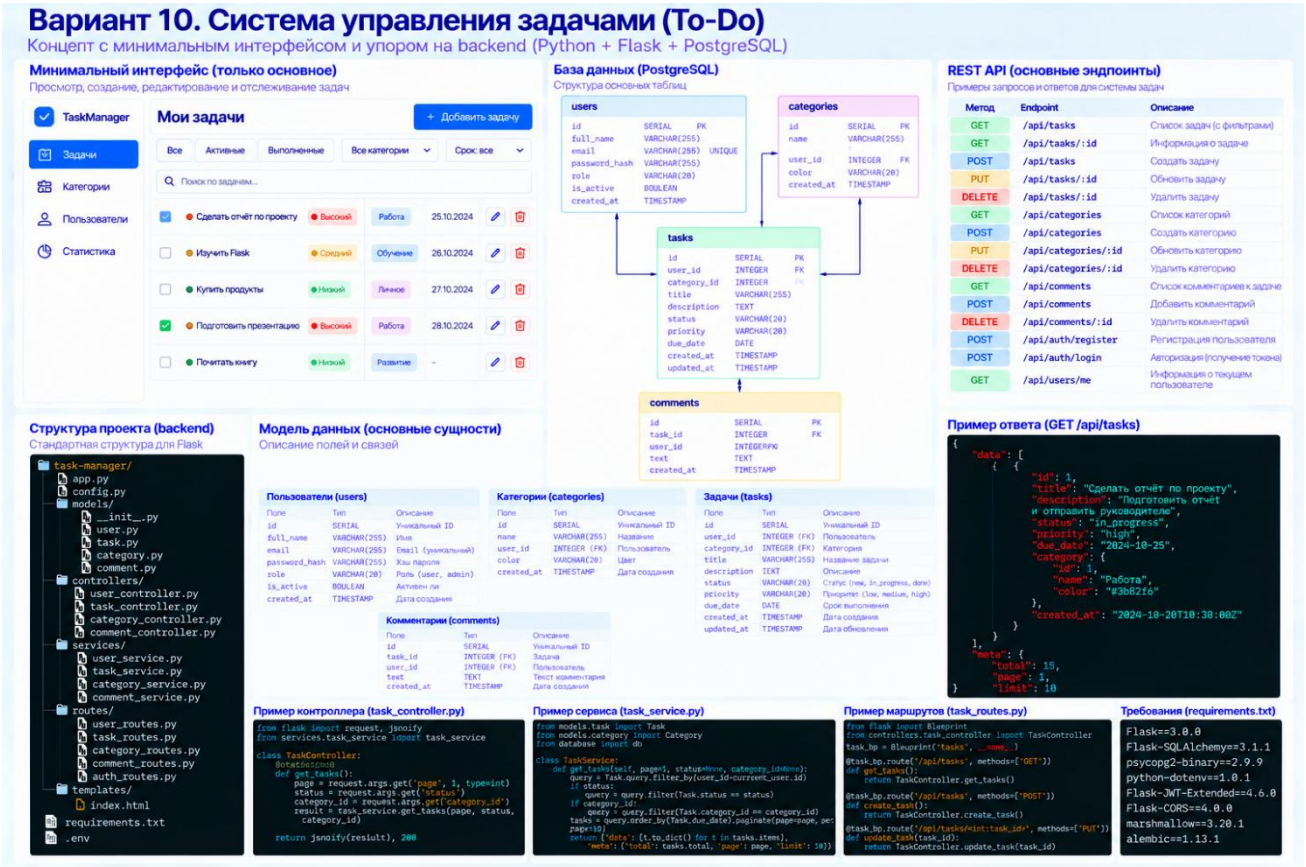
- расходы;
- категории;
- пользователи.

Необходимо реализовать:

- добавление расхода;
- получение расходов;
- изменение;
- удаление;
- фильтрацию по категории;
- получение расходов за период;
- расчет общей суммы;
- расчет расходов по категориям.

Вариант 10. Конструктор пиццы

Разработать Flask API сервиса заказа пиццы.



Сущности:

- пиццы;
- размеры;
- ингредиенты;
- заказы.

Необходимо реализовать:

- получение размеров;
- получение ингредиентов;
- создание заказа;
- добавление ингредиентов;
- изменение заказа;
- удаление заказа;
- расчет стоимости.

Стоимость заказа должна рассчитываться сервером.

Вариант 11. Диспетчер задач проекта

Разработать backend для управления проектами и задачами.

Сущности:

- проекты;
- пользователи;
- задачи;
- статусы.

Необходимо реализовать:

- создание проекта;
- получение проектов;
- создание задачи;
- изменение задачи;
- удаление задачи;
- изменение статуса;
- получение задач проекта;
- получение задач пользователя.

Дополнительно можно реализовать распределение задач между пользователями.

Вариант 12. Бронирование переговорных комнат

Разработать Flask API для бронирования переговорных.

Сущности:

- комнаты;
- пользователи;
- бронирования.

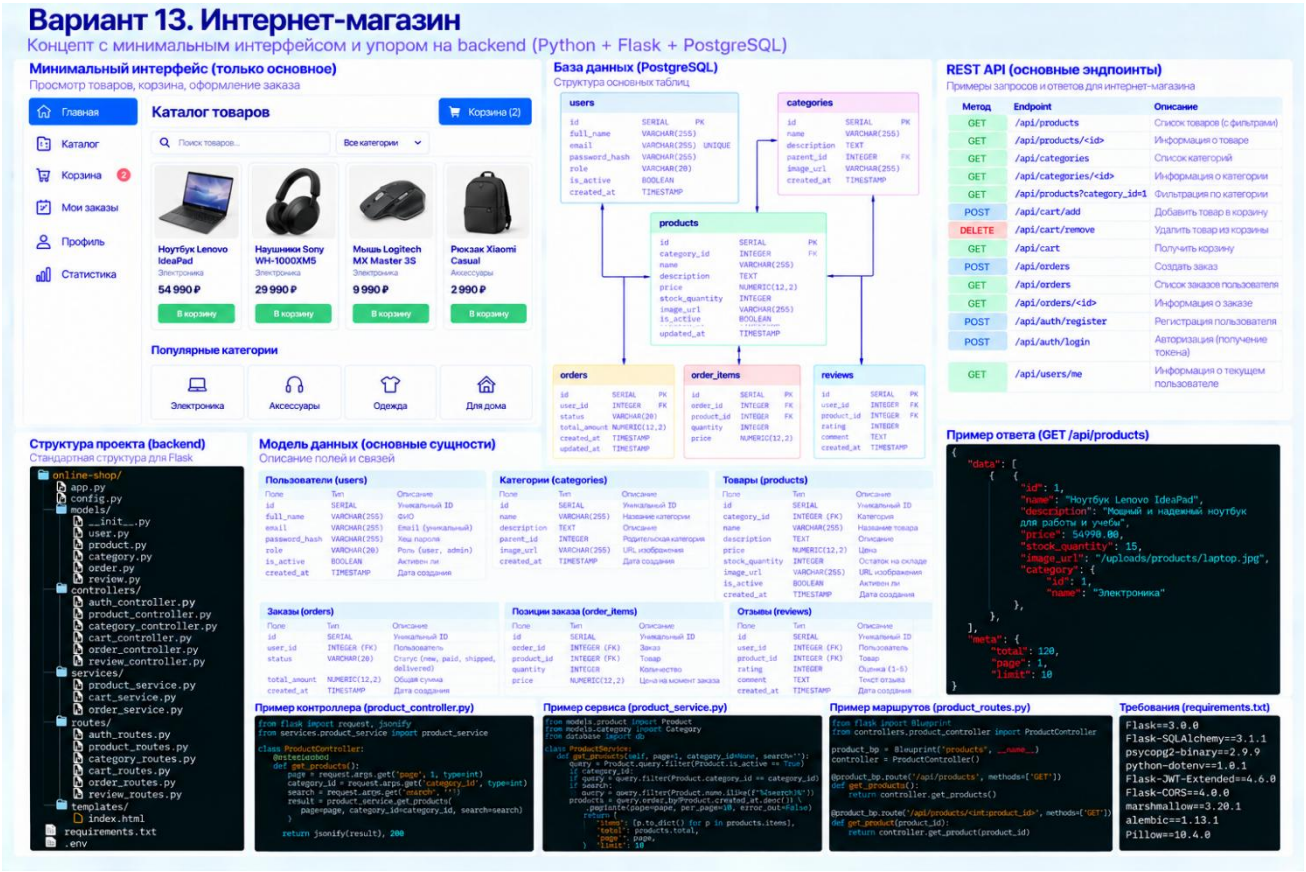
Необходимо реализовать:

- получение комнат;
- получение доступных комнат;
- создание бронирования;
- изменение бронирования;
- отмену бронирования;
- получение бронирований пользователя;
- получение бронирований комнаты.

Необходимо предусмотреть проверку пересечения бронирований.

Вариант 13. Панель мониторинга погоды

Разработать backend для хранения погодных данных.



Сущности:

- города;
- погодные показатели;
- прогнозы.

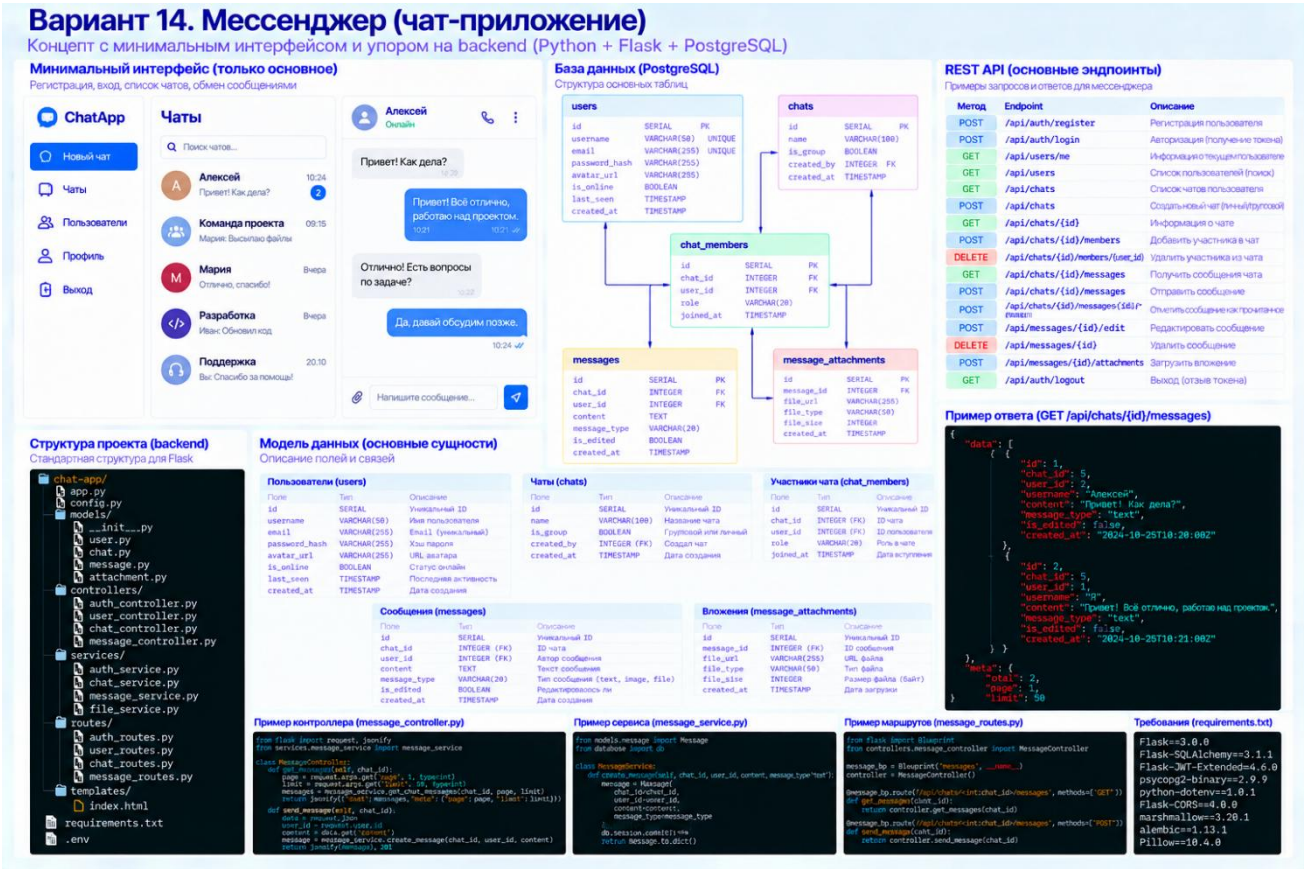
Необходимо реализовать:

- добавление города;
- получение городов;
- получение текущих данных;
- сохранение погодных показателей;
- получение истории;
- получение прогноза;
- удаление города.

Для практической работы допускается использовать тестовые данные без подключения внешнего API.

Вариант 14. Интерактивная викторина

Разработать Flask API для системы викторин.



Сущности:

- викторины;
- вопросы;
- варианты ответов;
- результаты.

Необходимо реализовать:

- создание викторины;
- получение списка викторин;
- получение вопросов;
- добавление вопросов;
- добавление вариантов ответов;
- удаление вопросов;
- сохранение результатов;
- получение результатов пользователя.

Вариант 15. Мини-магазин с корзиной

Разработать backend небольшого интернет-магазина.

Сущности:

- товары;
- категории;
- пользователи;
- корзины;
- заказы;
- позиции заказа.

Необходимо реализовать:

- получение каталога;
- получение товара;
- поиск;
- фильтрацию;
- добавление товара;
- изменение товара;
- удаление товара;
- работу с корзиной;

- создание заказа;
- получение истории заказов.

Стоимость заказа должна рассчитываться на стороне сервера.

Контрольные вопросы

1. Что такое Python Flask?
2. Почему Flask называют микрофреймворком?
3. Что такое backend?
4. Что такое маршрут Flask-приложения?
5. Что такое REST API?
6. Что такое endpoint?
7. Для чего используются GET, POST, PUT и DELETE?
8. Что такое CRUD?
9. Как Flask обрабатывает HTTP-запрос?
10. Как вернуть JSON из Flask?
11. Что такое PostgreSQL?
12. Что такое SQL?
13. Что такое первичный ключ?
14. Что такое внешний ключ?
15. Какие связи между таблицами существуют?
16. Как Flask взаимодействует с PostgreSQL?
17. Для чего используются переменные окружения?
18. Зачем нужен файл requirements.txt?
19. Для чего нужна валидация данных?
20. Для чего нужна обработка исключений?
21. Что такое HTTP-код 200?
22. Что означает код 201?
23. Что означает код 400?
24. Что означает код 404?
25. Что означает код 500?
26. Как передать данные из React в Flask?

27. Как Flask возвращает данные в React?
28. В чем отличие backend на Node.js от backend на Flask?
29. Какие преимущества дает использование PostgreSQL?
30. Почему нельзя хранить пароль базы данных в исходном коде?

ПРАКТИЧЕСКАЯ РАБОТА 13. РАЗРАБОТКА ПРОГРЕССИВНОГО WEB-ПРИЛОЖЕНИЯ (PWA)

Цель работы

- понятие и архитектуру PWA;
- требования к Progressive Web Applications;
- создание Web App Manifest;
- настройку Service Worker;
- кэширование ресурсов;
- работу приложения в условиях отсутствия или нестабильного интернет-соединения;
- создание адаптивного мобильного интерфейса;
- установку web-приложения на смартфон или планшет;
- работу с иконками и экраном запуска;
- использование различных стратегий кэширования;
- обновление содержимого PWA;
- тестирование PWA на мобильных устройствах;
- публикацию приложения по HTTPS.

Задания к практической работе

Необходимо разработать Progressive Web Application, предназначенное для работы на компьютере и мобильных устройствах. Приложение должно представлять собой полноценный интерактивный web-интерфейс выбранной предметной области и обладать основными свойствами PWA.

В зависимости от выбранного варианта необходимо:

- разработать интерфейс приложения;
- обеспечить адаптивность интерфейса для мобильных устройств;
- создать файл manifest.json;

- определить название приложения;
- настроить иконки приложения;
- реализовать Service Worker;
- организовать кэширование необходимых ресурсов;
- обеспечить возможность запуска приложения без повторной загрузки всех ресурсов;
- предусмотреть корректное поведение при отсутствии интернет-соединения;
- протестировать приложение в браузере;
- установить приложение на мобильное устройство;
- протестировать основные функции на смартфоне или планшете;
- опубликовать приложение по HTTPS.

Структура PWA-проекта

Рекомендуемая структура:

```
project/
│
├── src/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── App.jsx
│   └── main.jsx
│
├── public/
│   ├── icons/
│   │   ├── icon-192.png
│   │   └── icon-512.png
│   │
│   └── manifest.json
│
├── service-worker.js
├── index.html
├── package.json
└── README.md
```

Необходимо создать файл manifest.json. В нем должны быть указаны основные параметры приложения:

```
{
  "name": "Мой список покупок",
  "short_name": "Покупки",
  "start_url": "/",
  "display": "standalone",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
```

```
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Необходимо самостоятельно подобрать:

- название;
- короткое название;
- иконку;
- стартовую страницу;
- режим отображения приложения.

В приложении необходимо реализовать Service Worker. Он должен использоваться для:

- перехвата запросов;
- кэширования статических ресурсов;
- повторного использования ресурсов из кэша;
- обеспечения базовой работы приложения при отсутствии сети;
- обновления кэшированных ресурсов.

Необходимо продемонстрировать, что приложение действительно использует Service Worker.

Адаптивный интерфейс

Приложение должно корректно отображаться как минимум на:

- смартфоне;
- планшете;
- настольном компьютере.
- Необходимо использовать:
- адаптивную верстку;
- media queries;
- flexbox/grid;
- удобные размеры элементов управления;
- сенсорные элементы управления.

Особое внимание необходимо уделить кнопкам, полям ввода и навигации.

Установка на мобильное устройство

Одной из обязательных частей работы является реальная установка PWA на смартфон или планшет. Необходимо:

- открыть опубликованное приложение на мобильном устройстве;
- проверить возможность установки;
- установить приложение;
- запустить его с главного экрана;
- проверить отображение в режиме приложения;
- проверить работу основных функций;
- проверить работу после отключения сети.

В отчете необходимо привести скриншоты приложения на мобильном устройстве. Рекомендуется проверить работу как минимум на одном смартфоне.

Требования к результату

В результате выполнения практической работы необходимо получить PWA, которое:

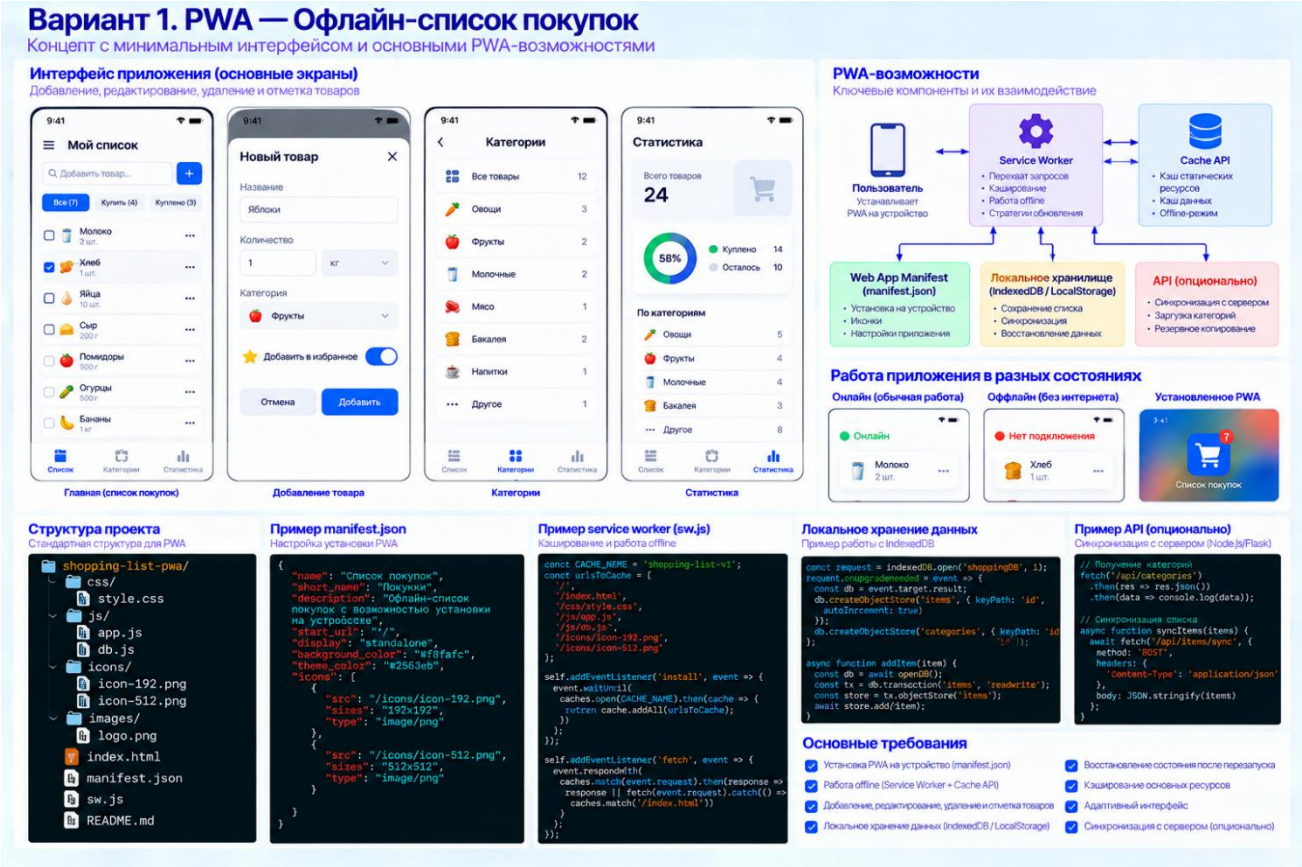
- имеет адаптивный интерфейс;
- содержит manifest.json;
- содержит и регистрирует Service Worker;
- использует кэширование;
- сохраняет необходимые данные;
- имеет минимум два размера иконок;
- может быть установлено на мобильное устройство;
- запускается как отдельное приложение;
- имеет базовую offline-функциональность;
- протестировано на смартфоне;
- опубликовано по HTTPS;
- имеет GitHub-репозиторий;
- содержит README.

Минимальные обязательные требования для всех вариантов:

- адаптивный интерфейс;
- Web App Manifest;
- Service Worker;
- кэширование ресурсов;
- offline-режим;
- локальное сохранение состояния/данных;
- установка на мобильное устройство;
- тестирование на смартфоне;
- публикация по HTTPS.

Вариант 1. Мобильный список покупок

Разработать PWA для ведения списка покупок.



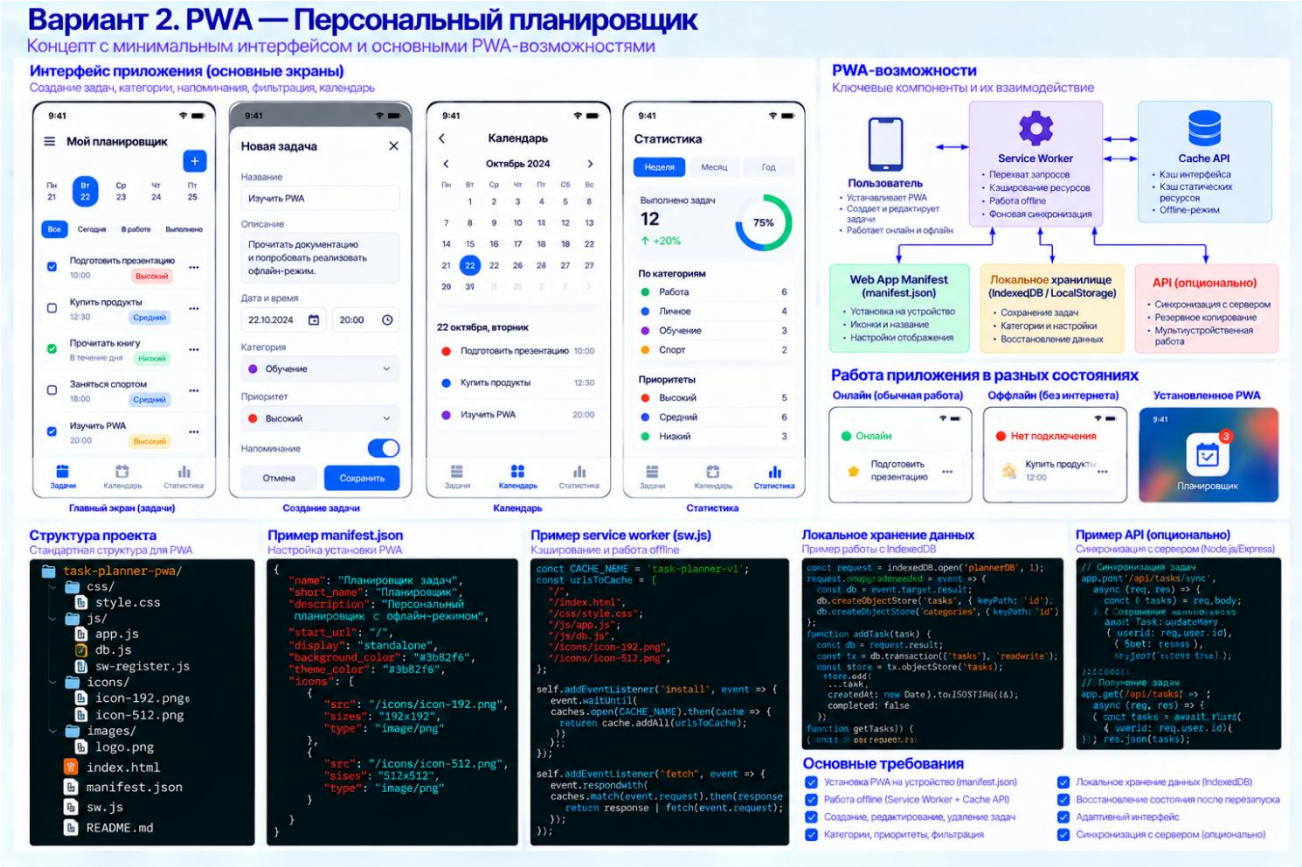
Необходимо реализовать:

- добавление товара;
- удаление товара;
- отметку купленных товаров;
- редактирование товара;
- фильтрацию;
- отображение количества оставшихся товаров;
- сохранение списка;
- работу приложения при временном отсутствии сети.

Дополнительно можно добавить категории товаров.

Вариант 2. Планировщик учебного дня

Создать мобильное PWA для планирования учебного дня.



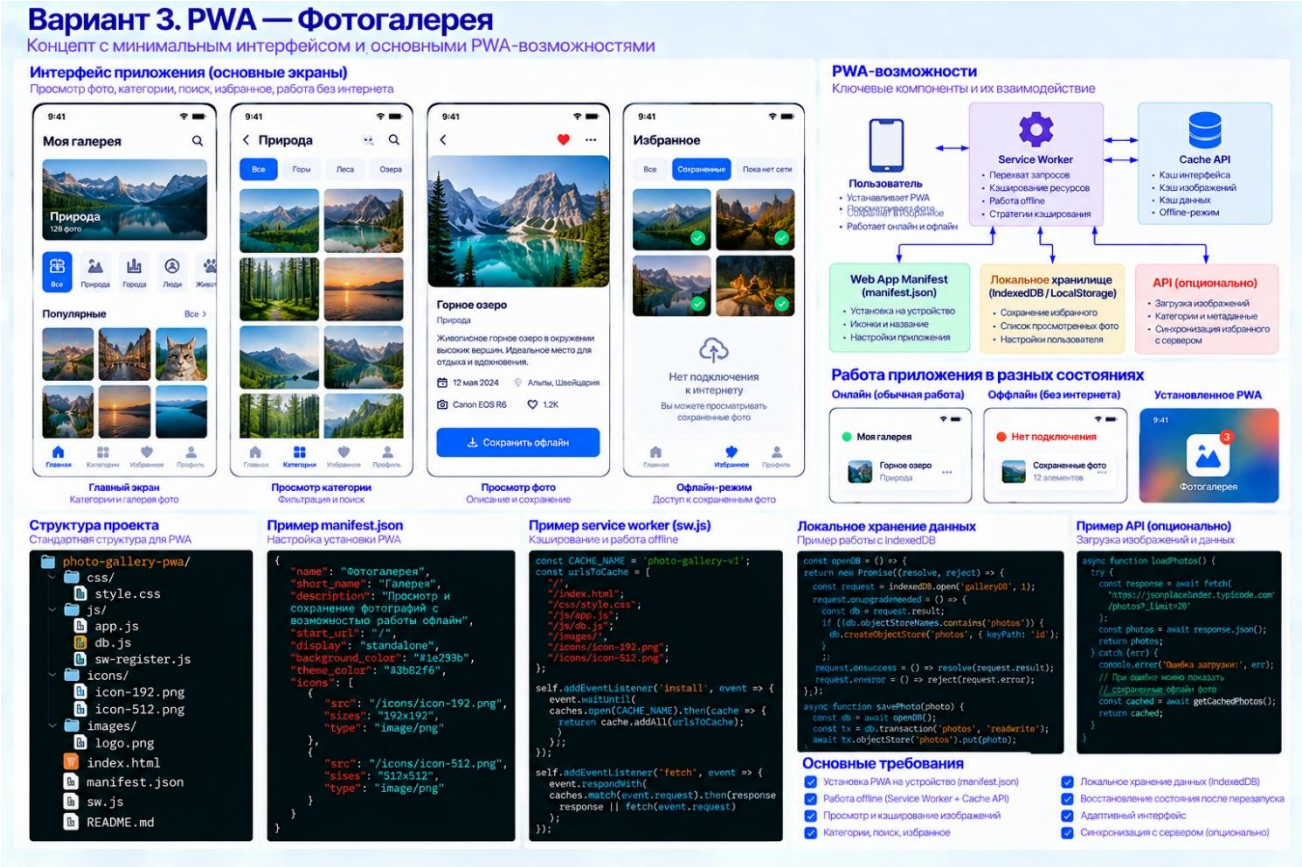
Необходимо реализовать:

- список занятий;
- добавление занятия;
- редактирование;
- удаление;
- отображение времени и аудитории;
- переключение между днями;
- отметку выполненных занятий.

Интерфейс должен быть удобен для использования одной рукой на смартфоне.

Вариант 3. Галерея путешествий

Разработать PWA для просмотра коллекции фотографий путешествий.



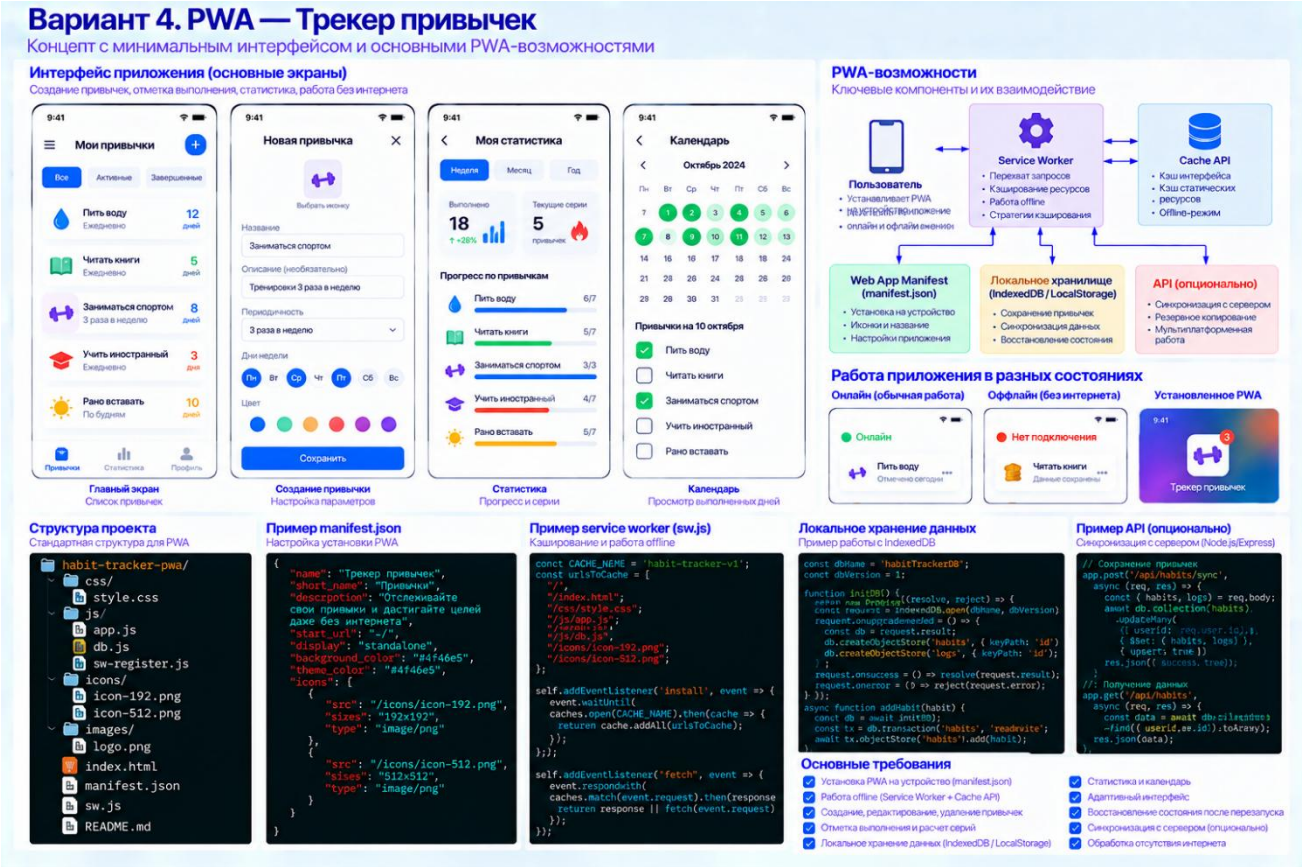
Необходимо реализовать:

- список фотографий;
- просмотр фотографии;
- категории или места;
- избранные фотографии;
- поиск;
- адаптивную галерею.

После первого посещения основные элементы интерфейса и доступные изображения должны загружаться из кэша.

Вариант 4. Трекер привычек

Создать мобильное приложение для контроля привычек.



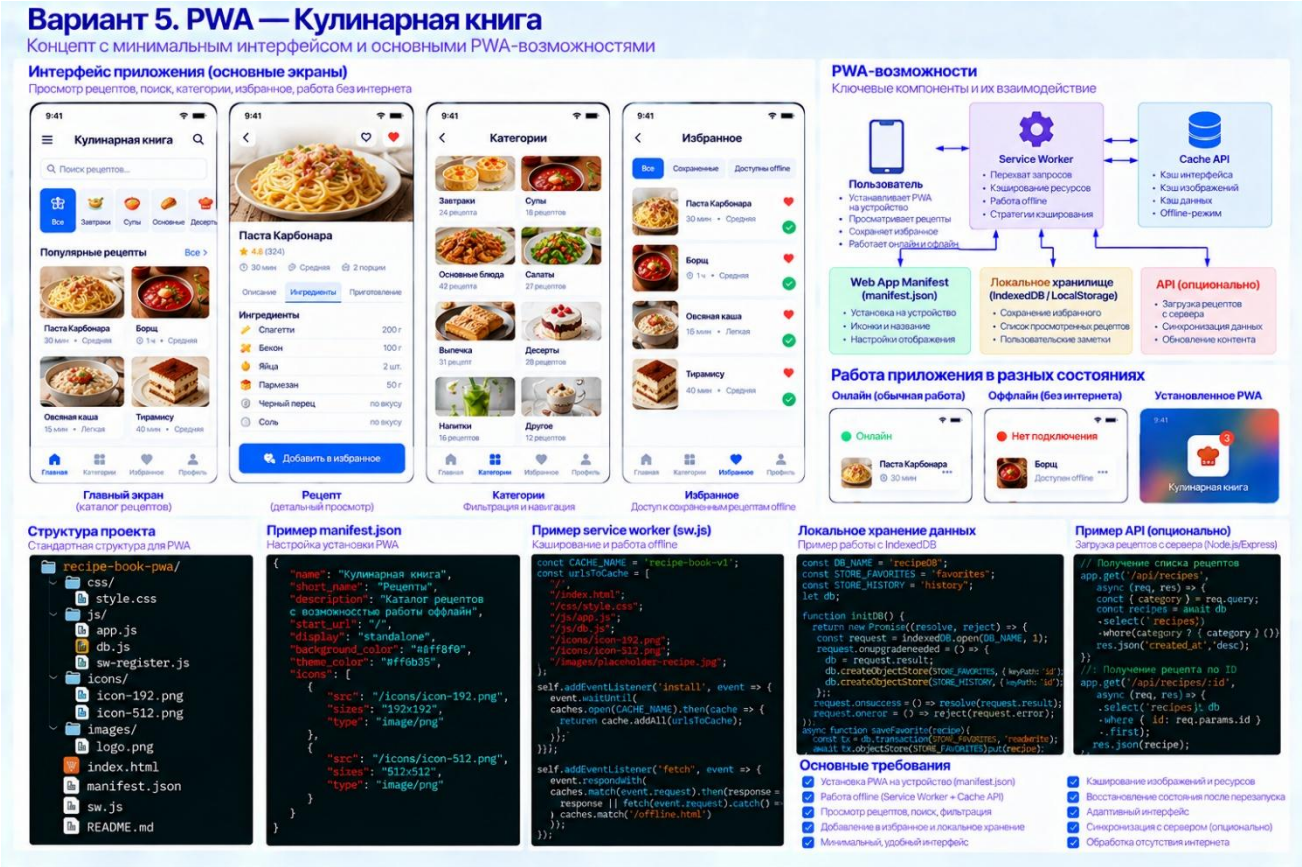
Необходимо реализовать:

- добавление привычки;
- отметку выполнения;
- календарь или историю;
- расчет прогресса;
- статистику;
- удаление и редактирование привычек.

PWA должно сохранять данные локально и корректно работать при отсутствии сети.

Вариант 5. Мобильное меню ресторана

Разработать PWA электронного меню ресторана.



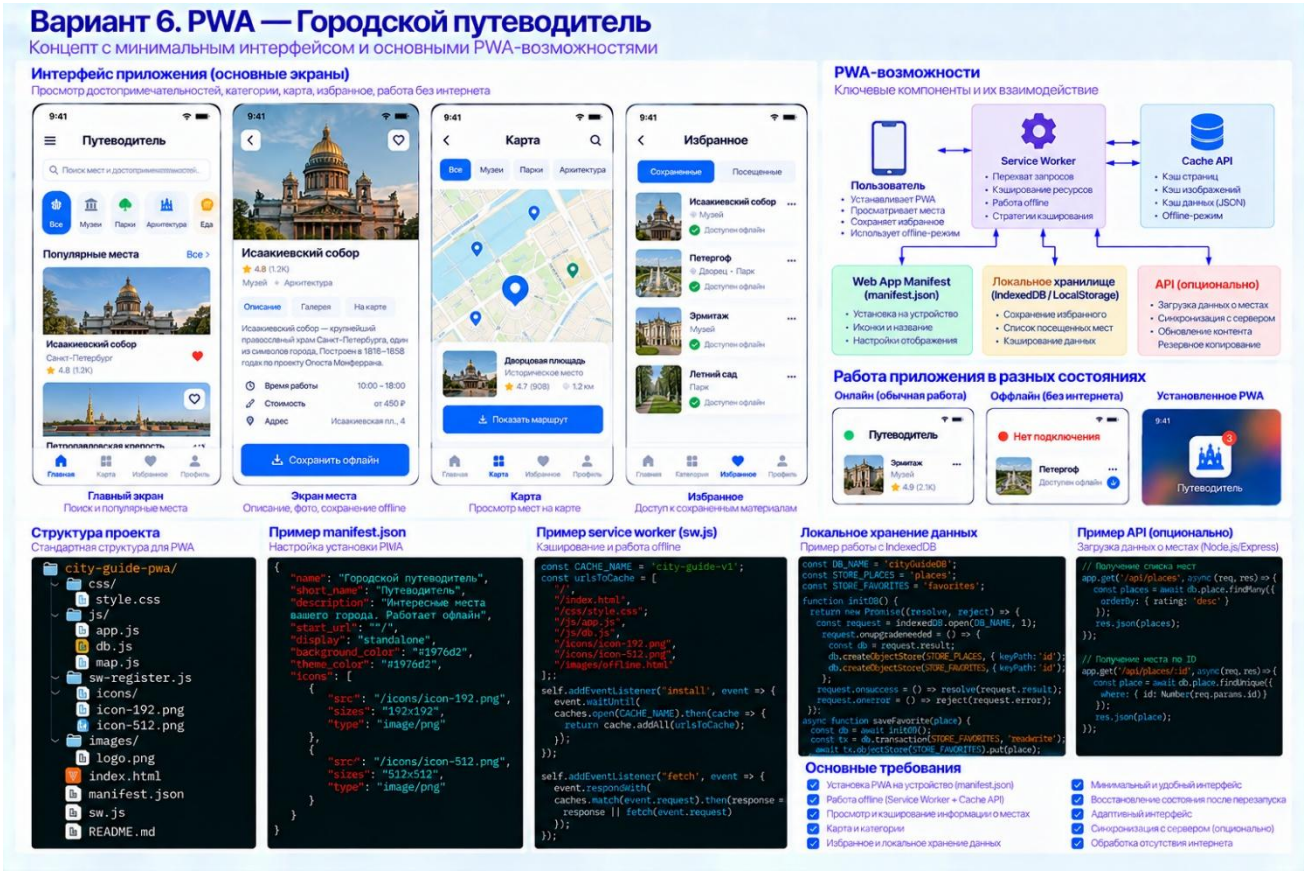
Необходимо реализовать:

- категории блюд;
- карточки блюд;
- поиск;
- фильтрацию;
- просмотр подробной информации;
- добавление блюда в заказ;
- отображение текущего заказа.

Меню должно оставаться доступным при временном отсутствии интернет-соединения.

Вариант 6. Мобильное расписание занятий

Создать PWA для просмотра университетского расписания.



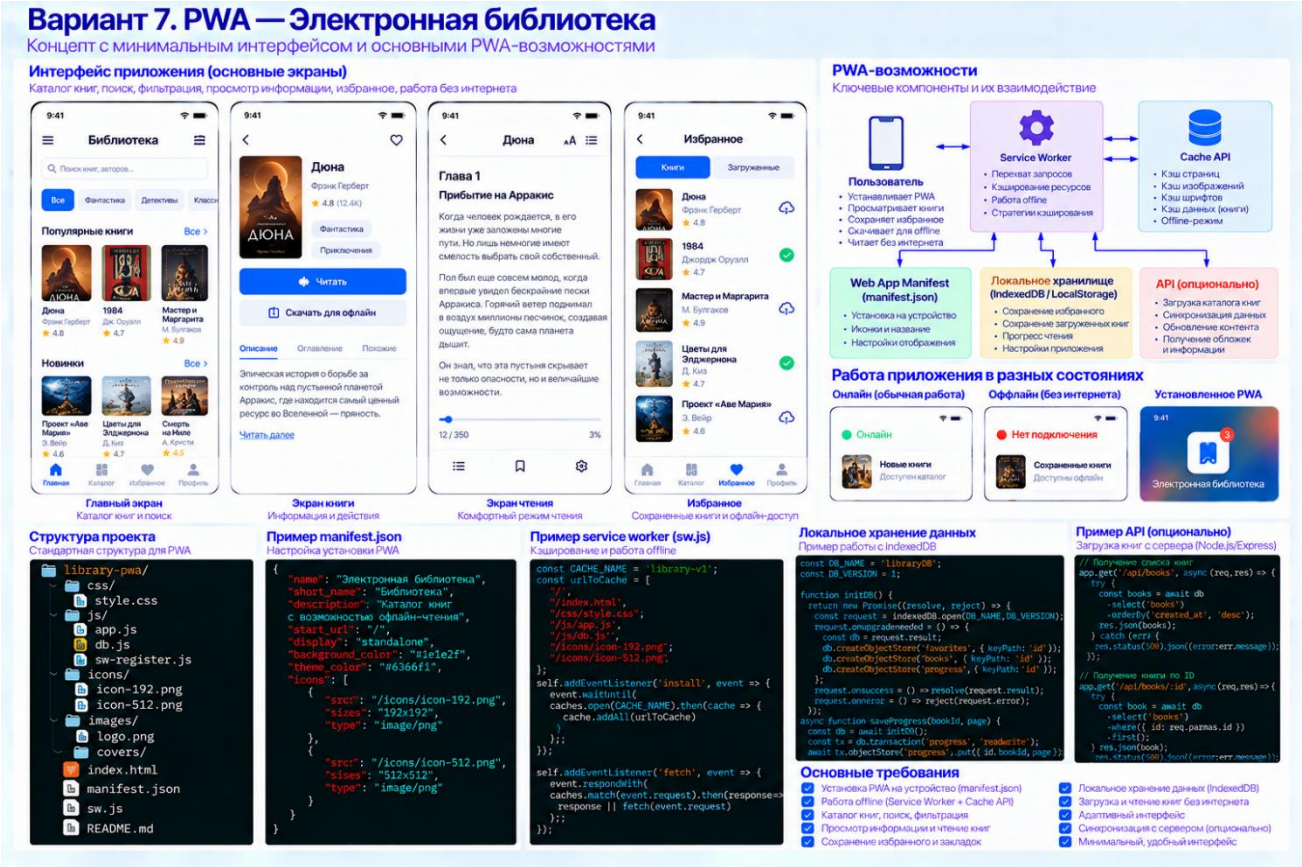
Необходимо реализовать:

- выбор дня;
- список занятий;
- информацию о преподавателе;
- аудиторию;
- время;
- выделение текущего занятия;
- просмотр расписания на неделю.

Особое внимание необходимо уделить компактному отображению информации на экране смартфона.

Вариант 7. Каталог компьютерной техники

Разработать PWA-каталог компьютерной техники.



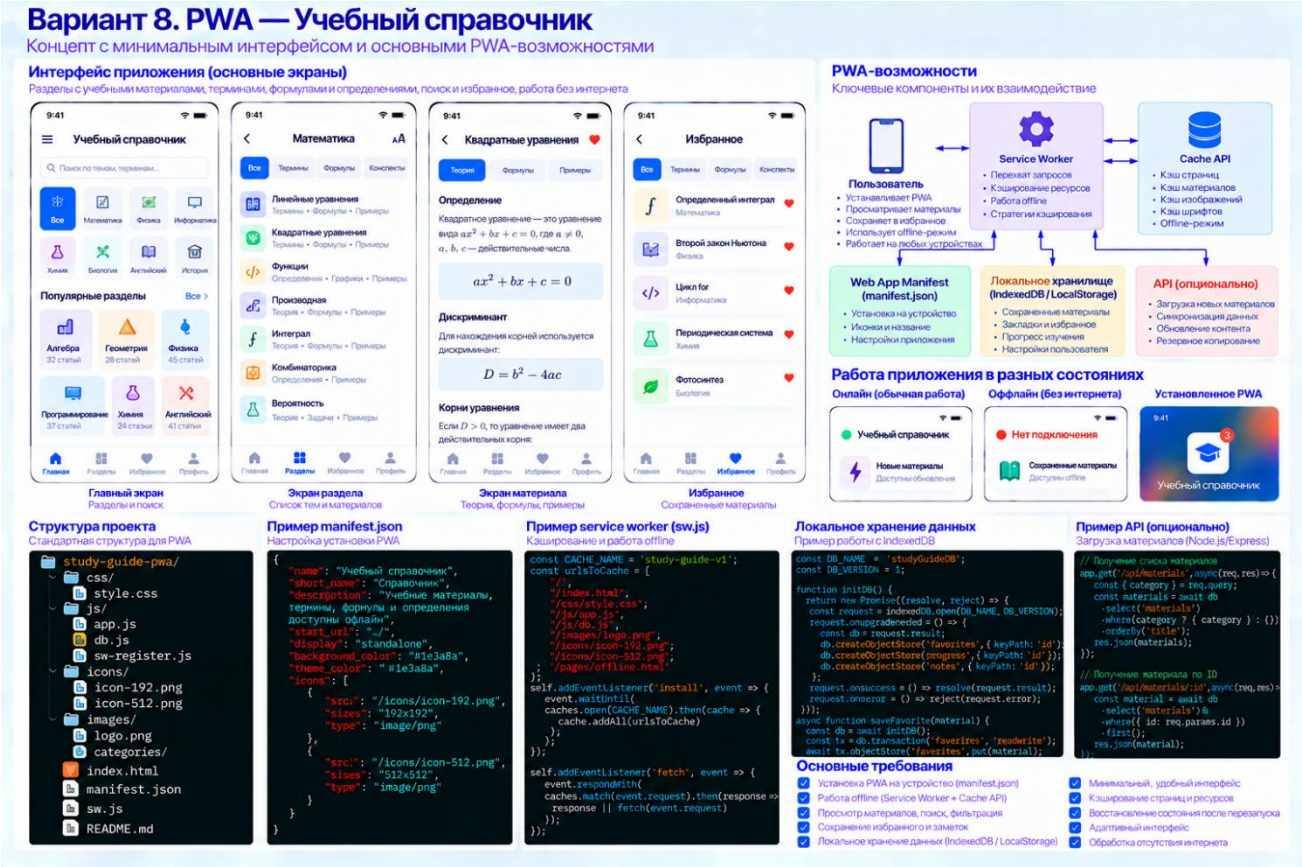
Необходимо реализовать:

- каталог товаров;
- поиск;
- фильтрацию;
- категории;
- карточку товара;
- избранные товары;
- сравнение выбранных товаров.

Каталог должен оставаться доступным при повторном запуске приложения без необходимости повторной загрузки всех статических ресурсов.

Вариант 8. Электронная библиотека

Создать PWA для работы с электронной библиотекой.



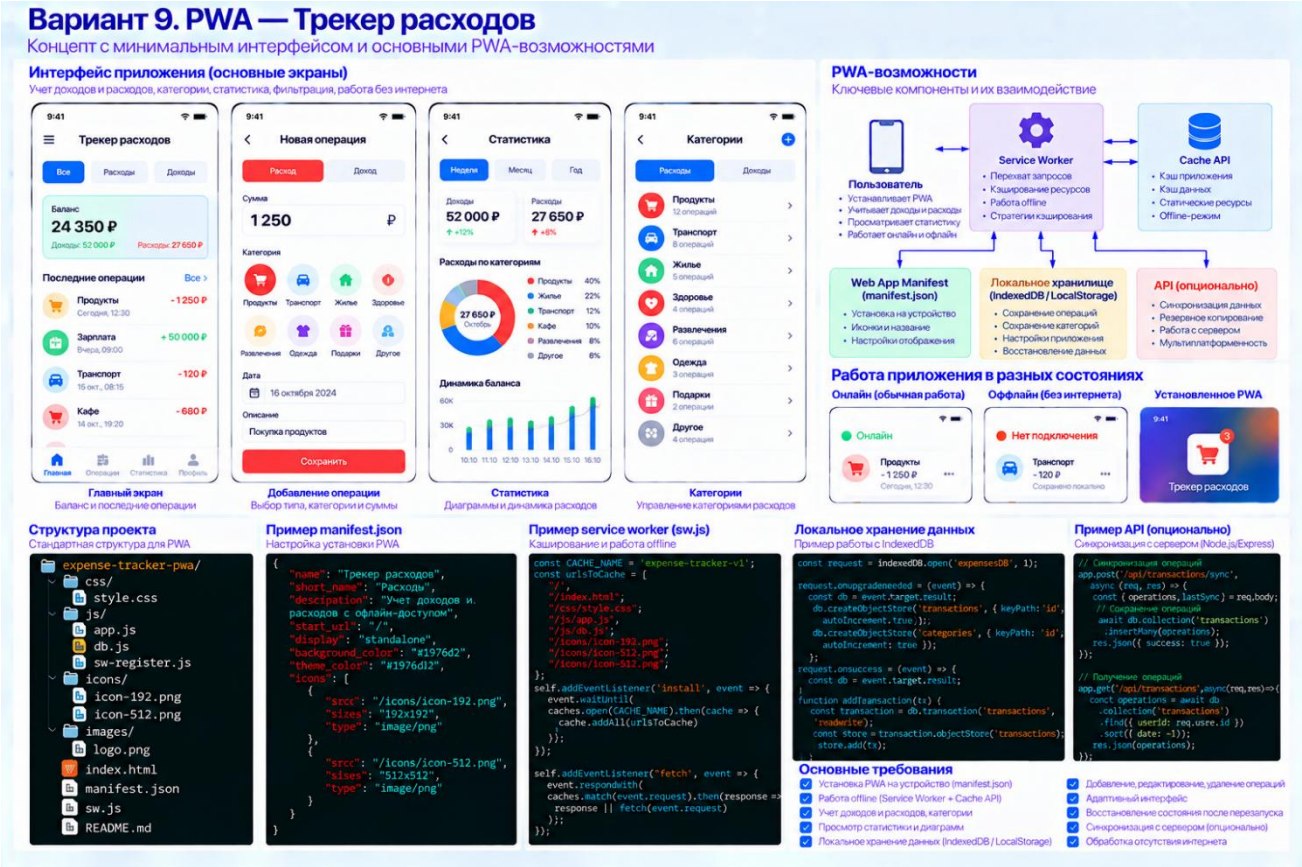
Необходимо реализовать:

- список книг;
- поиск;
- фильтрацию;
- категории;
- информацию об авторе;
- избранные книги;
- список прочитанных книг.

Необходимо предусмотреть возможность просмотра ранее загруженных данных без подключения к интернету.

Вариант 9. Трекер расходов

Разработать мобильное PWA для учета расходов.



Необходимо реализовать:

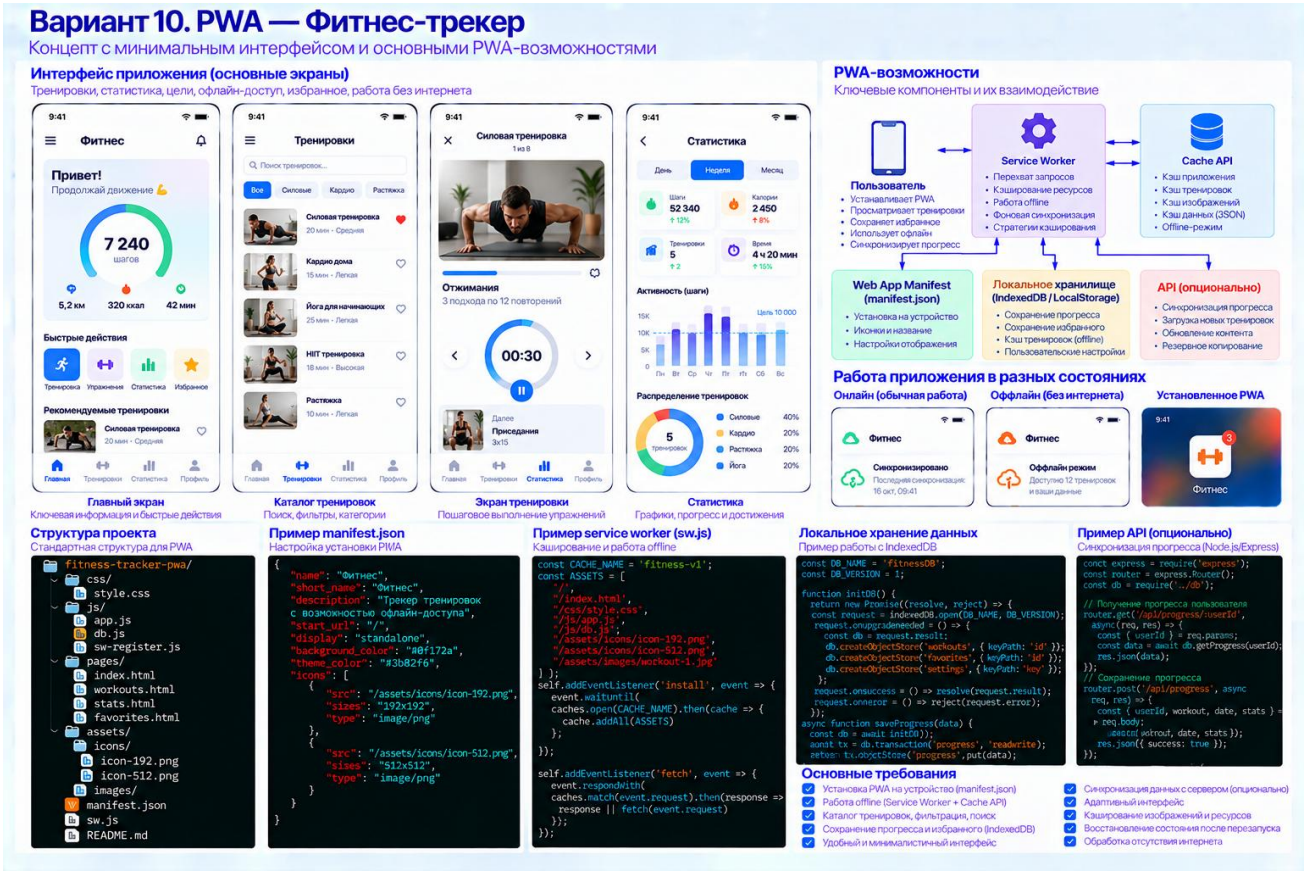
- добавление расхода;
- категорию;
- сумму;
- дату;
- список операций;
- фильтрацию;
- расчет общей суммы;
- статистику расходов.

Данные пользователя должны сохраняться между запусками приложения.

Дополнительно можно реализовать графическое отображение расходов.

Вариант 10. Конструктор пиццы

Создать PWA для формирования заказа пиццы.



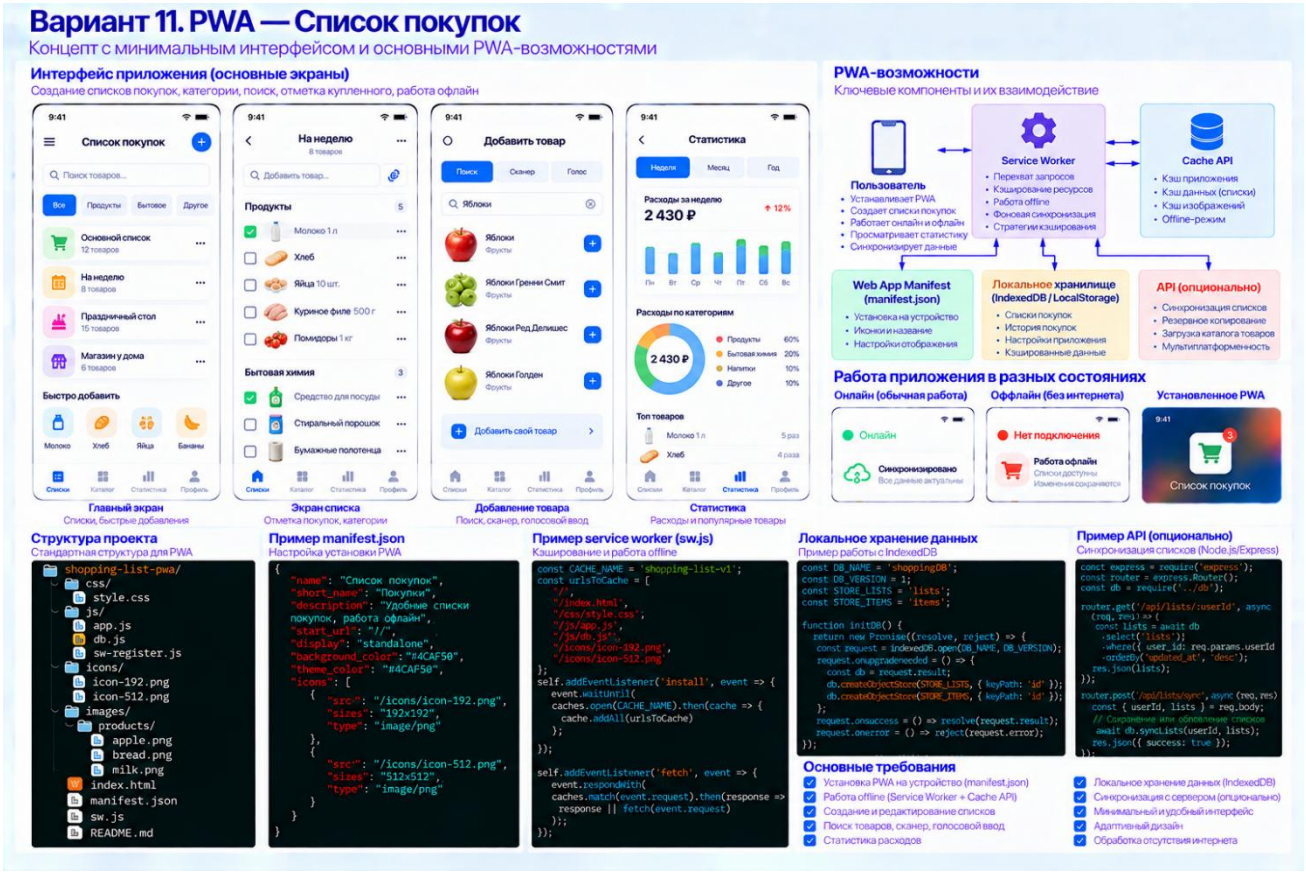
Необходимо реализовать:

- выбор размера;
- выбор теста;
- выбор ингредиентов;
- отображение выбранных ингредиентов;
- расчет стоимости;
- формирование заказа;
- отображение итоговой суммы.

Интерфейс должен быть рассчитан прежде всего на использование с сенсорного экрана.

Вариант 11. Диспетчер задач проекта

Разработать мобильное PWA для управления задачами проекта.



Необходимо реализовать:

- создание задачи;
- редактирование;
- удаление;
- изменение статуса;
- приоритет;
- срок выполнения;
- фильтрацию;
- поиск.

Можно использовать мобильную Kanban-доску:

Новые → В работе → Выполненные

Приложение должно сохранять состояние между запусками.

Вариант 12. Бронирование переговорных

Создать PWA для бронирования переговорных комнат.

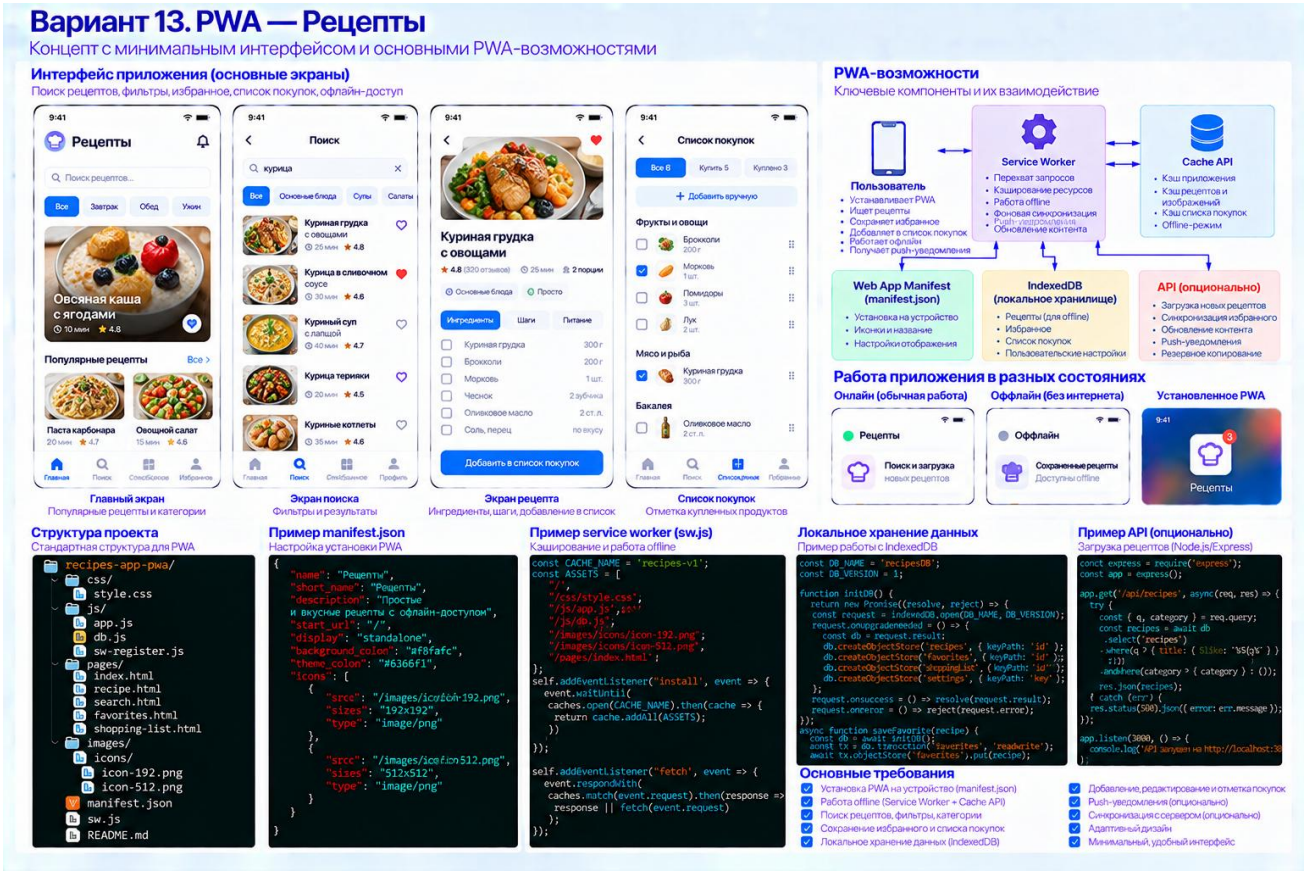
Необходимо реализовать:

- список комнат;
- дату;
- время;
- создание бронирования;
- отмену;
- просмотр текущих бронирований;
- отображение свободных временных интервалов.

Интерфейс должен быть оптимизирован для мобильного устройства.

Вариант 13. Панель мониторинга погоды

Разработать PWA для просмотра погодной информации.



Необходимо реализовать:

выбор города;

текущую температуру;

состояние погоды;

прогноз;

избранные города;

обновление данных;

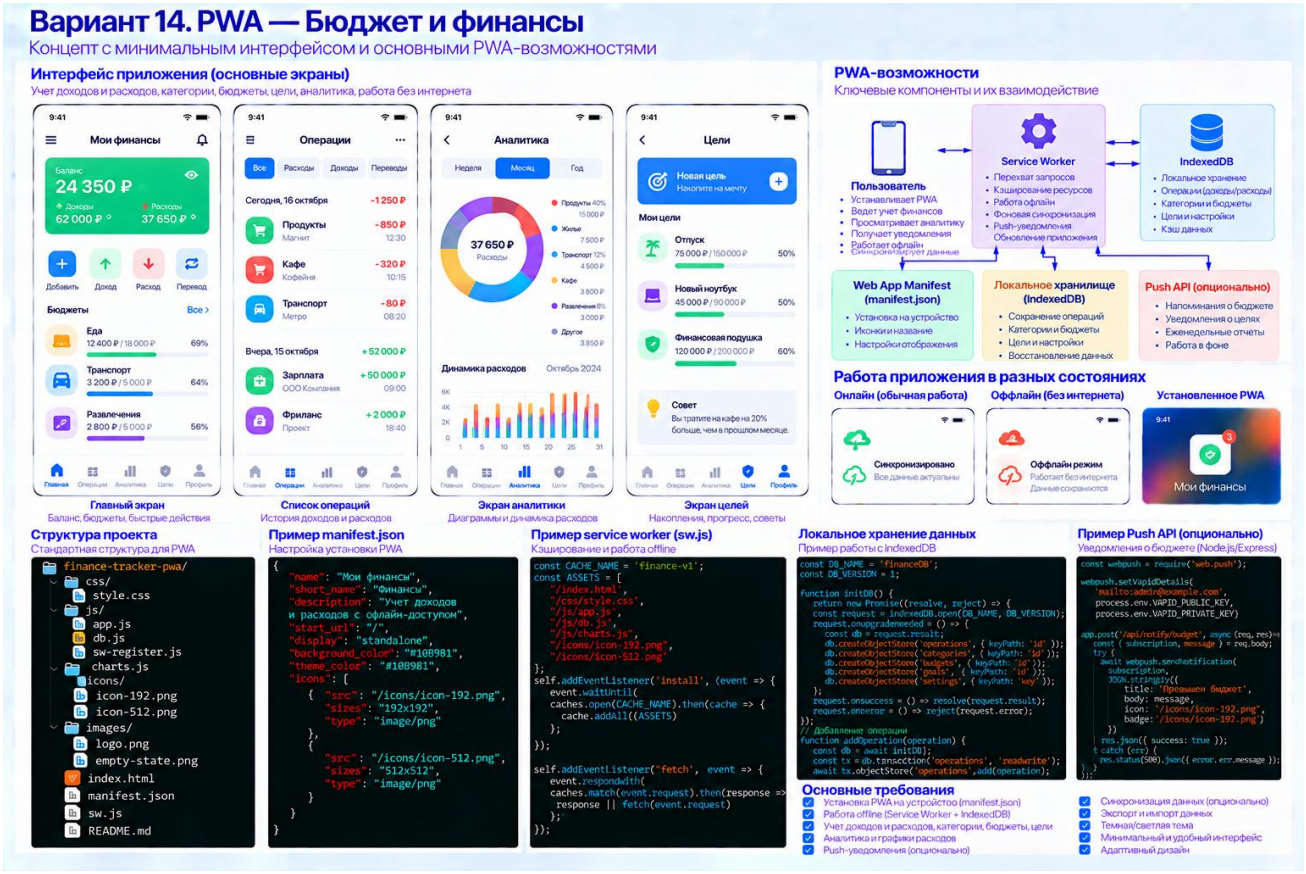
отображение времени последнего обновления.

При отсутствии подключения необходимо отображать последние доступные данные.

Допускается использовать тестовые данные либо внешний API.

Вариант 14. Интерактивная викторина

Создать PWA для прохождения викторин.



Необходимо реализовать:

список вопросов;

варианты ответов;

переход между вопросами;

отображение прогресса;

подсчет результата;

итоговый экран;

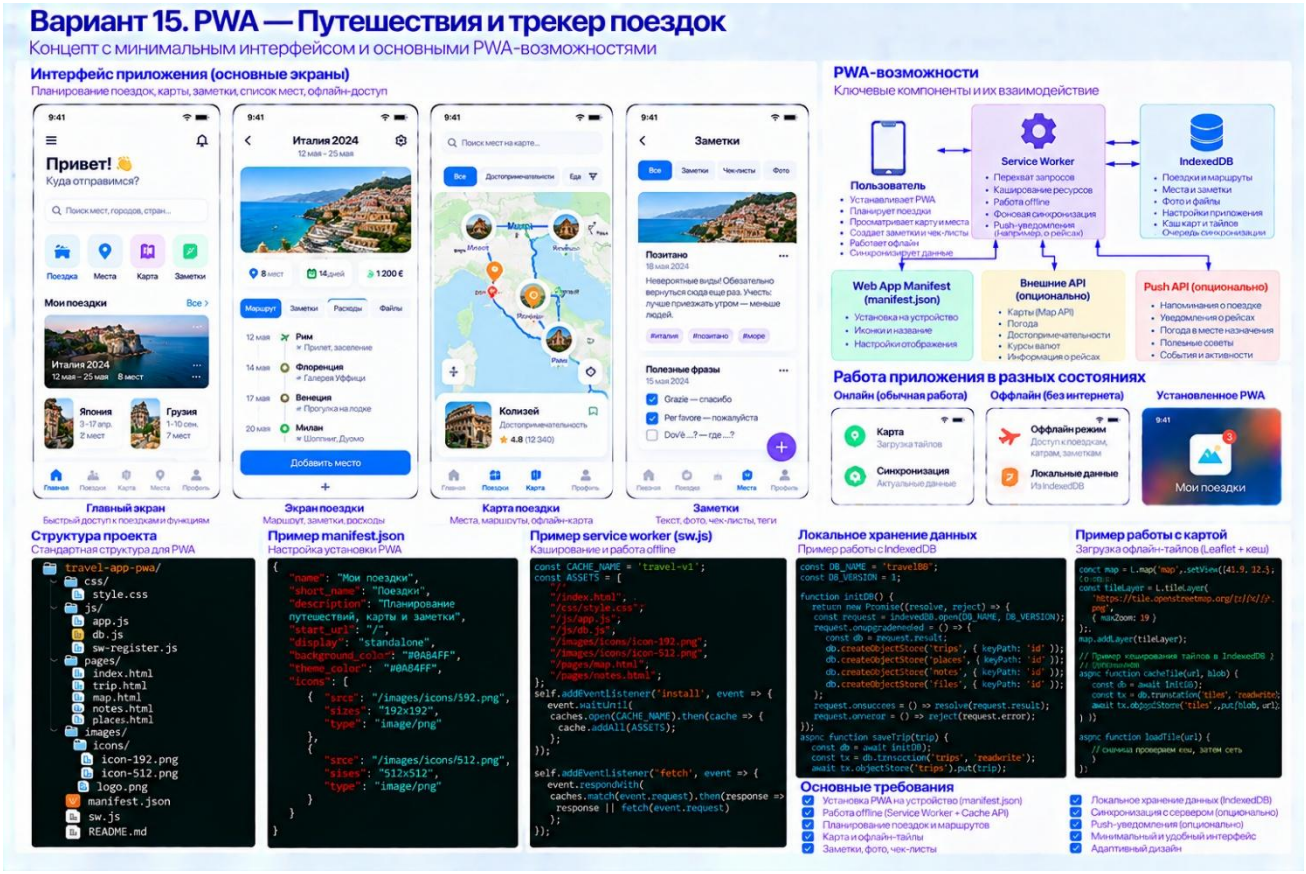
возможность начать викторину заново;

сохранение последнего результата.

Приложение должно корректно работать на смартфоне и планшете.

Вариант 15. Мини-магазин с корзиной

Разработать полноценное мобильное PWA небольшого интернет-магазина.



Необходимо реализовать:

каталог;

категории;

поиск;

фильтрацию;

карточку товара;

избранные товары;

корзину;

изменение количества;

расчет стоимости;

оформление заказа.

Необходимо обеспечить удобную навигацию на мобильном устройстве и сохранение состояния корзины между запусками приложения.

Контрольные вопросы

1. Что такое PWA?
2. Чем PWA отличается от обычного web-сайта?
3. Какие основные характеристики имеет PWA?

4. Что такое Web App Manifest?
5. Для чего нужен manifest.json?
6. Что такое Service Worker?
7. Где выполняется Service Worker?
8. Какие задачи может выполнять Service Worker?
9. Что такое Cache API?
10. Зачем необходимо кэширование?
11. Что такое offline-режим?
12. Что такое стратегия Cache First?
13. Что такое Network First?
14. Что такое Stale While Revalidate?
15. Для чего нужны иконки PWA?
16. Что означает режим standalone?
17. Что такое адаптивный web-дизайн?
18. Почему для PWA необходимо HTTPS?
19. Как установить PWA на смартфон?
20. Чем установленное PWA отличается от страницы, открытой в браузере?
21. Как проверить работу Service Worker?
22. Как проверить работу приложения без интернета?
23. Для чего необходимо локальное хранение данных?
24. Как PWA может взаимодействовать с REST API?
25. Какие преимущества PWA имеют перед обычными web-приложениями?
26. Какие ограничения существуют у PWA?
27. Как обновляются кэшированные ресурсы?
28. Что произойдет, если Service Worker содержит ошибку?
29. Как PWA может использовать backend из предыдущих практических работ?
30. Какие особенности необходимо учитывать при разработке интерфейса для смартфона?

ПРАКТИЧЕСКАЯ РАБОТА 14. РАЗРАБОТКА ПОЛНОСТЕКОВОГО WEB-ПРИЛОЖЕНИЯ (FULLSTACK)

Цель работы

- закрепить и систематизировать знания и практические навыки, полученные при изучении технологий разработки современных веб-приложений;
- освоить основные этапы проектирования и разработки полнофункционального веб-приложения на основе архитектуры Fullstack;
- научиться проектировать структуру приложения с разделением на клиентскую (Frontend) и серверную (Backend) части;
- закрепить навыки создания адаптивных и интерактивных пользовательских интерфейсов с использованием HTML, CSS, JavaScript, Bootstrap и React;
- освоить организацию взаимодействия клиентской и серверной частей приложения посредством REST API;
- закрепить навыки разработки серверной логики с использованием Node.js или Python Flask;
- научиться проектировать структуру реляционной базы данных PostgreSQL и организовывать взаимодействие приложения с базой данных;
- освоить реализацию основных операций обработки данных, включая создание, получение, изменение и удаление записей (CRUD);
- сформировать навыки интеграции отдельных программных компонентов в единую информационную систему;
- научиться организовывать обработку пользовательских действий, проверку вводимых данных и отображение результатов выполнения операций;
- закрепить навыки тестирования, отладки и проверки корректности работы веб-приложения;
- освоить основные подходы к обеспечению безопасности, надёжности и удобства использования веб-приложений;
- сформировать практические навыки самостоятельного выполнения комплексного проекта, включающего проектирование, программную реализацию, тестирование и подготовку к публикации.

Теоретическая часть

Современные веб-приложения представляют собой сложные программные системы, объединяющие пользовательский интерфейс, серверную логику, механизмы обработки данных и средства взаимодействия с внешними сервисами. В отличие от статических веб-сайтов, такие приложения обеспечивают интерактивное взаимодействие с пользователем, динамическое обновление информации, выполнение прикладных операций и централизованное хранение данных.

Одним из распространённых подходов к созданию подобных систем является **Fullstack-разработка**, предполагающая комплексную реализацию клиентской и серверной частей веб-приложения, а также организацию их взаимодействия с базой данных.

Fullstack-разработка охватывает несколько взаимосвязанных направлений:

- **Frontend** — разработка пользовательского интерфейса, обеспечивающего отображение информации и взаимодействие пользователя с приложением;
- **Backend** — разработка серверной части, отвечающей за обработку запросов, выполнение бизнес-логики и взаимодействие с базой данных;
- **Database** — проектирование и использование системы хранения данных, обеспечивающей сохранность, целостность и доступность информации;
- **API** — организация программного интерфейса для обмена данными между клиентской и серверной частями приложения;
- **Integration** — объединение отдельных компонентов в единую работоспособную систему;
- **Testing and Deployment** — проверка корректности работы приложения и подготовка его к развёртыванию.

В рамках предыдущих практических работ были рассмотрены отдельные технологии и инструменты, применяемые при разработке веб-приложений. Были изучены способы создания пользовательских интерфейсов, методы работы с объектной моделью документа (DOM), компонентный подход к разработке на React, создание серверных приложений на Node.js и Python Flask, взаимодействие с PostgreSQL, а также разработка прогрессивных веб-приложений (PWA).

Настоящая практическая работа направлена на **объединение полученных знаний и навыков в процессе создания законченного программного продукта**.

В отличие от предыдущих работ, ориентированных преимущественно на освоение отдельных технологий, данная работа предполагает выполнение комплексного проекта, в котором все основные компоненты должны взаимодействовать между собой.

Разрабатываемое приложение должно представлять собой единую информационную систему, реализующую определённый набор функциональных возможностей в соответствии с назначенным вариантом задания.

Тематика приложений может быть различной: информационно-аналитические системы, образовательные платформы, системы мониторинга, сервисы бронирования, электронная коммерция, системы управления ресурсами и другие прикладные веб-сервисы.

Независимо от предметной области приложение должно иметь логически организованную архитектуру, удобный пользовательский интерфейс, серверную часть и механизм постоянного хранения данных.

Архитектура разрабатываемого приложения

В качестве базовой архитектуры рекомендуется использовать трёхуровневую модель, включающую:

1. Клиентский уровень (Frontend).

Обеспечивает взаимодействие пользователя с приложением. На данном уровне реализуются:

- отображение страниц и компонентов интерфейса;
- навигация между разделами;
- ввод и проверка пользовательских данных;
- обработка действий пользователя;
- отправка запросов к серверу;
- получение и отображение результатов обработки данных;
- визуализация информации с помощью таблиц, карточек, диаграмм и других элементов.

Для разработки клиентской части рекомендуется использовать React совместно с HTML, CSS, JavaScript и Bootstrap.

2. Серверный уровень (Backend).

Обеспечивает обработку запросов от клиентского приложения и выполнение основной прикладной логики.

На данном уровне реализуются:

- приём и обработка HTTP-запросов;
- маршрутизация запросов;
- проверка корректности поступающих данных;
- выполнение бизнес-логики приложения;
- взаимодействие с базой данных;
- формирование ответов в формате JSON;
- обработка ошибок и исключительных ситуаций.

Для разработки серверной части допускается использование Node.js или Python Flask в зависимости от выбранного технологического стека.

3. Уровень хранения данных (Database).

Предназначен для долговременного хранения и обработки информации, необходимой для функционирования приложения.

На данном уровне предусматриваются:

- проектирование структуры базы данных;
- определение сущностей и связей между ними;

- создание таблиц и ограничений целостности;
- выполнение SQL-запросов;
- реализация операций CRUD;
- обеспечение согласованности хранимой информации.

В качестве системы управления базами данных используется PostgreSQL.

Взаимодействие между клиентской и серверной частями осуществляется через REST API с использованием HTTP-запросов. Обмен данными рекомендуется выполнять в формате JSON.

Клиентская часть не должна обращаться непосредственно к базе данных. Все операции получения и изменения информации выполняются через серверное API.

Особенности выполнения практической работы

Практическая работа носит комплексный характер и предполагает самостоятельное выполнение основных этапов разработки веб-приложения.

Студенту необходимо проанализировать назначенный вариант задания, определить функциональные возможности будущей системы, спроектировать структуру пользовательского интерфейса, разработать модель данных и реализовать взаимодействие между основными программными компонентами.

Особое внимание следует уделить не только внешнему виду приложения, но и корректности его функционирования.

Пользовательские действия должны приводить к соответствующим изменениям состояния приложения, а операции с данными — выполняться через серверную часть с сохранением результатов в базе данных.

Важным требованием является обеспечение согласованной работы всех уровней приложения. Изменения, выполненные пользователем, должны корректно обрабатываться сервером, сохраняться в базе данных и отображаться в интерфейсе.

При разработке необходимо учитывать принципы модульности, повторного использования компонентов, разделения ответственности и удобства сопровождения программного кода.

Допускается использование дополнительных библиотек, инструментов визуализации, внешних API и других технологий, если это необходимо для реализации функциональных возможностей конкретного варианта.

Использование дополнительных средств не должно заменять реализацию основных компонентов приложения, предусмотренных заданием.

Результатом выполнения практической работы должно стать работоспособное Fullstack веб-приложение, включающее клиентскую часть, серверное API и базу данных PostgreSQL, обеспечивающее выполнение предусмотренных вариантом пользовательских сценариев.

Разработанное приложение должно быть протестировано, сопровождаться краткой технической документацией и подготовлено к демонстрации основных функциональных возможностей.

Задания к практической работе

Разработать комплексное Fullstack веб-приложение в соответствии с назначенным вариантом задания и представленным графическим концептом.

Приложение должно представлять собой законченную информационную систему, предназначенную для мониторинга, анализа и управления объектами, событиями и процессами определённой предметной области.

Необходимо реализовать две взаимосвязанные части пользовательского интерфейса:

- **Веб-интерфейс администратора (оператора, руководителя)** — предназначен для централизованного мониторинга, просмотра аналитических показателей, управления объектами, обработки событий, формирования задач и отчётов.
- **Мобильный веб-интерфейс пользователя (сотрудника, инспектора)** — предназначен для оперативного получения информации, выполнения назначенных задач, регистрации событий, просмотра уведомлений и взаимодействия с системой непосредственно на рабочем месте.

Обе части должны использовать единый серверный API и общую базу данных, обеспечивая согласованность информации между различными категориями пользователей.

Разрабатываемое приложение должно включать клиентскую часть на React, серверную часть на Node.js или Python Flask и базу данных PostgreSQL.

Внешний вид и структура основных экранов должны соответствовать графическому концепту варианта. Допускается изменение отдельных декоративных элементов, цветовой схемы и расположения второстепенных компонентов при условии сохранения назначения и функциональности интерфейса.

В рамках работы необходимо реализовать основные пользовательские сценарии, включая просмотр данных, поиск и фильтрацию информации, регистрацию и обработку событий, создание и выполнение задач, изменение статусов, просмотр аналитических показателей и формирование отчётной информации.

Функции, связанные с компьютерным зрением, интеллектуальным распознаванием объектов, анализом видеопотоков и другими специализированными технологиями, допускается реализовать с использованием заранее подготовленных демонстрационных данных или имитации результатов обработки.

При этом **основные операции информационной системы должны быть реализованы полноценно**: данные должны передаваться через REST API, сохраняться в PostgreSQL и корректно отображаться в пользовательском интерфейсе.

Спецификация на разработку

Требования к архитектуре приложения

Приложение должно быть реализовано на основе трёхуровневой архитектуры:

- **Frontend:** React, JavaScript, HTML5, CSS3, Bootstrap.

- **Backend:** Node.js с Express либо Python с Flask.
- **Database:** PostgreSQL.
- **Взаимодействие компонентов:** REST API, HTTP, JSON.

Клиентская и серверная части должны быть организованы как самостоятельные программные модули. Обращение к базе данных должно выполняться исключительно через серверную часть приложения. Для обмена данными между клиентом и сервером необходимо использовать асинхронные HTTP-запросы.

Требования к веб-интерфейсу администратора

Веб-интерфейс должен быть ориентирован на использование на персональных компьютерах и ноутбуках. Необходимо предусмотреть следующие функциональные области.

Боковая панель должна содержать логотип приложения, основные разделы системы, дополнительные инструменты и информацию о текущем пользователе. Активный раздел должен иметь визуальное выделение.

В верхней части рабочей области необходимо разместить:

- строку поиска;
- выбор периода или контролируемого объекта;
- индикатор уведомлений;
- элементы управления профилем;
- дополнительные действия в соответствии с вариантом.

Главная страница должна содержать не менее четырёх карточек ключевых показателей. Каждая карточка должна включать название показателя, его текущее значение, тематическую иконку и, при необходимости, динамику изменения. Значения показателей должны формироваться на основе данных, полученных с сервера.

Предусмотреть центральную область для отображения контролируемого объекта или процесса. В зависимости от варианта это может быть изображение производственного помещения, торговой полки, дорожной ситуации или другого объекта. При наличии соответствующих данных необходимо отображать визуальные маркеры, обозначения состояний, обнаруженные события и другую контекстную информацию. Для имитации работы интеллектуального анализа допускается использовать подготовленные изображения с нанесёнными обозначениями.

Реализовать список последних событий с указанием:

- типа события;
- объекта или места возникновения;
- даты и времени;
- уровня важности;
- текущего статуса;
- краткого описания.

При выборе события должна открываться подробная информация.

Предусмотреть не менее двух различных способов визуализации данных, например линейный график, столбчатую или круговую диаграмму. Должна быть реализована возможность изменения периода отображения данных.

Реализовать интерфейс создания и просмотра задач, содержащих название, описание, исполнителя, приоритет, срок выполнения и статус. Предусмотреть возможность изменения статуса задачи и отображения результатов её выполнения.

Требования к мобильному веб-интерфейсу

Мобильный интерфейс должен быть адаптирован для экранов смартфонов и обеспечивать доступ к основным функциям системы. Необходимо реализовать следующие экраны.

Главный экран - содержит название приложения, краткую информацию о текущем пользователе и набор кнопок быстрого доступа к основным функциям.

Предусмотреть форму ввода данных с возможностью выбора типа события, объекта, добавления комментария и прикрепления изображения. В зависимости от варианта форма может использоваться для регистрации нарушения, проверки торговой полки или фиксации дорожного события.

Отображает изображения контролируемых объектов, обнаруженные события и результаты анализа. Реальная обработка видеопотока не является обязательной.

Реализовать список назначенных задач с отображением приоритета, сроков и статусов. Пользователь должен иметь возможность открыть задачу, ознакомиться с подробной информацией и отметить её выполнение.

Предусмотреть экран со списком уведомлений, их датами, типами и состоянием прочтения.

Отобразить основные показатели деятельности пользователя, включая количество выполненных задач, зарегистрированных событий или проведённых проверок. Данные должны поступать с сервера.

База данных PostgreSQL должна обеспечивать хранение информации, необходимой для функционирования системы. Минимально рекомендуется предусмотреть следующие логические сущности:

Сущность	**Назначение**
Users	Пользователи системы и их роли
Objects	Контролируемые объекты
Events	Зарегистрированные события
Tasks	Задачи и их статусы
Notifications	Уведомления пользователей
Media	Сведения об изображениях и видеоматериалах

В зависимости от варианта структура базы данных может расширяться дополнительными таблицами, например для камер, товаров, транспортных средств, производственных зон или результатов проверок. Необходимо определить первичные и внешние ключи, установить связи между таблицами и предусмотреть ограничения

целостности данных. Для основных сущностей должны быть реализованы операции создания, чтения, изменения и удаления записей.

Требования к REST API

Серверная часть должна предоставлять набор HTTP-эндпоинтов для взаимодействия с клиентским приложением. Минимальный набор API:

Метод	Эндпоинт	Назначение
GET	/api/dashboard	Получение сводных показателей
GET	/api/objects	Получение списка объектов
GET	/api/objects/	Получение информации об объекте
GET	/api/events	Получение списка событий
POST	/api/events	Регистрация нового события
PATCH	/api/events/	Изменение состояния события
GET	/api/tasks	Получение списка задач
POST	/api/tasks	Создание задачи
PATCH	/api/tasks/	Изменение задачи или её статуса
GET	/api/notifications	Получение уведомлений
GET	/api/statistics	Получение аналитических данных

При необходимости набор эндпоинтов расширяется в соответствии с функциональностью конкретного варианта. API должно возвращать данные в формате JSON, использовать соответствующие HTTP-коды состояния и обеспечивать обработку ошибочных запросов.

Требования к интерактивности

В приложении необходимо реализовать следующие интерактивные механизмы:

- переходы между основными разделами;
- поиск и фильтрацию записей;
- просмотр подробной информации;
- работу с формами ввода;
- проверку обязательных полей;
- создание и изменение записей;
- обновление статусов;
- отображение уведомлений;
- обновление показателей после изменения данных;
- обработку ошибок сервера и отображение сообщений пользователю.

Изменения данных должны сохраняться в базе данных и оставаться доступными после обновления страницы.

Требования к адаптивности и оформлению

Интерфейс должен соответствовать представленному графическому концепту и сохранять удобство использования на различных устройствах. Для веб-версии необходимо предусмотреть корректное отображение на широких экранах и ноутбуках.

Для мобильной версии необходимо разработать адаптивную компоновку с удобными элементами управления, читаемыми текстами и отсутствием горизонтальной прокрутки.

Рекомендуется использовать единую систему цветов, типографики, иконок, отступов и визуальных состояний компонентов.

Допустимые упрощения

С учётом учебного характера работы допускаются следующие упрощения:

- использование заранее подготовленных изображений и видеоматериалов;
- имитация результатов интеллектуального анализа;
- использование демонстрационных данных о работе камер и датчиков;
- периодическое обновление информации вместо настоящего потокового обмена;
- упрощённая модель авторизации и распределения ролей;
- формирование отчётов в виде таблиц или CSV-файлов;
- использование статических карт и схем вместо геоинформационных сервисов.

При этом имитация не должна заменять реализацию основных функций приложения, связанных с обработкой пользовательских запросов, управлением записями и сохранением данных.

Ожидаемый результат

В результате выполнения практической работы должно быть разработано работоспособное Fullstack веб-приложение, включающее:

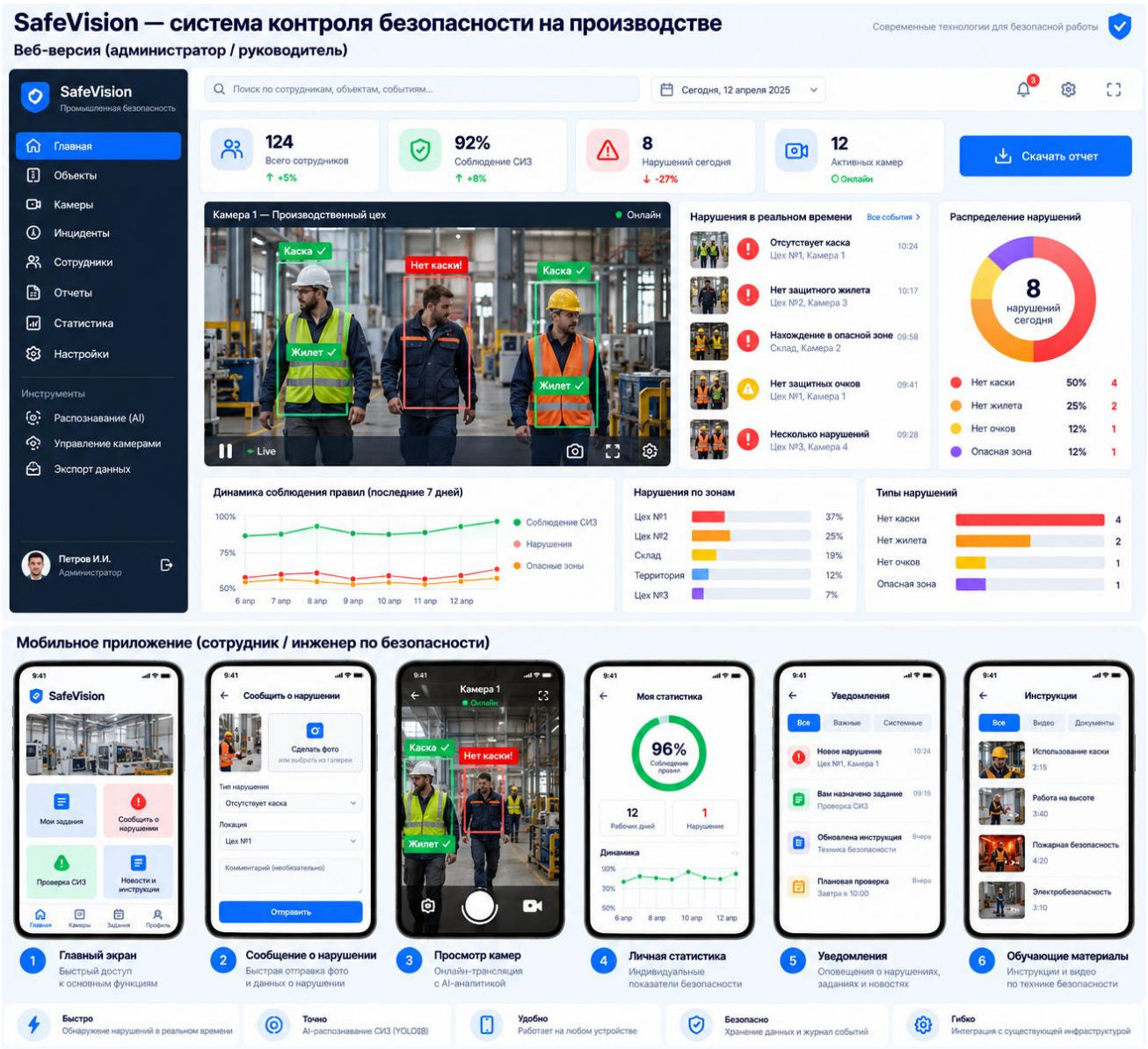
1. Веб-интерфейс администратора с навигацией, мониторингом и аналитикой.
2. Адаптивный мобильный интерфейс пользователя.
3. Серверное приложение с REST API.
4. Базу данных PostgreSQL с взаимосвязанными таблицами.
5. Механизмы регистрации и обработки событий.
6. Систему создания и выполнения задач.
7. Средства поиска, фильтрации и визуализации данных.
8. Обработку пользовательских действий и ошибок.
9. Демонстрационные данные, соответствующие тематике варианта.
10. Исходный код проекта и инструкцию по его запуску.

Разработанная система должна демонстрировать согласованную работу клиентской части, серверной логики и базы данных. Основные пользовательские сценарии должны выполняться без необходимости ручного изменения исходного кода или содержимого базы данных.

Вариант 1. SafeVision — система контроля промышленной безопасности

Разработать Fullstack веб-приложение **SafeVision**, предназначенное для мониторинга соблюдения требований промышленной безопасности на производственных объектах, регистрации выявленных нарушений, контроля использования средств индивидуальной защиты (СИЗ) и анализа состояния безопасности предприятия.

Система должна обеспечивать централизованное представление информации о производственных объектах, сотрудниках, камерах наблюдения и зарегистрированных нарушениях, а также предоставлять инструменты для оперативного реагирования на опасные ситуации.



Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для администратора и руководителя** — предназначена для мониторинга производственных объектов, просмотра камер, анализа нарушений, управления сотрудниками, контроля выполнения задач и формирования отчётов.
- **Мобильная версия для сотрудника и инженера по безопасности** — предназначена для просмотра информации о состоянии безопасности, регистрации нарушений, выполнения назначенных задач, получения уведомлений и изучения инструкций по охране труда.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт SafeVision, сохранив его основные композиционные решения, структуру навигации, информационные блоки и цветовую схему.

Функциональные требования к веб-версии

Главная информационная панель (Dashboard)

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления.

В боковом меню предусмотреть разделы:

- Главная;
- Объекты;
- Камеры;
- Инциденты;
- Сотрудники;
- Отчёты;
- Статистика;
- Настройки.

В дополнительной группе инструментов разместить пункты «Распознавание (AI)», «Управление камерами» и «Экспорт данных». В верхней панели расположить строку поиска по сотрудникам, объектам и событиям, выбор даты или периода, индикатор уведомлений и элементы управления профилем.

Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Всего сотрудников** — общее количество зарегистрированных сотрудников предприятия.
2. **Соблюдение СИЗ** — процент проверок, в которых не выявлено нарушений требований к средствам индивидуальной защиты.
3. **Нарушений сегодня** — количество зарегистрированных нарушений за текущую дату.
4. **Активных камер** — количество камер, имеющих статус «Онлайн».

Для числовых показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе информации из базы данных.

Мониторинг производственных объектов

Реализовать центральный блок просмотра камер наблюдения. В верхней части блока отображать название камеры, производственный объект и текущий статус подключения. Основная область должна содержать изображение производственного помещения с визуальными обозначениями обнаруженных объектов и возможных нарушений.

В соответствии с графическим концептом предусмотреть следующие типы обозначений:

- зелёная рамка и подпись «Каска» — наличие защитной каски;
- зелёная рамка и подпись «Жилет» — наличие сигнального жилета;
- красная рамка и подпись «Нет каски!» — отсутствие защитной каски;
- другие обозначения, соответствующие категориям нарушений.

Под изображением разместить элементы управления просмотром, созданием снимка, переходом в полноэкранный режим и настройками камеры. Предусмотреть возможность переключения между несколькими камерами и просмотра информации об их расположении. Для учебной реализации допускается использовать заранее подготовленные изображения с демонстрационными результатами распознавания. Реальное подключение к видеокамерам и разработка модели компьютерного зрения не являются обязательными.

Журнал нарушений

Разработать раздел регистрации и просмотра нарушений промышленной безопасности. Для каждого нарушения хранить и отображать:

- уникальный идентификатор;
- тип нарушения;
- производственный объект или зону;
- камеру, с которой связано событие;
- дату и время обнаружения;
- уровень опасности;
- описание;
- изображение события;
- текущий статус обработки.

Предусмотреть категории нарушений: отсутствие защитной каски, отсутствие сигнального жилета, отсутствие защитных очков, нахождение в опасной зоне и другие. Реализовать поиск и фильтрацию по типу нарушения, объекту, дате, уровню опасности и статусу. При выборе записи открывать подробную карточку инцидента. Администратор

должен иметь возможность изменить статус нарушения, добавить комментарий и создать задачу по его устранению. Предусмотреть статусы: «Новое», «В обработке», «Устранено», «Закрыто».

Аналитика промышленной безопасности

Разработать раздел статистики, содержащий несколько взаимосвязанных аналитических блоков.

Динамика соблюдения правил. Линейный график за последние семь дней с отображением уровня соблюдения требований СИЗ и количества нарушений. Предусмотреть выбор периода.

Распределение нарушений. Круговая диаграмма с отображением долей различных категорий нарушений и их абсолютного количества.

Нарушения по производственным зонам. Горизонтальная диаграмма, показывающая распределение нарушений между цехами, складами и другими контролируемыми объектами.

Типы нарушений. Столбчатая диаграмма, позволяющая сравнить количество случаев отсутствия касок, жилетов, очков и нарушений правил нахождения в опасных зонах.

Все аналитические показатели должны формироваться из данных, полученных через REST API. При изменении периода отображаемая информация должна обновляться.

Управление сотрудниками и объектами

Реализовать раздел сотрудников с возможностью просмотра, добавления, редактирования и поиска записей. Для каждого сотрудника предусмотреть ФИО, должность, подразделение, производственный объект, контактные сведения и статус активности.

В разделе объектов реализовать управление производственными площадками, цехами, складами и другими зонами предприятия. Для объекта указывать название, описание, местоположение, ответственного сотрудника и перечень связанных камер. Предусмотреть просмотр информации о зарегистрированных нарушениях и задачах, относящихся к выбранному объекту.

Отчёты и уведомления

Реализовать формирование отчётной информации за выбранный период. Отчёт должен включать количество нарушений, распределение по категориям и объектам, динамику соблюдения требований безопасности и сведения о выполненных мероприятиях. Предусмотреть экспорт табличных данных в CSV. Уведомления должны формироваться при регистрации нового нарушения, назначении задачи и изменении её статуса. Для каждого уведомления предусмотреть возможность просмотра подробностей и отметки о прочтении.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Главный экран

Разработать стартовую страницу с названием SafeVision, изображением производственного объекта и кнопками быстрого доступа:

- «Мои задания»;
- «Сообщить о нарушении»;
- «Проверка СИЗ»;
- «Новости и инструкции».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Камеры», «Задания», «Профиль».

Сообщение о нарушении

Реализовать форму регистрации нарушения. Форма должна содержать поле добавления фотографии, выбор типа нарушения, выбор производственного объекта или зоны, текстовое поле комментария и кнопку «Отправить». Предусмотреть проверку обязательных полей и отображение сообщений об ошибках. После отправки формы информация должна передаваться на сервер и сохраняться в PostgreSQL. Зарегистрированное нарушение должно появляться в журнале событий веб-версии администратора.

Просмотр камер

Создать экран просмотра выбранной камеры с отображением изображения производственного помещения. Предусмотреть название камеры, статус подключения, элементы управления снимком и переключением режима просмотра. На изображении должны отображаться демонстрационные результаты распознавания СИЗ и нарушений.

Личная статистика

Разработать экран с показателями соблюдения правил безопасности сотрудником. Предусмотреть круговой индикатор процента соблюдения требований, количество рабочих дней, зарегистрированных нарушений и график динамики за выбранный период. Значения должны рассчитываться на основе данных текущего пользователя.

Уведомления

Реализовать экран уведомлений с категориями «Все», «Важные» и «Системные». Предусмотреть отображение новых нарушений, назначенных задач, обновлений инструкций и плановых проверок. Каждое уведомление должно содержать тематическую

иконку, заголовок, краткое описание, дату и время. Реализовать отметку уведомления как прочитанного.

Обучающие материалы

Создать раздел инструкций по промышленной безопасности. Предусмотреть категории «Все», «Видео», «Документы». Для каждого материала отображать название, тематическое изображение, краткое описание, тип и продолжительность либо размер файла. В демонстрационном наборе должны присутствовать материалы по использованию касок, работе на высоте, пожарной безопасности и электробезопасности. Реализовать возможность открытия подробной информации и просмотра доступного материала.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать выполнение бизнес-логики, обработку HTTP-запросов, проверку поступающих данных и взаимодействие с PostgreSQL.

Минимальная структура базы данных должна включать следующие сущности:

Сущность	Основные данные
Users	Пользователи, роли, учётные данные
Employees	Сотрудники, должности, подразделения
Facilities	Производственные объекты и зоны
Cameras	Камеры, местоположение, статус
Violations	Нарушения, категории, дата, статус
Tasks	Задачи, исполнители, сроки, статусы
Notifications	Уведомления и отметки прочтения
TrainingMaterials	Инструкции и обучающие материалы
Media	Фотографии и сведения о медиафайлах

Необходимо определить первичные и внешние ключи, обеспечить связи между сущностями и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее получение статистики, управление объектами, сотрудниками, камерами, нарушениями, задачами и уведомлениями. Предусмотреть разграничение доступа по ролям администратора и сотрудника. Пароли должны храниться в виде защищённых хешей, а доступ к изменению данных — проверяться на сервере.

Основные пользовательские сценарии

Сценарий 1. Регистрация нарушения.

Сотрудник открывает мобильное приложение, выбирает функцию «Сообщить о нарушении», прикрепляет фотографию, указывает тип нарушения и производственную зону, после чего отправляет сообщение. Сервер проверяет данные, сохраняет запись и формирует уведомление для администратора. Новое нарушение отображается в журнале событий веб-версии.

Сценарий 2. Обработка нарушения администратором.

Администратор открывает уведомление о новом нарушении, просматривает его подробности и создаёт задачу ответственному сотруднику. Назначенная задача появляется в мобильном интерфейсе исполнителя.

Сценарий 3. Выполнение задачи.

Сотрудник открывает список заданий, выбирает назначенную задачу, знакомится с описанием и после выполнения изменяет её статус. Информация сохраняется в базе данных и становится доступной администратору.

Сценарий 4. Просмотр аналитики.

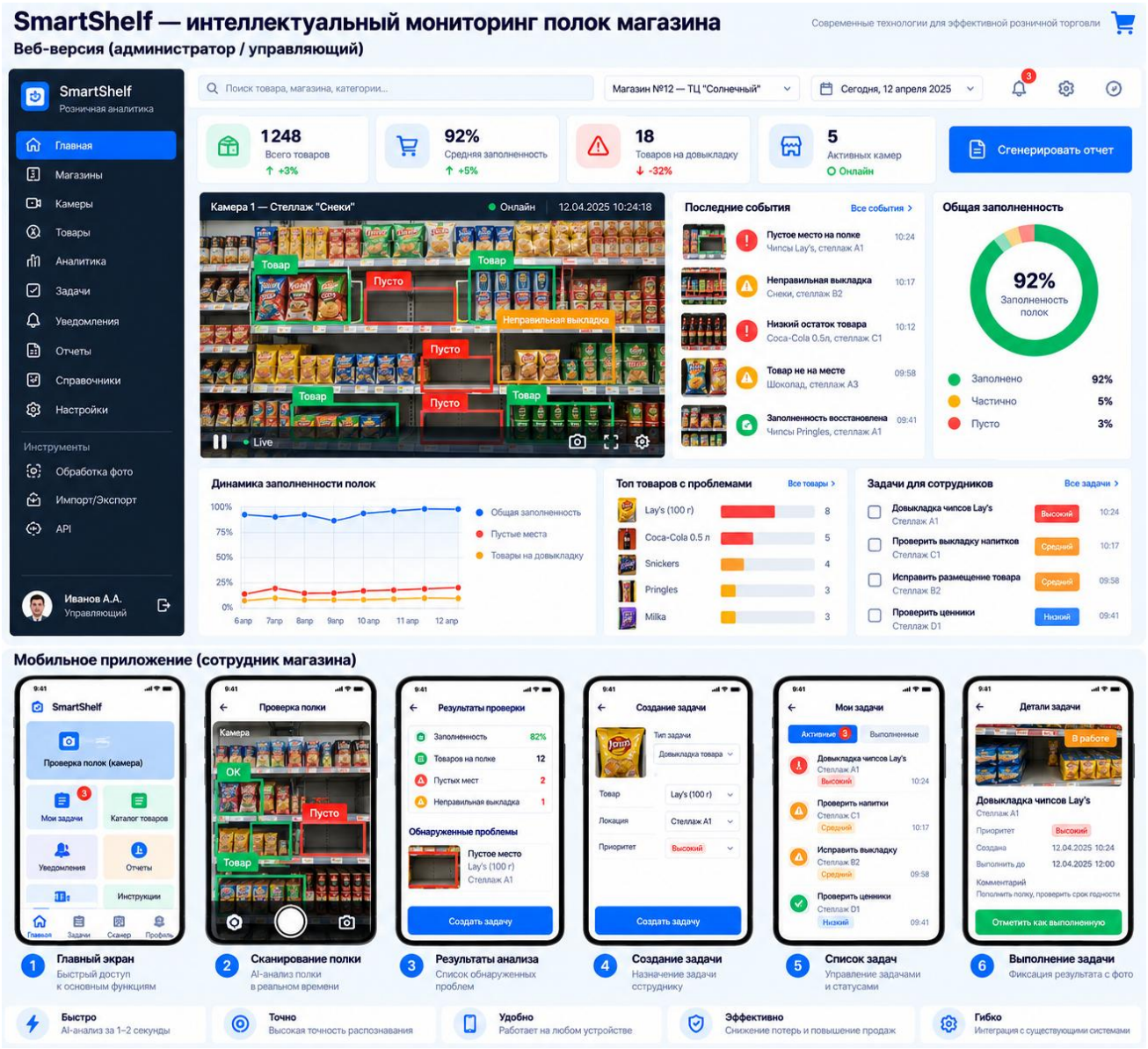
Руководитель открывает раздел статистики, выбирает производственный объект и временной период. Приложение запрашивает соответствующие данные у сервера и обновляет диаграммы, графики и показатели.

Сценарий 5. Просмотр камер и результатов анализа.

Администратор или сотрудник выбирает камеру и просматривает изображение с визуальными обозначениями обнаруженных нарушений. При выборе обозначенного события пользователь может перейти к связанной записи в журнале нарушений.

Вариант 2. SafeVision — система контроля промышленной безопасности

Разработать Fullstack веб-приложение **SmartShelf**, предназначенное для автоматизации контроля состояния торговых полок, мониторинга наличия товаров, выявления пустых мест и нарушений выкладки, управления задачами сотрудников и анализа эффективности использования торгового пространства магазина.



Система должна обеспечивать централизованное представление информации о торговых точках, товарах, стеллажах, камерах наблюдения и результатах проверок, а также предоставлять инструменты для оперативного выявления и устранения проблем с размещением товаров.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для администратора и управляющего** — предназначена для мониторинга торговых точек, просмотра состояния полок, анализа заполненности

стеллажей, управления товарным каталогом, контроля выполнения задач и формирования отчётов.
- **Мобильная версия для сотрудника магазина** — предназначена для проверки торговых полок, регистрации обнаруженных проблем, создания и выполнения задач, получения уведомлений и просмотра справочной информации о товарах.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт SmartShelf, сохранив основные композиционные решения, структуру навигации, информационные блоки и цветовую схему.

Функциональные требования к веб-версии

Главная информационная панель (Dashboard)

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления.

В боковом меню предусмотреть разделы:

- Главная;
- Магазины;
- Камеры;
- Товары;
- Аналитика;
- Задачи;
- Уведомления;
- Отчёты;
- Справочники;
- Настройки.

В дополнительной группе инструментов разместить пункты «Обработка фото», «Импорт/Экспорт» и «API».

В верхней панели расположить строку поиска по товарам, магазинам и категориям, выпадающий список выбора торговой точки, выбор даты или периода, индикатор уведомлений и элементы управления профилем.

Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Всего товаров** — количество товарных позиций, зарегистрированных в системе.
2. **Средняя заполненность** — средний процент заполненности контролируемых торговых полок.

3. **Товаров на довыкладку** — количество товарных позиций, для которых обнаружена необходимость пополнения.
4. **Активных камер** — количество камер наблюдения, имеющих статус «Онлайн».

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода.

Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL.

Мониторинг торговых полок

Реализовать центральный блок просмотра камер наблюдения за торговыми стеллажами. В верхней части блока отображать название камеры, торговую точку, контролируемый стеллаж, дату и время последнего обновления изображения, а также статус подключения. Основная область должна содержать изображение торговой полки с визуальными обозначениями результатов анализа.

В соответствии с графическим концептом предусмотреть следующие типы обозначений:

- зелёная рамка с подписью «Товар» — обнаруженная товарная позиция;
- красная рамка с подписью «Пусто» — свободное место на полке;
- оранжевая рамка с подписью «Неправильная выкладка» — товар, расположенный с нарушением установленной схемы размещения.

Под изображением разместить элементы управления просмотром, созданием снимка, переходом в полноэкранный режим и настройками камеры. Предусмотреть возможность выбора магазина, камеры и стеллажа. Для учебной реализации допускается использовать заранее подготовленные фотографии торговых полок с демонстрационными результатами анализа. Реальное подключение камер и разработка алгоритмов компьютерного зрения не являются обязательными.

При этом информация о выявленных проблемах должна храниться в базе данных и использоваться при формировании задач и аналитических показателей.

Журнал событий и проблем выкладки

Разработать раздел регистрации и просмотра событий, связанных с состоянием торговых полок. Для каждого события хранить и отображать:

- уникальный идентификатор;
- тип события;
- магазин;
- стеллаж и конкретную полку;
- товарную позицию, если она определена;

- камеру или источник проверки;
- дату и время обнаружения;
- приоритет;
- описание проблемы;
- изображение;
- текущий статус обработки.

Предусмотреть следующие основные категории событий:

- пустое место на полке;
- низкий остаток товара;
- неправильная выкладка;
- товар размещён не на своём месте;
- заполненность полки восстановлена.

Реализовать поиск и фильтрацию событий по магазину, стеллажу, товару, типу проблемы, дате и статусу. При выборе события должна открываться подробная карточка с фотографией, описанием и сведениями о месте возникновения. Администратор должен иметь возможность изменить статус события, добавить комментарий и создать задачу сотруднику. Предусмотреть статусы: «Новое», «В обработке», «Устранено», «Закрыто».

Аналитика заполненности торговых полок

Разработать раздел статистики, содержащий несколько взаимосвязанных аналитических блоков.

Динамика заполненности полок. Линейный график за последние семь дней, отображающий средний процент заполненности, количество пустых мест и количество товарных позиций, требующих пополнения. Предусмотреть выбор периода.

Общая заполненность. Круговая диаграмма с отображением долей полностью заполненных, частично заполненных и пустых участков полок.

Товары с наибольшим количеством проблем. Горизонтальная диаграмма, показывающая товары, для которых чаще всего регистрируются пустые места, низкие остатки или нарушения выкладки.

Аналитика по магазинам и стеллажам. Таблица или диаграмма со сравнением заполненности торговых точек и отдельных стеллажей.

Все показатели должны формироваться на основе данных, полученных через REST API. При изменении магазина или периода информация на графиках и в карточках должна обновляться.

Управление магазинами, товарами и стеллажами

Реализовать раздел управления торговыми точками с возможностью просмотра, добавления, редактирования и поиска магазинов. Для каждого магазина предусмотреть название, адрес, контактные сведения, ответственного сотрудника и статус активности. В разделе товаров реализовать каталог с возможностью выполнения CRUD-операций.

Для каждой товарной позиции хранить:

- наименование;
- артикул;
- категорию;
- производителя или бренд;
- изображение;
- цену;
- единицу измерения;
- минимально допустимый остаток;
- статус активности.

Предусмотреть поиск и фильтрацию товаров по названию, категории и бренду.

Реализовать справочник стеллажей с указанием магазина, названия или номера стеллажа, расположения, количества полок и связанных камер. Для каждого стеллажа предусмотреть просмотр закреплённых товаров, текущей заполненности и обнаруженных проблем. Дополнительно реализовать возможность определения планового размещения товаров на полках. Такая схема должна позволять указывать, какие товары должны находиться в определённых зонах стеллажа.

Управление задачами сотрудников

Разработать раздел создания и контроля задач, связанных с устранением проблем на торговых полках. Для каждой задачи предусмотреть:

- название и описание;
- тип задачи;
- магазин и стеллаж;
- товарную позицию;
- назначенного сотрудника;
- приоритет;
- дату создания;
- срок выполнения;
- статус;
- комментарий;
- фотографию результата при необходимости.

Основные типы задач:

- довыкладка товара;
- проверка наличия;
- исправление выкладки;
- проверка ценника;
- повторная проверка полки.

Предусмотреть приоритеты «Высокий», «Средний» и «Низкий». Задачи должны иметь статусы «Новая», «В работе», «Выполнена» и «Отменена». При изменении статуса информация должна сохраняться в базе данных и отображаться как в веб-версии управляющего, так и в мобильном интерфейсе сотрудника.

Отчёты и уведомления

Реализовать формирование отчётной информации за выбранный период. Отчёт должен содержать сведения о средней заполненности полок, количестве обнаруженных проблем, товарах с наибольшим количеством нарушений выкладки и выполненных задачах. Предусмотреть возможность экспорта табличных данных в CSV. Уведомления должны формироваться при обнаружении новой проблемы, создании задачи, изменении её статуса и восстановлении заполненности полки. Для каждого уведомления предусмотреть дату, тип, краткое описание и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу с логотипом SmartShelf, названием приложения и кнопками быстрого доступа:

- «Проверка полок (камера)»;
- «Мои задачи»;
- «Каталог товаров»;
- «Уведомления»;
- «Отчёты»;
- «Инструкции».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Задачи», «Сканер», «Профиль». Для раздела задач предусмотреть индикатор количества активных назначений.

Разработать экран проверки состояния торговой полки. Основную область должен занимать снимок стеллажа с визуальными обозначениями обнаруженных товаров и пустых мест. Предусмотреть отображение названия магазина, номера стеллажа и даты проверки. Разместить элементы управления созданием фотографии, выбором изображения из галереи и запуском демонстрационного анализа. После выполнения проверки приложение должно отображать результаты анализа выбранного изображения.

Допускается использовать подготовленные наборы изображений и соответствующие им результаты распознавания.

Разработать экран отображения результатов проверки торговой полки. В верхней части вывести основные показатели:

- процент заполненности;
- количество обнаруженных товаров;
- количество пустых мест;
- количество нарушений выкладки.

Ниже представить список обнаруженных проблем. Для каждой проблемы отображать тип, товар, расположение на стеллаже и фрагмент изображения. Предусмотреть кнопку «Создать задачу», позволяющую перейти к форме создания задания по выбранной проблеме. Результаты выполненных проверок должны сохраняться на сервере и быть доступны для последующего просмотра.

Реализовать форму создания задачи сотрудником магазина. Форма должна содержать:

- тип задачи;
- товарную позицию;
- магазин;
- стеллаж или полку;
- приоритет;
- текстовое описание;
- срок выполнения при необходимости.

Если создание задачи выполняется из результатов проверки, сведения о товаре, стеллаже и типе проблемы должны автоматически подставляться в соответствующие поля. Предусмотреть проверку обязательных данных. После отправки формы новая задача должна сохраняться в базе данных и появляться в общем списке задач.

Разработать экран «Мои задачи» с двумя основными категориями:

- «Активные»;
- «Выполненные».

Каждая задача должна содержать название, магазин или стеллаж, приоритет, время создания и текущий статус. Предусмотреть визуальное выделение задач высокого приоритета. Реализовать возможность перехода к подробной информации и изменения статуса задачи.

Разработать экран подробной информации о выбранной задаче. Отображать название задачи, фотографию проблемного участка полки, товар, местоположение, приоритет, дату создания, срок выполнения и комментарий. Предусмотреть кнопку «Отметить как

выполненную». При завершении задачи сотрудник должен иметь возможность добавить комментарий и, при необходимости, фотографию результата. После подтверждения информация должна сохраняться в базе данных, а задача — переходить в статус «Выполнена». Изменение статуса должно отображаться в веб-версии управляющего.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL.

Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, учётные данные
Stores	Магазины, адреса, ответственные сотрудники
Products	Товары, артикулы, категории, цены
Shelves	Стеллажи, полки и их расположение
ShelfPlacements	Плановое размещение товаров
Cameras	Камеры, местоположение, статус
Inspections	Проверки полок и результаты анализа
ShelfIssues	Обнаруженные проблемы выкладки
Tasks	Задачи, исполнители, приоритеты, статусы
Notifications	Уведомления и отметки прочтения
Media	Фотографии и сведения о медиафайлах

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление магазинами и товарами;
- получение информации о стеллажах и камерах;
- регистрацию и просмотр проверок;
- получение результатов анализа;
- регистрацию обнаруженных проблем;
- создание и выполнение задач;
- получение уведомлений;
- формирование аналитических данных и отчётов.

Предусмотреть разграничение доступа по ролям администратора и сотрудника магазина. Пароли должны храниться в виде защищённых хешей, а права на изменение данных — проверяться на сервере.

Основные пользовательские сценарии

Сценарий 1. Проверка торговой полки.

Сотрудник открывает мобильное приложение, выбирает функцию «Проверка полок», указывает магазин и стеллаж, после чего создаёт снимок или выбирает подготовленное изображение. Приложение отправляет информацию на сервер, получает демонстрационные результаты анализа и отображает обнаруженные товары, пустые места и нарушения выкладки. Результат проверки сохраняется в базе данных.

Сценарий 2. Создание задачи по обнаруженной проблеме.

После выполнения проверки сотрудник открывает список обнаруженных проблем, выбирает пустое место или нарушение выкладки и нажимает «Создать задачу». Приложение автоматически заполняет сведения о товаре и расположении. Сотрудник уточняет приоритет и сохраняет задачу. Новая задача появляется в веб-панели управляющего.

Сценарий 3. Выполнение задачи по довыкладке.

Сотрудник открывает список активных задач, выбирает задание на пополнение товара и знакомится с его подробностями. После выполнения работы сотрудник добавляет комментарий или фотографию и отмечает задачу как выполненную. Сервер сохраняет изменения и обновляет статус задачи в интерфейсе управляющего.

Сценарий 4. Анализ заполненности магазина.

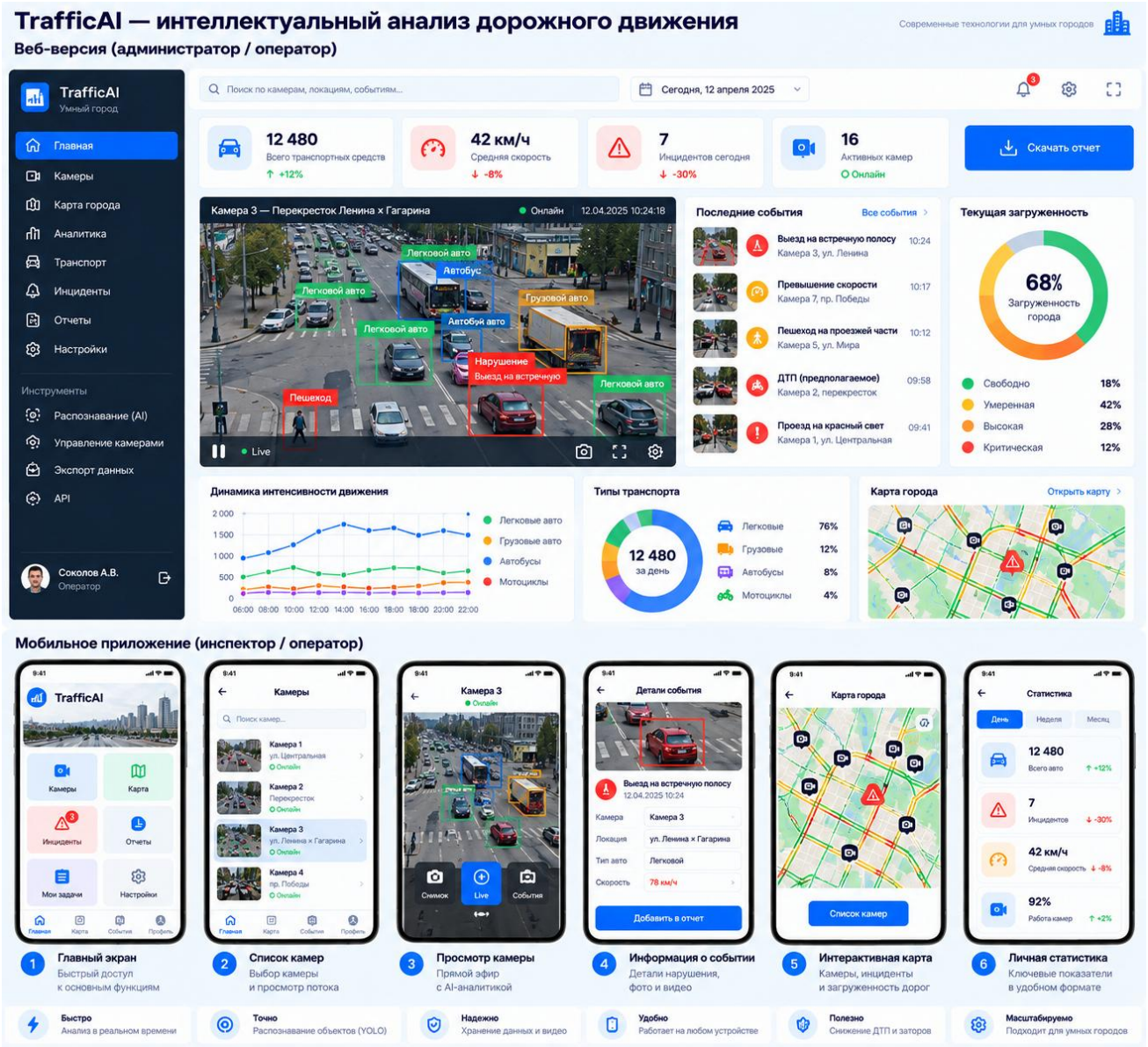
Управляющий открывает Dashboard, выбирает магазин и временной период. Приложение получает с сервера данные о заполненности полок, количестве пустых мест и товарах, требующих пополнения. Карточки показателей и диаграммы обновляются в соответствии с выбранными параметрами.

Сценарий 5. Обработка события мониторинга.

В системе регистрируется событие «Пустое место на полке». Управляющий открывает журнал событий, просматривает фотографию проблемного участка и назначает задачу ответственному сотруднику. После выполнения задачи её статус изменяется, а событие может быть отмечено как устранённое. Показатели заполненности обновляются после регистрации новой проверки, подтверждающей устранение проблемы.

Вариант 3. TrafficAI — система интеллектуального мониторинга и анализа дорожного движения

Разработать Fullstack веб-приложение **TrafficAI**, предназначенное для мониторинга дорожного движения, анализа транспортных потоков, выявления нарушений правил дорожного движения, регистрации дорожных инцидентов и оценки загруженности городской транспортной инфраструктуры.



Система должна обеспечивать централизованное представление информации о дорожных камерах, контролируемых участках улично-дорожной сети, транспортных средствах и зарегистрированных событиях, а также предоставлять инструменты для оперативного анализа дорожной обстановки и формирования отчётной информации.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для администратора и оператора** — предназначена для централизованного мониторинга дорожной обстановки, просмотра камер, анализа

транспортных потоков, регистрации и обработки дорожных инцидентов, управления контролируемыми участками и формирования аналитических отчётов.
- **Мобильная версия для инспектора и оператора** — предназначена для оперативного просмотра камер, получения информации о дорожных событиях, просмотра карты города, регистрации и обработки инцидентов, получения уведомлений и просмотра персональной статистики.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт TrafficAI, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации данных.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления.

В боковом меню предусмотреть разделы:

- Главная;
- Камеры;
- Карта города;
- Аналитика;
- Транспорт;
- Инциденты;
- Отчёты;
- Настройки.

В дополнительной группе инструментов разместить пункты «Распознавание (AI)», «Управление камерами», «Экспорт данных» и «API». В верхней панели расположить строку поиска по камерам, контролируемым участкам и событиям, выбор даты или периода, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Всего транспортных средств** — количество транспортных средств, зарегистрированных системой за выбранный период.
2. **Средняя скорость** — средняя скорость движения транспортных средств на контролируемых участках.
3. **Инцидентов сегодня** — количество дорожных событий и нарушений, зарегистрированных за текущую дату.
4. **Активных камер** — количество камер наблюдения, имеющих статус «Онлайн».

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL.

Реализовать центральный блок просмотра камер наблюдения за дорожной обстановкой. В верхней части блока отображать название камеры, адрес или наименование контролируемого участка, дату и время последнего обновления изображения, а также статус подключения. Основная область должна содержать изображение дорожной ситуации с визуальными обозначениями обнаруженных транспортных средств, пешеходов и возможных нарушений. В соответствии с графическим концептом предусмотреть следующие типы обозначений:

- зелёная рамка с подписью «Легковой автомобиль» — обнаруженный легковой транспорт;
- синяя рамка с подписью «Автобус» — обнаруженный пассажирский транспорт;
- оранжевая рамка с подписью «Грузовой автомобиль» — обнаруженный грузовой транспорт;
- красная рамка с подписью «Пешеход» — обнаруженный пешеход;
- красная рамка с подписью «Нарушение» — предполагаемое нарушение правил дорожного движения.

Для отдельных транспортных средств предусмотреть возможность отображения дополнительных параметров, например типа транспорта, условной скорости и идентификатора обнаруженного объекта. Под изображением разместить элементы управления просмотром, созданием снимка, переходом в полноэкранный режим и настройками камеры. Предусмотреть возможность переключения между камерами и просмотра информации об их местоположении.

Для учебной реализации допускается использовать заранее подготовленные фотографии дорожных ситуаций с демонстрационными результатами распознавания. Реальное подключение к видеокамерам и разработка алгоритмов компьютерного зрения не являются обязательными. При этом зарегистрированные события и результаты демонстрационного анализа должны храниться в базе данных и использоваться при формировании статистики.

Разработать раздел регистрации и просмотра дорожных событий и нарушений. Для каждого инцидента хранить и отображать:

- уникальный идентификатор;
- тип события;
- контролируемый участок дороги;
- камеру, с которой связано событие;
- дату и время обнаружения;
- тип транспортного средства при наличии;
- зафиксированную скорость при наличии;
- уровень важности;

- описание события;
- изображение;
- текущий статус обработки.

Предусмотреть следующие категории событий:

- превышение скорости;
- выезд на встречную полосу;
- проезд на запрещающий сигнал светофора;
- нарушение правил движения пешеходов;
- дорожно-транспортное происшествие;
- образование затора;
- остановка транспортного средства в запрещённом месте;
- другие события, связанные с дорожной обстановкой.

Реализовать поиск и фильтрацию инцидентов по типу, участку дороги, камере, дате, уровню важности и статусу. При выборе события должна открываться подробная карточка с изображением, описанием, местоположением и дополнительными параметрами. Оператор должен иметь возможность изменить статус инцидента, добавить комментарий и включить событие в отчёт. Предусмотреть статусы: «Новое», «На проверке», «Подтверждено», «Отклонено», «Закрыто». События, полученные в результате имитации интеллектуального распознавания, должны рассматриваться как предварительные до подтверждения оператором.

Разработать раздел статистики, содержащий несколько взаимосвязанных аналитических блоков.

Динамика интенсивности движения. Линейный график, отображающий изменение количества транспортных средств в течение суток или выбранного периода. Предусмотреть отдельные линии для легковых автомобилей, грузового транспорта, автобусов и мотоциклов.

Распределение транспортных средств по категориям. Круговая диаграмма с отображением доли и количества транспортных средств различных типов.

Текущая загруженность дорог. Круговой индикатор, отображающий общий уровень загруженности контролируемых участков. Предусмотреть категории «Свободно», «Умеренная», «Высокая» и «Критическая».

Средняя скорость движения. График или таблица изменения средней скорости по времени и контролируемым участкам.

Статистика дорожных инцидентов. Диаграмма количества событий по категориям, участкам и временным интервалам.

Все аналитические показатели должны формироваться на основе данных, полученных через REST API. При изменении периода или контролируемого участка информация на графиках и в карточках должна обновляться.

Разработать раздел «Карта города», предназначенный для отображения расположения камер наблюдения, контролируемых дорожных участков и зарегистрированных инцидентов. На карте необходимо предусмотреть следующие типы маркеров:

- камеры дорожного наблюдения;
- зарегистрированные инциденты;
- участки с повышенной загруженностью;
- участки с нормальной дорожной обстановкой.

Для обозначения загруженности дорог рекомендуется использовать цветовую схему:

- зелёный — свободное движение;
- жёлтый — умеренная загруженность;
- оранжевый — высокая загруженность;
- красный — критическая загруженность.

При выборе камеры должна отображаться информационная карточка с её названием, адресом, статусом и возможностью перехода к просмотру изображения. При выборе маркера инцидента должна открываться подробная информация о событии. Предусмотреть фильтрацию маркеров по типу и статусу. Для реализации интерактивной карты допускается использовать Leaflet или аналогичную библиотеку. При отсутствии доступа к внешним картографическим сервисам разрешается использовать подготовленную схему города с интерактивными маркерами. Координаты камер и инцидентов должны храниться в базе данных.

Реализовать раздел управления камерами наблюдения с возможностью просмотра, добавления, редактирования и поиска записей. Для каждой камеры предусмотреть:

- уникальный идентификатор;
- название;
- адрес установки;
- контролируемый участок;
- географические координаты;
- статус подключения;
- дату последнего обновления;
- демонстрационное изображение.

Предусмотреть статусы камер «Онлайн», «Офлайн» и «Обслуживание». Реализовать справочник контролируемых дорожных участков. Для каждого участка хранить название, описание, тип дороги, местоположение, допустимую скорость и связанные камеры. В разделе транспорта предусмотреть просмотр статистики по категориям транспортных средств, зарегистрированных на контролируемых участках.

Реализовать формирование отчётной информации за выбранный период. Отчёт должен содержать:

- общее количество зарегистрированных транспортных средств;
- среднюю скорость движения;
- распределение транспорта по категориям;
- количество и типы дорожных инцидентов;
- сведения о наиболее загруженных участках;
- динамику изменения интенсивности движения;
- информацию о состоянии камер наблюдения.

Предусмотреть экспорт табличных данных в CSV. Уведомления должны формироваться при регистрации нового инцидента, обнаружении критической загруженности участка, изменении статуса события и переходе камеры в состояние «Офлайн». Для каждого уведомления предусмотреть дату, тип, краткое описание и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу с логотипом TrafficAI, названием приложения и изображением городской дорожной инфраструктуры. Предусмотреть кнопки быстрого доступа:

- «Камеры»;
- «Карта»;
- «Инциденты»;
- «Отчёты»;
- «Мои задачи»;
- «Настройки».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Карта», «События», «Профиль». Для раздела инцидентов предусмотреть индикатор количества новых событий.

Разработать экран просмотра доступных камер дорожного наблюдения. В верхней части разместить строку поиска по названию камеры или адресу. Для каждой камеры отображать:

- миниатюру дорожной ситуации;
- название или номер камеры;
- адрес установки;
- статус подключения;
- краткое описание контролируемого участка.

Предусмотреть возможность выбора камеры и перехода к подробному просмотру. Список камер должен загружаться с сервера.

Разработать экран просмотра выбранной камеры. Основную область должен занимать снимок дорожной ситуации с визуальными обозначениями обнаруженных транспортных средств, пешеходов и событий. В верхней части отображать название камеры и статус подключения. В нижней части предусмотреть элементы управления:

- «Снимок»;
- «Live»;
- «События».

Кнопка «Снимок» должна обеспечивать сохранение или получение демонстрационного кадра. Режим «Live» может быть реализован как последовательное отображение заранее подготовленных изображений. Кнопка «События» должна открывать список инцидентов, связанных с выбранной камерой.

Разработать экран подробной информации о выбранном дорожном инциденте. Отображать:

- изображение события;
- тип нарушения;
- дату и время;
- камеру;
- местоположение;
- тип транспортного средства;
- зафиксированную скорость при наличии;
- уровень важности;
- статус обработки.

Для примера превышения скорости предусмотреть визуальное выделение зафиксированного значения. Разместить кнопку «Добавить в отчёт». При нажатии кнопки событие должно включаться в пользовательскую подборку для последующего формирования отчёта. Дополнительно предусмотреть возможность изменения статуса инцидента пользователем с соответствующими правами.

Разработать мобильный экран карты города с отображением камер и дорожных инцидентов. Предусмотреть масштабирование, перемещение карты и выбор маркеров. При выборе камеры отображать её название, адрес и статус. При выборе инцидента — тип события, дату и краткое описание. Использовать цветовое обозначение загруженности дорожных участков. В нижней части экрана разместить кнопку «Список камер» для перехода к перечню доступных устройств наблюдения. Информация о маркерах должна поступать с сервера.

Разработать экран персональной статистики оператора или инспектора. В верхней части предусмотреть переключение периода: «День», «Неделя», «Месяц». Отобразить следующие показатели:

- количество зарегистрированных транспортных средств;
- количество дорожных инцидентов;
- среднюю скорость движения;
- долю активных камер.

Для каждого показателя предусмотреть тематическую иконку, текущее значение и динамику изменения относительно предыдущего периода. Дополнительно допускается отображать количество инцидентов, проверенных или обработанных текущим пользователем. Значения должны формироваться на сервере на основе данных выбранного периода.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, учётные данные
RoadSegments	Контролируемые участки дорог, адреса, координаты, допустимая скорость
Cameras	Камеры, местоположение, статус, контролируемый участок
VehicleTypes	Категории транспортных средств
TrafficObservations	Результаты наблюдений, типы транспорта, количество, скорость, время
TrafficIncidents	Дорожные события, категории, местоположение, статусы
TrafficLoad	Показатели загруженности участков за определённые периоды
Reports	Сформированные отчёты и их параметры
ReportIncidents	Связи между отчётами и включёнными инцидентами
Notifications	Уведомления и отметки прочтения
Media	Изображения и сведения о медиафайлах

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление камерами и контролируемыми дорожными участками;
- получение изображений камер;
- регистрацию и просмотр дорожных инцидентов;
- изменение статусов событий;
- получение информации о транспортных потоках;
- получение данных о загруженности дорог;
- отображение камер и инцидентов на карте;

- получение уведомлений;
- формирование аналитических данных и отчётов.

Для хранения географического положения камер и событий допускается использовать отдельные поля широты и долготы в PostgreSQL. Применение PostGIS не является обязательным. Предусмотреть разграничение доступа по ролям администратора и оператора. Пароли должны храниться в виде защищённых хешей, а права на изменение данных — проверяться на сервере.

Основные пользовательские сценарии

Сценарий 1. Мониторинг дорожной обстановки.

Оператор открывает веб-приложение и выбирает камеру на контролируемом перекрёстке. Система отображает изображение дорожной ситуации с демонстрационными результатами распознавания транспортных средств и пешеходов. Оператор может перейти к списку событий, связанных с выбранной камерой, и просмотреть подробную информацию.

Сценарий 2. Регистрация и обработка дорожного инцидента.

В системе появляется новое событие «Выезд на встречную полосу». Сервер сохраняет запись об инциденте и формирует уведомление оператору. Оператор открывает карточку события, просматривает изображение, уточняет данные и изменяет статус на «Подтверждено» или «Отклонено». Результат обработки сохраняется в базе данных и отображается в журнале событий.

Сценарий 3. Анализ загруженности дорог.

Администратор открывает раздел аналитики, выбирает контролируемый участок и временной период. Приложение получает с сервера данные об интенсивности движения, средней скорости и уровне загруженности. Карточки показателей и диаграммы обновляются в соответствии с выбранными параметрами.

Сценарий 4. Просмотр дорожных событий на карте.

Инспектор открывает мобильное приложение и переходит в раздел «Карта». На экране отображаются контролируемые участки, камеры и зарегистрированные инциденты. Пользователь выбирает маркер события, просматривает его описание и переходит к подробной карточке.

Сценарий 5. Формирование отчёта по инцидентам.

Оператор открывает список дорожных событий, применяет фильтры по дате, типу и контролируемому участку. После выбора необходимых записей пользователь

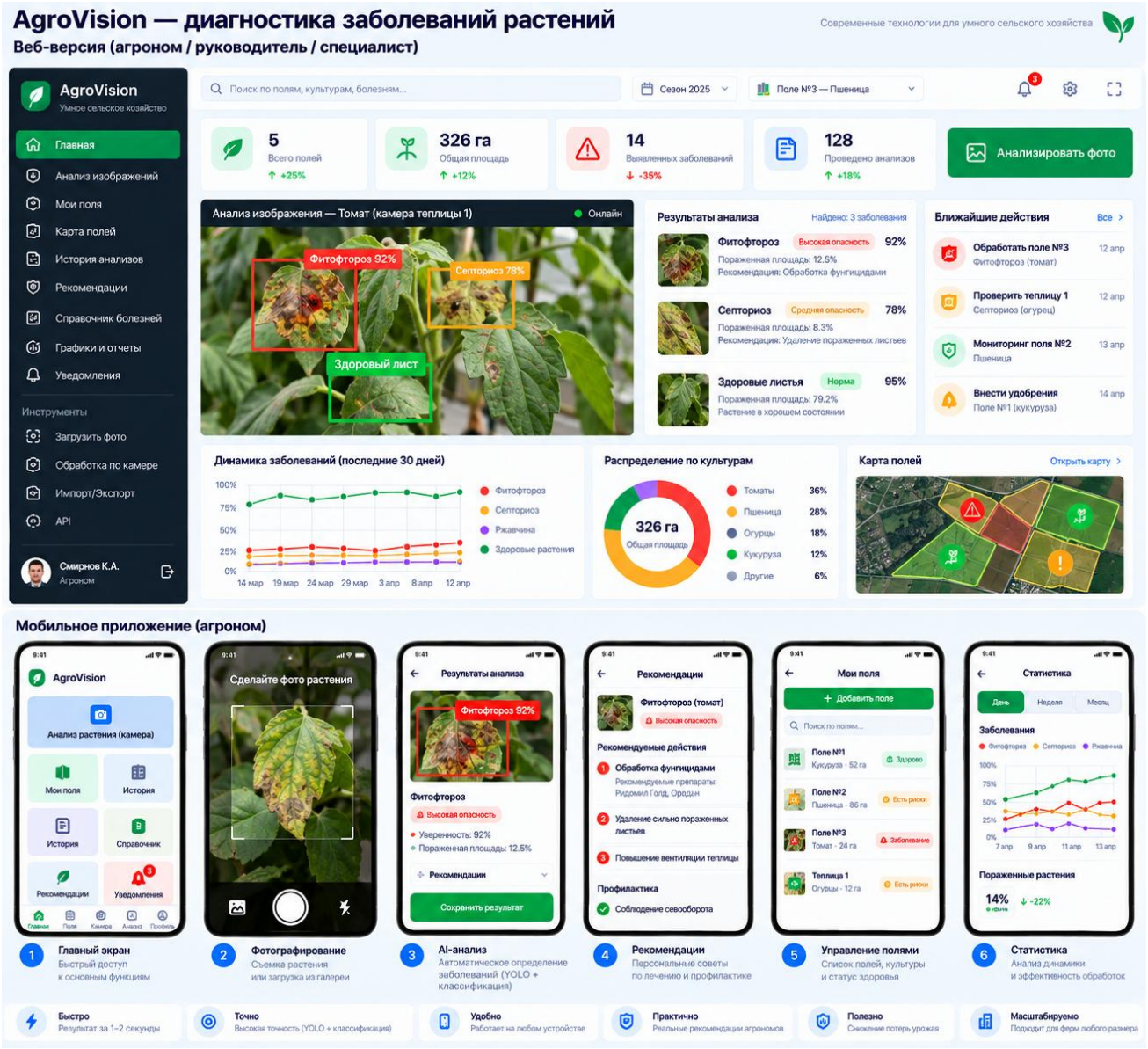
добавляет их в отчёт. Сервер сохраняет состав отчёта и формирует сводную информацию, доступную для просмотра и экспорта в CSV.

Сценарий 6. Изменение состояния камеры.

Администратор открывает раздел управления камерами и изменяет статус выбранного устройства на «Офлайн». Сервер сохраняет изменение, а приложение обновляет количество активных камер на Dashboard. В журнале уведомлений появляется сообщение об изменении состояния устройства.

Вариант 4. AgroVision — система интеллектуальной диагностики заболеваний растений

Разработать Fullstack веб-приложение **AgroVision**, предназначенное для мониторинга состояния сельскохозяйственных культур, диагностики заболеваний растений по изображениям, регистрации выявленных проблем, формирования рекомендаций по обработке растений и анализа фитосанитарного состояния сельскохозяйственных угодий.



Система должна обеспечивать централизованное представление информации о сельскохозяйственных полях, выращиваемых культурах, результатах обследований, выявленных заболеваниях и проведённых агротехнических мероприятиях, а также предоставлять инструменты для своевременного обнаружения проблем и принятия решений по защите растений.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для агронома, руководителя и специалиста** — предназначена для мониторинга состояния сельскохозяйственных угодий, анализа изображений растений, просмотра результатов диагностики, управления полями и

культурами, планирования мероприятий по защите растений и формирования аналитических отчётов.
- **Мобильная версия для агронома** — предназначена для фотографирования растений, получения результатов диагностики, просмотра рекомендаций по лечению и профилактике заболеваний, управления списком полей, изучения истории обследований и просмотра статистики.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт AgroVision, сохранив его основные композиционные решения, структуру навигации, информационные блоки и цветовую схему с преобладанием зелёных оттенков.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть разделы:

- Главная;
- Анализ изображений;
- Мои поля;
- Карта полей;
- История анализов;
- Рекомендации;
- Справочник болезней;
- Графики и отчёты;
- Уведомления.

В дополнительной группе инструментов разместить пункты «Загрузить фото», «Обработка по камере», «Импорт/Экспорт» и «API». В верхней панели расположить строку поиска по полям, культурам и заболеваниям, выбор сельскохозяйственного сезона, выпадающий список выбора поля, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Всего полей** — количество зарегистрированных сельскохозяйственных полей.
2. **Общая площадь** — суммарная площадь контролируемых сельскохозяйственных угодий в гектарах.
3. **Выявленных заболеваний** — количество зарегистрированных случаев обнаружения заболеваний растений за выбранный период.
4. **Проведено анализов** — общее количество выполненных диагностических исследований изображений растений.

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL.

Реализовать центральный блок анализа изображений сельскохозяйственных культур. В верхней части блока отображать название культуры, источник изображения, дату и время выполнения анализа, а также текущий статус обработки. Основная область должна содержать фотографию растения с визуальными обозначениями

обнаруженных заболеваний и здоровых участков. В соответствии с графическим концептом предусмотреть следующие типы обозначений:

- красная рамка с подписью «Фитофтороз» — участок растения с признаками фитофтороза;
- оранжевая рамка с подписью «Септориоз» — участок с признаками септориоза;
- зелёная рамка с подписью «Здоровый лист» — участок без выявленных признаков заболевания.

Для каждого обнаруженного заболевания предусмотреть отображение условной оценки уверенности распознавания в процентах. Реализовать возможность загрузки изображения с компьютера и выбора ранее сохранённой фотографии. После загрузки изображения пользователь должен иметь возможность запустить анализ и получить демонстрационные результаты распознавания.

Для учебной реализации допускается использовать заранее подготовленные изображения растений с соответствующими результатами классификации. Разработка собственной нейронной сети и реальной модели компьютерного зрения не является обязательной. При этом результаты анализа должны сохраняться в базе данных и использоваться для формирования истории обследований, рекомендаций и аналитических показателей.

Разработать раздел просмотра результатов анализа растений. Для каждого результата хранить и отображать:

- уникальный идентификатор анализа;
- дату и время проведения;
- название сельскохозяйственной культуры;
- поле или участок обследования;
- исходное изображение;
- обнаруженное заболевание;
- оценку уверенности распознавания;
- предполагаемую степень поражения;
- уровень опасности;
- краткое описание заболевания;
- рекомендации по дальнейшим действиям.

Предусмотреть категории результатов:

- обнаружено заболевание;
- обнаружены признаки нескольких заболеваний;
- растение здорово;
- результат требует дополнительной проверки.

Для заболеваний предусмотреть уровни опасности «Низкий», «Средний» и «Высокий». При обнаружении заболевания система должна отображать его название, характерные признаки, предполагаемую степень поражения и рекомендации по обработке. Если на одном изображении обнаружено несколько проблем, каждая из них должна отображаться отдельной записью в результатах анализа. Реализовать возможность сохранения результата, добавления комментария и перехода к подробной информации о заболевании. Результаты автоматической или имитированной диагностики должны

рассматриваться как предварительные и не заменять профессиональную оценку агронома.

Разработать раздел хранения и просмотра ранее выполненных диагностических исследований. История должна представлять собой таблицу или список с отображением даты анализа, культуры, поля, выявленного заболевания, уровня опасности и статуса. Предусмотреть поиск и фильтрацию по:

- сельскохозяйственной культуре;
- полю;
- типу заболевания;
- дате или периоду;
- уровню опасности;
- результату анализа.

При выборе записи должна открываться подробная карточка с исходной фотографией, результатами диагностики и сформированными рекомендациями. Предусмотреть возможность сравнения нескольких обследований одного поля, выполненных в разные даты. Это позволит отслеживать изменения состояния растений и оценивать результаты проведённых мероприятий.

Разработать раздел статистики, содержащий несколько взаимосвязанных аналитических блоков.

Динамика заболеваний. Линейный график за последние 30 дней, отображающий изменение количества или доли зарегистрированных случаев фитофтороза, септориоза, ржавчины и других заболеваний. Отдельной линией предусмотреть отображение доли здоровых растений.

Распределение по культурам. Круговая диаграмма, показывающая распределение общей площади сельскохозяйственных угодий между выращиваемыми культурами: томатами, пшеницей, огурцами, кукурузой и другими.

Статистика поражения растений. Диаграмма с отображением доли обследованных растений, для которых выявлены признаки заболеваний.

Заболевания по полям. Таблица или диаграмма, позволяющая сравнить количество зарегистрированных случаев заболеваний на различных полях.

Эффективность обработок. График изменения состояния растений до и после проведения зарегистрированных мероприятий по защите культур.

Все показатели должны формироваться на основе данных, полученных через REST API. При изменении сезона, поля, культуры или временного периода аналитическая информация должна обновляться.

Реализовать раздел «Мои поля» с возможностью просмотра, добавления, редактирования и поиска сельскохозяйственных участков. Для каждого поля предусмотреть:

- уникальный идентификатор;
- название или номер;

- местоположение;
- площадь в гектарах;
- выращиваемую культуру;
- сельскохозяйственный сезон;
- дату посева или посадки;
- ответственного агронома;
- текущий фитосанитарный статус;
- описание и дополнительные сведения.

Предусмотреть следующие статусы состояния поля:

- «Здорово» — при обследованиях не выявлено признаков заболеваний;
- «Есть риски» — обнаружены отдельные признаки возможных заболеваний;
- «Заболевание» — зарегистрированы подтверждённые или требующие обработки случаи поражения растений.

Для каждого поля реализовать просмотр истории обследований, обнаруженных заболеваний, рекомендаций и проведённых обработок. Предусмотреть возможность изменения текущей культуры и сельскохозяйственного сезона.

Разработать раздел «Карта полей», предназначенный для визуального отображения расположения сельскохозяйственных угодий и их текущего состояния. Каждое поле должно отображаться на карте в виде полигона или условного прямоугольного участка. В соответствии с графическим концептом предусмотреть цветовое обозначение:

- зелёный — поле находится в удовлетворительном состоянии;
- жёлтый или оранжевый — обнаружены риски или отдельные признаки заболеваний;
- красный — выявлено заболевание, требующее внимания.

На карте должны отображаться тематические маркеры, обозначающие состояние контролируемых участков. При выборе поля должна открываться информационная карточка с названием, культурой, площадью, текущим статусом и датой последнего обследования. Предусмотреть возможность перехода к подробной информации о поле.

Для реализации карты допускается использовать Leaflet или аналогичную библиотеку. При отсутствии доступа к внешним картографическим сервисам разрешается использовать подготовленное изображение территории с интерактивными областями. Координаты полей должны храниться в базе данных.

Разработать раздел рекомендаций по лечению и профилактике заболеваний растений. Для каждого заболевания предусмотреть:

- название;
- описание характерных признаков;
- причины и условия возникновения;
- перечень возможных профилактических мероприятий;
- рекомендуемые агротехнические действия;
- дополнительные сведения.

Рекомендации должны формироваться на основе заранее подготовленного справочника заболеваний и соответствующих мероприятий. Предусмотреть возможность создания задачи или запланированного мероприятия на основе полученной рекомендации. Для мероприятия хранить название, поле, связанную проблему, описание, ответственного сотрудника, плановую дату выполнения и статус. В соответствии с графическим концептом реализовать блок «Ближайшие действия», содержащий перечень запланированных мероприятий:

- обработка поля;
- проверка теплицы;
- повторный мониторинг;
- внесение удобрений;
- другие агротехнические мероприятия.

Предусмотреть статусы мероприятий «Запланировано», «В работе», «Выполнено» и «Отменено». Конкретные средства защиты растений и нормы их применения не должны автоматически назначаться системой без подтверждения специалистом.

Разработать справочник заболеваний растений с возможностью просмотра, поиска и фильтрации записей. Для каждого заболевания предусмотреть название, поражаемые культуры, описание симптомов, изображения характерных признаков, уровень опасности и рекомендации по профилактике. Реализовать формирование отчётов за выбранный период.

Отчёт должен включать:

- количество проведённых анализов;
- количество выявленных заболеваний;
- распределение заболеваний по культурам;
- сведения о состоянии полей;
- динамику выявления заболеваний;
- перечень проведённых мероприятий.

Предусмотреть экспорт табличных данных в CSV. Уведомления должны формироваться при обнаружении заболевания с высоким уровнем опасности, назначении мероприятия, приближении даты запланированной обработки и изменении статуса поля. Для каждого уведомления предусмотреть дату, тип, краткое описание и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте. Разработать стартовую страницу с логотипом AgroVision, названием приложения и кнопками быстрого доступа:

- «Анализ растения (камера)»;
- «Мои поля»;
- «История»;
- «Справочник»;
- «Рекомендации»;
- «Уведомления».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Поля», «Камера», «Анализ», «Профиль». Для уведомлений предусмотреть индикатор количества непрочитанных сообщений.

Фотографирование растения

Разработать экран получения изображения растения для последующего анализа. Основную область должен занимать интерфейс камеры с областью кадрирования листа или другого исследуемого участка растения. Предусмотреть следующие элементы управления:

- создание фотографии;
- выбор изображения из галереи;
- переключение режима камеры при наличии технической возможности;
- запуск анализа выбранного изображения.

После создания фотографии пользователь должен иметь возможность подтвердить снимок или выполнить повторное фотографирование. Для учебной реализации допускается использовать загрузку изображения из файловой системы вместо непосредственного подключения к камере устройства. Загруженное изображение должно передаваться на сервер для сохранения и демонстрационной обработки.

Разработать экран отображения результатов диагностики растения. В верхней части вывести исходную фотографию с визуальным выделением участка, на котором обнаружены признаки заболевания. Ниже отобразить:

- название обнаруженного заболевания;
- уровень опасности;
- условную уверенность распознавания;
- предполагаемую степень поражения;
- краткое описание результата.

Предусмотреть возможность раскрытия блока с рекомендациями. Разместить кнопку «Сохранить результат». После сохранения результат должен появляться в истории анализов и быть доступным в веб-версии приложения. Если система не обнаружила признаков заболевания, необходимо отображать соответствующее сообщение о здоровом состоянии растения.

Разработать экран персонализированных рекомендаций на основе выбранного результата диагностики. В верхней части отображать название заболевания, культуру и уровень опасности. Основная область должна содержать перечень рекомендуемых действий. В соответствии с графическим концептом предусмотреть следующие группы рекомендаций:

Рекомендуемые действия:

- обработка растений подходящими средствами защиты после согласования со специалистом;
- удаление сильно поражённых листьев при допустимости такого мероприятия;
- улучшение вентиляции теплицы;
- повторное обследование растений.

Профилактика:

- соблюдение севооборота;
- контроль влажности;
- регулярный осмотр растений;
- выполнение профилактических мероприятий.

Для каждого действия предусмотреть краткое описание. Дополнительно реализовать возможность создания запланированного мероприятия на основе рекомендации.

Разработать мобильный экран «Мои поля». В верхней части разместить кнопку «Добавить поле» и строку поиска. Список полей должен содержать карточки с отображением:

- названия или номера поля;
- выращиваемой культуры;
- площади;
- тематического изображения;
- текущего фитосанитарного статуса.

Предусмотреть визуальное выделение полей со статусами «Здорово», «Есть риски» и «Заболевание». При выборе поля должна открываться подробная информация с историей обследований, зарегистрированными заболеваниями и запланированными мероприятиями. Реализовать возможность добавления нового поля и редактирования основных сведений. Все изменения должны сохраняться на сервере.

Разработать экран просмотра статистики состояния сельскохозяйственных культур. В верхней части предусмотреть переключение периода: «День», «Неделя», «Месяц». Отобразить линейный график динамики выявленных заболеваний с отдельными линиями для фитофтороза, септориоза, ржавчины и других категорий. Ниже разместить карточку с показателем доли поражённых растений и динамикой изменения относительно предыдущего периода. Дополнительно предусмотреть отображение количества выполненных анализов и распределения результатов по категориям. Данные должны загружаться с сервера и обновляться при изменении выбранного периода.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, учётные данные
Fields	Поля, площади, координаты, статусы
Crops	Сельскохозяйственные культуры и их характеристики
Seasons	Сельскохозяйственные сезоны
Diseases	Заболевания, симптомы, уровни опасности
Analyses	Проведённые анализы изображений
AnalysisResults	Результаты диагностики, заболевания, уверенность распознавания, степень поражения
Recommendations	Рекомендации по лечению и профилактике
FieldTreatments	Запланированные и выполненные агротехнические мероприятия
Notifications	Уведомления и отметки прочтения

Media	Фотографии растений и сведения о медиафайлах

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление полями и культурами;
- загрузку изображений растений;
- запуск демонстрационного анализа;
- получение и сохранение результатов диагностики;
- просмотр истории анализов;
- получение информации о заболеваниях;
- формирование рекомендаций;
- управление агротехническими мероприятиями;
- получение информации о полях для отображения на карте;
- получение уведомлений;
- формирование аналитических данных и отчётов.

Для хранения географического положения полей допускается использовать координаты вершин полигонов в формате JSON или отдельной связанной таблице. Применение PostGIS не является обязательным. Предусмотреть разграничение доступа по ролям администратора, агронома и специалиста. Пароли должны храниться в виде защищённых хешей, а права на изменение данных — проверяться на сервере.

Основные пользовательские сценарии

Сценарий 1. Диагностика заболевания растения.

Агроном открывает мобильное приложение, выбирает функцию «Анализ растения» и фотографирует лист или загружает подготовленное изображение. Приложение отправляет изображение на сервер. Сервер выполняет демонстрационную обработку и возвращает результат с названием предполагаемого заболевания, уровнем опасности и условной оценкой уверенности. Мобильное приложение отображает фотографию с выделенной областью поражения и подробной информацией о результате. После подтверждения пользователь сохраняет анализ в базе данных.

Сценарий 2. Получение рекомендаций.

После диагностики фитофтороза агроном открывает раздел рекомендаций. Система отображает описание заболевания, возможные причины возникновения и перечень рекомендуемых мероприятий. Агроном выбирает подходящее действие и создаёт запланированное мероприятие для соответствующего поля. Информация сохраняется на сервере и появляется в списке ближайших действий веб-версии.

Сценарий 3. Управление состоянием полей.

Агроном открывает раздел «Мои поля», выбирает сельскохозяйственный участок и просматривает его текущий статус. При обнаружении заболевания пользователь добавляет результат обследования к выбранному полю. Сервер сохраняет информацию

и пересчитывает показатели состояния поля по установленным правилам. Обновлённый статус отображается в списке полей и на интерактивной карте.

Сценарий 4. Анализ фитосанитарной обстановки.

Руководитель открывает веб-приложение, выбирает сельскохозяйственный сезон и контролируемое поле. Система получает с сервера информацию о количестве выполненных анализов, зарегистрированных заболеваниях и состоянии растений. Карточки показателей, графики и диаграммы обновляются в соответствии с выбранными параметрами.

Сценарий 5. Мониторинг полей на карте.

Агроном открывает раздел «Карта полей». Приложение отображает расположение сельскохозяйственных участков с цветовым обозначением их текущего состояния. Пользователь выбирает поле с красным индикатором, просматривает информацию о выявленном заболевании и переходит к результатам последнего обследования.

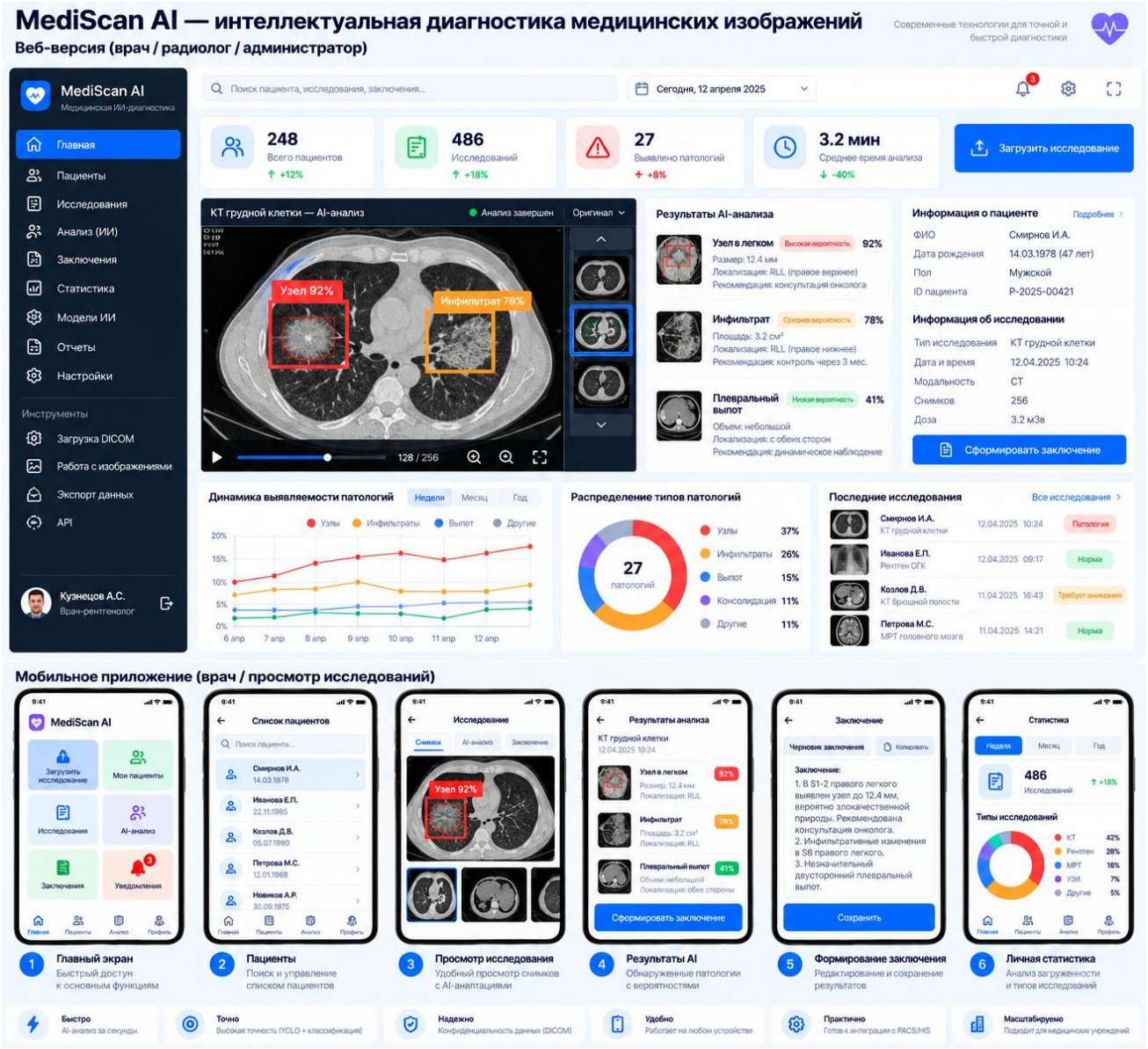
Сценарий 6. Контроль выполнения агротехнического мероприятия.

Специалист открывает список запланированных мероприятий и выбирает задачу по обработке поля. После выполнения работ пользователь добавляет комментарий, указывает дату проведения и изменяет статус мероприятия на «Выполнено». Сервер сохраняет изменения, а информация становится доступной в веб-панели руководителя.

Последующее изменение фитосанитарного статуса поля должно основываться на результатах повторного обследования, а не только на факте выполнения мероприятия.

Вариант 5. MediScan AI — система интеллектуального анализа медицинских изображений

Разработать Fullstack веб-приложение **MediScan AI**, предназначенное для организации хранения, просмотра и анализа медицинских изображений, ведения электронного реестра пациентов и диагностических исследований, регистрации предполагаемых патологических изменений и формирования медицинских заключений.



Система должна обеспечивать централизованное представление информации о пациентах, проведённых исследованиях, медицинских изображениях, результатах автоматизированного анализа и заключениях специалистов, а также предоставлять инструменты для удобного просмотра диагностических материалов и анализа статистических показателей медицинской организации.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для врача, рентгенолога и администратора** — предназначена для управления пациентами и исследованиями, просмотра медицинских изображений, анализа результатов демонстрационного AI-распознавания, подготовки диагностических заключений, ведения истории исследований и формирования отчётов.
- **Мобильная версия для врача** — предназначена для оперативного поиска пациентов, просмотра медицинских исследований и результатов анализа, подготовки и сохранения заключений, получения уведомлений и просмотра персональной статистики.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт MediScan AI, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации медицинских данных.

Приложение разрабатывается в учебных целях. Для его демонстрации необходимо использовать обезличенные, синтетические или специально подготовленные учебные медицинские данные и изображения.

Результаты интеллектуального анализа должны иметь демонстрационный характер и не должны использоваться для постановки реальных медицинских диагнозов или принятия клинических решений.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть разделы:

- Главная;
- Пациенты;
- Исследования;
- Анализ (ИИ);
- Заключения;
- Статистика;
- Модели ИИ;
- Отчёты;
- Настройки.

В дополнительной группе инструментов разместить пункты «Загрузка DICOM», «Работа с изображениями», «Экспорт данных» и «API». В верхней панели расположить строку поиска по пациентам, исследованиям и заключениям, выбор даты или периода, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Всего пациентов** — количество пациентов, зарегистрированных в учебной информационной системе.
2. **Исследований** — общее количество диагностических исследований за выбранный период.
3. **Выявлено патологий** — количество зарегистрированных предполагаемых патологических находок по результатам анализов.
4. **Среднее время анализа** — средняя продолжительность обработки исследования демонстрационным модулем анализа.

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL.

Реализовать центральный блок просмотра медицинских изображений. В верхней части блока отображать название исследования, тип исследования, статус анализа и доступные режимы просмотра. Основная область должна содержать медицинское изображение, например компьютерную томограмму грудной клетки, с возможностью отображения визуальных обозначений предполагаемых патологических изменений. В соответствии с графическим концептом предусмотреть следующие типы обозначений:

- красная рамка с подписью «Узел» — область предполагаемого узлового образования;
- оранжевая рамка с подписью «Инфильтрат» — область предполагаемых инфильтративных изменений;
- дополнительные обозначения для других категорий находок, предусмотренных учебным набором данных.

Для каждой выделенной области предусмотреть отображение условной оценки уверенности модели в процентах. В нижней части просмотрщика разместить элементы управления:

- переходом между изображениями или срезами;
- масштабированием;
- полноэкранным режимом;
- включением и отключением аннотаций;
- выбором режима отображения.

При работе с сериями изображений предусмотреть панель миниатюр с возможностью выбора отдельного среза. Для учебной реализации допускается использовать заранее подготовленные изображения компьютерной томографии, рентгенографии или магнитно-резонансной томографии. Поддержка полноценного диагностического DICOM-просмотрщика не является обязательной. Допускается отображение предварительно преобразованных изображений в форматах PNG или JPEG. При реализации загрузки DICOM необходимо предусмотреть проверку формата файла и исключить использование реальных идентифицирующих сведений пациентов.

Разработать раздел просмотра результатов демонстрационного AI-анализа медицинских изображений. Для каждого результата хранить и отображать:

- уникальный идентификатор;
- связанное исследование;
- дату и время анализа;
- тип обнаруженной находки;
- анатомическую локализацию;
- размеры или площадь выделенной области при наличии;
- условную оценку уверенности модели;
- краткое описание;
- визуальную аннотацию;
- статус проверки специалистом.

В соответствии с графическим концептом предусмотреть следующие категории находок:

- узловые образования;
- инфильтративные изменения;
- плевральный выпот;
- консолидация;
- другие демонстрационные категории.

Результаты должны отображаться в виде списка карточек с миниатюрами изображений, названиями находок, их характеристиками и цветными индикаторами. Для условной оценки уверенности модели предусмотреть визуальные категории «Высокая», «Средняя» и «Низкая». Важно учитывать, что уверенность модели не является достоверной оценкой вероятности заболевания. В учебном приложении она используется исключительно для демонстрации интерфейса. При выборе результата необходимо выделять соответствующую область на медицинском изображении. Предусмотреть возможность просмотра подробной информации, добавления комментария специалиста и изменения статуса проверки. Для результатов анализа использовать статусы «Не проверено», «Подтверждено специалистом» и «Отклонено специалистом».

Реализовать раздел «Пациенты» с возможностью просмотра, добавления, редактирования и поиска записей. Для каждого пациента предусмотреть:

- внутренний идентификатор;
- ФИО или условное обозначение;
- дату рождения;
- пол;
- контактные сведения при необходимости;
- дату регистрации;
- перечень связанных исследований.

В учебном проекте все сведения должны быть вымышленными или обезличенными. Предусмотреть поиск пациентов по ФИО или идентификатору. При выборе пациента должна открываться подробная карточка с основными сведениями и историей диагностических исследований. Реализовать возможность перехода от карточки пациента к просмотру выбранного исследования.

Разработать раздел регистрации и просмотра медицинских исследований. Для каждого исследования хранить:

- уникальный идентификатор;
- пациента;
- тип исследования;
- дату и время проведения;
- исследуемую анатомическую область;
- модальность;
- количество изображений или срезов;
- сведения о загруженных файлах;
- статус обработки;
- ответственного специалиста.

Предусмотреть следующие типы исследований:

- компьютерная томография (КТ);
- рентгенография;
- магнитно-резонансная томография (МРТ);
- ультразвуковое исследование (УЗИ);
- другие типы при необходимости.

Для статуса исследования предусмотреть значения «Загружено», «Ожидает анализа», «Анализ выполнен», «Заключение подготовлено». Реализовать загрузку изображений, просмотр списка исследований и фильтрацию по пациенту, модальности, дате и статусу. При выборе исследования должна открываться страница с медицинским просмотрщиком, результатами анализа и информацией о пациенте.

Разработать раздел подготовки и хранения заключений по диагностическим исследованиям. В соответствии с графическим концептом предусмотреть возможность формирования черновика заключения на основе результатов демонстрационного AI-анализа. Черновик должен содержать:

- сведения об исследовании;
- краткое описание изображений;
- перечень предполагаемых находок;
- их локализацию и характеристики;
- текстовое поле заключения;
- дополнительные комментарии специалиста.

Реализовать текстовый редактор с возможностью изменения автоматически подготовленного текста. Предусмотреть кнопки «Сформировать черновик», «Сохранить» и «Копировать». Заключение должно связываться с соответствующим пациентом, исследованием и автором. Для заключений предусмотреть статусы «Черновик», «На проверке» и «Подтверждено специалистом». Демонстрационная система не должна автоматически присваивать заключению статус подтверждённого без действия уполномоченного пользователя. Формирование текста должно основываться на подготовленных шаблонах и учебных данных. Реальная медицинская интерпретация изображений не требуется.

Разработать раздел статистики, содержащий несколько взаимосвязанных аналитических блоков.

Динамика выявляемости патологических находок. Линейный график за выбранный период, отображающий изменение количества или доли зарегистрированных находок различных категорий: узлов, инфильтратов, выпотов и других.

Распределение типов патологических находок. Круговая диаграмма с отображением долей различных категорий зарегистрированных результатов.

Типы исследований. Круговая или столбчатая диаграмма, показывающая распределение исследований по модальностям: КТ, рентгенография, МРТ, УЗИ и другим.

Количество выполненных исследований. График изменения количества исследований по дням, неделям или месяцам.

Среднее время обработки. Показатель средней продолжительности выполнения демонстрационного анализа.

Статистика заключений. Количество подготовленных, проверяемых и подтверждённых специалистами заключений.

Все аналитические показатели должны формироваться на основе данных, полученных через REST API. При изменении временного периода или типа исследования информация на графиках должна обновляться.

Реализовать демонстрационный раздел «Модели ИИ». Раздел должен содержать список доступных моделей или режимов анализа медицинских изображений. Для каждой модели предусмотреть:

- название;
- версию;
- поддерживаемые типы исследований;
- перечень определяемых категорий;
- дату добавления;
- текущий статус.

Предусмотреть статусы «Активна», «Неактивна» и «Тестирование». Администратор должен иметь возможность выбирать активную демонстрационную модель для определённого типа исследования. Реальное обучение, дообучение и развёртывание нейронных сетей не являются обязательными. Для учебной реализации допускается использовать заранее подготовленные результаты, связанные с выбранной моделью и демонстрационным исследованием.

Реализовать формирование отчётной информации за выбранный период. Отчёт должен включать:

- количество зарегистрированных пациентов;
- количество проведённых исследований;
- распределение исследований по модальностям;
- количество зарегистрированных патологических находок;
- распределение находок по категориям;
- среднее время обработки;
- количество подготовленных заключений.

Предусмотреть экспорт агрегированных табличных данных в CSV без персональных сведений пациентов. Уведомления должны формироваться при загрузке нового исследования, завершении демонстрационного анализа, появлении результата, требующего проверки, и изменении статуса заключения. Для каждого уведомления предусмотреть дату, тип, краткое описание и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу с логотипом MediScan AI, названием приложения и кнопками быстрого доступа:

- «Загрузить исследование»;
- «Мои пациенты»;
- «Исследования»;
- «AI-анализ»;
- «Заключения»;
- «Уведомления».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Пациенты», «Анализ», «Профиль». Для уведомлений предусмотреть индикатор количества непрочитанных сообщений.

Разработать экран просмотра зарегистрированных пациентов. В верхней части разместить строку поиска по ФИО или идентификатору пациента. Для каждой записи отображать:

- условную иконку пациента;
- ФИО;
- дату рождения;
- внутренний идентификатор при необходимости.

Предусмотреть возможность выбора пациента и перехода к списку связанных исследований. Список пациентов должен загружаться с сервера.

Разработать мобильный экран просмотра медицинского исследования. В верхней части отображать название исследования и вкладки:

- «Снимки»;
- «AI-анализ»;
- «Заключение».

Основную область должен занимать медицинский снимок с возможностью увеличения и просмотра выделенных областей. Под изображением предусмотреть горизонтальную ленту миниатюр для переключения между доступными изображениями или срезами. При выборе вкладки «AI-анализ» должна открываться информация о зарегистрированных находках. При выборе вкладки «Заключение» должен отображаться связанный с исследованием текст заключения или его черновик.

Разработать экран просмотра результатов демонстрационного анализа исследования. В верхней части отображать название исследования, дату выполнения анализа и его текущий статус. Ниже представить список обнаруженных находок. Для каждой находки предусмотреть:

- миниатюру изображения;
- название;
- анатомическую локализацию;
- размер или площадь при наличии;
- условную уверенность модели;
- цветовой индикатор.

В соответствии с графическим концептом предусмотреть отображение узлов, инфильтратов и плеврального выпота. Разместить кнопку «Сформировать заключение», позволяющую перейти к подготовке черновика. Результаты анализа должны загружаться через серверное API.

Разработать мобильный экран редактирования диагностического заключения. В верхней части разместить название раздела и кнопки «Черновик заключения» и «Копировать». Основная область должна содержать редактируемое текстовое поле с предварительно сформированным заключением. Текст может включать описание исследования, перечень обнаруженных находок и итоговую формулировку, предназначенную для проверки специалистом.

В нижней части разместить кнопку «Сохранить». После сохранения заключение должно появляться в истории соответствующего исследования и быть доступным в веб-версии приложения. Предусмотреть проверку пустого текста и отображение сообщений об успешном сохранении или ошибке.

Разработать экран персональной статистики врача. В верхней части предусмотреть переключение периода: «Неделя», «Месяц», «Год». Отобразить следующие показатели:

- количество обработанных исследований;
- динамику изменения относительно предыдущего периода;
- распределение исследований по типам;
- количество подготовленных заключений;
- количество результатов, ожидающих проверки.

Для распределения исследований по модальностям использовать круговую диаграмму. Дополнительно допускается отображать среднее время обработки исследований текущим пользователем. Значения должны формироваться на сервере на основе данных выбранного периода и идентификатора текущего пользователя.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Patients	Учебные пациенты, идентификаторы, основные сведения
Studies	Диагностические исследования, типы, даты, статусы
MedicalImages	Изображения, серии, номера срезов, пути к файлам
AIModels	Демонстрационные модели анализа, версии, статусы
AIAnalyses	Выполненные анализы, модель, время обработки
Findings	Обнаруженные находки, локализация, характеристики, уверенность модели
Annotations	Координаты выделенных областей на изображениях
Conclusions	Заключения, авторы, тексты, статусы
Notifications	Уведомления и отметки прочтения
Reports	Параметры и сведения о сформированных отчётах

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление учебными записями пациентов;
- регистрацию диагностических исследований;

- загрузку и получение медицинских изображений;
- запуск демонстрационного AI-анализа;
- получение результатов анализа;
- управление аннотациями изображений;
- создание и редактирование заключений;
- получение информации о моделях ИИ;
- получение уведомлений;
- формирование статистики и отчётов.

Для хранения изображений рекомендуется использовать файловую систему сервера или объектное хранилище, а в PostgreSQL сохранять метаданные и пути к файлам. Предусмотреть разграничение доступа по ролям администратора, врача и рентгенолога. Пароли должны храниться в виде защищённых хешей, а доступ к данным пациентов и исследованиям — проверяться на сервере. Для учебного проекта необходимо использовать только синтетические или обезличенные данные. Размещение реальных медицинских изображений и персональных медицинских сведений в публичном репозитории не допускается.

Основные пользовательские сценарии

Сценарий 1. Регистрация нового исследования.

Врач открывает веб-приложение, выбирает пациента и создаёт новое диагностическое исследование. Пользователь указывает тип исследования, дату, исследуемую область и загружает подготовленные медицинские изображения. Сервер проверяет данные, сохраняет информацию об исследовании и регистрирует связанные изображения. Новое исследование появляется в списке и становится доступным для просмотра в веб- и мобильной версиях.

Сценарий 2. Демонстрационный AI-анализ медицинского изображения.

Врач открывает исследование компьютерной томографии и запускает функцию AI-анализа. Сервер получает связанные с исследованием подготовленные результаты демонстрационной обработки и сохраняет сведения об обнаруженных находках. Приложение отображает медицинское изображение с выделенными областями, названиями находок и условными оценками уверенности модели. Результаты появляются в разделе анализа и сохраняются в истории исследования.

Сценарий 3. Проверка результатов специалистом.

Рентгенолог открывает исследование, просматривает изображение и список предполагаемых патологических находок. Специалист выбирает отдельную находку, изучает её характеристики и добавляет комментарий. После проверки пользователь изменяет статус результата на «Подтверждено специалистом» или «Отклонено специалистом».

Информация сохраняется в базе данных и становится доступной в обеих версиях приложения.

Сценарий 4. Формирование диагностического заключения.

Врач открывает результаты анализа и нажимает «Сформировать заключение». Приложение создаёт черновик на основе подготовленного шаблона и сохранённых результатов исследования. Врач редактирует текст, уточняет формулировки и сохраняет заключение. Сервер связывает заключение с пациентом, исследованием и автором. Сохранённый документ становится доступным в истории исследования.

Сценарий 5. Просмотр истории исследований пациента.

Врач открывает мобильное приложение и переходит в раздел «Пациенты». После выбора пациента система отображает список связанных диагностических исследований. Пользователь выбирает исследование, просматривает изображения, результаты анализа и сохранённое заключение. Все сведения загружаются через REST API.

Сценарий 6. Анализ статистики медицинской организации.

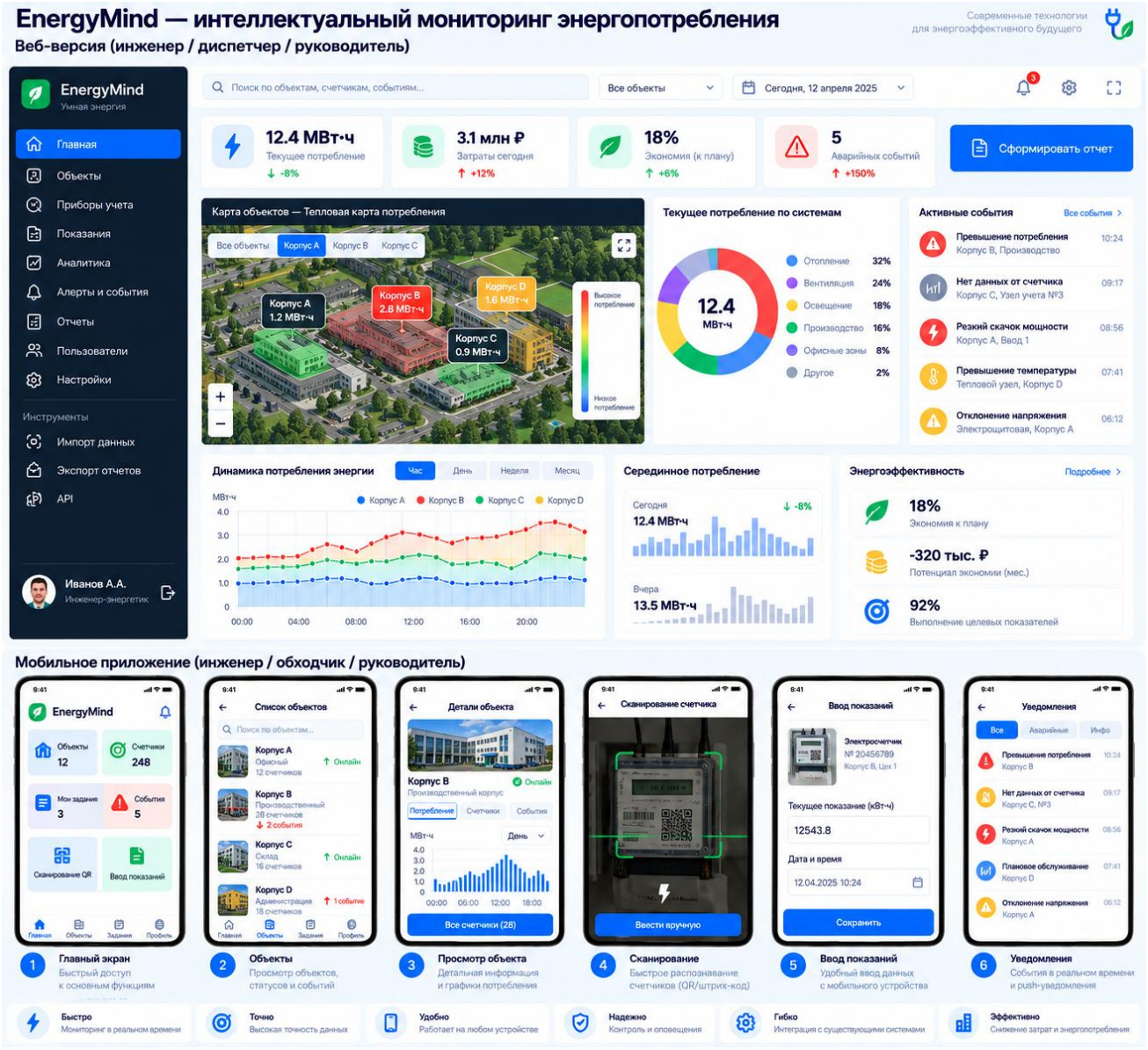
Администратор открывает раздел статистики и выбирает временной период. Система получает с сервера информацию о количестве проведённых исследований, распределении по модальностям, зарегистрированных находках и среднем времени анализа. Карточки показателей, графики и диаграммы обновляются в соответствии с выбранными параметрами.

Сценарий 7. Изменение статуса заключения.

Врач сохраняет черновик заключения и передаёт его на проверку. Уполномоченный специалист открывает документ, проверяет содержание и изменяет статус на «Подтверждено специалистом». Сервер сохраняет изменения, а информация о новом статусе отображается в списке исследований и мобильном приложении.

Вариант 3. EnergyMind — система интеллектуального мониторинга энергопотребления

Разработать Fullstack веб-приложение **EnergyMind**, предназначенное для централизованного мониторинга потребления электрической энергии, контроля состояния приборов учёта, регистрации показаний, выявления отклонений от нормативных или плановых значений, анализа энергоэффективности и формирования отчётной информации по объектам энергетической инфраструктуры.



Система должна обеспечивать сбор, хранение, обработку и визуализацию данных об энергопотреблении зданий, производственных корпусов, офисных помещений и других контролируемых объектов, а также предоставлять инструменты для оценки энергетических затрат, обнаружения нештатных ситуаций и принятия решений по повышению энергоэффективности.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для инженера, диспетчера и руководителя** — предназначена для централизованного мониторинга энергопотребления, просмотра состояния объектов и приборов учёта, анализа показателей, регистрации аварийных событий, контроля энергетических затрат и формирования отчётов.
- **Мобильная версия для инженера, обходчика и руководителя** — предназначена для оперативного просмотра объектов, получения информации о состоянии оборудования, сканирования идентификаторов приборов учёта, ручного ввода показаний, получения уведомлений и выполнения назначенных задач.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт EnergyMind, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации энергетических показателей.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть разделы:

- Главная;
- Объекты;
- Приборы учёта;
- Показания;
- Аналитика;
- Алерты и события;
- Отчёты;
- Пользователи;
- Настройки.

В дополнительной группе инструментов разместить пункты «Импорт данных», «Экспорт отчётов» и «API». В верхней панели расположить строку поиска по объектам, приборам учёта и событиям, выпадающий список выбора объекта, выбор даты или периода, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Текущее потребление** — суммарный объём электрической энергии, потреблённой контролируемыми объектами за выбранный период, в МВт·ч.
2. **Затраты сегодня** — расчётная стоимость потреблённой электрической энергии за текущие сутки в рублях.
3. **Экономия (к плану)** — процент отклонения фактического потребления от установленного планового значения.
4. **Аварийных событий** — количество зарегистрированных аварийных и критических событий за выбранный период.

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL. При расчёте энергетических показателей необходимо учитывать выбранный временной период и контролируемые объекты. Для расчёта стоимости использовать тарифы, заданные в системе.

Реализовать центральный блок визуального мониторинга объектов энергетической инфраструктуры. В верхней части блока отображать название раздела «Карта объектов — тепловая карта потребления» и элементы фильтрации по объектам. Основная область должна содержать схему территории предприятия, промышленного комплекса или группы зданий с обозначением контролируемых объектов. В соответствии с графическим концептом предусмотреть отображение нескольких корпусов, например:

- Корпус A;
- Корпус B;
- Корпус C;
- Корпус D.

Для каждого объекта отображать название и текущее энергопотребление в МВт·ч за выбранный период. Предусмотреть цветовое обозначение уровня потребления:

- синий или зелёный — низкое потребление;
- жёлтый — умеренное потребление;
- оранжевый — повышенное потребление;
- красный — высокое или критическое потребление.

Пороговые значения цветовых категорий должны определяться настройками системы. При выборе объекта должна открываться информационная карточка, содержащая:

- название объекта;
- назначение;
- текущее энергопотребление;
- количество подключённых приборов учёта;
- статус объекта;
- количество активных событий.

Предусмотреть возможность перехода к подробной странице объекта. Для учебной реализации допускается использовать заранее подготовленную схему территории или изображение комплекса зданий с интерактивными областями. Реализация полноценной геоинформационной системы не является обязательной. При этом информация об объектах, их координатах на схеме и показателях энергопотребления должна храниться в базе данных.

Разработать аналитический блок, отображающий распределение энергопотребления между основными инженерными и технологическими системами. В соответствии с графическим концептом предусмотреть следующие категории:

- отопление;
- вентиляция;
- освещение;
- производство;
- офисные зоны;
- другие потребители.

Информацию представить в виде круговой диаграммы с отображением процентных долей. В центральной части диаграммы отображать суммарное энергопотребление за выбранный период. При выборе категории должна отображаться дополнительная информация о количестве потреблённой энергии и её доле в общем объёме. Для каждой категории необходимо предусмотреть возможность хранения показателей потребления и связи с соответствующими объектами или приборами учёта. Все значения должны рассчитываться на сервере и передаваться через REST API.

Разработать раздел регистрации и просмотра событий, связанных с работой энергетической инфраструктуры. В соответствии с графическим концептом предусмотреть следующие типы событий:

- превышение установленного лимита энергопотребления;
- отсутствие данных от прибора учёта;
- резкий скачок мощности;
- превышение допустимой температуры;
- отклонение напряжения;
- плановое техническое обслуживание;
- другие нештатные ситуации.

Для каждого события хранить и отображать:

- уникальный идентификатор;
- тип события;
- связанный объект;
- прибор учёта при наличии;
- дату и время регистрации;
- описание;
- уровень важности;
- фактическое значение контролируемого параметра;
- установленное пороговое значение;
- статус обработки.

Предусмотреть уровни важности «Информационный», «Предупреждение», «Критический». Для событий реализовать статусы «Новое», «В работе», «Устранено» и «Закрыто». В главной панели должен отображаться список последних активных событий с тематическими иконками и цветовой индикацией.

При выборе события должна открываться подробная карточка с информацией об объекте, параметрах отклонения и истории обработки. Предусмотреть возможность добавления комментария и изменения статуса события. Для учебной реализации допускается имитировать обнаружение отклонений на основе заранее подготовленных данных и заданных пороговых значений.

Разработать раздел анализа изменения потребления электрической энергии. Основным элементом должен стать линейный график «Динамика потребления энергии». Предусмотреть переключение временного масштаба:

- «Час»;
- «День»;
- «Неделя»;
- «Месяц».

График должен отображать изменение энергопотребления выбранных объектов за указанный период. Для каждого объекта предусмотреть отдельную линию с индивидуальным цветовым обозначением. При наведении на точку графика отображать дату или время, название объекта и значение потребления. Реализовать возможность включения и отключения отдельных рядов данных. Дополнительно предусмотреть фильтрацию по объекту и типу инженерной системы. Данные для построения графиков должны поступать с сервера и формироваться на основе сохранённых показаний приборов учёта. При расчёте потребления за интервал необходимо использовать разность накопительных показаний либо сохранённые интервальные значения.

Разработать блок «Серединное потребление» или «Сравнение потребления», предназначенный для сопоставления энергетических показателей за разные периоды. В соответствии с графическим концептом предусмотреть две информационные карточки:

- «Сегодня» — объём потребления за текущие сутки;
- «Вчера» — объём потребления за предыдущие сутки.

В каждой карточке отображать числовое значение и компактную столбчатую диаграмму распределения потребления по часам. Дополнительно предусмотреть процент изменения потребления относительно предыдущего периода. При необходимости реализовать сравнение текущей и предыдущей недели или месяца. Для расчёта использовать данные, хранящиеся в PostgreSQL.

Разработать аналитический блок «Энергоэффективность». В соответствии с графическим концептом предусмотреть три основных показателя:

1. **Экономия к плану** — процент снижения фактического потребления относительно планового.
2. **Потенциал экономии** — расчётная величина возможного снижения энергетических затрат за выбранный период.

3. **Выполнение целевых показателей** — процент достижения установленных целей по энергоэффективности.

Для расчёта показателей предусмотреть хранение плановых значений потребления, тарифов и целевых параметров. Реализовать возможность просмотра динамики показателей за различные периоды. Дополнительно допускается формирование перечня объектов с наибольшим отклонением от плана. Расчёты должны выполняться на сервере по документированным формулам. Потенциал экономии в учебной реализации может определяться по заранее заданному сценарию или нормативу и должен отличаться от фактически достигнутой экономии.

Разработать раздел «Объекты» с возможностью просмотра, добавления, редактирования и поиска контролируемых зданий и сооружений. Для каждого объекта предусмотреть:

- уникальный идентификатор;
- название;
- тип или назначение;
- адрес или местоположение;
- описание;
- площадь при необходимости;
- ответственного сотрудника;
- количество установленных приборов учёта;
- текущий статус;
- плановый лимит энергопотребления.

Предусмотреть статусы объектов «Онлайн», «Предупреждение» и «Авария». При выборе объекта должна открываться подробная страница с изображением, основными характеристиками и вкладками:

- «Потребление»;
- «Счётчики»;
- «События».

На вкладке «Потребление» отображать графики и сводные показатели. На вкладке «Счётчики» — список установленных приборов учёта. На вкладке «События» — зарегистрированные отклонения и уведомления.

Реализовать раздел «Приборы учёта» с возможностью регистрации, просмотра, редактирования и поиска устройств. Для каждого прибора предусмотреть:

- уникальный идентификатор;
- заводской или условный номер;
- тип прибора;
- связанный объект;
- инженерную систему;

- дату установки;
- текущие показания;
- дату последней передачи данных;
- статус;
- QR-код или штрихкод.

Предусмотреть статусы «Активен», «Нет данных» и «На обслуживании». При выборе прибора должна открываться карточка с технической информацией и историей показаний. Реализовать возможность ручного добавления показаний. При регистрации нового показания сервер должен проверять корректность числового значения, принадлежность прибора выбранному объекту и допустимость изменения накопительных показаний. Для учебной реализации допускается использовать условные приборы учёта без подключения к реальному оборудованию.

Разработать раздел «Показания», предназначенный для хранения и обработки данных, получаемых от приборов учёта. Для каждой записи предусмотреть:

- идентификатор прибора;
- дату и время;
- значение показания;
- единицу измерения;
- источник данных;
- пользователя, зарегистрировавшего показание;
- статус проверки.

Предусмотреть источники данных «Ручной ввод», «Импорт файла» и «Демонстрационная автоматическая передача». Реализовать поиск и фильтрацию показаний по объекту, прибору и периоду. Предусмотреть импорт данных из CSV с проверкой структуры файла и корректности значений. При импорте необходимо исключать некорректные и дублирующиеся записи либо предоставлять пользователю информацию о найденных ошибках. После сохранения новых показаний связанные аналитические показатели должны обновляться.

Реализовать формирование отчётной информации за выбранный период. Отчёт должен включать:

- суммарное энергопотребление;
- распределение потребления по объектам;
- распределение по инженерным системам;
- динамику изменения потребления;
- расчётные энергетические затраты;
- отклонения от плановых значений;
- показатели энергоэффективности;
- количество и категории зарегистрированных событий;
- сведения о состоянии приборов учёта.

Предусмотреть экспорт табличных данных в CSV. Уведомления должны формироваться при регистрации критического события, превышении лимита потребления, отсутствии показаний от прибора, назначении задачи и изменении статуса оборудования. Для каждого уведомления предусмотреть дату, тип, краткое описание и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу с логотипом EnergyMind, названием приложения и карточками основных показателей. В соответствии с графическим концептом предусмотреть следующие блоки:

- «Объекты» — количество зарегистрированных объектов;
- «Счётчики» — количество приборов учёта;
- «Мои задачи» — количество назначенных пользователю задач;
- «События» — количество активных событий.

Ниже разместить кнопки быстрого доступа:

- «Сканирование QR»;
- «Ввод показаний».

В нижней части экрана предусмотреть навигационную панель с разделами «Главная», «Объекты», «Задания», «Профиль». Для активных событий предусмотреть цветовой индикатор и отображение количества новых уведомлений.

Разработать экран просмотра контролируемых объектов. В верхней части разместить строку поиска по названию или адресу. Для каждого объекта отображать:

- фотографию или тематическую миниатюру;
- название;
- назначение;
- количество установленных приборов учёта;
- текущий статус;
- индикатор наличия событий.

Предусмотреть визуальное выделение объектов с нормальным состоянием и объектов, требующих внимания. При выборе объекта должна открываться подробная информация. Список объектов должен загружаться с сервера.

Разработать экран подробной информации о выбранном объекте. В верхней части отображать фотографию здания, название, назначение и текущий статус. Ниже разместить вкладки:

- «Потребление»;
- «Счётчики»;
- «События».

На вкладке «Потребление» отображать график энергопотребления за выбранный период. Предусмотреть переключение периода и отображение значений в МВт·ч или кВт·ч в зависимости от масштаба. На вкладке «Счётчики» отображать перечень установленных приборов учёта с возможностью перехода к их карточкам. На вкладке «События» — список зарегистрированных отклонений и предупреждений. В нижней части экрана разместить кнопку «Все счётчики» с отображением количества приборов на объекте.

Разработать экран сканирования идентификатора прибора учёта. Основную область должен занимать интерфейс камеры с рамкой сканирования QR-кода или штрихкода. В верхней части разместить название «Сканирование счётчика». После распознавания идентификатора приложение должно отправить запрос на сервер для поиска соответствующего прибора. Если прибор найден, пользователь должен получить возможность перейти к его карточке или форме ввода показаний. Если идентификатор отсутствует в базе данных, необходимо вывести соответствующее сообщение. В нижней части разместить кнопку «Ввести вручную», позволяющую перейти к поиску прибора по номеру. Для учебной реализации допускается использовать заранее подготовленные QR-коды с идентификаторами зарегистрированных приборов.

Разработать экран ручного ввода показаний прибора учёта. В верхней части отображать фотографию или условное изображение прибора и его основные сведения:

- тип прибора;
- номер;
- объект установки;
- место установки.

Ниже разместить форму, содержащую:

- поле «Текущие показания»;
- единицу измерения;
- поле «Дата и время»;
- кнопку «Сохранить».

При открытии формы необходимо загружать последние сохранённые показания выбранного прибора. Предусмотреть проверку обязательных полей, корректности числового значения и допустимости даты. Если новое накопительное показание меньше предыдущего, система должна выдавать предупреждение и не принимать запись без предусмотренной процедуры корректировки. После сохранения данные должны передаваться на сервер и записываться в PostgreSQL. При успешном выполнении операции необходимо отображать подтверждающее сообщение.

Разработать экран просмотра уведомлений и событий. В верхней части предусмотреть переключение категорий:

- «Все»;
- «Аварийные»;
- «Инфо».

Для каждого уведомления отображать:

- тематическую иконку;
- название события;
- связанный объект;
- краткое описание;
- дату и время;
- уровень важности.

В соответствии с графическим концептом предусмотреть события:

- превышение потребления;
- отсутствие данных от счётчика;
- резкий скачок мощности;
- плановое обслуживание;
- отклонение напряжения.

При выборе уведомления должен выполняться переход к подробной карточке соответствующего события. Предусмотреть возможность отметки уведомления как прочитанного. Список уведомлений должен загружаться с сервера.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Objects	Контролируемые объекты, характеристики, координаты, статусы
EnergySystems	Инженерные системы и категории энергопотребления
Meters	Приборы учёта, номера, типы, статусы, связанные объекты
MeterReadings	Показания приборов, значения, даты, источники
ConsumptionData	Рассчитанные или импортированные интервальные значения энергопотребления
EnergyPlans	Плановые показатели и лимиты потребления
Tariffs	Тарифы на электрическую энергию и периоды действия
EnergyEvents	Аварийные события и отклонения
Tasks	Назначенные пользователям задачи
Notifications	Уведомления и отметки прочтения
Reports	Параметры и сведения о сформированных отчётах

Media	Изображения объектов и приборов учёта

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление объектами;
- управление приборами учёта;
- регистрацию и просмотр показаний;
- поиск прибора по QR-коду или номеру;
- получение данных об энергопотреблении;
- расчёт энергетических затрат;
- получение плановых показателей;
- получение данных для тепловой карты;
- регистрацию и обработку событий;
- управление задачами;
- получение уведомлений;
- формирование аналитических данных и отчётов.

Для расчёта потребления необходимо обеспечить корректную обработку временных интервалов и единиц измерения. Рекомендуется хранить накопительные показания приборов в кВт·ч, а агрегированные значения отображать в кВт·ч или МВт·ч в зависимости от масштаба. Для расчёта стоимости использовать действующий для соответствующего периода тариф, заданный в системе. Предусмотреть разграничение доступа по ролям администратора, инженера, диспетчера и руководителя. Пароли должны храниться в виде защищённых хешей, а права на изменение данных — проверяться на сервере.

Основные пользовательские сценарии

Сценарий 1. Мониторинг энергопотребления объектов.

Инженер открывает веб-приложение и выбирает контролируемый объект. Система отображает его текущее энергопотребление, статус и количество подключённых приборов учёта. На тепловой карте объект выделяется цветом в соответствии с уровнем потребления. При переходе к подробной странице пользователь получает доступ к графикам, списку счётчиков и зарегистрированным событиям.

Сценарий 2. Регистрация показаний прибора учёта.

Инженер открывает мобильное приложение и выбирает функцию «Сканирование QR». После распознавания QR-кода система определяет зарегистрированный прибор учёта и открывает форму ввода показаний. Пользователь вводит текущее значение, проверяет дату и нажимает «Сохранить». Сервер выполняет проверку корректности данных

и сохраняет новую запись в PostgreSQL. После сохранения обновляются история показаний и связанные показатели энергопотребления.

Сценарий 3. Обнаружение превышения энергопотребления.

Система получает новые показания прибора учёта или демонстрационные данные об энергопотреблении. Сервер рассчитывает фактическое потребление за заданный интервал и сравнивает его с установленным лимитом. При превышении порогового значения создаётся событие «Превышение потребления». Информация появляется в вебпанели и мобильном разделе уведомлений. Диспетчер открывает карточку события, изучает параметры отклонения и изменяет статус на «В работе».

Сценарий 4. Анализ динамики энергопотребления.

Руководитель открывает раздел аналитики, выбирает объект и временной период. Система получает с сервера агрегированные данные об энергопотреблении. На графике отображается динамика изменения показателей. Пользователь сравнивает значения за различные периоды и определяет объекты с наиболее высоким потреблением.

Сценарий 5. Контроль состояния приборов учёта.

Диспетчер открывает раздел «Приборы учёта» и просматривает список зарегистрированных устройств. Один из приборов имеет статус «Нет данных». Пользователь открывает его карточку и изучает дату последней передачи показаний. При необходимости создаётся задача на проверку прибора. Назначенный инженер получает информацию о задаче в мобильном приложении.

Сценарий 6. Выполнение задачи по проверке оборудования.

Инженер открывает мобильное приложение и переходит в раздел «Мои задачи». Пользователь выбирает задачу по проверке счётчика, просматривает сведения об объекте и приборе. После выполнения проверки инженер добавляет комментарий и изменяет статус задачи на «Выполнено». Сервер сохраняет результат, а информация становится доступной диспетчеру и руководителю.

Сценарий 7. Формирование отчёта об энергоэффективности.

Руководитель открывает раздел «Отчёты», выбирает объекты и период. Система рассчитывает суммарное потребление, стоимость энергии, отклонения от плановых значений и показатели энергоэффективности. Пользователь просматривает отчёт и экспортирует табличные данные в CSV.

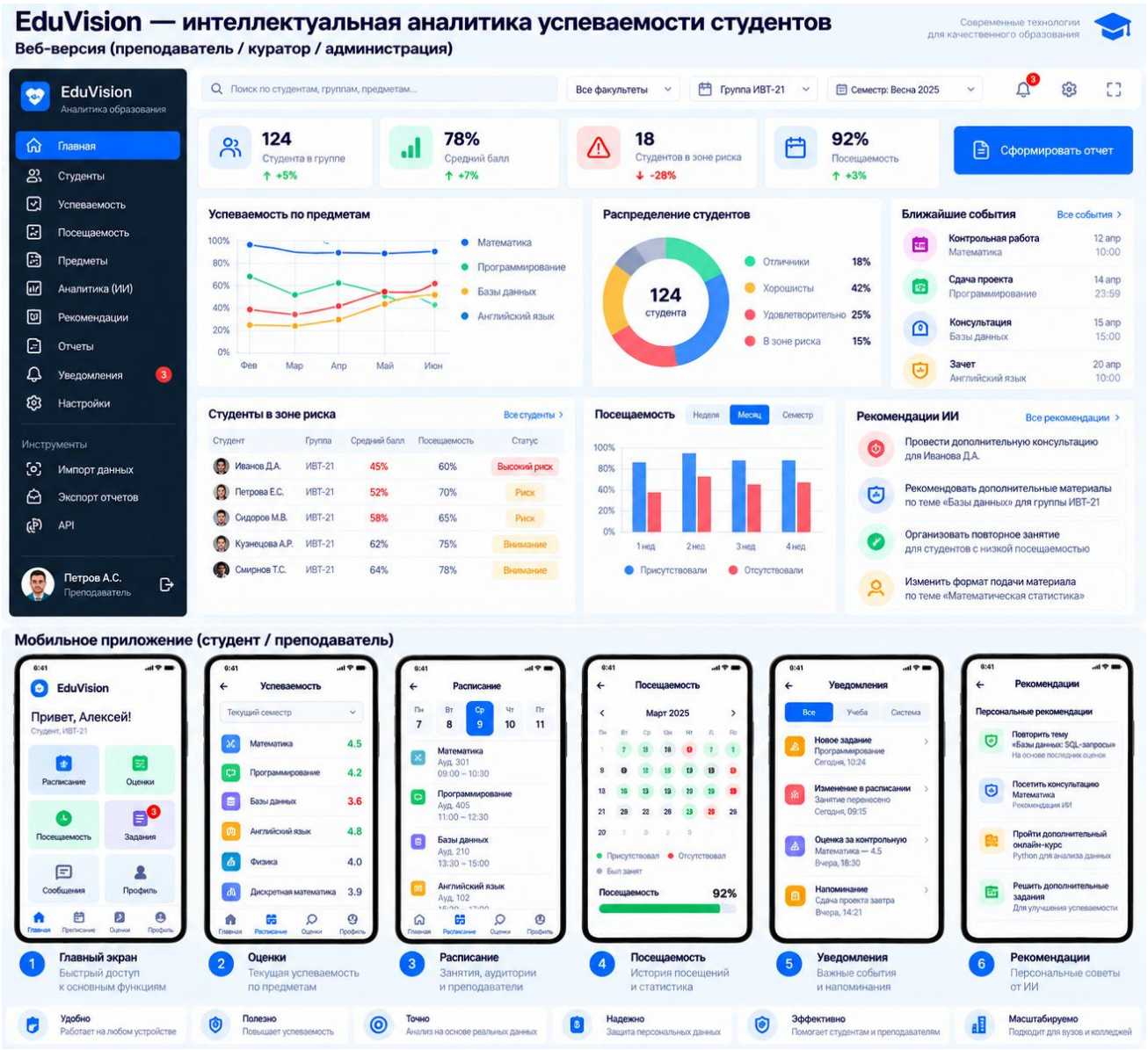
Сценарий 8. Изменение статуса аварийного события.

Диспетчер открывает список активных событий и выбирает уведомление об отклонении напряжения. После изучения информации пользователь назначает

ответственному инженеру задачу на проверку оборудования. Инженер выполняет проверку и фиксирует результат. Диспетчер переводит событие в статус «Устранено». Сервер сохраняет изменения и обновляет количество активных аварийных событий на Dashboard.

Вариант 7. EduVision — система интеллектуальной аналитики успеваемости студентов

Разработать Fullstack веб-приложение **EduVision**, предназначенное для централизованного мониторинга образовательного процесса, анализа академической успеваемости студентов, контроля посещаемости занятий, выявления обучающихся с риском возникновения академической задолженности и формирования персонализированных рекомендаций по улучшению результатов обучения.



Система должна обеспечивать сбор, хранение, обработку и визуализацию информации о студентах, учебных группах, дисциплинах, оценках, посещаемости, расписании занятий и предстоящих учебных мероприятиях, а также предоставлять инструменты для анализа образовательных показателей и своевременного выявления проблем в освоении учебных дисциплин.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для преподавателя, куратора и администрации** — предназначена для управления учебными группами, просмотра и редактирования результатов обучения, контроля посещаемости, анализа успеваемости, выявления студентов группы риска, формирования рекомендаций и аналитических отчётов.
- **Мобильная версия для студента и преподавателя** — предназначена для оперативного просмотра оценок, расписания занятий, истории посещаемости, учебных заданий, уведомлений и персонализированных рекомендаций по улучшению успеваемости.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт EduVision, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации образовательных показателей.

Для учебной реализации допускается использовать демонстрационные сведения о студентах, группах, дисциплинах и результатах обучения.

2. Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть разделы:

- Главная;
- Студенты;
- Успеваемость;
- Посещаемость;
- Предметы;
- Аналитика (ИИ);
- Рекомендации;
- Отчёты;
- Уведомления;
- Настройки.

В дополнительной группе инструментов разместить пункты «Импорт данных», «Экспорт отчётов» и «API». В верхней панели расположить строку поиска по студентам, учебным группам и дисциплинам, выпадающий список выбора факультета, выбор учебной группы, выбор семестра, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Студентов в группе** — количество обучающихся в выбранной учебной группе.
2. **Средний балл** — средний показатель академической успеваемости студентов за выбранный период.

3. **Студентов в зоне риска** — количество обучающихся, для которых выявлены признаки возможных академических затруднений.
4. **Посещаемость** — средний процент посещённых занятий в выбранной группе за указанный период.

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL. При расчёте среднего балла необходимо учитывать используемую систему оценивания. Для графиков и сводных показателей допускается преобразование оценок в процентную шкалу.

Разработать центральный аналитический блок «Успеваемость по предметам». Основным элементом должен стать линейный график, отображающий изменение средней успеваемости студентов по нескольким учебным дисциплинам. В соответствии с графическим концептом предусмотреть следующие дисциплины:

- Математика;
- Программирование;
- Базы данных;
- Английский язык.

Для каждой дисциплины предусмотреть отдельную линию с индивидуальным цветовым обозначением. По горизонтальной оси отображать месяцы или другие временные интервалы, по вертикальной — средний показатель успеваемости в процентах. При наведении на точку графика отображать название дисциплины, период и значение показателя. Реализовать возможность включения и отключения отдельных дисциплин на графике. Предусмотреть фильтрацию данных по учебной группе, факультету и семестру. Дополнительно реализовать возможность перехода к подробной информации об успеваемости по выбранной дисциплине. Данные для построения графика должны поступать с сервера и рассчитываться на основе оценок, сохранённых в PostgreSQL.

Разработать аналитический блок «Распределение студентов». Информацию представить в виде круговой диаграммы с отображением количества и процентных долей студентов различных категорий. В соответствии с графическим концептом предусмотреть следующие категории:

- **Отличники** — студенты с высокими показателями успеваемости;
- **Хорошисты** — студенты со стабильными положительными результатами;
- **Удовлетворительно** — студенты с минимально допустимыми положительными результатами;
- **В зоне риска** — студенты, у которых выявлены академические затруднения.

В центральной части диаграммы отображать общее количество студентов выбранной группы. Для каждой категории предусмотреть цветовое обозначение и процентную долю. Правила отнесения студентов к категориям должны быть определены в

настройках системы или прикладной логике сервера. При изменении группы или семестра распределение должно пересчитываться автоматически.

Разработать раздел «Студенты в зоне риска», предназначенный для выявления обучающихся, которым требуется дополнительное внимание преподавателя или куратора. В соответствии с графическим концептом представить информацию в виде таблицы со следующими столбцами:

- студент;
- учебная группа;
- средний балл;
- посещаемость;
- статус риска.

Для определения уровня риска предусмотреть анализ следующих показателей:

- средний балл студента;
- количество неудовлетворительных оценок;
- процент посещаемости;
- наличие невыполненных заданий;
- динамика изменения успеваемости.

Предусмотреть следующие категории:

- «Норма»;
- «Внимание»;
- «Риск»;
- «Высокий риск».

Для каждой категории использовать соответствующий цветовой индикатор. При выборе студента должна открываться подробная карточка с информацией об оценках, посещаемости, учебных заданиях и выявленных проблемах. Реализовать возможность сортировки списка по среднему баллу, посещаемости и уровню риска. Предусмотреть переход к формированию рекомендаций для выбранного студента. Для учебной реализации допускается использовать набор заранее определённых правил оценки академического риска без разработки модели машинного обучения. Оценка риска должна быть объяснимой: преподаватель должен иметь возможность увидеть, какие показатели повлияли на присвоение категории. Автоматическая оценка не должна использоваться как единственное основание для принятия решений об отчислении или иных существенных образовательных последствиях.

Разработать раздел «Посещаемость», предназначенный для регистрации и анализа посещения учебных занятий. Для каждого занятия предусмотреть:

- учебную дисциплину;
- преподавателя;

- учебную группу;
- дату и время;
- аудиторию;
- список студентов;
- отметки посещаемости.

Для студентов предусмотреть следующие состояния:

- «Присутствовал»;
- «Отсутствовал»;
- «Опоздал»;
- «Уважительная причина».

Реализовать возможность внесения и редактирования отметок преподавателем. В аналитическом блоке отображать столбчатую диаграмму посещаемости за выбранный период. В соответствии с графическим концептом предусмотреть переключение:

- «Неделя»;
- «Месяц»;
- «Семестр».

На диаграмме отображать доли присутствовавших и отсутствовавших студентов. Дополнительно предусмотреть расчёт индивидуального процента посещаемости каждого студента. При изменении отметок информация должна сохраняться в PostgreSQL, а статистические показатели — обновляться.

Реализовать раздел «Студенты» с возможностью просмотра, добавления, редактирования и поиска записей. Для каждого студента предусмотреть:

- уникальный идентификатор;
- ФИО;
- учебную группу;
- факультет или направление подготовки;
- курс;
- дату зачисления;
- контактные сведения при необходимости;
- текущий статус обучения;
- средний балл;
- показатель посещаемости.

При выборе студента должна открываться подробная карточка с вкладками:

- «Успеваемость»;
- «Посещаемость»;
- «Расписание»;
- «Задания»;

- «Рекомендации».

Реализовать справочник учебных групп с возможностью добавления, редактирования и просмотра состава. Для каждой группы хранить название, факультет, курс, учебный год, куратора и список студентов. Предусмотреть поиск и фильтрацию студентов по группе, факультету, курсу и академическому статусу.

Разработать раздел «Предметы», предназначенный для ведения справочника учебных дисциплин. Для каждой дисциплины предусмотреть:

- уникальный идентификатор;
- название;
- краткое описание;
- преподавателя;
- учебную группу;
- семестр;
- количество учебных часов;
- форму итогового контроля.

Предусмотреть возможность просмотра списка студентов, изучающих выбранную дисциплину. Реализовать журнал успеваемости с возможностью внесения и редактирования оценок. Для каждой оценки хранить:

- студента;
- дисциплину;
- вид учебной работы;
- дату получения;
- значение оценки;
- максимальный возможный балл;
- комментарий преподавателя.

Предусмотреть следующие виды учебных работ:

- практическая работа;
- лабораторная работа;
- контрольная работа;
- тестирование;
- проект;
- зачёт;
- экзамен.

Средний балл должен рассчитываться по установленным правилам с учётом шкалы оценивания и весов учебных работ, если они предусмотрены. Все изменения оценок должны сохраняться на сервере и отображаться в мобильной версии соответствующего студента.

Разработать информационный блок «Ближайшие события», предназначенный для отображения предстоящих учебных мероприятий. В соответствии с графическим концептом предусмотреть следующие типы событий:

- контрольная работа;
- сдача проекта;
- консультация;
- зачёт;
- экзамен;
- другие учебные мероприятия.

Для каждого события отображать:

- название;
- учебную дисциплину;
- дату;
- время;
- тип мероприятия;
- краткое описание.

Предусмотреть цветовые иконки для различных категорий событий. При выборе события должна открываться подробная информация. Реализовать возможность добавления и редактирования учебных мероприятий преподавателем или администратором. Для мероприятий, связанных с выполнением заданий, предусмотреть срок сдачи. События должны отображаться в расписании и уведомлениях студентов соответствующих учебных групп.

Разработать раздел «Аналитика (ИИ)», предназначенный для анализа образовательных показателей и формирования рекомендаций. Система должна анализировать данные об успеваемости, посещаемости и выполнении учебных заданий. В соответствии с графическим концептом предусмотреть следующие типы рекомендаций:

- проведение дополнительной консультации для студента с низкой успеваемостью;
- предоставление дополнительных учебных материалов по сложной теме;
- организация повторного занятия для студентов с низкой посещаемостью;
- изменение формата подачи учебного материала;
- выполнение дополнительных практических заданий;
- повторение отдельных тем перед контрольной работой.

Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- тип;
- связанного студента, группу или дисциплину;
- основание формирования;

- текст рекомендации;
- дату создания;
- приоритет;
- статус выполнения.

Предусмотреть статусы «Новая», «Принята», «В работе», «Выполнена» и «Отклонена». Рекомендации должны формироваться на основе заранее определённых правил. Например, при снижении среднего балла по дисциплине ниже установленного порога система может предложить дополнительные учебные материалы или консультацию. При снижении посещаемости ниже установленного значения может формироваться рекомендация по повторному изучению пропущенных тем. Реализация полноценной модели искусственного интеллекта не является обязательной. Для учебного проекта допускается использовать демонстрационный алгоритм интеллектуальной аналитики, основанный на правилах, пороговых значениях и статистических показателях. Рекомендации должны носить вспомогательный характер и предоставлять преподавателю возможность их проверки и корректировки.

Реализовать формирование отчётной информации за выбранный период. Отчёт должен включать:

- количество студентов;
- средний балл группы;
- распределение студентов по категориям успеваемости;
- показатели посещаемости;
- динамику успеваемости по дисциплинам;
- количество студентов в зоне риска;
- сведения о выполнении учебных заданий;
- перечень сформированных рекомендаций.

Предусмотреть экспорт табличных данных в CSV. Для преподавателя и куратора дополнительно предусмотреть формирование отчёта по отдельной учебной группе или дисциплине. Уведомления должны формироваться при:

- выставлении новой оценки;
- изменении расписания;
- назначении учебного задания;
- приближении срока сдачи работы;
- публикации результата контрольной работы;
- появлении новой рекомендации;
- назначении консультации.

Для каждого уведомления предусмотреть дату, тип, краткое описание, связанный объект и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу с логотипом EduVision, названием приложения и персональным приветствием пользователя. Для студента отображать его имя и учебную группу. В соответствии с графическим концептом предусмотреть кнопки быстрого доступа:

- «Расписание»;
- «Оценки»;
- «Посещаемость»;
- «Задания»;
- «Сообщения»;
- «Профиль».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Расписание», «Оценки», «Профиль». Для раздела заданий предусмотреть индикатор количества новых или невыполненных работ. Основные показатели и уведомления должны загружаться с сервера с учётом текущего пользователя.

Разработать экран «Успеваемость», предназначенный для просмотра текущих результатов обучения. В верхней части разместить выпадающий список выбора семестра. Основная область должна содержать список учебных дисциплин. Для каждой дисциплины отображать:

- тематическую иконку;
- название;
- средний балл;
- цветовой индикатор уровня успеваемости.

В соответствии с графическим концептом предусмотреть отображение дисциплин «Математика», «Программирование», «Базы данных», «Английский язык» и других. При выборе дисциплины должна открываться подробная информация с перечнем полученных оценок, датами, видами учебных работ и комментариями преподавателя. Предусмотреть расчёт среднего балла по каждой дисциплине. Информация должна загружаться с сервера и обновляться после внесения оценок преподавателем.

Разработать экран «Расписание». В верхней части разместить горизонтальный календарь с возможностью выбора дня недели и даты. Для выбранного дня отображать список учебных занятий. Каждая карточка занятия должна содержать:

- название дисциплины;
- аудиторию;
- время начала и окончания;
- преподавателя;
- тип занятия.

В соответствии с графическим концептом предусмотреть цветовые иконки дисциплин. При выборе занятия должна открываться подробная информация. Предусмотреть возможность переключения между датами и просмотра расписания на неделю. При изменении расписания преподавателем или администратором информация должна обновляться в мобильном приложении.

Разработать экран просмотра индивидуальной посещаемости студента. В верхней части разместить календарь выбранного месяца. Для каждого дня предусмотреть цветовую индикацию:

- зелёный — студент присутствовал;
- красный — студент отсутствовал;
- серый — занятия не проводились или сведения отсутствуют.

При необходимости дополнительно предусмотреть обозначение опозданий и отсутствий по уважительной причине. При выборе даты должен отображаться список занятий и соответствующие отметки посещаемости. В нижней части экрана разместить показатель общего процента посещаемости за выбранный период. В соответствии с графическим концептом предусмотреть горизонтальный индикатор выполнения. Данные должны загружаться с сервера и рассчитываться на основе журнала посещаемости.

Разработать экран просмотра персональных уведомлений. В верхней части предусмотреть переключение категорий:

- «Все»;
- «Учёба»;
- «Система».

В соответствии с графическим концептом предусмотреть следующие типы уведомлений:

- новое задание;
- изменение расписания;
- оценка за контрольную работу;
- напоминание о сроке сдачи проекта;
- назначение консультации;
- новая персональная рекомендация.

Для каждого уведомления отображать тематическую иконку, название, краткое описание, дату и время. При выборе уведомления должен выполняться переход к соответствующему заданию, оценке, занятию или рекомендации. Предусмотреть возможность отметки уведомления как прочитанного. Количество непрочитанных уведомлений должно отображаться на главном экране.

Разработать экран «Рекомендации», предназначенный для отображения персонализированных советов по улучшению успеваемости. В верхней части разместить заголовок

«Персональные рекомендации». Основная область должна содержать карточки рекомендаций с тематическими иконками. В соответствии с графическим концептом предусмотреть следующие примеры:

- повторить тему «Базы данных: SQL-запросы»;
- посетить консультацию по математике;
- пройти дополнительный онлайн-курс;
- решить дополнительные задания.

Для каждой рекомендации предусмотреть:

- название;
- связанную дисциплину;
- краткое описание;
- причину формирования;
- приоритет;
- статус выполнения.

При выборе рекомендации должна открываться подробная информация с перечнем рекомендуемых действий и доступными учебными материалами. Предусмотреть возможность отметки рекомендации как выполненной. После изменения статуса информация должна сохраняться в PostgreSQL и отображаться в веб-версии преподавателя.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Students	Студенты, учебные группы, основные сведения
Teachers	Преподаватели, кафедры, дисциплины
Faculties	Факультеты и направления подготовки
Groups	Учебные группы, курсы, кураторы
Subjects	Учебные дисциплины и характеристики
Semesters	Учебные семестры, даты начала и окончания
GroupSubjects	Связи учебных групп, дисциплин, семестров и преподавателей
Grades	Оценки студентов, виды работ, даты
Schedule	Расписание занятий, аудитории, время
Attendance	Отметки посещаемости
Assignments	Учебные задания и сроки выполнения
AssignmentSubmissions	Результаты выполнения заданий студентами
AcademicRisks	Рассчитанные уровни академического риска
Recommendations	Персональные и групповые рекомендации
EducationalEvents	Контрольные работы, консультации, зачёты и другие события
Notifications	Уведомления и отметки прочтения

Reports	Параметры и сведения о сформированных отчётах

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление студентами и учебными группами;
- управление преподавателями и дисциплинами;
- регистрацию и редактирование оценок;
- получение индивидуальной и групповой успеваемости;
- управление расписанием занятий;
- регистрацию и просмотр посещаемости;
- управление учебными заданиями;
- получение статистики образовательных показателей;
- определение уровня академического риска;
- формирование и просмотр рекомендаций;
- получение учебных событий;
- получение уведомлений;
- формирование аналитических отчётов.

Для расчёта среднего балла и распределения студентов по категориям необходимо использовать единые правила оценивания, реализованные на сервере. Предусмотреть разграничение доступа по ролям администратора, преподавателя, куратора и студента. Студент должен иметь доступ только к собственным персональным данным, оценкам, посещаемости и рекомендациям. Преподаватель должен иметь возможность изменять оценки и посещаемость только в рамках назначенных ему дисциплин и учебных групп. Куратор должен иметь доступ к аналитике закреплённых за ним групп. Администратор должен управлять справочниками, пользователями и настройками системы. Пароли должны храниться в виде защищённых хешей, а права на просмотр и изменение данных — проверяться на сервере.

Для демонстрации приложения рекомендуется использовать синтетические сведения о студентах. Необходимо исключить публикацию реальных персональных и образовательных данных без соответствующего правового основания.

Основные пользовательские сценарии

Сценарий 1. Просмотр успеваемости учебной группы.

Преподаватель открывает веб-приложение и выбирает факультет, учебную группу и семестр. Система получает с сервера сведения об успеваемости студентов. На Dashboard отображаются средний балл группы, количество студентов, распределение по категориям успеваемости и динамика результатов по дисциплинам. Преподаватель может выбрать отдельную дисциплину и перейти к подробному журналу оценок.

Сценарий 2. Регистрация оценки студента.

Преподаватель открывает раздел «Успеваемость», выбирает учебную группу, дисциплину и студента. Пользователь указывает вид учебной работы, дату и полученную оценку. Сервер проверяет корректность данных и сохраняет новую запись в PostgreSQL. После сохранения пересчитываются средний балл студента, показатели группы и связанные аналитические данные. Новая оценка появляется в мобильном приложении студента.

Сценарий 3. Выявление студента в зоне риска.

Куратор открывает раздел «Аналитика (ИИ)» и запускает анализ академических показателей выбранной группы. Система получает данные об оценках, посещаемости и выполненных заданиях. На основе установленных правил сервер определяет студентов, для которых выявлены признаки академических затруднений. В таблице «Студенты в зоне риска» отображаются соответствующие записи с указанием среднего балла, посещаемости и уровня риска. Куратор выбирает студента и просматривает факторы, повлиявшие на результат анализа.

Сценарий 4. Формирование персональной рекомендации.

После выявления снижения успеваемости по дисциплине «Базы данных» система формирует рекомендацию о повторении темы SQL-запросов. Преподаватель открывает рекомендацию, проверяет её содержание и при необходимости корректирует текст. После подтверждения рекомендация сохраняется в базе данных и становится доступной в мобильном приложении студента. Студент открывает рекомендацию, изучает предложенные материалы и отмечает выполнение.

Сценарий 5. Учёт посещаемости занятия.

Преподаватель открывает раздел «Посещаемость», выбирает занятие и отмечает присутствие студентов. После сохранения сервер обновляет журнал посещаемости. В мобильном приложении студента соответствующая дата получает цветовой индикатор. Общий процент посещаемости пересчитывается автоматически.

Сценарий 6. Просмотр расписания и уведомлений.

Студент открывает мобильное приложение и переходит в раздел «Расписание». Приложение загружает с сервера список занятий на выбранную дату. После изменения времени консультации преподавателем система обновляет расписание и создаёт уведомление. Студент получает информацию об изменении в разделе уведомлений.

Сценарий 7. Выполнение учебного задания.

Преподаватель создаёт новое задание по дисциплине «Программирование» и устанавливает срок сдачи. Система сохраняет задание и формирует уведомления студентам

соответствующей группы. Студент открывает карточку задания, изучает описание и отправляет результат выполнения. Преподаватель проверяет работу и выставляет оценку. После сохранения результата обновляются журнал успеваемости и персональная статистика студента.

Сценарий 8. Формирование отчёта по учебной группе.

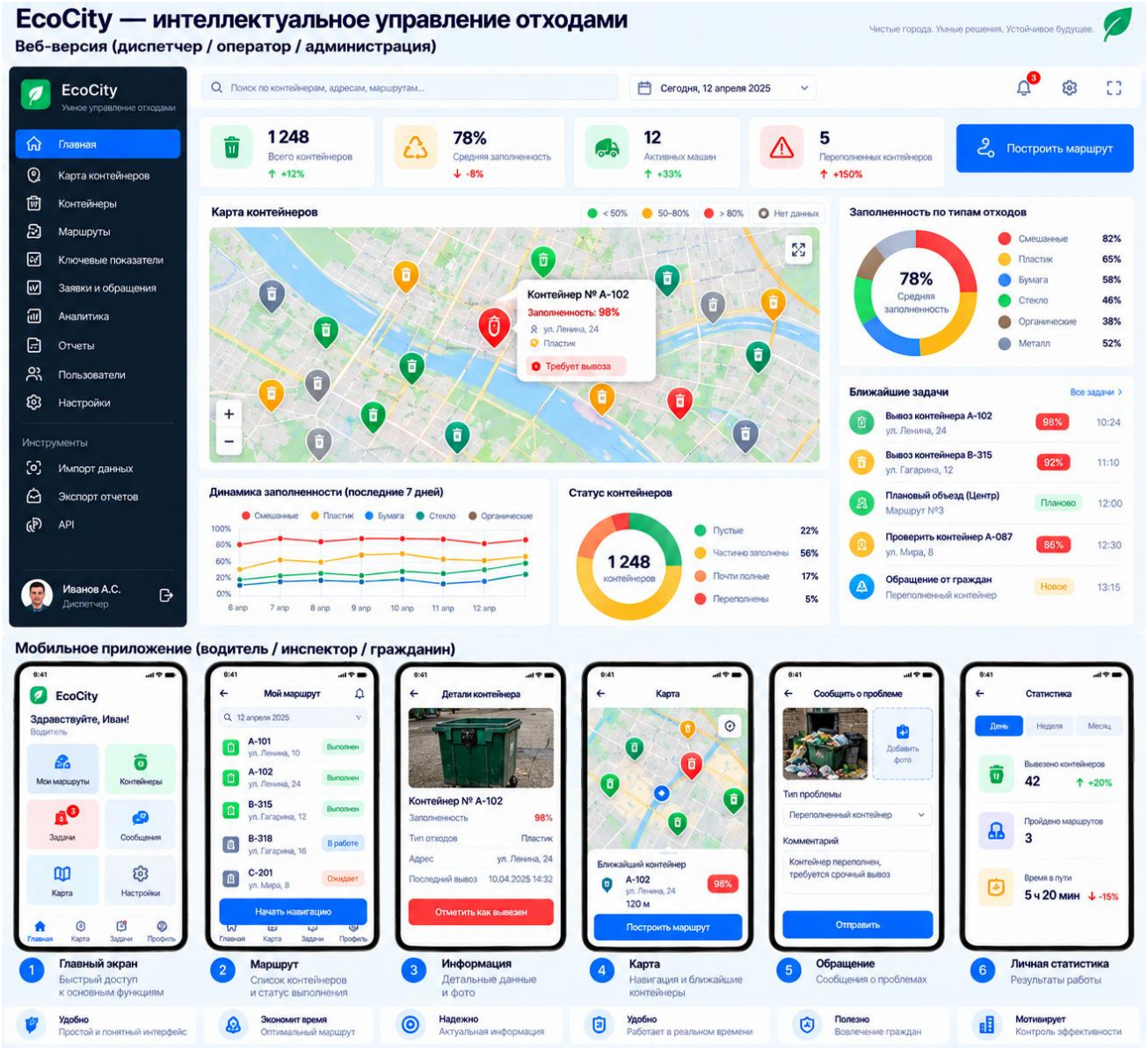
Куратор открывает раздел «Отчёты», выбирает группу и семестр. Система рассчитывает средний балл, показатели посещаемости, распределение студентов по категориям и количество обучающихся в зоне риска. Пользователь просматривает сформированный отчёт и экспортирует табличные данные в CSV.

Сценарий 9. Просмотр персональной статистики студентом.

Студент открывает мобильное приложение и переходит в раздел «Оценки». Система отображает список дисциплин и текущие средние баллы. После выбора дисциплины пользователь просматривает историю оценок. Затем студент открывает раздел «Посещаемость» и изучает календарь посещений. Все данные загружаются через REST API и соответствуют информации, сохранённой в PostgreSQL.

Вариант 8. EcoCity — система интеллектуального управления отходами

Разработать Fullstack веб-приложение **EcoCity**, предназначенное для централизованного мониторинга и управления процессами сбора, вывоза и учёта бытовых и коммунальных отходов, контроля состояния мусорных контейнеров, планирования маршрутов специализированного транспорта, обработки обращений граждан и анализа эффективности работы коммунальных служб.



Система должна обеспечивать сбор, хранение, обработку и визуализацию информации о контейнерных площадках, отдельных контейнерах, их местоположении, типах отходов, текущем уровне заполненности, графиках обслуживания, маршрутах вывоза, транспортных средствах и зарегистрированных проблемах.

Приложение должно предоставлять инструменты для своевременного выявления переполненных контейнеров, определения приоритетности вывоза отходов, оптимизации

работы коммунального транспорта и повышения качества обслуживания городской инфраструктуры.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для диспетчера, оператора и администрации** — предназначена для централизованного мониторинга контейнеров, просмотра их расположения на карте, анализа заполненности, управления маршрутами вывоза отходов, распределения задач между водителями, обработки обращений граждан и формирования аналитических отчётов.
- **Мобильная версия для водителя, инспектора и гражданина** — предназначена для просмотра назначенных маршрутов, получения информации о контейнерах, подтверждения выполнения вывоза, навигации по карте, регистрации проблем с контейнерными площадками и просмотра персональной статистики.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт EcoCity, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации данных.

Для учебной реализации допускается использовать демонстрационные сведения о контейнерах, транспортных средствах, маршрутах и уровнях заполненности, а также имитацию поступления данных от датчиков.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть разделы:

- Главная;
- Карта контейнеров;
- Контейнеры;
- Маршруты;
- Ключевые показатели;
- Заявки и обращения;
- Аналитика;
- Отчёты;
- Пользователи;
- Настройки.

В дополнительной группе инструментов разместить пункты «Импорт данных», «Экспорт отчётов» и «API». В верхней панели расположить строку поиска по контейнерам, адресам и маршрутам, элемент выбора даты, индикатор уведомлений и элементы

управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Всего контейнеров** — общее количество зарегистрированных мусорных контейнеров.
2. **Средняя заполненность** — средний процент заполнения контейнеров за выбранный период.
3. **Активных машин** — количество транспортных средств, выполняющих маршруты вывоза отходов.
4. **Переполненных контейнеров** — количество контейнеров, заполненность которых превышает установленное критическое значение.

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Значения карточек должны рассчитываться на сервере на основе данных, хранящихся в PostgreSQL. В правой части верхней области разместить кнопку «Построить маршрут», предназначенную для запуска процедуры формирования маршрута вывоза отходов. При изменении даты, фильтров или данных о заполненности соответствующие показатели должны обновляться.

Разработать центральный блок «Карта контейнеров», предназначенный для визуального мониторинга расположения мусорных контейнеров на территории города. Основная область должна содержать интерактивную карту с маркерами контейнеров. Для каждого контейнера предусмотреть отображение маркера с цветовым обозначением текущего уровня заполненности. В соответствии с графическим концептом использовать следующие категории:

- **Зелёный** — заполненность менее 50%;
- **Жёлтый** — заполненность от 50% до 80%;
- **Красный** — заполненность более 80%;
- **Серый** — данные о заполненности отсутствуют.

Пороговые значения должны определяться настройками системы. На карте предусмотреть:

- изменение масштаба;
- перемещение по территории;
- отображение контейнеров;
- фильтрацию по типу отходов;
- фильтрацию по уровню заполненности;
- выбор отдельного контейнера;
- переход к подробной информации.

При выборе маркера должна открываться информационная карточка, содержащая:

- номер контейнера;
- адрес расположения;

- тип отходов;
- текущий уровень заполненности;
- дату последнего обновления;
- статус обслуживания.

Для переполненных контейнеров предусмотреть предупреждение «Требует вывоза». Реализовать возможность перехода от карточки контейнера к подробной странице. Координаты контейнеров и связанные сведения должны храниться в PostgreSQL. Для реализации карты допускается использовать Leaflet с OpenStreetMap или другую доступную картографическую библиотеку.

Разработать раздел «Контейнеры», предназначенный для регистрации, просмотра и управления мусорными контейнерами. Для каждого контейнера предусмотреть:

- уникальный идентификатор;
- номер или обозначение;
- адрес расположения;
- географические координаты;
- тип отходов;
- номинальную вместимость;
- текущий уровень заполненности;
- дату последнего вывоза;
- дату последнего обновления показаний;
- техническое состояние;
- статус обслуживания.

Предусмотреть возможность добавления, редактирования и удаления контейнеров с учётом ограничений целостности данных. Реализовать поиск по номеру и адресу. Предусмотреть фильтрацию по типу отходов, уровню заполненности, техническому состоянию и району города. При выборе контейнера должна открываться подробная карточка с фотографией, характеристиками, текущим уровнем заполненности и историей обслуживания. Для отображения заполненности использовать процентный индикатор. Предусмотреть возможность ручного обновления уровня заполненности оператором или инспектором. При достижении критического уровня система должна автоматически создавать событие о необходимости вывоза.

Разработать аналитический блок «Заполненность по типам отходов». В соответствии с графическим концептом предусмотреть следующие категории:

- смешанные отходы;
- пластик;
- бумага;
- стекло;
- органические отходы;
- металл.

Информацию представить в виде круговой диаграммы с отображением средней заполненности по каждой категории. В центральной части диаграммы отображать общий показатель средней заполненности контейнеров. Для каждой категории предусмотреть индивидуальное цветовое обозначение. При выборе категории должна отображаться дополнительная информация:

- количество контейнеров;
- средняя заполненность;
- количество переполненных контейнеров;
- объём доступной вместимости.

Показатели должны рассчитываться на сервере на основе данных PostgreSQL. Для учебной реализации допускается использовать демонстрационные значения заполненности.

Разработать блок «Динамика заполненности», предназначенный для анализа изменения уровня заполненности контейнеров за выбранный период. Основным элементом должен стать линейный график. В соответствии с графическим концептом предусмотреть отображение динамики за последние семь дней. Для каждого типа отходов использовать отдельную линию с индивидуальным цветовым обозначением. По горизонтальной оси отображать даты, по вертикальной — среднюю заполненность в процентах. При наведении на точку графика отображать дату, тип отходов и значение показателя. Реализовать возможность включения и отключения отдельных категорий отходов. Предусмотреть фильтрацию по району города, типу отходов и временному периоду. Данные для построения графика должны поступать с сервера и формироваться на основе истории показаний заполненности контейнеров.

Разработать аналитический блок «Статус контейнеров». В соответствии с графическим концептом предусмотреть следующие категории:

- «Пустые»;
- «Частично заполнены»;
- «Почти полные»;
- «Переполнены».

Информацию представить в виде круговой диаграммы с отображением количества контейнеров и процентного распределения по категориям. В центральной части диаграммы отображать общее количество контейнеров. Предусмотреть возможность перехода к списку контейнеров выбранной категории. Правила определения статусов должны быть реализованы на сервере на основе установленных пороговых значений. При обновлении заполненности контейнера его статус должен пересчитываться автоматически.

Разработать раздел «Маршруты», предназначенный для планирования и контроля вывоза отходов. Для каждого маршрута предусмотреть:

- уникальный идентификатор;
- название или номер;
- дату выполнения;
- назначенное транспортное средство;
- водителя;
- список контейнеров;
- последовательность посещения;
- плановое время начала и завершения;
- фактическое время выполнения;
- статус маршрута.

Предусмотреть следующие статусы:

- «Запланирован»;
- «В работе»;
- «Выполнен»;
- «Отменён».

Реализовать возможность создания маршрута вручную путём выбора контейнеров на карте. Дополнительно предусмотреть функцию автоматического формирования маршрута на основе контейнеров, требующих вывоза. При построении маршрута система должна учитывать:

- уровень заполненности контейнеров;
- приоритет обслуживания;
- географическое расположение;
- доступность транспортных средств;
- максимальное количество точек маршрута.

Для учебной реализации допускается использовать упрощённый алгоритм определения последовательности посещения контейнеров, например метод ближайшего соседа. При наличии картографического сервиса маршрутизации допускается отображение маршрута по дорожной сети. Если такой сервис не используется, необходимо явно обозначать рассчитанную последовательность точек как демонстрационный маршрут, а не как оптимальный автомобильный путь. После формирования маршрута он должен сохраняться в PostgreSQL и становиться доступным назначенному водителю в мобильном приложении.

Разработать раздел учёта специализированного транспорта, используемого для вывоза отходов. Для каждого транспортного средства предусмотреть:

- уникальный идентификатор;
- государственный регистрационный номер;
- марку и модель;
- грузоподъёмность;
- допустимый объём перевозимых отходов;

- текущий статус;
- назначенного водителя;
- текущий маршрут;
- дату последнего обслуживания.

Предусмотреть статусы:

- «Свободен»;
- «На маршруте»;
- «На обслуживании»;
- «Недоступен».

Реализовать возможность назначения транспортного средства на маршрут. При назначении необходимо проверять отсутствие конфликтующих активных маршрутов. Предусмотреть просмотр истории выполненных маршрутов. Информация о количестве активных транспортных средств должна использоваться при расчёте показателей Dashboard.

Разработать блок «Ближайшие задачи», предназначенный для отображения запланированных и текущих работ. В соответствии с графическим концептом предусмотреть следующие типы задач:

- вывоз переполненного контейнера;
- плановый вывоз отходов;
- проверка состояния контейнера;
- выполнение планового объезда;
- обработка обращения граждан.

Для каждой задачи отображать:

- название;
- связанный контейнер или маршрут;
- адрес;
- приоритет;
- плановое время выполнения;
- текущий статус;
- ответственного сотрудника.

Предусмотреть цветовую индикацию приоритетов и статусов. При выборе задачи должна открываться подробная карточка. Реализовать возможность назначения задачи водителю или инспектору. Предусмотреть изменение статуса задачи и сохранение результата выполнения. После завершения задачи соответствующая информация должна обновляться в веб-версии и мобильном приложении.

Разработать раздел «Заявки и обращения», предназначенный для регистрации и обработки сообщений о проблемах с контейнерами и контейнерными площадками. Предусмотреть следующие типы обращений:

- переполненный контейнер;
- повреждённый контейнер;
- мусор рядом с контейнерной площадкой;
- отсутствие контейнера;
- нарушение графика вывоза;
- другая проблема.

Для каждого обращения хранить:

- уникальный идентификатор;
- тип проблемы;
- адрес;
- связанный контейнер при наличии;
- описание;
- фотографию;
- дату и время регистрации;
- автора обращения;
- приоритет;
- статус обработки;
- ответственного сотрудника.

Предусмотреть статусы:

- «Новое»;
- «Принято»;
- «В работе»;
- «Выполнено»;
- «Отклонено».

Оператор должен иметь возможность просматривать обращения, назначать ответственных сотрудников, изменять статусы и добавлять комментарии. При завершении обработки обращения система должна сохранять результат и дату выполнения. Для гражданина должна быть предусмотрена возможность просмотра статуса собственного обращения.

Разработать раздел «Ключевые показатели», предназначенный для анализа эффективности работы системы вывоза отходов. Предусмотреть расчёт следующих показателей:

- количество обслуженных контейнеров;
- количество выполненных маршрутов;
- среднее время выполнения маршрута;

- количество переполненных контейнеров;
- доля контейнеров, обслуженных в установленный срок;
- среднее время обработки обращений;
- количество активных транспортных средств;
- средний уровень заполненности контейнеров.

Информацию представить в виде карточек, графиков и таблиц. Предусмотреть фильтрацию по дате, району, типу отходов и транспортному средству. Все показатели должны рассчитываться на сервере на основе фактических или демонстрационных данных.

Реализовать формирование отчётной информации за выбранный период. Отчёт должен включать:

- общее количество контейнеров;
- распределение контейнеров по типам отходов;
- среднюю заполненность;
- количество переполненных контейнеров;
- динамику заполненности;
- количество выполненных маршрутов;
- количество обслуженных контейнеров;
- сведения о работе транспортных средств;
- количество зарегистрированных обращений;
- показатели эффективности обслуживания.

Предусмотреть экспорт табличных данных в CSV. Уведомления должны формироваться при:

- превышении критического уровня заполненности;
- назначении нового маршрута;
- изменении статуса задачи;
- регистрации нового обращения;
- обнаружении контейнера без актуальных данных;
- приближении планового времени вывоза;
- завершении маршрута.

Для каждого уведомления предусмотреть дату, тип, краткое описание, связанный объект и состояние прочтения.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу с логотипом EcoCity, названием приложения и персональным приветствием пользователя. Для водителя отображать его имя и роль. В соответствии с графическим концептом предусмотреть кнопки быстрого доступа:

- «Мои маршруты»;
- «Контейнеры»;
- «Задачи»;
- «Сообщения»;
- «Карта»;
- «Настройки».

В нижней части экрана разместить навигационную панель с разделами «Главная», «Карта», «Задачи», «Профиль». Для раздела задач предусмотреть индикатор количества новых или невыполненных заданий. Содержимое главного экрана должно зависеть от роли пользователя. Водителю должны быть доступны назначенные маршруты и задачи. Инспектору — информация о контейнерах, проверках и обращениях. Гражданину — карта контейнеров и возможность сообщить о проблеме.

Разработать экран «Мой маршрут», предназначенный для просмотра последовательности контейнеров, которые необходимо обслужить. В верхней части разместить дату маршрута. Основная область должна содержать список контейнеров в порядке посещения. Для каждого контейнера отображать:

- номер;
- адрес;
- текущий статус обслуживания;
- порядковый номер в маршруте.

Предусмотреть следующие статусы:

- «Ожидает»;
- «В работе»;
- «Выполнен»;
- «Пропущен».

В соответствии с графическим концептом использовать цветовую индикацию статусов. В нижней части экрана разместить кнопку «Начать навигацию». При выборе контейнера должна открываться его подробная карточка. После выполнения обслуживания статус контейнера в маршруте должен обновляться. Все изменения должны передаваться на сервер и сохраняться в PostgreSQL.

Разработать экран «Детали контейнера». В верхней части разместить фотографию контейнера. Ниже отображать:

- номер контейнера;
- текущую заполненность;

- тип отходов;
- адрес;
- дату и время последнего вывоза;
- текущий статус.

Для уровня заполненности предусмотреть цветовой индикатор. В соответствии с графическим концептом в нижней части разместить кнопку «Отметить как вывезен». При нажатии кнопки система должна:

1. Проверить права пользователя и принадлежность контейнера активному маршруту.
2. Зарегистрировать дату и время обслуживания.
3. Сохранить запись о выполненном вывозе.
4. Обновить статус контейнера.
5. Установить демонстрационный уровень заполненности после вывоза, например 0%.
6. Обновить статус соответствующей точки маршрута.

После успешного выполнения операции необходимо отображать подтверждающее сообщение. Повторное подтверждение одного и того же вывоза не должно создавать дублирующие записи.

Разработать мобильный экран «Карта». Основная область должна содержать карту с расположением контейнеров. Для каждого контейнера предусмотреть маркер с цветовым обозначением уровня заполненности. Предусмотреть возможность:

- изменения масштаба;
- перемещения по карте;
- выбора контейнера;
- просмотра ближайших контейнеров;
- отображения назначенного маршрута;
- перехода к карточке контейнера.

При выборе маркера должна открываться информационная карточка с номером, адресом, заполненностью и расстоянием до контейнера при наличии данных о местоположении пользователя. В нижней части разместить кнопку «Построить маршрут». Для водителя кнопка должна отображать маршрут обслуживания выбранных контейнеров. Для инспектора допускается построение маршрута к выбранному контейнеру. Для учебной реализации допускается использовать демонстрационные координаты и заранее подготовленные маршруты.

Разработать экран «Сообщить о проблеме», предназначенный для регистрации обращений граждан и инспекторов. В верхней части разместить область добавления фотографии. Ниже расположить форму, содержащую:

- тип проблемы;

- адрес или выбранный контейнер;
- комментарий;
- фотографию при наличии.

В соответствии с графическим концептом предусмотреть выпадающий список типов проблем.

Например:

- переполненный контейнер;
- повреждённый контейнер;
- мусор рядом с площадкой;
- отсутствие контейнера;
- другая проблема.

Для текстового описания предусмотреть многострочное поле. В нижней части разместить кнопку «Отправить». При отправке система должна проверить обязательные поля и передать данные на сервер. Сервер должен сохранить обращение в PostgreSQL, присвоить ему статус «Новое» и сформировать уведомление для оператора. После успешной отправки необходимо отображать подтверждение и номер обращения.

Разработать экран «Статистика», предназначенный для просмотра результатов работы пользователя. В верхней части предусмотреть переключение периода:

- «День»;
- «Неделя»;
- «Месяц».

Для водителя отображать следующие показатели:

1. **Вывезено контейнеров** — количество обслуженных контейнеров за выбранный период.
2. **Пройдено маршрутов** — количество завершённых маршрутов.
3. **Время в пути** — суммарная продолжительность выполнения маршрутов.

Для показателей предусмотреть отображение динамики изменения относительно предыдущего периода. Для инспектора допускается отображать количество проверенных контейнеров, зарегистрированных обращений и выполненных задач. Для гражданина допускается отображать количество отправленных обращений и число решённых проблем. Все значения должны рассчитываться на сервере на основе данных о выполненных действиях пользователя.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Districts	Районы города и территориальные зоны
ContainerSites	Контейнерные площадки, адреса, координаты
WasteTypes	Типы отходов и категории сортировки
Containers	Контейнеры, вместимость, тип отходов, заполненность, статусы
FillLevelHistory	История изменения заполненности контейнеров
Vehicles	Транспортные средства, характеристики, статусы
Routes	Маршруты вывоза, даты, водители, транспорт
RoutePoints	Контейнеры маршрута, порядок посещения, статусы
CollectionHistory	История обслуживания и вывоза контейнеров
Tasks	Задачи водителей и инспекторов
Complaints	Обращения граждан, описания, статусы
ComplaintMedia	Фотографии, прикреплённые к обращениям
Events	Системные события и предупреждения
Notifications	Уведомления и отметки прочтения
Reports	Параметры и сведения о сформированных отчётах

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- получение показателей Dashboard;
- управление контейнерами и контейнерными площадками;
- получение координат контейнеров для отображения на карте;
- регистрацию и просмотр истории заполненности;
- получение данных о переполненных контейнерах;
- управление транспортными средствами;
- создание и редактирование маршрутов;
- получение назначенных маршрутов водителя;
- регистрацию выполнения вывоза контейнера;
- управление задачами;
- регистрацию и обработку обращений граждан;
- получение аналитических показателей;
- получение уведомлений;
- формирование отчётов.

Для операций с координатами предусмотреть хранение широты и долготы контейнеров. Для расчёта расстояний и формирования демонстрационных маршрутов допускается использовать географические координаты и упрощённые алгоритмы. Предусмотреть разграничение доступа по ролям администратора, диспетчера, оператора, водителя, инспектора и гражданина. Администратор должен иметь доступ к управлению пользователями, справочниками и настройками системы. Диспетчер должен управлять

маршрутами, транспортными средствами и назначением задач. Оператор должен просматривать контейнеры, контролировать заполненность и обрабатывать обращения. Водитель должен иметь доступ к назначенным маршрутам и возможность подтверждать выполнение вывоза. Инспектор должен иметь возможность проверять контейнеры, обновлять сведения об их состоянии и регистрировать проблемы. Гражданин должен иметь возможность просматривать общедоступную информацию о контейнерах и управлять собственными обращениями. Пароли должны храниться в виде защищённых хешей, а права на просмотр и изменение данных — проверяться на сервере.

Основные пользовательские сценарии

Сценарий 1. Мониторинг заполненности контейнеров.

Диспетчер открывает веб-приложение и переходит на главную информационную панель. Система загружает с сервера сведения о контейнерах и отображает их расположение на карте. Контейнеры выделяются цветами в соответствии с текущим уровнем заполненности. Диспетчер выбирает контейнер с красным маркером и просматривает его номер, адрес, тип отходов и текущую заполненность. При необходимости пользователь создаёт задачу на вывоз.

Сценарий 2. Автоматическое выявление переполненного контейнера.

Система получает новое демонстрационное значение заполненности контейнера. Сервер сравнивает полученное значение с установленным критическим порогом. При превышении порога контейнер переводится в категорию «Переполнен». На Dashboard обновляется количество переполненных контейнеров. Система создаёт событие о необходимости вывоза, а диспетчер получает уведомление.

Сценарий 3. Формирование маршрута вывоза отходов.

Диспетчер открывает раздел «Маршруты» и выбирает функцию «Построить маршрут». Система формирует список контейнеров, требующих обслуживания. На основе координат и заданных приоритетов сервер определяет последовательность посещения контейнеров. Диспетчер выбирает транспортное средство и назначает водителя. После сохранения маршрут появляется в мобильном приложении водителя.

Сценарий 4. Выполнение маршрута водителем.

Водитель открывает мобильное приложение и переходит в раздел «Мои маршруты». Система отображает назначенный маршрут и список контейнеров в порядке посещения. Водитель выбирает контейнер и переходит к его подробной карточке. После выполнения вывоза пользователь нажимает кнопку «Отметить как вывезен». Сервер сохраняет информацию об обслуживании, обновляет уровень заполненности контейнера и статус точки маршрута. Изменения становятся доступны диспетчеру в веб-версии.

Сценарий 5. Регистрация обращения гражданина.

Гражданин открывает мобильное приложение и выбирает функцию «Сообщить о проблеме». Пользователь добавляет фотографию переполненного контейнера, выбирает тип проблемы и вводит комментарий. После отправки сервер создаёт новое обращение в PostgreSQL. Оператор получает уведомление и открывает карточку обращения. После проверки информации оператор назначает ответственному сотруднику задачу на устранение проблемы.

Сценарий 6. Обработка обращения оператором.

Оператор открывает раздел «Заявки и обращения» и выбирает новое сообщение. Система отображает описание, фотографию, адрес и дату регистрации. Оператор проверяет информацию, определяет приоритет и назначает задачу водителю или инспектору. После выполнения работ ответственный сотрудник фиксирует результат. Оператор переводит обращение в статус «Выполнено». Гражданину становится доступна информация об изменении статуса.

Сценарий 7. Анализ динамики заполненности.

Руководитель открывает раздел «Аналитика» и выбирает временной период. Система получает с сервера историю заполненности контейнеров. На графике отображается динамика изменения заполненности по типам отходов. Пользователь сравнивает показатели и определяет категории отходов с наиболее высокой средней заполненностью. На основе результатов анализа допускается корректировка графиков вывоза.

Сценарий 8. Контроль эффективности работы транспорта.

Диспетчер открывает раздел «Ключевые показатели». Система отображает количество активных транспортных средств, выполненных маршрутов и обслуженных контейнеров. Пользователь выбирает отдельный автомобиль и просматривает историю его маршрутов. На основе данных рассчитываются показатели эффективности работы транспорта.

Сценарий 9. Просмотр личной статистики водителем.

Водитель открывает мобильное приложение и переходит в раздел «Статистика». Система отображает количество вывезенных контейнеров, выполненных маршрутов и суммарное время работы за выбранный период. Пользователь переключает период с дня на неделю или месяц. Все показатели пересчитываются на сервере и отображаются в мобильном интерфейсе.

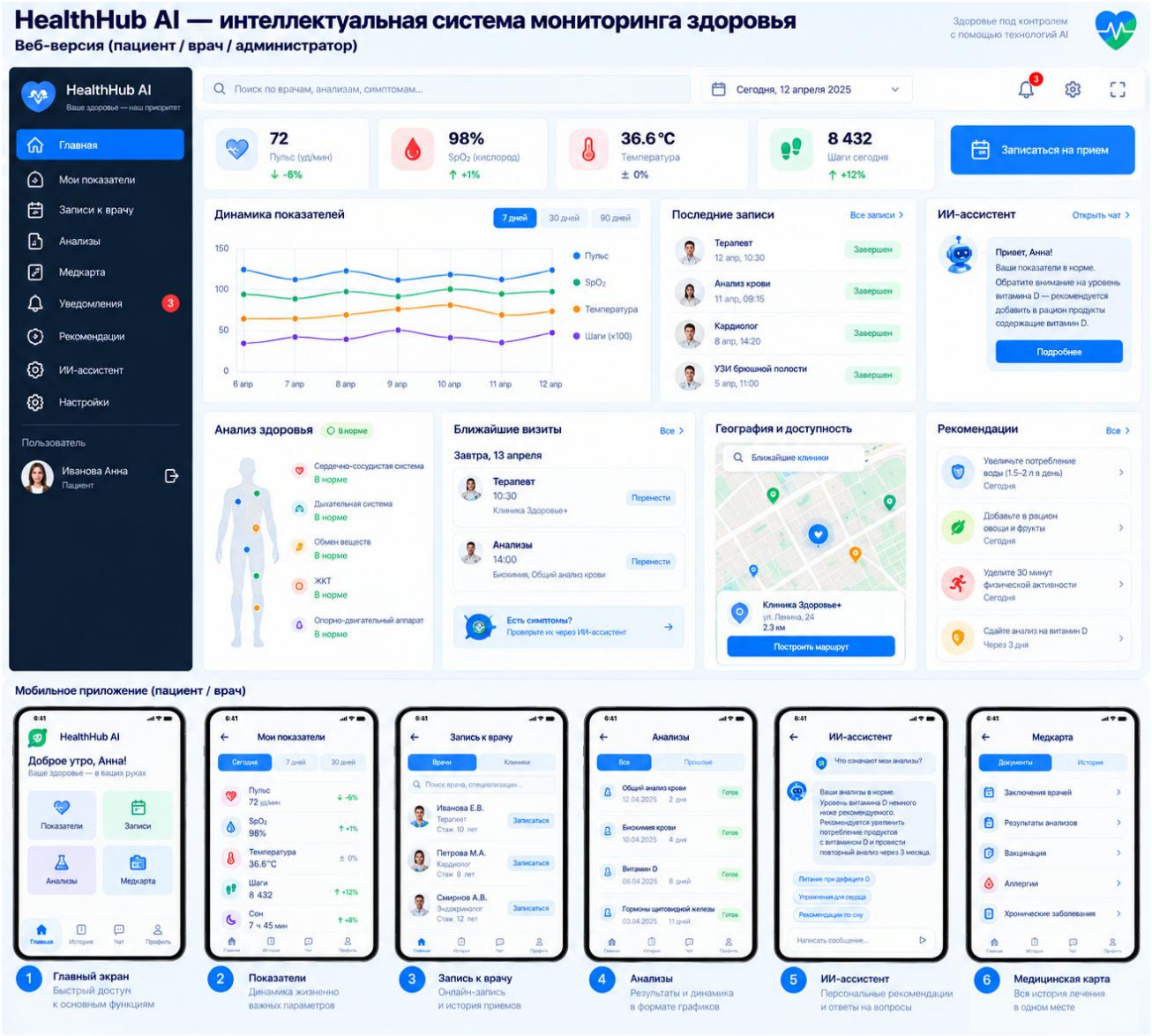
Сценарий 10. Формирование отчёта о работе коммунальных служб.

Руководитель открывает раздел «Отчёты», выбирает район города и временной период. Система рассчитывает количество обслуженных контейнеров, выполненных маршрутов, среднюю заполненность, число переполненных контейнеров и количество

обработанных обращений. Пользователь просматривает сформированный отчёт и экспортирует табличные данные в CSV.

Вариант 9. HealthHub AI — интеллектуальная система мониторинга здоровья

Разработать Fullstack веб-приложение **HealthHub AI**, предназначенное для комплексного мониторинга состояния здоровья пользователей, хранения и анализа медицинских показателей, организации записи на приём к врачам, управления результатами лабораторных исследований, ведения электронной медицинской карты и предоставления персонализированных рекомендаций по поддержанию здорового образа жизни.



Система должна обеспечивать централизованный сбор, хранение, обработку и визуализацию информации о физиологических показателях пользователя, истории медицинских обследований, посещениях врачей, результатах анализов, рекомендациях специалистов и запланированных медицинских мероприятиях.

Приложение должно предоставлять инструменты для отслеживания динамики основных показателей здоровья, выявления отклонений от заданных референсных значений, своевременного информирования пользователей о предстоящих медицинских

визитах и формирования персонализированных рекомендаций на основе имеющихся данных.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для пациента, врача и администратора** — предназначена для комплексного просмотра показателей здоровья, анализа динамики физиологических параметров, управления записями на приём, просмотра медицинских анализов, ведения электронной медицинской карты, взаимодействия с интеллектуальным помощником и формирования отчётной информации.
- **Мобильная версия для пациента и врача** — предназначена для оперативного просмотра показателей здоровья, регистрации новых измерений, записи на приём к специалистам, получения результатов анализов, использования ИИ-ассистента, просмотра медицинской карты и получения уведомлений.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт HealthHub AI, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации данных.

Для учебной реализации допускается использование демонстрационных медицинских данных, имитация поступления показателей от носимых устройств и программная реализация интеллектуального помощника на основе заранее подготовленных правил и рекомендаций.

Система является учебным прототипом и не должна позиционироваться как медицинское изделие или средство постановки диагноза. Автоматические рекомендации должны носить информационный характер и не заменять консультацию врача.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть следующие разделы:

- Главная;
- Мои показатели;
- Записи к врачу;
- Анализы;
- Медкарта;
- Уведомления;
- Рекомендации;
- ИИ-ассистент;
- Настройки.

В нижней части боковой панели разместить информацию об авторизованном пользователе, его фотографию, имя и роль. В верхней панели расположить строку поиска по врачам, анализам и симптомам, элемент выбора даты, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Пульс** — текущее значение частоты сердечных сокращений в ударах в минуту.
2. **SpO_2** — уровень насыщения крови кислородом в процентах.
3. **Температура** — последнее зарегистрированное значение температуры тела.
4. **Шаги сегодня** — количество шагов за текущие сутки.

Для каждого показателя предусмотреть отображение единицы измерения, текущего значения и динамики изменения относительно предыдущего измерения или сопоставимого периода. В соответствии с графическим концептом использовать цветовые индикаторы изменения показателей. Необходимо учитывать, что увеличение или уменьшение значения само по себе не всегда означает улучшение либо ухудшение состояния здоровья. В правой части верхней области разместить кнопку «Записаться на приём», обеспечивающую переход к форме выбора врача и времени посещения. Значения карточек должны загружаться с сервера и формироваться на основе последних измерений, хранящихся в PostgreSQL.

Разработать раздел «Мои показатели», предназначенный для просмотра, регистрации и анализа основных физиологических параметров пользователя. В соответствии с графическим концептом предусмотреть следующие показатели:

- пульс;
- насыщение крови кислородом (SpO_2);
- температура тела;
- количество шагов;
- продолжительность сна.

Для каждого показателя необходимо хранить:

- уникальный идентификатор измерения;
- идентификатор пациента;
- тип показателя;
- числовое значение;
- единицу измерения;
- дату и время измерения;
- источник данных;
- комментарий при наличии.

Предусмотреть возможность ручного добавления измерений. Для каждого типа показателя реализовать проверку корректности вводимых значений и допустимого формата данных. При просмотре показателей пользователь должен иметь возможность выбирать временной период:

- «Сегодня»;
- «7 дней»;
- «30 дней»;
- «90 дней».

Для каждого параметра отображать текущее значение, историю измерений и динамику изменения. Предусмотреть возможность редактирования и удаления ошибочно внесённых пользователем измерений с сохранением истории изменений, если это требуется правилами учебного проекта. Для имитации работы носимых устройств допускается реализовать программный генератор демонстрационных показателей.

Разработать аналитический блок «Динамика показателей». Основным элементом должен стать линейный график, отображающий изменение физиологических параметров за выбранный период. В соответствии с графическим концептом предусмотреть переключатели:

- «7 дней»;
- «30 дней»;
- «90 дней».

На графике должны отображаться следующие показатели:

- пульс;
- SpO_2;
- температура тела;
- количество шагов.

Для каждого параметра использовать отдельную цветовую линию. По горизонтальной оси отображать даты измерений. При наведении на точку графика необходимо показывать дату, название показателя, значение и единицу измерения. Поскольку показатели имеют различные единицы измерения и диапазоны значений, для совместного отображения необходимо предусмотреть нормализацию данных либо возможность переключения между отдельными графиками. Пользователь должен иметь возможность включать и отключать отображение отдельных показателей.

Предусмотреть расчёт:

- среднего значения за период;
- минимального значения;
- максимального значения;
- изменения относительно предыдущего периода;
- количества зарегистрированных измерений.

Данные для построения графиков должны поступать с сервера и формироваться на основе записей PostgreSQL.

Разработать блок «Анализ здоровья», предназначенный для обобщённого представления информации о состоянии различных систем организма. В соответствии с графическим концептом предусмотреть отображение схематичного изображения человеческого тела с цветными маркерами. Рядом со схемой расположить список основных направлений мониторинга:

- сердечно-сосудистая система;
- дыхательная система;
- обмен веществ;
- желудочно-кишечный тракт;
- опорно-двигательный аппарат.

Для каждого направления предусмотреть отображение условного статуса:

- «В норме»;
- «Требует внимания»;
- «Недостаточно данных».

Статусы должны определяться на основе заранее заданных учебных правил и имеющихся показателей. При выборе направления пользователь должен получать подробную информацию о показателях, использованных для формирования статуса. Если данных недостаточно, система должна отображать соответствующее уведомление вместо предположительного заключения. Предусмотреть возможность перехода к истории измерений и связанным медицинским документам. Для учебной реализации допускается использование демонстрационной модели оценки состояния здоровья, не предназначенной для постановки медицинских диагнозов.

Разработать раздел «Записи к врачу», предназначенный для поиска медицинских специалистов, просмотра доступного времени и создания записей на приём. Для каждого врача предусмотреть хранение следующей информации:

- уникальный идентификатор;
- фамилия, имя и отчество;
- фотография;
- специальность;
- стаж работы;
- медицинская организация;
- краткое описание;
- расписание приёма.

В соответствии с графическим концептом предусмотреть возможность записи к следующим специалистам:

- терапевт;
- кардиолог;
- эндокринолог.

При необходимости список специальностей должен расширяться через справочник. Разработать форму записи, содержащую:

- выбор специальности;
- выбор врача;
- выбор даты;
- выбор доступного времени;
- краткое описание причины обращения.

При сохранении записи сервер должен проверять доступность выбранного временного интервала. Предусмотреть следующие статусы записи:

- «Запланирована»;
- «Подтверждена»;
- «Завершена»;
- «Отменена».

Пользователь должен иметь возможность просматривать будущие и прошедшие записи, отменять предстоящие визиты и переносить запись на другое свободное время. Изменения расписания должны сохраняться в PostgreSQL и отображаться в веб- и мобильной версиях.

Разработать блок «Ближайшие визиты», предназначенный для отображения запланированных медицинских мероприятий. В соответствии с графическим концептом предусмотреть карточки предстоящих записей. Для каждого мероприятия отображать:

- название или тип;
- имя специалиста;
- специальность;
- дату;
- время;
- медицинскую организацию;
- текущий статус.

Предусмотреть возможность перехода к подробной информации о записи. Для предстоящих визитов разместить кнопку «Перенести». При переносе необходимо отображать доступные временные интервалы и сохранять новое время посещения. Дополнительно предусмотреть напоминания о приближении даты медицинского мероприятия. Для учебной реализации достаточно реализовать уведомления внутри приложения.

Разработать раздел «Анализы», предназначенный для хранения и просмотра результатов лабораторных исследований. Для каждого исследования предусмотреть:

- уникальный идентификатор;
- наименование анализа;
- дату проведения;

- медицинскую организацию;
- статус готовности;
- список исследуемых показателей;
- полученные значения;
- единицы измерения;
- референсные интервалы при наличии;
- прикреплённый документ.

В соответствии с графическим концептом предусмотреть следующие примеры исследований:

- общий анализ крови;
- биохимический анализ крови;
- витамин D;
- гормоны щитовидной железы.

Разработать список исследований с фильтрацией по дате, типу и статусу готовности. Предусмотреть статусы:

- «Назначен»;
- «В обработке»;
- «Готов».

При выборе исследования должна открываться подробная карточка с результатами. Для числовых показателей предусмотреть визуальное выделение значений, выходящих за пределы указанного лабораторией референсного интервала. Необходимо учитывать, что референсные интервалы могут зависеть от метода исследования, лаборатории и характеристик пациента. Реализовать возможность прикрепления демонстрационных PDF-документов с результатами анализов. Результаты должны сохраняться в PostgreSQL, а файлы — в защищённом файловом хранилище или другой предусмотренной серверной системе хранения.

Разработать раздел «Медкарта», предназначенный для централизованного хранения медицинской информации пользователя. В соответствии с графическим концептом предусмотреть следующие категории:

- заключения врачей;
- результаты анализов;
- вакцинация;
- аллергии;
- хронические заболевания.

Для каждой категории разработать отдельную страницу или вкладку. В медицинской карте предусмотреть хранение:

- персональных данных пациента;

- истории посещений врачей;
- медицинских заключений;
- результатов исследований;
- сведений о вакцинации;
- зарегистрированных аллергий;
- сведений о хронических заболеваниях;
- прикреплённых медицинских документов.

Для каждой записи предусмотреть дату создания, тип документа, автора и краткое описание. Реализовать поиск и фильтрацию медицинских записей. Предусмотреть возможность просмотра и скачивания прикреплённых документов пользователями, имеющими соответствующие права доступа. Редактирование медицинских заключений должно быть доступно только уполномоченным пользователям. Для учебной реализации использовать исключительно вымышленные персональные данные и демонстрационные медицинские документы.

Разработать раздел «ИИ-ассистент», предназначенный для предоставления пользователю справочной информации о показателях здоровья, медицинских исследованиях и общих принципах здорового образа жизни. Интерфейс должен быть реализован в формате чата. В соответствии с графическим концептом предусмотреть:

- область истории сообщений;
- сообщения пользователя;
- ответы ассистента;
- поле ввода текста;
- кнопку отправки;
- быстрые варианты вопросов.

Примеры быстрых вопросов:

- «Что означают мои анализы?»;
- «Питание при дефиците витамина D»;
- «Упражнения для сердца»;
- «Рекомендации по сну».

Ассистент должен иметь возможность использовать разрешённые пользователем данные о показателях здоровья и результатах исследований. Для учебной реализации допускаются два варианта:

1. Правиловая система, формирующая ответы на основе заранее подготовленных сценариев, справочных материалов и условных операторов.
2. Интеграция с внешним API языковой модели с обязательной серверной обработкой запросов и соблюдением требований к защите данных.

Для работы ассистента предусмотреть следующие ограничения:

- ответы должны носить информационно-справочный характер;
- система не должна самостоятельно устанавливать диагноз;
- ассистент не должен назначать лечение или изменять врачебные назначения;
- при недостатке информации необходимо сообщать об ограничениях ответа;
- при описании потенциально опасных симптомов необходимо рекомендовать обращение за медицинской помощью.

История переписки должна сохраняться в PostgreSQL, если пользователь согласился на её хранение.

Разработать раздел «Рекомендации», предназначенный для отображения индивидуальных советов по поддержанию здорового образа жизни. В соответствии с графическим концептом предусмотреть рекомендации по следующим направлениям:

- питьевой режим;
- питание;
- физическая активность;
- сон и восстановление;
- своевременное прохождение медицинских исследований.

Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- категорию;
- заголовок;
- описание;
- дату формирования;
- основание рекомендации;
- статус выполнения.

Рекомендации должны формироваться на основе демонстрационных правил с учётом доступных показателей пользователя. Например, при недостаточном количестве шагов система может предложить обратить внимание на регулярную физическую активность. При приближении даты запланированного анализа система должна сформировать напоминание. Пользователь должен иметь возможность отмечать рекомендацию как выполненную, откладывать её или скрывать. Необходимо предусмотреть разделение общих рекомендаций по образу жизни и рекомендаций, внесённых медицинским специалистом.

Разработать блок «География и доступность», предназначенный для отображения медицинских организаций на интерактивной карте. В соответствии с графическим концептом предусмотреть карту с маркерами клиник. Для каждой медицинской организации хранить:

- уникальный идентификатор;
- название;

- адрес;
- географические координаты;
- контактную информацию;
- список медицинских специальностей;
- режим работы.

На карте предусмотреть:

- отображение маркеров клиник;
- изменение масштаба;
- выбор медицинской организации;
- просмотр информационной карточки;
- поиск ближайших клиник;
- построение маршрута.

При выборе клиники должна открываться карточка с названием, адресом и расстоянием до пользователя при наличии разрешения на определение местоположения. В нижней части карточки разместить кнопку «Построить маршрут». Для реализации допускается использовать Leaflet и OpenStreetMap. Если маршрутизация по дорожной сети недоступна, допускается открывать внешний картографический сервис с выбранным пунктом назначения.

Разработать систему уведомлений о событиях, связанных с состоянием здоровья и медицинскими мероприятиями. Предусмотреть следующие типы уведомлений:

- приближение даты записи к врачу;
- изменение времени приёма;
- отмена записи;
- готовность результатов анализа;
- появление нового медицинского документа;
- напоминание о запланированном исследовании;
- новая рекомендация;
- сообщение от врача.

Для каждого уведомления хранить дату, тип, содержание, связанный объект и состояние прочтения. Предусмотреть индикатор количества непрочитанных уведомлений. Реализовать возможность просмотра уведомлений и перехода к связанным разделам. Дополнительно предусмотреть формирование отчёта о динамике показателей здоровья за выбранный период. Отчёт должен содержать:

- сведения о пользователе;
- период наблюдения;
- список отслеживаемых показателей;
- средние, минимальные и максимальные значения;
- динамику изменения;
- перечень проведённых исследований;

- историю посещений врачей.

Для учебной реализации достаточно предусмотреть экспорт табличных данных в CSV и формирование печатной версии отчёта.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте. Разработать стартовую страницу мобильного приложения с логотипом HealthHub AI и персональным приветствием пользователя. В верхней части разместить название приложения и краткое информационное сообщение. Основная область должна содержать кнопки быстрого доступа:

- «Показатели»;
- «Записи»;
- «Анализы»;
- «Медкарта».

В нижней части разместить навигационную панель с разделами:

- «Главная»;
- «История»;
- «Чат»;
- «Профиль».

При выборе элемента необходимо выполнять переход к соответствующему экрану. Содержимое главного экрана должно формироваться на основе роли авторизованного пользователя. Для пациента отображать персональные показатели и медицинские события. Для врача предусмотреть доступ к расписанию приёмов и разрешённым медицинским записям пациентов.

Разработать мобильный экран «Мои показатели», предназначенный для оперативного просмотра физиологических параметров. В верхней части предусмотреть переключение временного периода:

- «Сегодня»;
- «7 дней»;
- «30 дней».

В основной области разместить карточки:

- пульс;
- SpO_2;
- температура;
- шаги;
- сон.

Для каждого показателя отображать:

- тематическую иконку;
- название;
- последнее значение;
- единицу измерения;
- динамику изменения.

Предусмотреть возможность выбора показателя для просмотра истории измерений. Дополнительно реализовать форму ручного добавления нового измерения. После сохранения данных необходимо обновлять показатели в мобильной и веб-версиях.

Разработать экран «Запись к врачу». В верхней части предусмотреть переключатель:

- «Врачи»;
- «Клиники».

Ниже расположить строку поиска по фамилии врача и медицинской специальности. Основная область должна содержать список врачей. Для каждого специалиста отображать:

- фотографию;
- фамилию и инициалы;
- специальность;
- стаж работы;
- кнопку «Записаться».

При нажатии кнопки должна открываться форма выбора даты и времени приёма. После выбора времени необходимо отправить запрос на сервер. Сервер должен проверить доступность временного интервала и создать запись в PostgreSQL. При успешном сохранении пользователь должен получить подтверждение. Созданная запись должна отображаться в разделе «Ближайшие визиты».

Разработать экран «Анализы», предназначенный для просмотра лабораторных исследований пользователя. В верхней части предусмотреть переключатели:

- «Все»;
- «Прошлые».

Основная область должна содержать список анализов. Для каждого исследования отображать:

- название;
- дату проведения;
- срок с момента получения результата;
- статус готовности.

В соответствии с графическим концептом использовать цветовые индикаторы статуса. При выборе анализа должна открываться подробная карточка с результатами исследования. Предусмотреть просмотр отдельных лабораторных показателей и их референсных интервалов. При наличии прикреплённого документа реализовать возможность его открытия. Дополнительно предусмотреть отображение истории результатов одного и того же исследования.

Разработать мобильный экран «ИИ-ассистент» в формате диалогового интерфейса. В верхней части разместить название раздела. Основная область должна содержать историю сообщений пользователя и ответов интеллектуального помощника. В нижней части расположить текстовое поле ввода и кнопку отправки сообщения. Предусмотреть отображение сообщений в виде отдельных диалоговых блоков. Реализовать быстрые варианты вопросов:

- «Что означают мои анализы?»;
- «Питание при дефиците витамина D»;
- «Упражнения для сердца»;
- «Рекомендации по сну».

После отправки сообщения мобильный клиент должен передавать запрос на сервер. Сервер должен формировать ответ на основе выбранного механизма интеллектуального помощника. При использовании персональных медицинских данных необходимо проверять права доступа и согласие пользователя. Ответ должен отображаться в истории переписки. Для учебной реализации допускается использование заранее подготовленных ответов и сценариев.

Разработать экран «Медкарта». В верхней части предусмотреть переключение:

- «Документы»;
- «История».

Основная область должна содержать категории медицинских записей:

- заключения врачей;
- результаты анализов;
- вакцинация;
- аллергии;
- хронические заболевания.

При выборе категории должен открываться список соответствующих записей. Для каждой записи отображать название, дату и краткое описание. Предусмотреть возможность просмотра подробной информации и прикреплённых документов. В разделе «История» отображать медицинские события в хронологическом порядке. Все данные должны загружаться с сервера с учётом прав доступа авторизованного пользователя.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку входных данных, управление доступом и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
PatientProfiles	Профили пациентов, основные сведения
Doctors	Врачи, специальности, стаж, медицинские организации
Specialties	Справочник медицинских специальностей
Clinics	Медицинские организации, адреса, координаты
DoctorSchedules	Расписание врачей и доступные интервалы
Appointments	Записи на приём, даты, время, статусы
HealthMetrics	Типы физиологических показателей и единицы измерения
HealthMeasurements	История измерений пользователя
LabTests	Лабораторные исследования, даты, статусы
LabTestResults	Отдельные показатели и результаты исследований
MedicalRecords	Медицинские записи и заключения
MedicalDocuments	Сведения о прикреплённых документах
Vaccinations	Сведения о вакцинации
Allergies	Зарегистрированные аллергии
ChronicConditions	Сведения о хронических заболеваниях
Recommendations	Рекомендации, категории, статусы
AIConversations	Диалоги пользователя с ассистентом
AIMessages	Сообщения и ответы ассистента
Notifications	Уведомления и отметки прочтения
AccessGrants	Разрешения на доступ к медицинским данным
AuditLogs	Журнал доступа и изменения медицинских записей

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение персональных показателей здоровья;
- добавление и изменение измерений;
- получение истории показателей;
- расчёт аналитических значений;
- получение списка врачей;
- управление расписанием;
- создание и отмену записей на приём;
- просмотр медицинских анализов;
- управление медицинскими документами;
- получение электронной медицинской карты;
- формирование рекомендаций;
- взаимодействие с ИИ-ассистентом;
- получение списка клиник;
- работу с уведомлениями;

- формирование отчётов.

Предусмотреть разграничение доступа по ролям пациента, врача и администратора.

Пациент должен иметь доступ к собственным показателям, анализам, медицинским документам, записям на приём, рекомендациям и истории взаимодействия с ассистентом.

Врач должен иметь доступ к своему расписанию, записям на приём и медицинским данным пациентов, доступ к которым ему предоставлен в рамках учебной модели.

Администратор должен управлять учётными записями, справочниками, медицинскими организациями и настройками системы. По умолчанию административная роль не должна предоставлять доступ к содержимому медицинских документов и переписке пациентов.

Для обеспечения безопасности необходимо:

- хранить пароли в виде защищённых хешей;
- использовать авторизацию на основе сессий или JWT;
- проверять права доступа на сервере;
- ограничивать доступ к медицинским данным;
- проверять типы и размеры загружаемых файлов;
- валидировать поступающие данные;
- вести журнал доступа к медицинским записям;
- исключить передачу медицинских данных сторонним сервисам без соответствующего разрешения;
- использовать HTTPS при развёртывании приложения.

В учебном проекте необходимо использовать только синтетические данные вымышленных пациентов.

Основные пользовательские сценарии

Сценарий 1. Просмотр текущих показателей здоровья.

Пациент открывает веб-приложение HealthHub AI и авторизуется. Система загружает последние зарегистрированные физиологические показатели. На Dashboard отображаются пульс, SpO_2, температура тела и количество шагов за текущие сутки. Пользователь выбирает интересующий показатель и переходит к его подробной истории. Система отображает график изменения значения за выбранный период.

Сценарий 2. Регистрация нового измерения.

Пациент открывает мобильное приложение и переходит в раздел «Мои показатели». Пользователь выбирает показатель, например пульс, и вводит новое значение. После

отправки формы сервер проверяет корректность данных. Измерение сохраняется в PostgreSQL. На экране обновляется текущее значение, а новая запись появляется в истории измерений.

Сценарий 3. Анализ динамики показателей здоровья.

Пациент открывает раздел «Динамика показателей». Система загружает историю измерений за последние семь дней. На графике отображаются изменения физиологических параметров. Пользователь переключает период на 30 дней. Сервер выполняет расчёт средних, минимальных и максимальных значений. График и аналитические показатели обновляются.

Сценарий 4. Запись на приём к врачу.

Пациент открывает раздел «Записи к врачу». Пользователь выбирает специальность «Кардиолог». Система отображает список доступных специалистов. Пациент выбирает врача, дату и свободное время. После подтверждения сервер проверяет доступность временного интервала и сохраняет запись. Новая запись появляется в списке ближайших визитов.

Сценарий 5. Перенос медицинского визита.

Пациент открывает блок «Ближайшие визиты» и выбирает запланированный приём. Пользователь нажимает кнопку «Перенести». Система отображает доступные даты и время. Пациент выбирает новый временной интервал. Сервер проверяет доступность и обновляет запись. Врач и пациент получают уведомления об изменении времени приёма.

Сценарий 6. Просмотр результатов лабораторного исследования.

Пациент получает уведомление о готовности анализа крови. Пользователь открывает раздел «Анализы». Система отображает список проведённых исследований. Пациент выбирает нужный анализ. Открывается карточка с результатами, единицами измерения и референсными интервалами. При наличии прикреплённого документа пользователь может открыть его для просмотра.

Сценарий 7. Использование интеллектуального помощника.

Пациент открывает раздел «ИИ-ассистент». Пользователь выбирает быстрый вопрос «Что означают мои анализы?». Система формирует запрос на сервер. При наличии разрешения на использование результатов исследований сервер получает необходимые данные. Ассистент формирует информационный ответ с пояснением лабораторных показателей и указанием ограничений автоматической интерпретации. Ответ отображается в диалоговом окне. При необходимости пользователь может перейти к записи на консультацию врача.

Сценарий 8. Просмотр электронной медицинской карты.

Пациент открывает раздел «Медкарта». Система отображает категории медицинских документов. Пользователь выбирает «Заключения врачей». Сервер возвращает список доступных заключений. Пациент открывает нужный документ и просматривает подробную информацию. Действие фиксируется в журнале доступа, если это предусмотрено настройками системы.

Сценарий 9. Поиск медицинской организации.

Пациент открывает блок «География и доступность». Система отображает карту с расположением медицинских организаций. Пользователь выбирает ближайшую клинику. На экране появляется карточка с названием, адресом и контактной информацией. Пациент нажимает кнопку «Построить маршрут». Система открывает маршрут к выбранной медицинской организации.

Сценарий 10. Формирование персональных рекомендаций.

Пациент открывает раздел «Рекомендации». Система получает сохранённые показатели здоровья и сведения о запланированных мероприятиях. На основе демонстрационных правил сервер формирует список рекомендаций по физической активности, питанию, сну и прохождению исследований. Пользователь открывает подробное описание рекомендации. После выполнения соответствующего действия пользователь отмечает рекомендацию как выполненную. Сервер сохраняет изменение статуса.

Сценарий 11. Работа врача с расписанием и медицинскими записями.

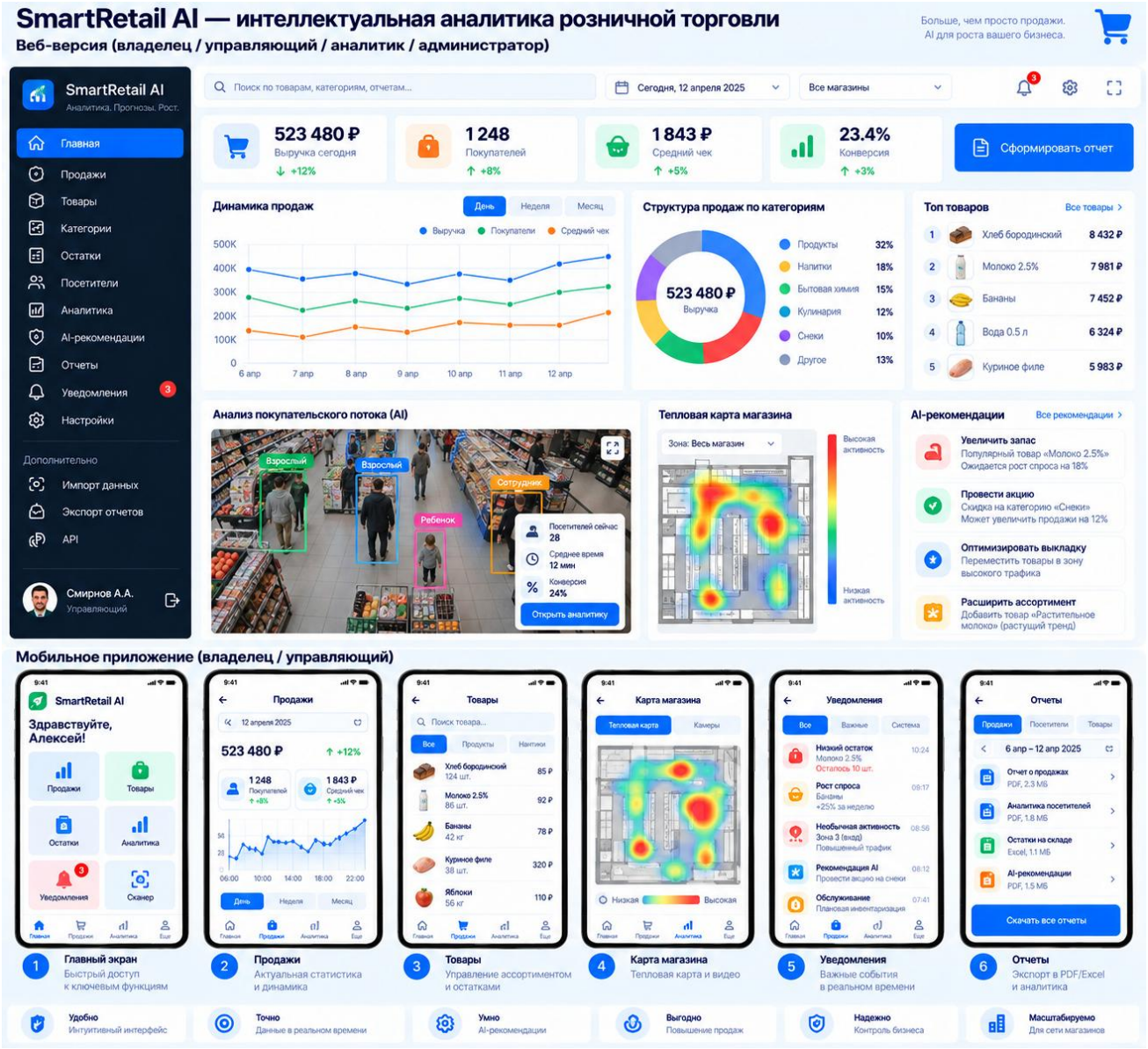
Врач авторизуется в системе и открывает расписание приёмов. Система отображает список пациентов, записанных на выбранную дату. Врач выбирает предстоящий приём. При наличии разрешённого доступа система отображает необходимые медицинские сведения пациента. После завершения приёма врач добавляет демонстрационное заключение и переводит запись в статус «Завершена». Информация сохраняется в PostgreSQL и становится доступной пациенту в разделе «Медкарта».

Сценарий 12. Формирование отчёта о состоянии здоровья.

Пациент открывает раздел аналитики и выбирает период наблюдения. Система загружает историю физиологических показателей, проведённых исследований и медицинских посещений. Сервер рассчитывает сводные статистические значения. Пользователь просматривает сформированный отчёт с таблицами и графиками. При необходимости данные экспортируются в CSV или выводятся в печатной форме.

Вариант 10. SmartRetail AI — интеллектуальная аналитика розничной торговли

Разработать Fullstack веб-приложение **SmartRetail AI**, предназначенное для комплексного мониторинга и анализа деятельности розничных магазинов, управления товарным ассортиментом, контроля складских остатков, анализа покупательского потока, оценки эффективности торговых площадей и формирования интеллектуальных рекомендаций по повышению продаж.



Система должна обеспечивать централизованный сбор, хранение, обработку и визуализацию информации о продажах, товарах, категориях, товарных остатках, покупательской активности, посещаемости магазинов, среднем чеке, конверсии посетителей в покупателей и результатах работы торговых точек.

Приложение должно предоставлять инструменты для оперативного контроля ключевых показателей розничной торговли, выявления товаров с высоким и низким спросом, анализа динамики выручки, определения наиболее посещаемых зон магазина,

обнаружения потенциального дефицита товаров и формирования рекомендаций по оптимизации ассортимента, выкладки и проведения маркетинговых мероприятий.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для владельца, управляющего, аналитика и администратора** — предназначена для централизованного мониторинга продаж, управления ассортиментом, анализа покупательского потока, просмотра тепловых карт торговых залов, контроля товарных остатков, работы с интеллектуальными рекомендациями и формирования аналитических отчётов.
- **Мобильная версия для владельца и управляющего** — предназначена для оперативного просмотра показателей продаж, контроля ассортимента и остатков, просмотра тепловой карты магазина, получения уведомлений о важных событиях и доступа к отчётам.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт SmartRetail AI, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации данных.

Для учебной реализации допускается использование демонстрационных данных о продажах, товарах, остатках и посещаемости, имитация поступления событий от камер видеонаблюдения, программное построение тепловых карт и формирование интеллектуальных рекомендаций на основе заранее подготовленных правил. Реализация полноценной промышленной системы компьютерного зрения и интеграции с реальными кассовыми терминалами не является обязательной.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть следующие разделы:

- Главная;
- Продажи;
- Товары;
- Категории;
- Остатки;
- Посетители;
- Аналитика;
- AI-рекомендации;
- Отчёты;
- Уведомления;
- Настройки.

В дополнительной группе инструментов разместить пункты «Импорт данных», «Экспорт отчётов» и «API». В нижней части боковой панели разместить информацию об авторизованном пользователе, его фотографию, имя и роль. В верхней панели расположить строку поиска по товарам, категориям и отчётам, элемент выбора даты, фильтр по магазинам, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Выручка сегодня** — суммарная стоимость продаж за выбранный день.
2. **Покупателей** — количество покупателей, совершивших покупки за выбранный период.
3. **Средний чек** — средняя стоимость одной покупки.
4. **Конверсия** — процент посетителей магазина, совершивших покупку.

Для каждого показателя предусмотреть отображение текущего значения и динамики изменения относительно предыдущего сопоставимого периода. В соответствии с графическим концептом использовать цветовые индикаторы динамики показателей. Значения карточек должны рассчитываться на сервере на основе данных PostgreSQL. Для среднего чека использовать отношение общей выручки к количеству чеков. Конверсию рассчитывать как отношение количества покупателей к количеству посетителей, умноженное на 100%. При изменении выбранной даты или магазина показатели должны обновляться. В правой части верхней области разместить кнопку «Сформировать отчёт», предназначенную для перехода к формированию аналитической отчётности.

Разработать раздел «Продажи», предназначенный для просмотра, анализа и контроля торговых операций. Для каждой продажи предусмотреть хранение:

- уникального идентификатора;
- даты и времени;
- идентификатора магазина;
- номера чека;
- списка проданных товаров;
- количества каждой товарной позиции;
- цены продажи;
- суммы скидки;
- итоговой стоимости покупки.

Основная область раздела должна содержать таблицу продаж с возможностью поиска, сортировки и фильтрации. Предусмотреть фильтрацию по:

- дате;
- магазину;
- товарной категории;
- конкретному товару;
- диапазону суммы покупки.

При выборе продажи должна открываться подробная карточка чека. В карточке отображать список товаров, количество, стоимость каждой позиции, скидки и итоговую сумму. Реализовать возможность добавления демонстрационных продаж через специальную форму или импорт подготовленного набора данных. После регистрации новой продажи система должна обновлять показатели выручки, количества покупателей, среднего чека и товарных остатков. Необходимо обеспечить согласованность данных о продажах и остатках на уровне серверной логики и транзакций PostgreSQL.

Разработать аналитический блок «Динамика продаж». Основным элементом должен стать линейный график, отображающий изменение ключевых показателей торговой деятельности. В соответствии с графическим концептом предусмотреть переключение периода:

- «День»;
- «Неделя»;
- «Месяц».

На графике должны отображаться три показателя:

- выручка;
- количество покупателей;
- средний чек.

Для каждого показателя использовать отдельную цветовую линию. По горизонтальной оси отображать временные интервалы выбранного периода. При наведении на точку графика показывать дату или время, название показателя, значение и единицу измерения. Поскольку показатели имеют различные единицы измерения, предусмотреть нормализацию значений либо возможность переключения между отдельными графиками. Дополнительно реализовать расчёт:

- общей выручки за период;
- количества покупок;
- среднего чека;
- максимальной дневной выручки;
- минимальной дневной выручки;
- темпа изменения продаж.

Данные для построения графиков должны поступать с сервера и формироваться на основе сохранённых торговых операций.

Разработать аналитический блок «Структура продаж по категориям». В соответствии с графическим концептом предусмотреть следующие категории:

- продукты;
- напитки;
- бытовая химия;

- кулинария;
- снеки;
- другие товары.

Информацию представить в виде кольцевой диаграммы с отображением процентного распределения выручки по категориям. В центральной части диаграммы отображать общую сумму выручки за выбранный период. Для каждой категории использовать индивидуальное цветовое обозначение. При выборе категории должна отображаться дополнительная информация:

- сумма выручки;
- доля в общей выручке;
- количество проданных единиц;
- количество чеков, содержащих товары категории;
- средняя стоимость проданной единицы товара.

Предусмотреть возможность перехода от выбранной категории к списку входящих в неё товаров. Значения должны рассчитываться на сервере на основе данных о продажах.

Разработать раздел «Товары», предназначенный для ведения каталога товаров и контроля ассортимента. Для каждого товара предусмотреть:

- уникальный идентификатор;
- артикул;
- штрихкод;
- название;
- категорию;
- описание;
- изображение;
- единицу измерения;
- розничную цену;
- закупочную цену при наличии;
- текущий остаток;
- минимальный допустимый остаток;
- статус активности.

Реализовать основные CRUD-операции:

- добавление товара;
- просмотр подробной информации;
- редактирование характеристик;
- архивирование или удаление товара при отсутствии связанных ограничений.

Предусмотреть поиск по названию, артикулу и штрихкоду. Реализовать фильтрацию по категории, наличию, ценовому диапазону и статусу. Для каждого товара

предусмотреть подробную карточку с изображением, характеристиками, остатками и историей продаж. При изменении цены необходимо сохранять актуальную стоимость товара. Исторические продажи должны сохранять цену, действовавшую на момент совершения операции.

Разработать раздел «Категории», предназначенный для структурирования ассортимента. Для каждой категории предусмотреть:

- уникальный идентификатор;
- название;
- описание;
- родительскую категорию при наличии;
- количество товаров;
- статус активности.

Предусмотреть возможность создания и редактирования категорий. Допускается организация вложенной структуры категорий. При выборе категории должен отображаться список относящихся к ней товаров. Дополнительно предусмотреть отображение аналитических показателей:

- количество товаров;
- общая выручка;
- доля категории в продажах;
- количество проданных единиц;
- средний уровень товарных остатков.

Изменение категории товара должно отражаться в каталоге и соответствующих аналитических разделах.

Разработать раздел «Остатки», предназначенный для мониторинга наличия товаров в магазинах. Для каждого товара и магазина предусмотреть хранение:

- идентификатора товара;
- идентификатора магазина;
- текущего количества;
- минимального допустимого остатка;
- даты последнего обновления;
- статуса наличия.

Предусмотреть следующие статусы:

- «В наличии»;
- «Низкий остаток»;
- «Нет в наличии».

Статус должен определяться автоматически на основе количества товара и установленного минимального порога. Реализовать таблицу остатков с поиском и фильтрацией. Предусмотреть возможность регистрации поступления товара, списания и корректировки остатков. Для каждой операции необходимо сохранять:

- тип операции;
- товар;
- магазин;
- количество;
- дату;
- ответственного пользователя;
- комментарий.

При достижении минимального уровня остатка система должна формировать уведомление. В соответствии с графическим концептом предусмотреть пример события «Низкий остаток — молоко 2,5%». После регистрации поступления товара соответствующий остаток и статус должны обновляться.

Разработать аналитический блок «Топ товаров», предназначенный для отображения наиболее продаваемых товарных позиций. В соответствии с графическим концептом предусмотреть ранжированный список из пяти товаров. Для каждого товара отображать:

- порядковый номер;
- изображение;
- название;
- сумму выручки за выбранный период.

Примеры товаров:

1. Хлеб бородинский.
2. Молоко 2,5%.
3. Бананы.
4. Вода 0,5 л.
5. Куриное филе.

Список должен формироваться автоматически на основе фактических или демонстрационных продаж. Предусмотреть возможность переключения критерия сортировки:

- по выручке;
- по количеству продаж;
- по количеству проданных единиц.

При выборе товара должна открываться подробная карточка. Дополнительно предусмотреть переход к полному рейтингу товаров.

Разработать раздел «Посетители», предназначенный для мониторинга покупательской активности в торговом зале. В соответствии с графическим концептом основная область должна содержать изображение торгового зала с визуальными обозначениями обнаруженных посетителей. Для каждого обнаруженного объекта предусмотреть прямоугольную рамку и условную категорию. В учебной версии допускается использовать следующие демонстрационные категории:

- взрослый посетитель;
- ребёнок;
- сотрудник магазина.

Для реального применения автоматическое определение возраста или статуса сотрудника должно выполняться только при наличии технически обоснованного и допустимого способа получения таких сведений. В учебном прототипе категории могут назначаться заранее в демонстрационных данных. В правой части изображения разместить информационный блок со следующими показателями:

- посетителей сейчас;
- среднее время пребывания;
- конверсия.

Предусмотреть возможность просмотра аналитической информации о посетителях. Система должна обеспечивать регистрацию агрегированных событий посещаемости, включая:

- дату и время;
- идентификатор магазина;
- количество вошедших посетителей;
- количество вышедших посетителей;
- количество посетителей в торговом зале;
- среднюю продолжительность пребывания.

Для учебной реализации допускается использование заранее подготовленных изображений и видеозаписей с демонстрационной разметкой объектов. Реализация реального распознавания посетителей с использованием нейросетевых моделей не является обязательной. Система не должна выполнять идентификацию личности посетителей или создавать биометрические профили.

Разработать аналитический блок «Тепловая карта магазина», предназначенный для визуализации распределения покупательской активности по территории торгового зала. В соответствии с графическим концептом предусмотреть отображение плана магазина с наложенной цветовой тепловой картой. Использовать цветовую шкалу:

- синий — низкая активность;
- зелёный — умеренная активность;
- жёлтый — повышенная активность;

- красный — высокая активность.

В верхней части блока предусмотреть выбор магазина или зоны. Тепловая карта должна отображать участки торгового зала с различной интенсивностью посещения. При выборе зоны необходимо отображать:

- название зоны;
- количество посещений;
- долю общего покупательского потока;
- среднее время пребывания;
- уровень активности.

Предусмотреть возможность переключения временного периода. Для формирования тепловой карты использовать данные о количестве посещений отдельных зон. Для учебной реализации допускается использовать демонстрационный план магазина и заранее подготовленные координаты активности посетителей. Реализация автоматического отслеживания перемещений посетителей по видеопотоку не является обязательной.

Разработать раздел «AI-рекомендации», предназначенный для формирования предложений по повышению эффективности розничной торговли. В соответствии с графическим концептом предусмотреть следующие типы рекомендаций:

- увеличение товарных запасов;
- проведение рекламных акций;
- оптимизация выкладки товаров;
- расширение ассортимента.

Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- категорию;
- заголовок;
- описание;
- основание формирования;
- предполагаемый эффект;
- дату создания;
- приоритет;
- статус.

Примеры рекомендаций:

Увеличить запас товара. На основе динамики продаж система выявляет рост спроса на молоко 2,5% и предлагает увеличить товарный запас.

Провести акцию. При снижении продаж категории «Снеки» система предлагает рассмотреть проведение рекламной акции.

Оптимизировать выкладку. На основе тепловой карты система определяет зону с высокой посещаемостью и рекомендует разместить в ней товары, требующие дополнительного продвижения.

Расширить ассортимент. При устойчивом росте продаж определённой категории система предлагает рассмотреть добавление новых товаров.

Для каждой рекомендации предусмотреть возможность просмотра подробного обоснования. Реализовать статусы:

- «Новая»;
- «На рассмотрении»;
- «Принята»;
- «Отклонена»;
- «Выполнена».

Для учебной реализации допускается использовать правиловую систему рекомендаций на основе SQL-запросов, агрегированных показателей и заранее заданных условий. Подключение внешней языковой модели или модели машинного обучения не является обязательным. Предполагаемый эффект рекомендации должен обозначаться как демонстрационная оценка, а не гарантированный результат.

Разработать раздел «Уведомления», предназначенный для информирования пользователей о важных событиях торговой деятельности. В соответствии с графическим концептом предусмотреть следующие типы уведомлений:

- низкий товарный остаток;
- рост спроса;
- необычная покупательская активность;
- новая AI-рекомендация;
- плановое обслуживание оборудования;
- изменение ключевых показателей продаж.

Для каждого уведомления хранить:

- уникальный идентификатор;
- тип события;
- заголовок;
- описание;
- дату и время;
- связанный товар, магазин или аналитический показатель;
- приоритет;
- состояние прочтения.

Предусмотреть фильтрацию уведомлений по категориям:

- «Все»;
- «Важные»;
- «Системные».

В верхней панели отображать количество непрочитанных уведомлений. При выборе уведомления должен выполняться переход к связанному объекту. Предусмотреть возможность отметить уведомление как прочитанное.

Разработать раздел «Аналитика», предназначенный для комплексного анализа эффективности розничной торговли. Предусмотреть расчёт следующих показателей:

- общая выручка;
- количество чеков;
- количество покупателей;
- средний чек;
- конверсия посетителей;
- количество проданных товаров;
- выручка по категориям;
- рейтинг наиболее продаваемых товаров;
- количество товаров с низким остатком;
- динамика покупательского потока;
- распределение активности по зонам магазина.

Для каждого показателя предусмотреть фильтрацию по дате и магазину. Реализовать графическое представление данных в виде линейных, столбчатых и круговых диаграмм. Разработать раздел «Отчёты», предназначенный для формирования отчётной информации. В соответствии с графическим концептом предусмотреть следующие виды отчётов:

- отчёт о продажах;
- аналитика посетителей;
- остатки на складе;
- AI-рекомендации.

Для каждого отчёта предусмотреть выбор магазина и временного периода. Реализовать формирование отчётов на сервере. Предусмотреть экспорт отчётной информации в PDF и Excel. Для учебной реализации допускается формирование PDF-документов с таблицами и диаграммами и Excel-файлов с табличными данными.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу мобильного приложения с логотипом SmartRetail AI и персональным приветствием пользователя. В верхней части разместить название приложения и имя авторизованного пользователя. Основная область должна содержать кнопки быстрого доступа:

- «Продажи»;
- «Товары»;
- «Остатки»;
- «Аналитика»;
- «Уведомления»;
- «Сканер».

Для раздела уведомлений предусмотреть индикатор количества новых событий. В нижней части разместить навигационную панель с разделами:

- «Главная»;
- «Продажи»;
- «Аналитика»;
- «Ещё».

При выборе элемента необходимо выполнять переход к соответствующему экрану. Информация должна загружаться с сервера с учётом выбранного магазина и роли пользователя.

Разработать мобильный экран «Продажи», предназначенный для оперативного просмотра торговых показателей. В верхней части разместить элемент выбора даты. Основная область должна содержать:

- общую выручку;
- количество покупателей;
- средний чек;
- график динамики продаж.

Для ключевых показателей предусмотреть отображение изменения относительно предыдущего сопоставимого периода. В соответствии с графическим концептом реализовать переключатели:

- «День»;
- «Неделя»;
- «Месяц».

При изменении периода график и значения показателей должны обновляться. Предусмотреть возможность просмотра подробной информации о продажах. Все данные должны поступать с серверного API.

Разработать мобильный экран «Товары», предназначенный для просмотра ассортимента и контроля остатков. В верхней части разместить строку поиска по названию товара. Предусмотреть переключатели категорий:

- «Все»;
- «Продукты»;
- «Напитки».

Список категорий должен формироваться динамически на основе справочника. Для каждого товара отображать:

- изображение;
- название;
- текущий остаток;
- розничную цену.

При выборе товара должна открываться подробная карточка. Предусмотреть возможность просмотра информации о товаре и его наличии в выбранном магазине. Дополнительно реализовать поиск по штрихкоду через экран «Сканер». Для учебной реализации допускается использовать ввод штрихкода вручную или демонстрационную имитацию сканирования.

Разработать мобильный экран «Карта магазина». В верхней части предусмотреть переключатели:

- «Тепловая карта»;
- «Камеры».

Основная область должна содержать план торгового зала. В режиме «Тепловая карта» отображать распределение покупательской активности. В режиме «Камеры» предусмотреть отображение списка доступных камер и демонстрационных изображений торгового зала. Под картой разместить цветовую шкалу интенсивности активности от низкой до высокой. При выборе зоны магазина отображать краткую информацию о посещаемости. Предусмотреть возможность изменения масштаба и просмотра отдельных участков торгового зала. Для учебной реализации допускается использовать статический план магазина и демонстрационные данные активности.

Разработать мобильный экран «Уведомления». В верхней части предусмотреть переключатели:

- «Все»;
- «Важные»;
- «Система».

Основная область должна содержать список событий. В соответствии с графическим концептом предусмотреть следующие примеры:

- низкий остаток товара;
- рост спроса;
- необычная активность посетителей;
- новая AI-рекомендация;
- плановое обслуживание оборудования.

Для каждого уведомления отображать:

- тематическую иконку;
- заголовок;
- краткое описание;
- дату и время.

При выборе уведомления должна открываться подробная информация. Предусмотреть возможность отметить уведомление как прочитанное. Новые уведомления должны отображаться в мобильной и веб-версиях.

Разработать мобильный экран «Отчёты», предназначенный для просмотра и экспорта аналитической информации. В верхней части предусмотреть категории:

- «Продажи»;
- «Посетители»;
- «Товары».

Ниже разместить элемент выбора временного периода. Основная область должна содержать список доступных отчётов. Для каждого отчёта отображать:

- название;
- тип файла;
- размер;
- дату формирования.

В соответствии с графическим концептом предусмотреть отчёты:

- «Отчёт о продажах»;
- «Аналитика посетителей»;
- «Остатки на складе»;
- «AI-рекомендации».

При выборе отчёта должна открываться подробная информация или предварительный просмотр. В нижней части разместить кнопку «Скачать все отчёты». Предусмотреть возможность экспорта отчётов в PDF и Excel. Для групповой загрузки допускается формирование ZIP-архива.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку поступающих данных, управление доступом и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Stores	Магазины, адреса, характеристики
Categories	Товарные категории и иерархия
Products	Товары, артикулы, штрихкоды, цены
Inventory	Остатки товаров по магазинам
InventoryTransactions	Поступления, списания, корректировки
Sales	Продажи, даты, магазины, итоговые суммы
SaleItems	Товарные позиции чеков
VisitorStatistics	Количество посетителей и показатели посещаемости
StoreZones	Зоны торгового зала
ZoneActivity	Агрегированные данные активности по зонам
Cameras	Камеры, расположение, статусы
DetectionEvents	Демонстрационные события обнаружения объектов
HeatmapData	Данные для построения тепловых карт
AIRecommendations	Интеллектуальные рекомендации и статусы
Notifications	Уведомления и отметки прочтения
Reports	Сформированные отчёты и параметры
AuditLogs	Журнал действий пользователей

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение показателей Dashboard;
- управление магазинами;
- управление товарами;
- управление категориями;
- получение и изменение остатков;
- регистрацию продаж;
- получение истории торговых операций;
- расчёт динамики выручки;
- получение статистики посетителей;
- получение данных о зонах магазина;
- построение тепловых карт;
- получение рейтингов товаров;
- формирование AI-рекомендаций;
- управление уведомлениями;
- формирование и экспорт отчётов.

Предусмотреть разграничение доступа по ролям владельца, управляющего, аналитика и администратора.

Владелец должен иметь доступ к сводным показателям всех принадлежащих ему магазинов, аналитике, рекомендациям и отчётам.

Управляющий должен иметь доступ к данным назначенного магазина, управлению товарами, остатками, продажами и уведомлениями.

Аналитик должен иметь доступ к статистике продаж, покупательскому потоку, тепловым картам, рекомендациям и отчётам.

Администратор должен управлять пользователями, магазинами, справочниками, настройками и техническими параметрами системы.

Необходимо реализовать проверку прав доступа на сервере. Для обеспечения безопасности предусмотреть:

- хранение паролей в виде защищённых хешей;
- авторизацию на основе сессий или JWT;
- проверку корректности входных данных;
- разграничение доступа к магазинам;
- защиту API от несанкционированных запросов;
- журналирование критически важных операций;
- безопасную обработку загружаемых файлов.

Демонстрационные данные о посетителях должны использоваться в агрегированном или обезличенном виде.

Основные пользовательские сценарии

Сценарий 1. Просмотр ключевых показателей магазина.

Управляющий открывает веб-приложение SmartRetail AI и авторизуется. Система загружает сведения о продажах и покупательской активности. На Dashboard отображаются выручка, количество покупателей, средний чек и конверсия. Пользователь выбирает конкретный магазин и дату. Система обновляет показатели на основе выбранных фильтров.

Сценарий 2. Анализ динамики продаж.

Управляющий открывает блок «Динамика продаж». Система отображает график изменения выручки, количества покупателей и среднего чека. Пользователь переключает период с дня на неделю. Сервер выполняет агрегацию торговых операций за выбранный период. График обновляется, позволяя оценить изменение показателей.

Сценарий 3. Просмотр структуры продаж по категориям.

Аналитик открывает блок «Структура продаж по категориям». Система рассчитывает распределение выручки по категориям товаров. На экране отображается кольцевая диаграмма. Пользователь выбирает категорию «Напитки». Система показывает выручку, долю категории и количество проданных товаров.

Сценарий 4. Контроль товарных остатков.

Управляющий открывает раздел «Остатки». Система отображает список товаров и текущее количество на складе. Пользователь выбирает фильтр «Низкий остаток». На экране отображаются товары, количество которых достигло установленного порога. Управляющий выбирает товар и просматривает историю изменения остатка.

Сценарий 5. Автоматическое уведомление о низком остатке.

В системе регистрируется новая продажа молока 2,5%. Сервер сохраняет торговую операцию и уменьшает остаток товара. После обновления количества система сравнивает остаток с минимальным допустимым значением. При достижении порога создаётся уведомление. Управляющий получает сообщение «Низкий остаток» в веб- и мобильной версиях.

Сценарий 6. Анализ покупательского потока.

Аналитик открывает раздел «Посетители». Система отображает демонстрационное изображение торгового зала с размеченными объектами. В информационном блоке показываются количество посетителей, среднее время пребывания и конверсия. Пользователь выбирает временной период. Система обновляет агрегированные показатели посещаемости.

Сценарий 7. Просмотр тепловой карты магазина.

Управляющий открывает раздел «Тепловая карта магазина». Система загружает план торгового зала и данные активности посетителей. На плане отображаются цветовые зоны различной интенсивности. Пользователь выбирает участок с высокой активностью. Система показывает количество посещений и среднее время пребывания в выбранной зоне. На основе полученной информации управляющий может принять решение об изменении выкладки товаров.

Сценарий 8. Формирование интеллектуальной рекомендации.

Система анализирует динамику продаж молока 2,5% за последние несколько дней. Сервер выявляет увеличение спроса и снижение товарного остатка. На основе заданных правил формируется рекомендация «Увеличить запас». Управляющий открывает раздел «AI-рекомендации». Система отображает рекомендацию и её обоснование. Пользователь переводит рекомендацию в статус «Принята».

Сценарий 9. Оптимизация товарной выкладки.

Аналитик открывает тепловую карту торгового зала. Система отображает зоны с различной посещаемостью. Пользователь выбирает участок с высокой активностью. Система формирует демонстрационную рекомендацию по размещению товаров, требующих дополнительного продвижения. Управляющий просматривает рекомендацию и принимает решение о её выполнении.

Сценарий 10. Просмотр рейтинга товаров.

Управляющий открывает блок «Топ товаров». Система формирует рейтинг товаров по выручке. На экране отображаются пять наиболее продаваемых позиций. Пользователь переключает критерий сортировки на количество проданных единиц. Список обновляется. При выборе товара открывается его подробная карточка.

Сценарий 11. Работа с мобильным приложением.

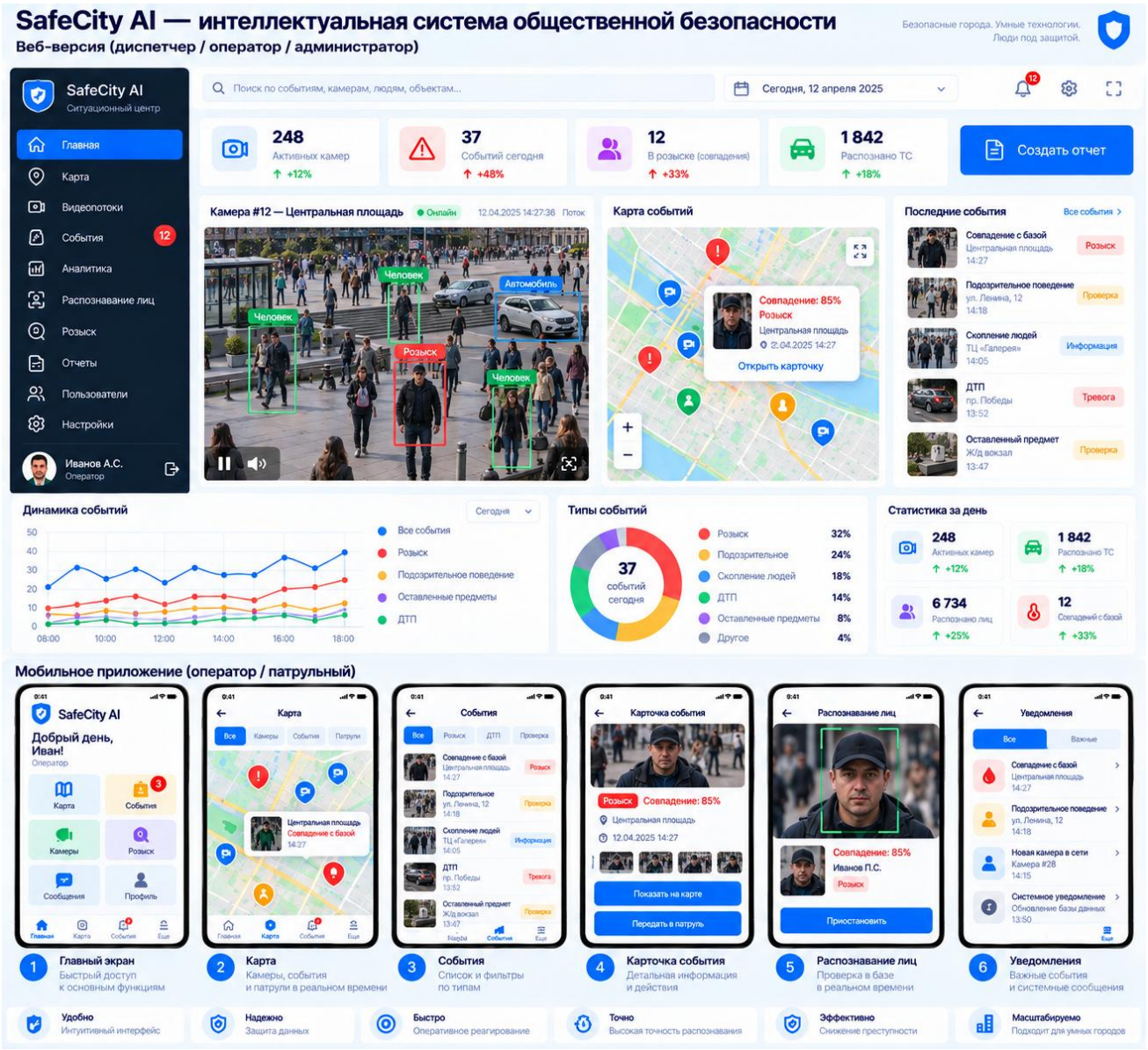
Владелец магазина открывает мобильное приложение SmartRetail AI. На главном экране отображаются основные разделы. Пользователь переходит в раздел «Продажи». Система загружает текущую выручку, количество покупателей и средний чек. Владелец переключает временной период и просматривает динамику продаж. После этого пользователь открывает уведомления и проверяет сообщения о низких остатках и новых рекомендациях.

Сценарий 12. Формирование аналитического отчёта.

Управляющий открывает раздел «Отчёты». Пользователь выбирает магазин, временной период и тип отчёта. Система загружает необходимые данные и выполняет расчёт показателей. На экране отображается предварительный просмотр отчёта. Пользователь выбирает экспорт в PDF или Excel. Сервер формирует файл и предоставляет его для скачивания.

Вариант 11. SafeCity AI — интеллектуальная система общественной безопасности

Разработать Fullstack веб-приложение **SafeCity AI**, предназначенное для комплексного мониторинга общественной безопасности в городской среде, централизованного управления информацией с камер видеонаблюдения, регистрации и анализа происшествий, визуализации событий на интерактивной карте города и координации действий операторов ситуационного центра.



Система должна обеспечивать централизованный сбор, хранение, обработку и визуализацию информации о городских камерах видеонаблюдения, зарегистрированных событиях, потенциально опасных ситуациях, дорожно-транспортных происшествиях, подозрительных и оставленных предметах, скоплениях людей, а также результатах проверки автоматических уведомлений.

Приложение должно предоставлять инструменты для оперативного контроля обстановки, просмотра изображений и видеопотоков, анализа динамики происшествий,

определения местоположения событий, ведения карточек инцидентов, организации проверки поступающих сообщений и формирования аналитической отчётности.

В соответствии с представленным графическим концептом система предусматривает возможность демонстрационного обнаружения объектов на изображениях, включая людей и транспортные средства, а также сопоставления изображений с учебной базой контрольных образцов.

Функции автоматического распознавания и сопоставления лиц в рамках учебного проекта должны реализовываться исключительно на синтетических или специально подготовленных демонстрационных данных, не позволяющих идентифицировать реальных людей. Результаты сопоставления являются условными и не должны использоваться для установления личности или принятия решений о задержании.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для диспетчера, оператора и администратора** — предназначена для централизованного мониторинга городской обстановки, просмотра видеопотоков, работы с интерактивной картой, регистрации и обработки событий, анализа статистики, управления камерами и формирования отчётов.
- **Мобильная версия для оператора и патрульного сотрудника** — предназначена для оперативного просмотра карты, получения информации о событиях, работы с карточками происшествий, просмотра доступных камер, получения уведомлений и передачи информации о результатах проверки.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт SafeCity AI, сохранив его основные композиционные решения, структуру навигации, информационные блоки, цветовую схему и способы визуализации данных.

Для учебной реализации допускается использование демонстрационных изображений и видеозаписей городской среды, заранее подготовленных событий, имитация поступления информации от камер, программная разметка обнаруженных объектов и генерация уведомлений на основе заданных правил.

Реализация промышленной системы компьютерного зрения, реального биометрического поиска, интеграции с государственными базами данных и действующими системами экстренного реагирования не является обязательной и не входит в минимальный объём учебного проекта.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью и верхней областью управления. В боковом меню предусмотреть следующие разделы:

- Главная;
- Карта;
- Видеопотоки;
- События;
- Аналитика;
- Распознавание лиц (демонстрационный режим);
- Розыск (учебные карточки);
- Отчёты;
- Пользователи;
- Настройки.

В нижней части боковой панели разместить информацию об авторизованном пользователе, его фотографию, имя и роль. В верхней панели расположить строку поиска по событиям, камерам, людям и объектам, элемент выбора даты, индикатор уведомлений и элементы управления профилем. Основная область Dashboard должна содержать четыре карточки ключевых показателей:

1. **Активных камер** — количество камер, находящихся в состоянии «Онлайн».
2. **Событий сегодня** — количество зарегистрированных событий за текущий день.
3. **В розыске (совпадения)** — количество демонстрационных событий сопоставления с учебными контрольными образцами.
4. **Распознано ТС** — количество обнаруженных транспортных средств за выбранный период.

Для каждого показателя предусмотреть отображение текущего значения и динамики изменения относительно предыдущего сопоставимого периода. В соответствии с графическим концептом использовать цветовые индикаторы динамики показателей. Значения карточек должны рассчитываться на сервере на основе данных PostgreSQL. При изменении выбранной даты показатели должны обновляться. В правой части верхней области разместить кнопку «Создать отчёт», предназначенную для перехода к формированию аналитической отчётности.

Разработать раздел «Видеопотоки», предназначенный для централизованного просмотра изображений и видеозаписей городских камер наблюдения. Для каждой камеры предусмотреть хранение:

- уникального идентификатора;
- названия;
- описания;
- адреса размещения;
- географических координат;
- типа камеры;
- источника видеопотока;
- текущего статуса;
- даты последнего подключения;

- связанной городской зоны.

Предусмотреть следующие статусы камер:

- «Онлайн»;
- «Офлайн»;
- «Обслуживание».

В соответствии с графическим концептом на главной странице необходимо разместить крупный блок просмотра видеопотока с заголовком «Камера №12 — Центральная площадь». В верхней части блока отображать название камеры, статус подключения, дату и время. Основная область должна содержать изображение или видеозапись городской площади. Поверх изображения предусмотреть визуальные обозначения обнаруженных объектов:

- человек;
- автомобиль;
- демонстрационное событие сопоставления.

Для каждого обнаруженного объекта отображать прямоугольную рамку и подпись с типом объекта. В нижней части видеоплеера разместить элементы управления:

- воспроизведение и пауза;
- включение и отключение звука;
- переход в полноэкранный режим.

В разделе «Видеопотоки» реализовать список доступных камер с возможностью поиска и фильтрации по статусу и расположению. При выборе камеры должен открываться соответствующий видеопоток. Для учебной реализации допускается использование заранее подготовленных видеозаписей и изображений с демонстрационной разметкой объектов. Реализация подключения к реальным IP-камерам не является обязательной.

Разработать раздел «Карта», предназначенный для отображения расположения камер видеонаблюдения и зарегистрированных событий на городской карте. В соответствии с графическим концептом предусмотреть отображение интерактивной карты с цветными маркерами. Использовать следующие категории маркеров:

- красный — происшествие или событие повышенного приоритета;
- синий — камера видеонаблюдения;
- оранжевый — событие, требующее проверки;
- зелёный — информационное событие или доступный сотрудник.

Карта должна поддерживать:

- изменение масштаба;

- перемещение;
- выбор маркера;
- фильтрацию по типу события;
- фильтрацию по статусу;
- отображение информации об объекте.

При выборе маркера должна открываться информационная карточка. Для события отображать:

- название;
- тип;
- адрес;
- дату и время;
- приоритет;
- текущий статус;
- краткое описание;
- изображение при наличии.

В карточке предусмотреть кнопку «Открыть карточку», выполняющую переход к подробной информации. Для камер отображать название, местоположение и статус подключения. При выборе камеры предусмотреть переход к соответствующему видеопотоку. Информация о событиях и камерах должна загружаться с серверного API. Для учебной реализации допускается использование демонстрационной карты с заранее подготовленными координатами.

Разработать раздел «События», предназначенный для регистрации, просмотра, классификации и обработки происшествий. Для каждого события предусмотреть:

- уникальный идентификатор;
- тип;
- название;
- описание;
- дату и время;
- адрес;
- географические координаты;
- идентификатор камеры;
- изображение или видеоматериал;
- приоритет;
- статус;
- ответственного оператора;
- комментарии;
- историю изменения статуса.

В соответствии с графическим концептом предусмотреть следующие типы событий:

- совпадение с учебным контрольным образцом;

- подозрительное поведение;
- скопление людей;
- дорожно-транспортное происшествие;
- оставленный предмет.

Дополнительно допускается категория «Другое». Для каждого события предусмотреть один из следующих статусов:

- «Новое»;
- «На проверке»;
- «Подтверждено»;
- «Отклонено»;
- «В работе»;
- «Завершено».

Реализовать таблицу или список событий с поиском, сортировкой и фильтрацией. Предусмотреть фильтрацию по:

- типу;
- дате;
- статусу;
- приоритету;
- камере;
- городской зоне.

При выборе события должна открываться подробная карточка. Оператор должен иметь возможность изменить статус, добавить комментарий, назначить ответственного и прикрепить материалы проверки. Все изменения должны сохраняться в PostgreSQL.

Разработать информационный блок «Последние события», расположенный на главной панели. В соответствии с графическим концептом предусмотреть отображение пяти последних зарегистрированных событий. Для каждого элемента отображать:

- миниатюру изображения;
- название;
- адрес или местоположение;
- время регистрации;
- категорию или статус.

В качестве демонстрационных событий использовать:

1. Совпадение с учебной базой — Центральная площадь.
2. Подозрительное поведение — ул. Ленина, 12.
3. Скопление людей — ТЦ «Галерея».
4. ДТП — проспект Победы.
5. Оставленный предмет — железнодорожный вокзал.

Для различных типов событий использовать индивидуальные цветовые обозначения. При выборе элемента должна открываться подробная карточка события. Предусмотреть ссылку «Все события», выполняющую переход к полному списку. Список должен формироваться автоматически на основе данных сервера и сортироваться по времени регистрации.

Разработать демонстрационный модуль анализа изображений и видеоматериалов. Модуль должен обеспечивать отображение результатов обнаружения объектов следующих категорий:

- человек;
- автомобиль;
- группа людей;
- оставленный предмет.

Для каждого результата предусмотреть:

- идентификатор;
- тип объекта;
- идентификатор камеры;
- дату и время;
- координаты прямоугольной области на изображении;
- условную оценку уверенности;
- связанное событие при наличии.

В соответствии с графическим концептом обнаруженные объекты должны выделяться цветными прямоугольниками. Для разных типов объектов использовать различные цвета и подписи. Предусмотреть возможность просмотра исходного изображения и изображения с нанесённой разметкой. Для учебной реализации допускается хранение заранее подготовленных координат объектов в PostgreSQL и отображение разметки средствами Frontend. Реализация полноценного алгоритма обнаружения объектов на основе нейронной сети не является обязательной.

Разработать раздел «Распознавание лиц», предназначенный для демонстрации интерфейса сопоставления изображений с учебным набором контрольных образцов. В соответствии с графическим концептом предусмотреть экран с крупным изображением лица, выделенной прямоугольной областью и информационной карточкой результата. В учебной реализации использовать только синтетические изображения персонажей либо специально подготовленные демонстрационные изображения, не связанные с реальными разыскиваемыми лицами. Для каждого учебного образца предусмотреть:

- уникальный идентификатор;
- условное имя персонажа;
- изображение;
- описание;
- статус учебной карточки.

Для результата сопоставления предусмотреть:

- идентификатор события;
- изображение;
- идентификатор контрольного образца;
- условный процент совпадения;
- дату и время;
- местоположение;
- статус проверки.

В соответствии с графическим концептом предусмотреть отображение демонстрационного результата «Совпадение: 85%». Указанное значение должно использоваться исключительно как заранее заданный учебный показатель и не должно трактоваться как достоверная вероятность установления личности. В интерфейсе предусмотреть возможность просмотра учебной карточки и отметки результата как проверенного или отклонённого. Необходимо исключить автоматическое принятие решений о задержании, ограничении доступа или установлении личности на основе результатов модуля. Для учебной реализации достаточно реализовать демонстрационное сопоставление заранее подготовленных записей без обработки биометрических данных реальных людей.

Разработать раздел «Розыск», предназначенный для демонстрационного ведения учебных контрольных карточек. Раздел должен использовать исключительно вымышленные данные и синтетические изображения. Для каждой карточки предусмотреть:

- уникальный идентификатор;
- условное имя персонажа;
- изображение;
- описание;
- дату создания;
- статус;
- связанные демонстрационные события.

Предусмотреть просмотр списка карточек и подробной информации. Реализовать поиск по условному имени и идентификатору. При выборе карточки должна открываться подробная информация о персонаже и связанных учебных событиях. Для администратора предусмотреть возможность добавления, редактирования и архивирования демонстрационных карточек. Связь между карточкой и событием должна сохраняться в PostgreSQL.

Разработать аналитический блок «Динамика событий», предназначенный для отображения изменения количества зарегистрированных событий во времени. В соответствии с графическим концептом использовать линейный график с несколькими цветовыми рядами. Предусмотреть следующие показатели:

- все события;

- учебные события сопоставления;
- подозрительное поведение;
- оставленные предметы;
- ДТП.

По горизонтальной оси отображать временные интервалы. По вертикальной оси — количество зарегистрированных событий. В верхней части блока предусмотреть выбор периода анализа. При наведении на точку графика отображать время, тип события и количество. Данные должны рассчитываться сервером на основе сохранённых событий. При изменении периода график должен обновляться. Дополнительно предусмотреть расчёт:

- общего количества событий;
- количества событий по категориям;
- количества событий повышенного приоритета;
- количества завершённых событий;
- количества событий, находящихся на проверке.

Разработать аналитический блок «Типы событий». В соответствии с графическим концептом представить данные в виде кольцевой диаграммы. Предусмотреть категории:

- учебные события сопоставления;
- подозрительное поведение;
- скопление людей;
- ДТП;
- оставленные предметы;
- другие события.

В центральной части диаграммы отображать общее количество событий за выбранный период. Рядом с диаграммой разместить легенду с названиями категорий и процентным распределением. При выборе категории должна отображаться дополнительная информация:

- количество событий;
- доля от общего количества;
- количество подтверждённых событий;
- количество событий в работе;
- количество завершённых событий.

Предусмотреть возможность перехода к отфильтрованному списку событий. Данные должны рассчитываться на сервере.

Разработать информационный блок «Статистика за день». В соответствии с графическим концептом предусмотреть четыре карточки:

1. **Активных камер** — количество камер со статусом «Онлайн».

2. **Распознано ТС** — количество обнаруженных транспортных средств.
3. **Распознано лиц** — количество демонстрационных обнаружений лиц на синтетических изображениях.
4. **Совпадений с базой** — количество учебных событий сопоставления с контрольными образцами.

Для каждого показателя предусмотреть отображение текущего значения и динамики изменения относительно предыдущего дня. Показатели должны рассчитываться на основе данных PostgreSQL. Для демонстрационных распознаваний необходимо использовать условные записи и заранее подготовленные результаты.

Разработать раздел «Пользователи», предназначенный для управления учётными записями сотрудников. Для каждого пользователя предусмотреть:

- уникальный идентификатор;
- фамилию и имя;
- логин;
- адрес электронной почты;
- роль;
- статус учётной записи;
- дату создания;
- дату последнего входа.

Предусмотреть следующие роли:

- диспетчер;
- оператор;
- патрульный сотрудник;
- администратор.

Диспетчер должен иметь доступ к общей карте событий, просмотру инцидентов, назначению ответственных и контролю обработки происшествий.

Оператор должен иметь доступ к видеопотокам, регистрации событий, проверке демонстрационных результатов анализа и работе с карточками происшествий.

Патрульный сотрудник должен иметь доступ к назначенным событиям, карте, подробной информации о происшествиях и возможности передачи результатов проверки.

Администратор должен управлять пользователями, камерами, справочниками, настройками и техническими параметрами системы.

Реализовать создание, редактирование и блокировку учётных записей. Предусмотреть разграничение прав доступа на сервере.

Разработать раздел «Аналитика», предназначенный для комплексного анализа городской обстановки. Предусмотреть расчёт следующих показателей:

- количество активных камер;
- количество зарегистрированных событий;
- распределение событий по типам;
- динамика событий;
- количество событий повышенного приоритета;
- количество событий по районам;
- количество подтверждённых событий;
- среднее время обработки событий;
- количество завершённых событий;
- статистика обнаружения транспортных средств.

Для каждого показателя предусмотреть фильтрацию по дате, району и типу события. Реализовать графическое представление данных в виде линейных, столбчатых и кольцевых диаграмм. Разработать раздел «Отчёты», предназначенный для формирования отчётной информации. Предусмотреть следующие виды отчётов:

- сводка событий за период;
- статистика по категориям;
- отчёт по камерам;
- отчёт по обработке происшествий;
- отчёт по районам;
- отчёт по демонстрационным результатам анализа изображений.

Для каждого отчёта предусмотреть выбор временного периода и необходимых фильтров. Реализовать формирование отчётов на сервере. Предусмотреть экспорт в PDF и Excel.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу мобильного приложения с логотипом SafeCity AI и персональным приветствием пользователя. В верхней части разместить название приложения, приветствие и роль авторизованного пользователя. Основная область должна содержать кнопки быстрого доступа:

- «Карта»;
- «События»;
- «Камеры»;
- «Розыск»;
- «Сообщения»;
- «Профиль».

Для раздела событий предусмотреть индикатор количества новых происшествий. В нижней части разместить навигационную панель:

- «Главная»;
- «Карта»;
- «События»;
- «Ещё».

При выборе элемента должен выполняться переход к соответствующему экрану. Информация должна загружаться с сервера с учётом роли пользователя.

Разработать мобильный экран «Карта», предназначенный для оперативного просмотра городской обстановки. В верхней части предусмотреть фильтры:

- «Все»;
- «Камеры»;
- «События»;
- «Патрули».

Основная область должна содержать интерактивную карту. На карте отображать маркеры камер, событий и демонстрационных патрульных групп. При выборе маркера должна открываться информационная карточка. В карточке события отображать:

- название;
- адрес;
- время;
- тип;
- приоритет.

При выборе карточки должен выполняться переход к подробной информации. Предусмотреть возможность изменения масштаба карты и обновления данных. Для учебной реализации допускается использование демонстрационных координат патрулей без отслеживания реального местоположения сотрудников.

Разработать мобильный экран «События». В верхней части предусмотреть переключатели:

- «Все»;
- «Розыск»;
- «ДТП»;
- «Проверка».

Основная область должна содержать список зарегистрированных событий. Для каждого события отображать:

- изображение;

- название;
- адрес;
- время;
- тип или статус.

В соответствии с графическим концептом предусмотреть демонстрационные события:

- совпадение с учебной базой;
- подозрительное поведение;
- скопление людей;
- ДТП;
- оставленный предмет.

При выборе элемента должна открываться подробная карточка. Предусмотреть обновление списка и фильтрацию по типу события.

Разработать мобильный экран «Карточка события», предназначенный для просмотра подробной информации о происшествии. В верхней части разместить изображение или кадр видеозаписи. Ниже отображать:

- тип события;
- статус;
- условный показатель совпадения при наличии;
- адрес;
- дату и время;
- описание;
- идентификатор камеры.

Предусмотреть блок миниатюр связанных изображений. В нижней части разместить кнопки:

- «Показать на карте»;
- «Передать в патруль».

Кнопка «Показать на карте» должна открывать карту с выделенным местоположением события. Кнопка «Передать в патруль» должна создавать назначение на проверку для выбранного сотрудника или группы в рамках учебной системы. После назначения события его статус должен обновляться. Предусмотреть возможность добавления комментария и фиксации результата проверки.

Разработать мобильный экран «Распознавание лиц», предназначенный для просмотра демонстрационных результатов сопоставления синтетических изображений. В верхней части разместить изображение учебного персонажа с выделенной областью лица. Ниже отображать:

- условный процент совпадения;

- изображение контрольного образца;
- условное имя;
- статус учебной карточки.

В соответствии с графическим концептом предусмотреть демонстрационный показатель «Совпадение: 85%». В нижней части разместить кнопку «Приостановить», предназначенную для приостановки демонстрационного просмотра или проверки результата. Результат должен иметь информационный характер и не использоваться для установления личности реального человека. Предусмотреть возможность перехода к связанной учебной карточке.

Разработать мобильный экран «Уведомления». В верхней части предусмотреть переключатели:

- «Все»;
- «Важные».

Основная область должна содержать список уведомлений. В соответствии с графическим концептом предусмотреть следующие типы сообщений:

- учебное совпадение с контрольным образцом;
- подозрительное поведение;
- новая камера в сети;
- системное уведомление.

Для каждого уведомления отображать:

- тематическую иконку;
- заголовок;
- краткое описание;
- дату и время.

При выборе уведомления должна открываться подробная информация или связанная карточка события. Предусмотреть возможность отметить уведомление как прочитанное. Новые уведомления должны отображаться в веб- и мобильной версиях.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение прикладной логики, проверку входных данных, управление доступом и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
CityZones	Городские районы и зоны мониторинга
Cameras	Камеры, расположение, статусы, источники видео

VideoRecords	Демонстрационные видеозаписи и изображения
Events	События, типы, приоритеты, статусы
EventTypes	Справочник типов событий
EventMedia	Изображения и видеоматериалы событий
DetectedObjects	Обнаруженные объекты и координаты разметки
DemoPersons	Синтетические персонажи учебной базы
DemoMatches	Условные результаты сопоставления изображений
PatrolUnits	Демонстрационные патрульные группы
EventAssignments	Назначения событий сотрудникам
EventHistory	История изменения статусов событий
Notifications	Уведомления и отметки прочтения
Reports	Сформированные отчёты и параметры
AuditLogs	Журнал действий пользователей

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение показателей Dashboard;
- управление камерами;
- получение списка видеозаписей;
- получение демонстрационных результатов обнаружения объектов;
- получение событий;
- создание и редактирование событий;
- изменение статусов;
- получение данных для карты;
- управление учебными карточками;
- получение результатов демонстрационного сопоставления;
- назначение ответственных;
- получение статистики;
- управление уведомлениями;
- формирование и экспорт отчётов.

Предусмотреть разграничение доступа по ролям диспетчера, оператора, патрульного сотрудника и администратора. Необходимо реализовать проверку прав доступа на сервере. Для обеспечения безопасности предусмотреть:

- хранение паролей в виде защищённых хешей;
- авторизацию на основе сессий или JWT;
- проверку корректности входных данных;
- разграничение доступа к служебной информации;
- защиту API от несанкционированных запросов;
- журналирование критически важных операций;
- безопасную обработку загружаемых файлов;
- ограничение доступа к изображениям и видеоматериалам.

Демонстрационные данные о людях должны быть синтетическими или обезличенными. Учебная система не должна подключаться к реальным базам розыска, выполнять биометрическую идентификацию граждан или принимать автоматизированные решения, затрагивающие права людей.

Основные пользовательские сценарии

Сценарий 1. Просмотр городской обстановки.

Диспетчер открывает веб-приложение SafeCity AI и авторизуется. Система загружает сведения о камерах и зарегистрированных событиях. На Dashboard отображаются количество активных камер, число событий за день, количество демонстрационных совпадений и обнаруженных транспортных средств. Пользователь выбирает дату. Система обновляет показатели на основе выбранного периода.

Сценарий 2. Просмотр видеопотока.

Оператор открывает раздел «Видеопотоки». Система отображает список доступных камер. Пользователь выбирает камеру «Центральная площадь». Открывается демонстрационная видеозапись с разметкой обнаруженных объектов. Оператор может приостановить воспроизведение, включить полноэкранный режим и перейти к связанным событиям.

Сценарий 3. Просмотр карты событий.

Диспетчер открывает раздел «Карта». Система отображает расположение камер и зарегистрированных происшествий. Пользователь выбирает маркер события. Открывается информационная карточка с названием, адресом, временем и статусом. Диспетчер переходит к подробной карточке происшествия.

Сценарий 4. Регистрация нового события.

Оператор обнаруживает на демонстрационной видеозаписи оставленный предмет. Пользователь открывает форму создания события. Вводит тип, описание, адрес, дату и время. Прикрепляет изображение. Система сохраняет событие в PostgreSQL со статусом «Новое». Событие появляется в общем списке и на карте.

Сценарий 5. Обработка происшествия.

Диспетчер открывает список событий. Выбирает событие «Подозрительное поведение». Просматривает описание, местоположение и приложенные материалы. Изменяет статус на «На проверке». Назначает ответственного оператора. Система сохраняет изменения и создаёт соответствующее уведомление.

Сценарий 6. Просмотр демонстрационного результата сопоставления.

Оператор открывает раздел «Распознавание лиц». Система отображает синтетическое изображение персонажа и условный результат сопоставления с учебным образцом. Пользователь открывает связанную учебную карточку. Система отображает условное имя, изображение и описание персонажа. Оператор отмечает демонстрационный результат как проверенный или отклонённый.

Сценарий 7. Передача события в патруль.

Диспетчер открывает карточку события. Выбирает действие «Передать в патруль». Система отображает список доступных демонстрационных патрульных групп. Пользователь выбирает ответственную группу. Сервер создаёт запись о назначении. Событие получает статус «В работе». Назначенному пользователю отправляется уведомление в мобильном интерфейсе.

Сценарий 8. Работа патрульного сотрудника.

Патрульный сотрудник открывает мобильное приложение SafeCity AI. На главном экране отображается уведомление о назначенном событии. Пользователь открывает карточку происшествия. Просматривает описание, изображения и местоположение. Нажимает кнопку «Показать на карте». После учебной проверки добавляет комментарий и изменяет статус события. Информация сохраняется на сервере и становится доступной диспетчеру.

Сценарий 9. Анализ динамики событий.

Аналитик или диспетчер открывает раздел «Аналитика». Система отображает график динамики событий. Пользователь выбирает временной период. Сервер выполняет агрегацию данных. График обновляется. Пользователь анализирует изменение количества событий по категориям.

Сценарий 10. Просмотр распределения событий по типам.

Диспетчер открывает блок «Типы событий». Система отображает кольцевую диаграмму распределения происшествий. Пользователь выбирает категорию «ДТП». На экране отображается количество событий выбранного типа. Пользователь переходит к отфильтрованному списку происшествий.

Сценарий 11. Получение уведомления.

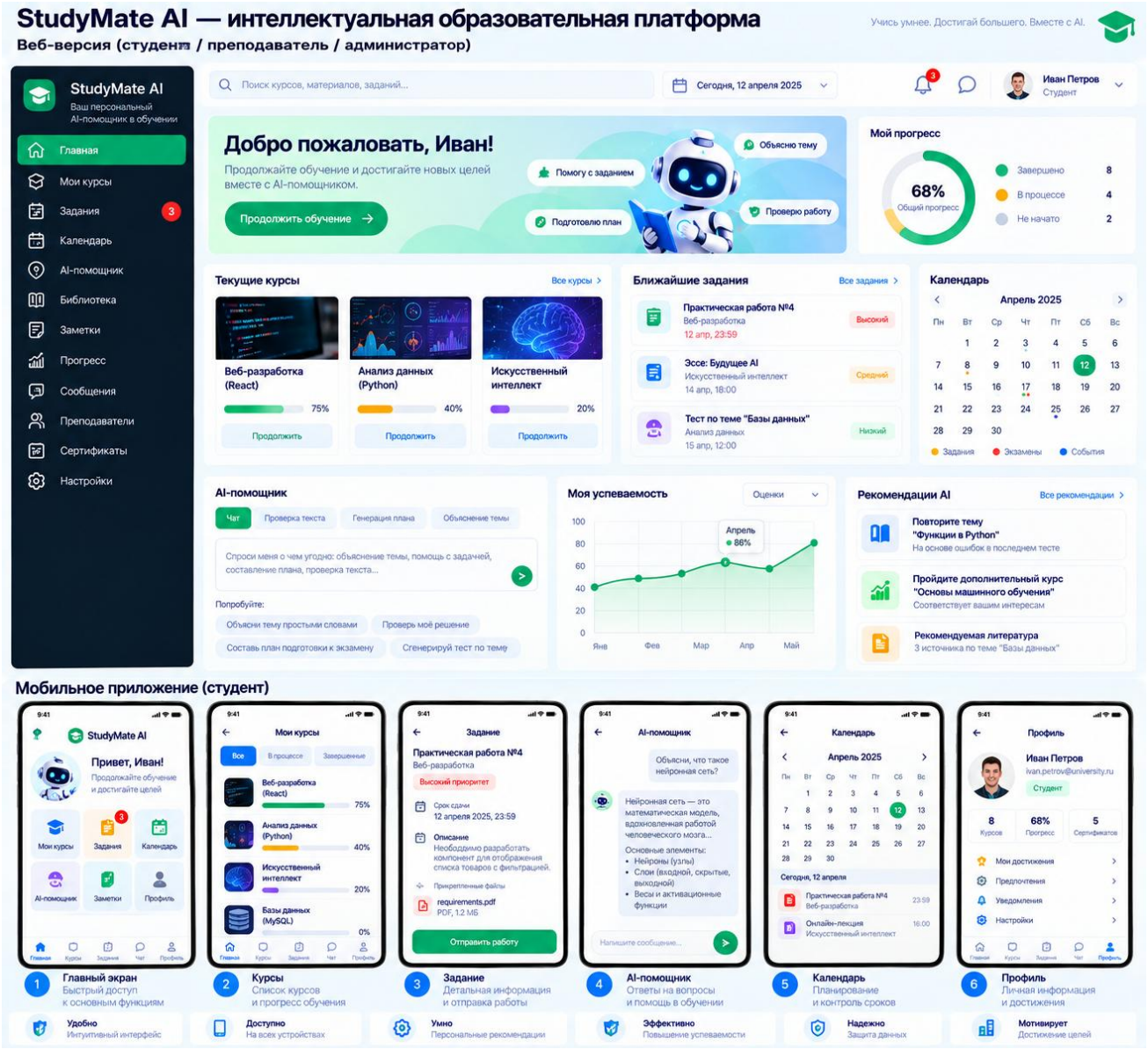
В системе регистрируется новое событие повышенного приоритета. Сервер создаёт уведомление для соответствующих пользователей. В верхней панели веб-приложения обновляется счётчик непрочитанных сообщений. В мобильной версии появляется новое уведомление. Пользователь открывает сообщение и переходит к карточке события.

Сценарий 12. Формирование аналитического отчёта.

Диспетчер открывает раздел «Отчёты». Выбирает период и тип отчёта. Система загружает необходимые данные и выполняет расчёт показателей. На экране отображается предварительный просмотр. Пользователь выбирает экспорт в PDF или Excel. Сервер формирует файл и предоставляет его для скачивания.

Вариант 12. StudyMate AI — интеллектуальная образовательная платформа

Разработать Fullstack веб-приложение **StudyMate AI**, предназначенное для организации персонализированного образовательного процесса, управления учебными курсами, выполнения и проверки заданий, отслеживания успеваемости студентов, планирования учебной деятельности и предоставления интеллектуальной помощи в обучении.



Система должна обеспечивать централизованное хранение и обработку информации о пользователях, учебных курсах, образовательных материалах, практических работах, тестированиях, сроках выполнения заданий, результатах обучения, индивидуальном прогрессе и образовательных рекомендациях.

Приложение должно предоставлять студентам удобные инструменты для самостоятельного обучения, просмотра учебных материалов, выполнения заданий,

взаимодействия с преподавателями, контроля академических результатов и получения персонализированной помощи от AI-ассистента.

Преподаватели должны иметь возможность создавать и редактировать учебные курсы, публиковать материалы, назначать задания, устанавливать сроки выполнения, проверять отправленные работы, выставлять оценки и анализировать успеваемость обучающихся.

Администраторы должны обеспечивать управление пользователями, учебными курсами, образовательными категориями, настройками системы и правами доступа.

В соответствии с представленным графическим концептом приложение должно включать интеллектуального помощника, который предоставляет объяснения учебных тем, помогает разобраться в заданиях, формирует индивидуальные планы подготовки, предлагает дополнительные материалы и генерирует рекомендации по дальнейшему обучению.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для студента, преподавателя и администратора** — предназначена для управления образовательным процессом, просмотра курсов, выполнения и проверки заданий, работы с AI-помощником, анализа успеваемости, планирования учебных мероприятий и формирования отчётности.
- **Мобильная версия для студента** — предназначена для быстрого доступа к курсам, просмотра заданий, отправки выполненных работ, общения с AI-помощником, работы с календарём и контроля личного прогресса.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт StudyMate AI, сохранив его основные композиционные решения, структуру навигации, цветовую схему, расположение информационных блоков и способы визуализации образовательных показателей.

Для учебной реализации допускается использование заранее подготовленных образовательных материалов, демонстрационных курсов, тестовых заданий, условных оценок, имитации ответов AI-помощника и генерации рекомендаций на основе заданных правил.

Реализация собственной большой языковой модели, полноценной интеллектуальной системы автоматической проверки сложных работ и промышленной рекомендательной системы не является обязательной.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью, верхней областью управления и персонализированным информационным содержимым.

В боковом меню предусмотреть следующие разделы:

- Главная;
- Мои курсы;
- Задания;
- Календарь;
- AI-помощник;
- Библиотека;
- Заметки;
- Прогресс;
- Сообщения;
- Преподаватели;
- Сертификаты;
- Настройки.

В верхней панели расположить строку поиска по курсам, учебным материалам и заданиям, элемент выбора даты, индикатор уведомлений, кнопку сообщений и информацию об авторизованном пользователе. Основная область Dashboard должна содержать персональный приветственный блок с именем студента, кратким мотивационным сообщением и кнопкой «Продолжить обучение». В соответствии с графическим концептом в приветственном блоке разместить изображение виртуального AI-помощника и информационные элементы:

- «Помогу с заданием»;
- «Подготовлю план»;
- «Объясню тему»;
- «Проверю работу».

При выборе соответствующего действия должен открываться AI-помощник с предварительно выбранным режимом работы. На главной странице также необходимо разместить следующие информационные блоки:

1. **Мой прогресс** — общий процент завершения обучения и распределение курсов по статусам.
2. **Текущие курсы** — карточки активных образовательных курсов с процентом выполнения.
3. **Ближайшие задания** — список предстоящих учебных работ с указанием сроков и приоритетов.
4. **Календарь** — отображение учебных событий и контрольных дат.
5. **AI-помощник** — интерактивная область для отправки запросов.
6. **Моя успеваемость** — график изменения образовательных результатов.
7. **Рекомендации AI** — персонализированные предложения по дальнейшему обучению.

Информация на Dashboard должна загружаться с сервера и формироваться на основе данных конкретного пользователя. При изменении статусов заданий, оценок или прогресса соответствующие показатели должны обновляться.

Разработать раздел «Мои курсы», предназначенный для просмотра и управления образовательными курсами. Для каждого курса предусмотреть хранение:

- уникального идентификатора;
- названия;
- описания;
- категории;
- изображения обложки;
- преподавателя;
- даты начала;
- даты окончания;
- общего количества учебных модулей;
- количества завершённых модулей;
- статуса;
- процента выполнения.

В соответствии с графическим концептом предусмотреть демонстрационные курсы:

- «Веб-разработка (React)»;
- «Анализ данных (Python)»;
- «Искусственный интеллект»;
- «Базы данных (MySQL)».

Для каждого курса отображать карточку с изображением, названием, индикатором прогресса и кнопкой «Продолжить». Предусмотреть следующие статусы курсов:

- «Не начат»;
- «В процессе»;
- «Завершён».

Реализовать фильтрацию курсов по статусу, категории и преподавателю. При выборе курса должна открываться подробная страница с описанием, списком модулей, учебными материалами, заданиями и информацией о прогрессе. Преподаватель должен иметь возможность создавать, редактировать и архивировать курсы, добавлять учебные модули и назначать студентов. Данные курсов должны храниться в PostgreSQL.

Разработать функциональность управления структурой учебных курсов. Каждый курс должен включать набор учебных модулей. Для каждого модуля предусмотреть:

- идентификатор;
- название;
- описание;

- порядковый номер;
- список учебных материалов;
- список заданий;
- статус изучения.

Внутри модуля предусмотреть возможность размещения:

- текстовых материалов;
- изображений;
- PDF-документов;
- ссылок на видеолекции;
- презентаций;
- практических заданий;
- контрольных вопросов.

Студент должен иметь возможность открывать материалы, отмечать их как изученные и переходить к следующему модулю. Прогресс курса должен рассчитываться на основе завершённых учебных элементов по выбранному правилу, одинаковому для веб- и мобильной версий. Преподаватель должен иметь возможность добавлять, редактировать и удалять материалы. Предусмотреть проверку допустимых форматов и размеров загружаемых файлов.

Разработать раздел «Задания», предназначенный для просмотра, выполнения и проверки учебных работ. Для каждого задания предусмотреть:

- уникальный идентификатор;
- название;
- описание;
- связанный курс;
- преподавателя;
- дату публикации;
- срок выполнения;
- приоритет;
- максимальный балл;
- статус выполнения;
- прикреплённые материалы.

В соответствии с графическим концептом предусмотреть демонстрационные задания:

1. «Практическая работа №4» — курс «Веб-разработка», высокий приоритет.
2. «Эссе: Будущее AI» — курс «Искусственный интеллект», средний приоритет.
3. «Тест по теме "Базы данных"» — курс «Анализ данных», низкий приоритет.

Для каждого задания отображать название, курс, срок сдачи и цветовой индикатор приоритета.

Предусмотреть следующие статусы:

- «Не начато»;
- «В процессе»;
- «Отправлено»;
- «На проверке»;
- «Проверено»;
- «Требует доработки».

Реализовать поиск, сортировку и фильтрацию заданий по курсу, сроку, приоритету и статусу. При выборе задания должна открываться подробная страница с описанием, требованиями, вложениями и формой отправки результата. Студент должен иметь возможность прикрепить файл, добавить текстовый комментарий и отправить работу на проверку. После отправки сервер должен сохранить результат, дату отправки и обновить статус. Преподаватель должен иметь возможность просматривать отправленные работы, выставлять оценки и оставлять комментарии.

Разработать раздел «AI-помощник», предназначенный для интеллектуальной поддержки студентов в процессе обучения. В соответствии с графическим концептом предусмотреть интерактивный чат с виртуальным помощником. В верхней части блока разместить переключатели режимов:

- «Чат»;
- «Проверка текста»;
- «Генерация плана»;
- «Объяснение темы».

Основная область должна содержать историю сообщений пользователя и ответы помощника. В нижней части разместить поле ввода и кнопку отправки сообщения. Предусмотреть следующие возможности:

Режим «Чат». Студент вводит вопрос по учебной теме. Система обрабатывает запрос и отображает ответ. Предусмотреть сохранение истории переписки.

Режим «Проверка текста». Студент вводит текст или вставляет фрагмент выполненного задания. Система формирует рекомендации по улучшению структуры, логики изложения и понятности текста. Для учебной реализации допускается использование шаблонных рекомендаций.

Режим «Генерация плана». Студент указывает тему и желаемую продолжительность подготовки. Система формирует структурированный учебный план с этапами изучения материала.

Режим «Объяснение темы». Студент вводит название учебной темы или вопрос. Система отображает объяснение, основные понятия и пример.

В соответствии с графическим концептом предусмотреть кнопки быстрых запросов:

- «Объясни тему простыми словами»;
- «Проверь моё решение»;
- «Составь план подготовки к экзамену»;
- «Сгенерируй тест по теме».

Для учебной реализации допускается использование заранее подготовленной базы вопросов и ответов, шаблонной генерации и правил выбора рекомендаций. При наличии технической возможности допускается интеграция с внешним API языковой модели. Ключи доступа должны храниться только на сервере. Ответы AI-помощника должны иметь рекомендательный характер и не заменять проверку преподавателя.

Разработать раздел «Календарь», предназначенный для планирования учебной деятельности. В соответствии с графическим концептом предусмотреть отображение календаря с переключением месяцев. На календаре должны отображаться:

- сроки сдачи заданий;
- экзамены;
- контрольные работы;
- онлайн-лекции;
- консультации;
- личные учебные события.

Для разных типов событий использовать цветовые индикаторы. В соответствии с графическим концептом предусмотреть:

- оранжевый — задания;
- красный — экзамены;
- синий — другие учебные события.

При выборе даты должен отображаться список запланированных мероприятий. Для каждого события предусмотреть:

- название;
- описание;
- тип;
- дату;
- время;
- связанный курс;
- статус.

Студент должен иметь возможность создавать личные события и напоминания. Преподаватель должен иметь возможность публиковать учебные мероприятия для участников курса. Предусмотреть отображение ближайших событий на главной панели.

Разработать раздел «Прогресс», предназначенный для отображения индивидуальных результатов обучения. В соответствии с графическим концептом на Dashboard предусмотреть кольцевую диаграмму общего прогресса. В центральной части диаграммы отображать процент выполнения образовательной программы. Рядом разместить легенду с количеством курсов по статусам:

- завершено;
- в процессе;
- не начато.

Для демонстрационных данных допускается использовать значения:

- общий прогресс — 68%;
- завершено — 8;
- в процессе — 4;
- не начато — 2.

Общий прогресс должен рассчитываться на основе выбранной методики и данных о завершении курсов и учебных элементов.

В разделе «Прогресс» предусмотреть отображение:

- количества изучаемых курсов;
- количества завершённых курсов;
- количества выполненных заданий;
- среднего балла;
- процента выполнения;
- динамики результатов;
- полученных сертификатов.

Предусмотреть фильтрацию по курсам и временным периодам. Данные должны рассчитываться на сервере.

Разработать аналитический блок «Моя успеваемость». В соответствии с графическим концептом представить данные в виде линейного графика. По горизонтальной оси отображать месяцы или другие выбранные временные интервалы. По вертикальной оси — показатель успеваемости в процентах. В верхней части блока предусмотреть выбор типа отображаемого показателя:

- оценки;
- выполненные задания;
- прогресс обучения.

При наведении на точку графика отображать период и соответствующее значение. Предусмотреть расчёт:

- среднего балла;
- количества выполненных заданий;
- доли успешно выполненных заданий;
- динамики успеваемости;
- количества просроченных работ.

Для демонстрационного примера допускается отображение показателя 86% за апрель. При изменении периода данные графика должны обновляться. Все аналитические показатели должны формироваться на сервере.

Разработать блок «Рекомендации AI», предназначенный для формирования индивидуальных предложений по обучению. В соответствии с графическим концептом предусмотреть карточки рекомендаций с иконками, названиями и пояснениями. Использовать следующие демонстрационные рекомендации:

1. «Повторите тему "Функции в Python"» — на основе ошибок в последнем тесте.
2. «Пройдите дополнительный курс "Основы машинного обучения"» — соответствует образовательным интересам студента.
3. «Рекомендуемая литература» — дополнительные источники по теме «Базы данных».

Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- тип;
- название;
- описание;
- основание формирования;
- связанный курс;
- дату создания;
- статус просмотра.

При выборе рекомендации должен выполняться переход к соответствующему материалу или курсу. Предусмотреть возможность отметить рекомендацию как выполненную или скрыть её. Для учебной реализации допускается использовать алгоритм на основе правил. Например, при низком результате тестирования по определённой теме система должна предложить повторение материала. При завершении курса система может рекомендовать следующий курс соответствующей категории.

Разработать раздел «Библиотека», предназначенный для централизованного хранения учебных ресурсов. Предусмотреть следующие категории материалов:

- учебники;
- методические пособия;
- лекции;
- презентации;

- видеоматериалы;
- справочники;
- дополнительные ресурсы.

Для каждого материала предусмотреть:

- название;
- описание;
- категорию;
- автора;
- связанный курс;
- тип файла;
- дату добавления;
- ссылку на файл или ресурс.

Реализовать поиск по названию, категории и курсу. Предусмотреть фильтрацию по типу материала. Студент должен иметь возможность открывать материалы и добавлять их в избранное. Преподаватель должен иметь возможность загружать и редактировать материалы. Администратор должен управлять категориями и доступностью ресурсов.

Разработать раздел «Заметки», предназначенный для хранения личных учебных записей. Для каждой заметки предусмотреть:

- идентификатор;
- заголовок;
- содержимое;
- дату создания;
- дату изменения;
- связанный курс;
- категорию.

Студент должен иметь возможность:

- создавать заметки;
- редактировать содержимое;
- удалять заметки;
- выполнять поиск;
- группировать записи по курсам.

Предусмотреть возможность добавления заметки непосредственно со страницы учебного материала. Все заметки должны сохраняться в PostgreSQL и быть доступны только владельцу.

Разработать раздел «Сообщения», предназначенный для обмена информацией между студентами и преподавателями. Предусмотреть список диалогов и область просмотра сообщений. Для каждого сообщения хранить:

- идентификатор;
- отправителя;
- получателя;
- текст;
- дату и время;
- статус прочтения;
- вложения при наличии.

Студент должен иметь возможность отправлять вопросы преподавателю. Преподаватель должен иметь возможность отвечать на сообщения студентов. В разделе «Преподаватели» предусмотреть отображение списка преподавателей с указанием имени, дисциплин и контактной информации в пределах учебной системы. При выборе преподавателя должна открываться его карточка. Предусмотреть возможность начать диалог. Новые сообщения должны отображаться в системе уведомлений.

Разработать раздел «Сертификаты», предназначенный для просмотра документов о завершении учебных курсов. Для каждого сертификата предусмотреть:

- уникальный номер;
- название курса;
- имя студента;
- дату выдачи;
- преподавателя;
- итоговый результат;
- ссылку на документ.

Сертификат должен формироваться после выполнения установленных требований курса. Предусмотреть возможность просмотра и скачивания сертификата в PDF. Для учебной реализации допускается использование демонстрационного шаблона сертификата. В профиле пользователя отображать общее количество полученных сертификатов.

Разработать систему авторизации и управления учётными записями. Предусмотреть следующие роли:

- студент;
- преподаватель;
- администратор.

Студент должен иметь доступ к своим курсам, заданиям, оценкам, календарю, AI-помощнику, библиотеке, заметкам, сообщениям и личному прогрессу.

Преподаватель должен иметь возможность управлять собственными курсами, публиковать материалы, создавать задания, проверять работы, выставлять оценки, взаимодействовать со студентами и просматривать аналитику по своим курсам.

Администратор должен управлять пользователями, курсами, категориями, настройками и правами доступа.

Для каждого пользователя предусмотреть:

- уникальный идентификатор;
- фамилию и имя;
- адрес электронной почты;
- логин;
- защищённые учётные данные;
- роль;
- изображение профиля;
- статус;
- дату регистрации.

Предусмотреть редактирование профиля и смену пароля. Разграничение прав доступа должно выполняться на сервере.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу мобильного приложения StudyMate AI. В верхней части разместить логотип приложения, персональное приветствие студента и изображение виртуального помощника. Ниже разместить кнопки быстрого доступа:

- «Мои курсы»;
- «Задания»;
- «Календарь»;
- «AI-помощник»;
- «Заметки»;
- «Профиль».

Для раздела «Задания» предусмотреть индикатор количества актуальных заданий.

В нижней части разместить навигационную панель:

- «Главная»;
- «Курсы»;
- «Задания»;
- «Чат»;

- «Профиль».

При выборе элемента должен выполняться переход к соответствующему экрану. Информация должна загружаться с сервера с учётом авторизованного пользователя.

Разработать мобильный экран «Мои курсы». В верхней части предусмотреть переключатели:

- «Все»;
- «В процессе»;
- «Завершённые».

Основная область должна содержать список учебных курсов. Для каждого курса отображать:

- изображение;
- название;
- процент выполнения;
- цветовой индикатор прогресса.

В соответствии с графическим концептом предусмотреть демонстрационные значения:

- «Веб-разработка (React)» — 75%;
- «Анализ данных (Python)» — 40%;
- «Искусственный интеллект» — 20%;
- «Базы данных (MySQL)» — 0%.

При выборе курса должна открываться подробная информация с учебными модулями и материалами. Студент должен иметь возможность продолжить обучение с последнего изученного материала.

Разработать мобильный экран «Задание», предназначенный для просмотра и отправки выполненной работы. В верхней части отображать название задания и связанный курс. Ниже предусмотреть:

- индикатор приоритета;
- срок сдачи;
- описание;
- требования;
- список приложенных файлов.

В соответствии с графическим концептом использовать демонстрационное задание «Практическая работа №4» по курсу «Веб-разработка». Предусмотреть отображение срока сдачи и файла с требованиями. В нижней части разместить кнопку «Отправить работу». При выборе кнопки должна открываться форма загрузки результата. Студент должен иметь возможность прикрепить файл и добавить комментарий. После отправки

данные должны сохраняться на сервере. Статус задания должен изменяться на «Отправлено».

Разработать мобильный экран «AI-помощник». Основная область должна содержать историю диалога между студентом и виртуальным помощником. В соответствии с графическим концептом предусмотреть пример запроса: «Объясни, что такое нейронная сеть?» Система должна отображать ответ в виде текстового сообщения. Ответ может содержать:

- определение;
- объяснение;
- список основных понятий;
- пример;
- рекомендации по дальнейшему изучению.

В нижней части разместить поле ввода сообщения и кнопку отправки. Предусмотреть прокрутку истории диалога. Сообщения должны сохраняться на сервере. Для учебной реализации допускается использование заранее подготовленных ответов.

Разработать мобильный экран «Календарь». В верхней части отображать текущий месяц и кнопки переключения. Основная область должна содержать календарную сетку. Для дат, на которые запланированы задания или другие учебные события, предусмотреть цветовые индикаторы. При выборе даты ниже календаря должен отображаться список событий. В соответствии с графическим концептом предусмотреть демонстрационные события:

- «Практическая работа №4»;
- «Онлайн-лекция — Искусственный интеллект».

Для каждого события отображать название, связанный курс и время. При выборе элемента должен выполняться переход к соответствующему заданию или мероприятию.

Разработать мобильный экран «Профиль». В верхней части разместить фотографию, имя, адрес электронной почты и роль пользователя. В соответствии с графическим концептом предусмотреть три информационные карточки:

- количество курсов;
- общий прогресс;
- количество сертификатов.

Ниже разместить пункты меню:

- «Мои достижения»;
- «Предпочтения»;
- «Уведомления»;
- «Настройки».

При выборе соответствующего пункта должен открываться отдельный экран. Предусмотреть возможность редактирования личных данных и изменения настроек. Показатели профиля должны загружаться с сервера.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение бизнес-логики, проверку входных данных, управление правами доступа и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Courses	Учебные курсы, описания, категории
CourseModules	Модули и структура курсов
LearningMaterials	Учебные материалы и файлы
Enrollments	Записи студентов на курсы
Assignments	Задания, сроки, приоритеты
Submissions	Отправленные работы студентов
Grades	Оценки и комментарии преподавателей
Tests	Тесты и параметры прохождения
TestQuestions	Вопросы и варианты ответов
TestResults	Результаты тестирования
CalendarEvents	Учебные события и сроки
LearningProgress	Индивидуальный прогресс
AIConversations	Диалоги с AI-помощником
AIMessages	Сообщения в диалогах
AIRecommendations	Персональные рекомендации
LibraryResources	Ресурсы библиотеки
Notes	Личные заметки
Messages	Сообщения между пользователями
Notifications	Уведомления
Certificates	Сертификаты о завершении курсов
AuditLogs	Журнал действий

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение данных профиля;
- управление курсами;
- управление учебными модулями;
- получение образовательных материалов;
- запись студентов на курсы;
- получение списка заданий;
- отправку выполненных работ;
- проверку заданий и выставление оценок;
- получение результатов тестирования;

- расчёт прогресса обучения;
- получение календарных событий;
- обработку запросов AI-помощника;
- сохранение истории диалогов;
- формирование рекомендаций;
- управление заметками;
- обмен сообщениями;
- получение уведомлений;
- формирование сертификатов;
- получение аналитических данных.

Предусмотреть разграничение доступа по ролям студента, преподавателя и администратора. Для обеспечения безопасности необходимо реализовать:

- хранение паролей в виде защищённых хешей;
- авторизацию на основе сессий или JWT;
- проверку корректности входных данных;
- ограничение доступа к персональным данным;
- проверку прав на просмотр и изменение оценок;
- защиту загружаемых файлов;
- безопасное хранение ключей внешних API;
- журналирование значимых операций.

Студент должен иметь доступ только к собственным оценкам, результатам, заметкам и личным данным. Преподаватель должен иметь доступ к образовательной информации студентов только в пределах назначенных ему курсов.

Основные пользовательские сценарии

Сценарий 1. Авторизация и просмотр главной страницы.

Студент открывает приложение StudyMate AI и авторизуется. Система загружает сведения о пользователе, его курсах, заданиях и прогрессе. На Dashboard отображается персональное приветствие, список текущих курсов, ближайшие задания и календарь. Студент просматривает общую информацию и выбирает действие «Продолжить обучение».

Сценарий 2. Продолжение изучения курса.

Студент открывает раздел «Мои курсы». Система отображает список курсов с процентами выполнения. Пользователь выбирает курс «Веб-разработка (React)». Открывается страница курса с учебными модулями. Студент выбирает следующий доступный материал. После изучения отмечает материал как завершённый. Система сохраняет результат и пересчитывает прогресс.

Сценарий 3. Выполнение практического задания.

Студент открывает раздел «Задания». Выбирает «Практическая работа №4». Просматривает описание, требования и срок сдачи. Скачивает приложенный файл с требованиями. Выполняет работу и загружает результат. Нажимает кнопку «Отправить работу». Система сохраняет отправленный файл и изменяет статус задания на «Отправлено».

Сценарий 4. Проверка задания преподавателем.

Преподаватель авторизуется в системе. Открывает список заданий своего курса. Выбирает отправленную студентом работу. Просматривает приложенный файл. Выставляет оценку и добавляет комментарий. Система сохраняет результат проверки. Студент получает уведомление о выставленной оценке.

Сценарий 5. Использование AI-помощника.

Студент открывает раздел «AI-помощник». Выбирает режим «Объяснение темы». Вводит вопрос «Что такое нейронная сеть?». Система обрабатывает запрос. На экране отображается объяснение с основными понятиями и примерами. История диалога сохраняется в PostgreSQL.

Сценарий 6. Формирование плана подготовки.

Студент открывает AI-помощника. Выбирает режим «Генерация плана». Указывает тему «Основы Python» и продолжительность подготовки. Система формирует последовательность учебных этапов. Студент просматривает план и сохраняет его. Сохранённый план становится доступным в личном кабинете.

Сценарий 7. Просмотр календаря.

Студент открывает раздел «Календарь». Система отображает текущий месяц. На датах учебных событий появляются цветовые индикаторы. Студент выбирает дату сдачи практической работы. Открывается список мероприятий. Пользователь переходит к соответствующему заданию.

Сценарий 8. Анализ личного прогресса.

Студент открывает раздел «Прогресс». Система отображает общий процент выполнения обучения. Пользователь просматривает количество завершённых курсов, выполненных заданий и полученных сертификатов. Выбирает конкретный курс. Система отображает детальную статистику его прохождения.

Сценарий 9. Просмотр успеваемости.

Студент открывает блок «Моя успеваемость». Система отображает график изменения результатов. Пользователь выбирает показатель «Оценки». График обновляется. Студент просматривает динамику успеваемости по месяцам.

Сценарий 10. Получение персональной рекомендации.

Студент завершает тест по теме «Функции в Python» с низким результатом. Система анализирует результат тестирования. На основе заданного правила формируется рекомендация повторить соответствующую тему. Рекомендация появляется на Dashboard. Студент выбирает её и переходит к учебному материалу.

Сценарий 11. Работа с заметками.

Студент открывает учебный материал. Выбирает действие «Добавить заметку». Вводит заголовок и текст. Сохраняет запись. Система связывает заметку с соответствующим курсом. Позднее студент открывает раздел «Заметки» и редактирует сохранённую информацию.

Сценарий 12. Получение сертификата.

Студент завершает все обязательные модули курса и выполняет итоговые требования. Система проверяет условия завершения. Курс получает статус «Завершён». Формируется демонстрационный сертификат. Документ появляется в разделе «Сертификаты». Студент скачивает сертификат в формате PDF.

Сценарий 13. Общение с преподавателем.

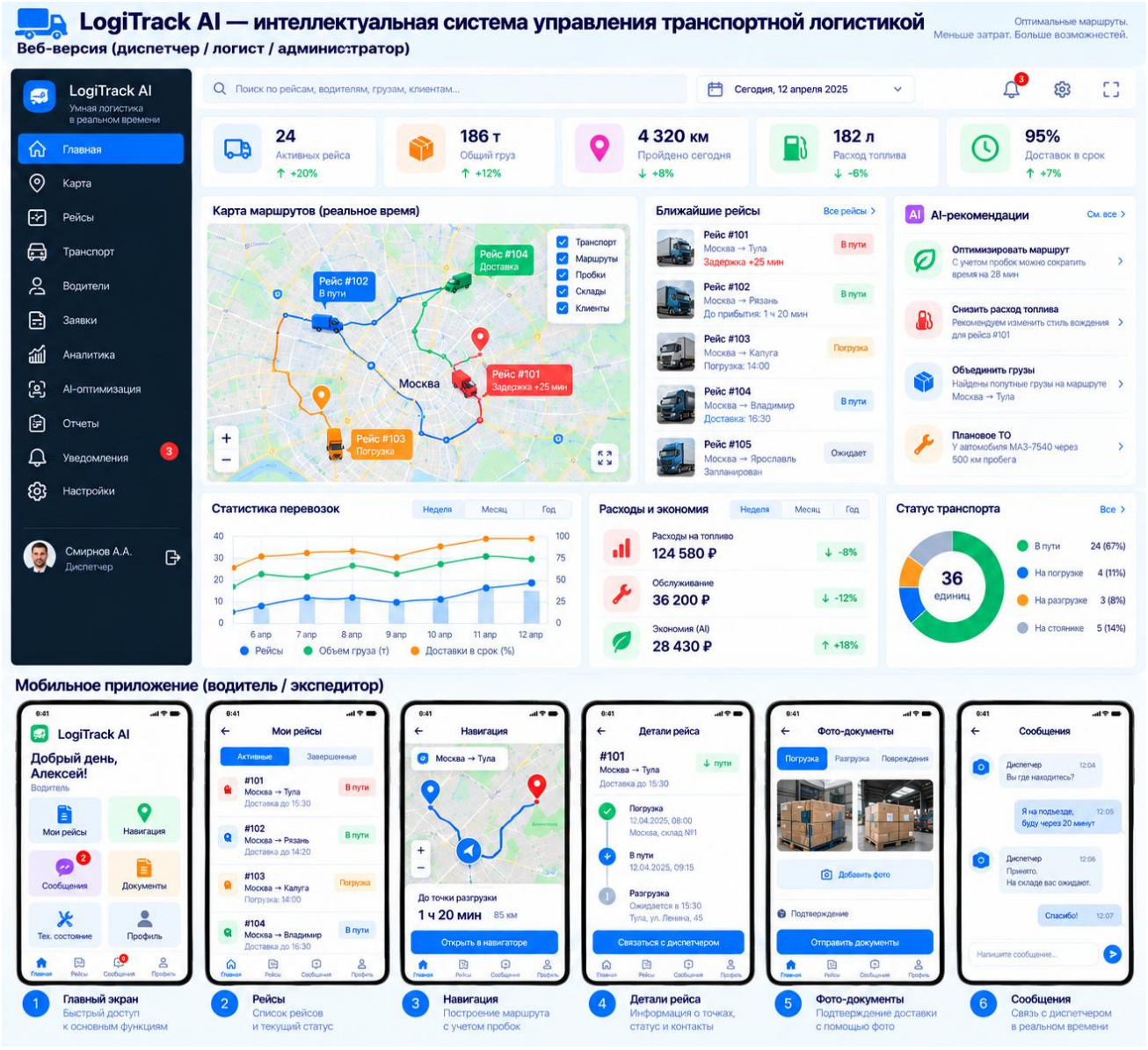
Студент открывает раздел «Преподаватели». Выбирает преподавателя курса. Переходит к диалогу. Отправляет вопрос по практическому заданию. Преподаватель получает уведомление. После ответа преподавателя студент видит новое сообщение.

Сценарий 14. Работа с мобильным приложением.

Студент открывает мобильную версию StudyMate AI. На главном экране отображаются текущие курсы, задания и календарь. Пользователь выбирает задание с ближайшим сроком сдачи. Просматривает описание и прикреплённые материалы. Загружает выполненную работу. Система сохраняет результат на сервере. После обновления веб-версии статус задания отображается как «Отправлено».

Вариант 13. LogiTrack AI — интеллектуальная система управления транспортной логистикой

Разработать Fullstack веб-приложение **LogiTrack AI**, предназначенное для автоматизации управления транспортной логистикой, планирования грузоперевозок, мониторинга транспортных средств, контроля выполнения рейсов, управления заявками на доставку, анализа эксплуатационных расходов и интеллектуальной оптимизации логистических процессов.



Система должна обеспечивать централизованное хранение и обработку информации о транспортных средствах, водителях, грузах, клиентах, заявках, маршрутах, рейсах, пунктах погрузки и разгрузки, расходе топлива, техническом состоянии автопарка и результатах перевозок.

Приложение должно предоставлять диспетчерам и логистам инструменты для оперативного контроля транспортных операций, отслеживания местоположения

автомобилей, выявления отклонений от маршрутов и графиков, распределения заявок между транспортными средствами и формирования аналитической отчётности.

Водители и экспедиторы должны иметь возможность просматривать назначенные рейсы, получать маршрутные инструкции, контролировать этапы доставки, взаимодействовать с диспетчером, передавать информацию о выполнении перевозки и загружать подтверждающие документы.

Администраторы должны обеспечивать управление пользователями, транспортными средствами, справочниками, настройками системы и правами доступа.

В соответствии с представленным графическим концептом приложение должно включать интеллектуальный модуль **AI-оптимизации**, который анализирует логистические показатели, выявляет потенциальные проблемы и формирует рекомендации по сокращению времени доставки, снижению расхода топлива, объединению совместимых грузов и планированию технического обслуживания.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для диспетчера, логиста и администратора** — предназначена для управления транспортными перевозками, просмотра интерактивной карты маршрутов, контроля рейсов и автопарка, обработки заявок, анализа расходов, получения AI-рекомендаций и формирования отчётности.
- **Мобильная версия для водителя и экспедитора** — предназначена для просмотра назначенных рейсов, навигации по маршруту, изменения статусов перевозки, передачи фотоотчётов, обмена сообщениями с диспетчером и получения оперативных уведомлений.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL.

В качестве основы оформления необходимо использовать представленный графический концепт LogiTrack AI, сохранив основные композиционные решения, структуру навигации, цветовую схему, расположение информационных блоков, способы визуализации логистических показателей и организацию мобильных экранов.

Для учебной реализации допускается использование заранее подготовленных маршрутов, демонстрационных транспортных средств, тестовых заявок, имитации GPS-перемещения автомобилей, условных показателей расхода топлива и алгоритмов формирования рекомендаций на основе заданных правил.

Реализация собственной системы спутникового мониторинга, промышленного навигационного сервиса, сложных алгоритмов машинного обучения и интеграции с реальными телематическими устройствами не является обязательной.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью, верхней областью управления и информационными блоками, отражающими текущее состояние транспортной логистики. В боковом меню предусмотреть следующие разделы:

- Главная;
- Карта;
- Рейсы;
- Транспорт;
- Водители;
- Заявки;
- Аналитика;
- AI-оптимизация;
- Отчёты;
- Уведомления;
- Настройки.

В верхней панели разместить строку поиска по рейсам, водителям, грузам и клиентам, элемент выбора даты, индикатор уведомлений, кнопку настроек и элемент управления режимом отображения. Основная область Dashboard должна содержать информационные карточки ключевых показателей:

1. **Активные рейсы** — количество перевозок, находящихся в процессе выполнения.
2. **Общий груз** — суммарная масса грузов, участвующих в перевозках.
3. **Пройденное расстояние** — общий пробег транспортных средств за выбранный период.
4. **Расход топлива** — суммарное количество израсходованного топлива.
5. **Доставки в срок** — процент перевозок, выполненных в установленное время.

В соответствии с графическим концептом для демонстрационных данных допускается использовать следующие значения:

- активные рейсы — 24;
- общий груз — 186 т;
- пройденное расстояние — 4 320 км;
- расход топлива — 182 л;
- доставки в срок — 95%.

Под каждым показателем необходимо предусмотреть отображение изменения относительно предыдущего периода. Основная область Dashboard должна включать следующие блоки:

- «Карта маршрутов (реальное время)»;
- «Ближайшие рейсы»;
- «AI-рекомендации»;
- «Статистика перевозок»;

- «Расходы и экономия»;
- «Статус транспорта».

Информация должна загружаться с сервера и формироваться на основе данных о рейсах, транспортных средствах, заявках и логистических операциях. При изменении статуса рейса, местоположения автомобиля или других показателей соответствующие информационные блоки должны обновляться.

Разработать раздел «Карта», предназначенный для визуального отображения транспортных средств, маршрутов перевозок и логистических объектов. В соответствии с графическим концептом основную область карты должна занимать интерактивная схема города или региона с нанесёнными маршрутами. На карте предусмотреть отображение:

- транспортных средств;
- активных маршрутов;
- пунктов погрузки;
- пунктов разгрузки;
- складов;
- клиентов;
- контрольных точек;
- дорожных затруднений.

Для каждого активного рейса использовать отдельный цвет маршрута. В соответствии с графическим концептом предусмотреть демонстрационные рейсы:

- рейс №101 — Москва → Тула;
- рейс №102 — Москва → Рязань;
- рейс №103 — Москва → Калуга;
- рейс №104 — Москва → Владимир;
- рейс №105 — Москва → Ярославль.

На карте должны отображаться маркеры транспортных средств с указанием номера рейса и текущего статуса. При выборе маркера должна открываться информационная карточка, содержащая:

- номер рейса;
- транспортное средство;
- водителя;
- пункт отправления;
- пункт назначения;
- текущий статус;
- плановое время прибытия;
- предполагаемое время задержки.

В верхней части карты предусмотреть переключатели отображаемых слоёв:

- «Транспорт»;
- «Маршруты»;
- «Пробки»;
- «Склады»;
- «Клиенты».

Реализовать возможность включения и отключения отдельных слоёв. Предусмотреть изменение масштаба карты и просмотр подробной информации об объектах. Для учебной реализации допускается использование Leaflet с OpenStreetMap либо другого доступного картографического решения. Перемещение транспортных средств допускается имитировать последовательным изменением координат по заранее подготовленным маршрутам. Обновление положения автомобилей должно выполняться через серверный API с заданной периодичностью.

Разработать раздел «Рейсы», предназначенный для создания, просмотра, редактирования и контроля транспортных перевозок. Для каждого рейса предусмотреть хранение:

- уникального идентификатора;
- номера рейса;
- транспортного средства;
- водителя;
- пункта отправления;
- пункта назначения;
- промежуточных точек;
- даты и времени отправления;
- планового времени прибытия;
- фактического времени прибытия;
- перевозимого груза;
- массы груза;
- протяжённости маршрута;
- текущего статуса;
- информации о задержке.

В соответствии с графическим концептом предусмотреть демонстрационные рейсы:

1. Рейс №101 — Москва → Тула, статус «В пути», задержка 25 минут.
2. Рейс №102 — Москва → Рязань, статус «В пути», прибытие в 14:20.
3. Рейс №103 — Москва → Калуга, статус «Погрузка», отправление в 14:00.
4. Рейс №104 — Москва → Владимир, статус «В пути», доставка в 16:30.
5. Рейс №105 — Москва → Ярославль, статус «Ожидает».

Предусмотреть следующие статусы рейсов:

- «Запланирован»;
- «Ожидает»;

- «Погрузка»;
- «В пути»;
- «Разгрузка»;
- «Завершён»;
- «Отменён».

Реализовать поиск, сортировку и фильтрацию рейсов по статусу, водителю, транспортному средству, дате и маршруту. При выборе рейса должна открываться подробная карточка с информацией о перевозке, маршруте, водителе, грузе и истории изменения статусов. Диспетчер должен иметь возможность создавать рейсы, назначать транспортные средства и водителей, изменять плановые параметры и контролировать выполнение перевозки. Изменения должны сохраняться в PostgreSQL.

Разработать раздел «Транспорт», предназначенный для учёта и контроля автопарка. Для каждого транспортного средства предусмотреть хранение:

- уникального идентификатора;
- государственного регистрационного номера;
- марки и модели;
- типа транспортного средства;
- грузоподъёмности;
- года выпуска;
- текущего пробега;
- среднего расхода топлива;
- технического состояния;
- даты последнего обслуживания;
- даты следующего обслуживания;
- текущего статуса.

Предусмотреть следующие статусы транспорта:

- «В пути»;
- «На погрузке»;
- «На разгрузке»;
- «На стоянке»;
- «На обслуживании».

В соответствии с графическим концептом на Dashboard предусмотреть кольцевую диаграмму распределения транспорта по статусам. Для демонстрационных данных использовать:

- всего транспортных средств — 36;
- в пути — 24;
- на погрузке — 4;
- на разгрузке — 3;
- на стоянке — 5.

В разделе «Транспорт» предусмотреть таблицу транспортных средств с возможностью поиска, фильтрации и сортировки. При выборе автомобиля должна открываться подробная карточка с техническими характеристиками, текущим рейсом, историей перевозок, пробегом и информацией о техническом обслуживании. Предусмотреть возможность добавления и редактирования транспортных средств.

Разработать раздел «Водители», предназначенный для хранения и обработки информации о сотрудниках, выполняющих транспортные перевозки. Для каждого водителя предусмотреть:

- уникальный идентификатор;
- фамилию, имя и отчество;
- контактные данные;
- номер водительского удостоверения;
- категории допуска;
- дату начала работы;
- закреплённое транспортное средство;
- текущий рейс;
- статус занятости;
- количество выполненных рейсов.

Предусмотреть следующие статусы:

- «Свободен»;
- «Назначен на рейс»;
- «В пути»;
- «Отдых»;
- «Недоступен».

Реализовать поиск и фильтрацию водителей по статусу, транспортному средству и количеству выполненных рейсов. При назначении водителя на рейс сервер должен проверять его доступность и отсутствие конфликтующих назначений. Предусмотреть отображение истории рейсов водителя. Диспетчер должен иметь возможность назначать и изменять водителей для запланированных рейсов.

Разработать раздел «Заявки», предназначенный для регистрации и обработки запросов клиентов на транспортировку грузов. Для каждой заявки предусмотреть хранение:

- уникального номера;
- клиента;
- контактной информации;
- наименования груза;
- типа груза;
- массы;
- объёма;
- пункта отправления;

- пункта назначения;
- желаемой даты доставки;
- приоритета;
- статуса;
- назначенного рейса.

Предусмотреть следующие статусы заявок:

- «Новая»;
- «На рассмотрении»;
- «Подтверждена»;
- «Назначена на рейс»;
- «Выполняется»;
- «Завершена»;
- «Отменена».

Логист должен иметь возможность создавать заявку, редактировать её параметры и назначать на рейс. Предусмотреть проверку соответствия массы груза грузоподъёмности выбранного автомобиля. Реализовать поиск и фильтрацию заявок по клиенту, дате, статусу и направлению перевозки. После завершения доставки заявка должна получать соответствующий статус.

Разработать функциональность контроля текущего состояния активных рейсов. Для каждого рейса предусмотреть отображение:

- текущего местоположения;
- последней контрольной точки;
- текущего статуса;
- пройденного расстояния;
- оставшегося расстояния;
- планового времени прибытия;
- расчётного времени прибытия;
- отклонения от графика.

При изменении местоположения автомобиля сервер должен сохранять новую запись координат и время обновления. Предусмотреть выявление следующих ситуаций:

- задержка прибытия;
- отклонение от маршрута;
- длительная остановка;
- нарушение установленного графика;
- отсутствие обновления координат.

При обнаружении отклонения система должна создавать событие и уведомление для диспетчера. Для учебной реализации допускается использовать заранее подготовленные координаты и имитацию перемещения транспортных средств.

Разработать раздел «Аналитика», предназначенный для анализа эффективности транспортных операций. В соответствии с графическим концептом на Dashboard разместить блок «Статистика перевозок». Визуализацию выполнить в виде комбинированного графика. Предусмотреть следующие показатели:

- количество выполненных рейсов;
- общий объём перевезённых грузов;
- процент доставок в срок.

Для каждого показателя использовать отдельный цвет. По горизонтальной оси отображать даты выбранного периода. В верхней части блока предусмотреть переключатели:

- «Неделя»;
- «Месяц»;
- «Год».

При изменении периода график должен обновляться. Дополнительно в разделе аналитики предусмотреть:

- количество завершённых рейсов;
- среднюю продолжительность перевозки;
- общий пробег;
- среднюю загрузку транспорта;
- процент своевременных доставок;
- количество задержек;
- распределение перевозок по направлениям.

Аналитические показатели должны рассчитываться на сервере на основе сохранённых данных.

Разработать блок «Расходы и экономия», предназначенный для контроля затрат на транспортные перевозки. В соответствии с графическим концептом предусмотреть три основные карточки:

1. Расходы на топливо.
2. Обслуживание транспорта.
3. Экономия за счёт оптимизации.

Для демонстрационных данных допускается использовать:

- расходы на топливо — 124 580 ₽;
- обслуживание — 36 200 ₽;
- экономия AI — 28 430 ₽.

Для каждого показателя отображать изменение относительно предыдущего периода. В верхней части блока предусмотреть переключение временного периода:

- «Неделя»;
- «Месяц»;
- «Год».

Для каждого расхода хранить:

- категорию;
- транспортное средство;
- связанный рейс при наличии;
- сумму;
- дату;
- описание.

Предусмотреть расчёт общей суммы расходов по выбранному периоду. Экономию от оптимизации рассчитывать как разницу между базовым и оптимизированным сценарием по заданной методике. Для учебной реализации допускается использовать условные тарифы и заранее подготовленные данные.

Разработать раздел «AI-оптимизация», предназначенный для формирования рекомендаций по повышению эффективности транспортной логистики. В соответствии с графическим концептом предусмотреть следующие категории рекомендаций:

Оптимизация маршрутов. Система анализирует маршрут перевозки и предлагает альтернативный вариант, позволяющий сократить расстояние или предполагаемое время доставки. Для демонстрационного примера использовать рекомендацию: «С учётом пробок можно сократить время доставки на 28 минут».

Снижение расхода топлива. Система анализирует показатели расхода топлива и предлагает способы уменьшения затрат. Для демонстрационного примера: «Рекомендуем изменить стиль вождения для рейса №101».

Объединение грузов. Система анализирует заявки с близкими направлениями доставки. При обнаружении совместимых заявок формируется предложение объединить грузы в один рейс. Для демонстрационного примера: «Найдены попутные грузы на маршруте Москва → Тула».

Планирование технического обслуживания. Система анализирует пробег и даты технического обслуживания автомобилей. При приближении планового обслуживания формируется рекомендация. Для демонстрационного примера: «У автомобиля МАЗ-7540 через 500 км пробега необходимо провести плановое ТО».

Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- тип;
- описание;

- связанный объект;
- дату создания;
- приоритет;
- предполагаемый эффект;
- статус.

Предусмотреть возможность просмотра подробной информации и изменения статуса рекомендации. Для учебной реализации допускается использовать алгоритмы на основе заданных правил. Например:

- при превышении порога задержки система предлагает проверить альтернативный маршрут;
- при повышенном расходе топлива формируется рекомендация по снижению затрат;
- при наличии нескольких заявок с близкими маршрутами предлагается объединение;
- при достижении порогового пробега формируется уведомление о техническом обслуживании.

Интеграция с внешними AI-сервисами допускается, но не является обязательной.

Разработать раздел «Отчёты», предназначенный для формирования сводной информации о транспортной деятельности. Предусмотреть следующие виды отчётов:

- отчёт по выполненным рейсам;
- отчёт по транспортным средствам;
- отчёт по водителям;
- отчёт по расходу топлива;
- отчёт по затратам на обслуживание;
- отчёт по своевременности доставок;
- отчёт по эффективности маршрутов.

Пользователь должен иметь возможность выбрать тип отчёта и временной период. После выбора параметров система должна сформировать таблицу с соответствующими данными. Предусмотреть возможность экспорта отчётов в CSV или PDF. Данные отчётов должны формироваться серверной частью на основе информации PostgreSQL.

Разработать раздел «Уведомления», предназначенный для информирования пользователей о значимых логистических событиях. Предусмотреть следующие типы уведомлений:

- назначение нового рейса;
- изменение маршрута;
- задержка доставки;
- начало погрузки;
- начало разгрузки;

- завершение рейса;
- отклонение от маршрута;
- необходимость технического обслуживания;
- новое сообщение диспетчера;
- новая AI-рекомендация.

Для каждого уведомления предусмотреть:

- идентификатор;
- тип;
- заголовок;
- текст;
- дату и время;
- связанный объект;
- статус прочтения.

В верхней панели отображать количество непрочитанных уведомлений. При выборе уведомления должен выполняться переход к соответствующему рейсу, транспортному средству или сообщению. Предусмотреть возможность отметки уведомления как прочитанного.

Разработать систему регистрации, авторизации и разграничения прав доступа. Предусмотреть следующие роли:

- диспетчер;
- логист;
- водитель;
- экспедитор;
- администратор.

Диспетчер должен иметь возможность просматривать карту, контролировать рейсы, отслеживать транспортные средства, получать уведомления и взаимодействовать с водителями.

Логист должен иметь возможность управлять заявками, формировать маршруты, назначать транспорт и водителей, анализировать расходы и работать с AI-рекомендациями.

Водитель должен иметь доступ к назначенным рейсам, маршрутам, этапам перевозки, сообщениям и функциям передачи информации о доставке.

Экспедитор должен иметь возможность просматривать сопровождаемые перевозки, контролировать погрузку и разгрузку, загружать фотографии и подтверждающие документы.

Администратор должен управлять пользователями, ролями, транспортными средствами, справочниками и настройками системы.

Для каждого пользователя предусмотреть:

- уникальный идентификатор;
- фамилию и имя;
- адрес электронной почты;
- логин;
- защищённые учётные данные;
- роль;
- контактный телефон;
- изображение профиля;
- статус;
- дату регистрации.

Разграничение прав доступа должно выполняться на сервере.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу мобильного приложения LogiTrack AI. В верхней части разместить логотип приложения, персональное приветствие и информацию о роли пользователя. В соответствии с графическим концептом предусмотреть приветствие: «Добрый день, Алексей!»

Ниже разместить кнопки быстрого доступа:

- «Мои рейсы»;
- «Навигация»;
- «Сообщения»;
- «Документы»;
- «Тех. состояние»;
- «Профиль».

Для раздела «Сообщения» предусмотреть индикатор количества непрочитанных сообщений. В нижней части разместить навигационную панель:

- «Главная»;
- «Рейсы»;
- «Сообщения»;
- «Профиль».

При выборе элемента должен выполняться переход к соответствующему экрану. Данные должны загружаться с сервера с учётом авторизованного пользователя.

Разработать мобильный экран «Мои рейсы». В верхней части предусмотреть переключатели:

- «Активные»;
- «Завершённые».

Основная область должна содержать список назначенных пользователю рейсов. Для каждого рейса отображать:

- номер;
- пункт отправления;
- пункт назначения;
- плановое время доставки;
- текущий статус.

В соответствии с графическим концептом предусмотреть следующие демонстрационные рейсы:

- №101 — Москва → Тула, «В пути»;
- №102 — Москва → Рязань, «В пути»;
- №103 — Москва → Калуга, «Погрузка»;
- №104 — Москва → Владимир, «В пути».

Для статусов использовать цветовые индикаторы. При выборе рейса должна открываться подробная карточка. Предусмотреть возможность просмотра завершённых рейсов.

Разработать мобильный экран «Навигация». Основную область должна занимать карта с отображением текущего маршрута. В верхней части предусмотреть информацию о направлении движения. В соответствии с графическим концептом использовать маршрут: «Москва → Тула».

На карте должны отображаться:

- текущее местоположение автомобиля;
- линия маршрута;
- пункт назначения;
- промежуточные точки.

В нижней части предусмотреть информационный блок:

- оставшееся время;
- оставшееся расстояние;

- предполагаемое время прибытия.

Для демонстрационного примера использовать:

- оставшееся время — 1 ч 20 мин;
- расстояние — 85 км.

В нижней части разместить кнопку «Открыть в навигаторе». При выборе кнопки должен выполняться переход во внешнее навигационное приложение или картографический сервис с передачей координат пункта назначения. Для учебной реализации допускается использовать имитацию GPS-перемещения и заранее подготовленный маршрут.

Разработать мобильный экран «Детали рейса». В верхней части отображать номер рейса, направление перевозки, плановое время доставки и текущий статус. Ниже предусмотреть вертикальную временную шкалу этапов перевозки. В соответствии с графическим концептом использовать следующие этапы:

1. Погрузка.
2. В пути.
3. Разгрузка.

Для каждого этапа отображать:

- название;
- дату и время;
- местоположение;
- статус выполнения.

Завершённые этапы отмечать зелёным индикатором. Текущий этап выделять синим цветом. Предстоящие этапы отображать нейтральным цветом. В нижней части разместить кнопку «Связаться с диспетчером». При выборе кнопки должен открываться экран сообщений. Предусмотреть возможность изменения статуса рейса в соответствии с допустимой последовательностью этапов. Изменения должны передаваться на сервер и отображаться в веб-версии.

Разработать мобильный экран «Фото-документы», предназначенный для фиксации выполнения транспортных операций. В верхней части предусмотреть переключатели:

- «Погрузка»;
- «Разгрузка»;
- «Повреждения».

Основная область должна содержать фотографии, связанные с выбранным рейсом. Предусмотреть возможность:

- сделать фотографию;

- выбрать изображение из галереи;
- добавить комментарий;
- удалить фотографию до отправки;
- загрузить документы на сервер.

В соответствии с графическим концептом предусмотреть блок «Подтверждение». В нижней части разместить кнопку «Отправить документы». После отправки сервер должен сохранить информацию о загруженных файлах и связать их с соответствующим рейсом. Предусмотреть отображение статуса загрузки. Для учебной реализации допускается использование заранее подготовленных изображений грузов, складов и транспортных средств.

Разработать мобильный экран «Сообщения», предназначенный для оперативного взаимодействия водителя с диспетчером. В верхней части отображать название раздела. Основная область должна содержать историю переписки. Сообщения диспетчера и водителя должны иметь различное визуальное оформление. В соответствии с графическим концептом предусмотреть демонстрационный диалог:

Диспетчер: «Вы где находитесь?»

Водитель: «Я на подъезде, буду через 20 минут».

Диспетчер: «Принято. На складе вас ожидают».

Водитель: «Спасибо!»

Для каждого сообщения отображать время отправки. В нижней части разместить поле ввода и кнопку отправки. Предусмотреть:

- отправку текстовых сообщений;
- получение новых сообщений;
- сохранение истории;
- отображение статуса прочтения;
- уведомления о новых сообщениях.

Для учебной реализации допускается периодическое получение новых сообщений через REST API.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение бизнес-логики, проверку входных данных, управление правами доступа и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	Основные данные

Users	Пользователи, роли, защищённые учётные данные
Vehicles	Транспортные средства и технические характеристики
Drivers	Водители и их доступность
Clients	Клиенты и контактная информация
Cargo	Грузы, масса, объём и тип
TransportRequests	Заявки на перевозку
Trips	Рейсы, маршруты, сроки и статусы
Routes	Маршруты и параметры движения
RoutePoints	Контрольные точки маршрутов
GPSPositions	Координаты транспортных средств
Warehouses	Склады и пункты погрузки
TripEvents	События и этапы выполнения рейсов
FuelRecords	Данные о расходе топлива
MaintenanceRecords	Техническое обслуживание транспорта
Expenses	Эксплуатационные расходы
AIRecommendations	Рекомендации по оптимизации
TripDocuments	Фотографии и документы перевозок
Messages	Сообщения между пользователями
Notifications	Уведомления
AuditLogs	Журнал действий пользователей

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение данных профиля;
- управление транспортными средствами;
- управление водителями;
- управление клиентами;
- регистрацию заявок;
- создание и изменение рейсов;
- назначение водителей и транспорта;
- получение маршрутов;
- получение и обновление координат;
- изменение статусов перевозок;
- регистрацию логистических событий;
- учёт расхода топлива;
- учёт технического обслуживания;
- расчёт логистических показателей;
- формирование AI-рекомендаций;
- загрузку фото-документов;
- обмен сообщениями;
- получение уведомлений;
- формирование отчётов.

Предусмотреть разграничение доступа по ролям диспетчера, логиста, водителя, экспедитора и администратора. Для обеспечения безопасности необходимо реализовать:

- хранение паролей в виде защищённых хешей;
- авторизацию на основе сессий или JWT;
- проверку корректности входных данных;
- ограничение доступа к персональным данным;
- проверку прав на изменение рейсов и заявок;
- проверку принадлежности рейса водителю;
- защиту загружаемых файлов;
- безопасное хранение ключей внешних API;
- журналирование значимых операций.

Водитель должен иметь доступ только к назначенным ему рейсам и связанным с ними документам. Диспетчер и логист должны получать доступ к данным в пределах своих полномочий.

Основные пользовательские сценарии

Сценарий 1. Авторизация и просмотр Dashboard.

Диспетчер открывает веб-приложение LogiTrack AI и авторизуется. Система загружает сведения о текущих рейсах, транспортных средствах и логистических показателях. На главной странице отображаются карточки ключевых показателей, интерактивная карта, ближайшие рейсы и AI-рекомендации. Диспетчер просматривает общую ситуацию и выбирает рейс, требующий внимания.

Сценарий 2. Создание заявки на перевозку.

Логист открывает раздел «Заявки». Выбирает действие «Создать заявку». Указывает клиента, характеристики груза, пункт отправления, пункт назначения и желаемую дату доставки. Система проверяет корректность введённых данных. После сохранения заявка получает статус «Новая». Логист может назначить её на запланированный рейс.

Сценарий 3. Создание и назначение рейса.

Логист открывает раздел «Рейсы». Создаёт новый рейс. Указывает маршрут, дату отправления и плановое время прибытия. Выбирает доступное транспортное средство и водителя. Система проверяет отсутствие конфликтующих назначений и соответствие грузоподъёмности. После сохранения рейс появляется в общем списке. Назначенный водитель получает уведомление.

Сценарий 4. Мониторинг рейса на карте.

Диспетчер открывает раздел «Карта». Система отображает активные маршруты и местоположение автомобилей. Диспетчер выбирает маркер рейса №101. Открывается карточка с информацией о направлении Москва → Тула, текущем статусе и задержке. Диспетчер просматривает маршрут и предполагаемое время прибытия.

Сценарий 5. Обнаружение задержки.

Система получает обновлённые координаты транспортного средства. Расчётное время прибытия превышает плановое. Сервер создаёт событие задержки. На Dashboard появляется предупреждение «Задержка +25 мин». Диспетчер получает уведомление и открывает подробную информацию о рейсе.

Сценарий 6. Формирование AI-рекомендации по маршруту.

Система анализирует данные активного рейса. На основе демонстрационных данных обнаруживается альтернативный маршрут с меньшим предполагаемым временем движения. Формируется рекомендация «Оптимизировать маршрут». Диспетчер открывает раздел «AI-оптимизация». Просматривает исходный и предлагаемый варианты. При подтверждении изменения обновлённый маршрут сохраняется и становится доступным водителю.

Сценарий 7. Объединение грузов.

Логист регистрирует несколько заявок с близкими направлениями доставки. Система сравнивает пункты назначения, сроки и характеристики грузов. При выполнении заданных условий формируется рекомендация об объединении. Логист просматривает предложение и выбирает подходящий транспорт. После подтверждения заявки связываются с одним рейсом.

Сценарий 8. Контроль технического обслуживания.

Система анализирует текущий пробег автомобиля. До планового технического обслуживания остаётся 500 км. Формируется соответствующее уведомление. Диспетчер открывает карточку транспортного средства. Просматривает информацию и создаёт запись о предстоящем обслуживании.

Сценарий 9. Анализ статистики перевозок.

Диспетчер открывает раздел «Аналитика». Выбирает период «Неделя». Система формирует график количества рейсов, объёма грузов и своевременности доставок. Пользователь изменяет период на «Месяц». График и сводные показатели обновляются.

Сценарий 10. Анализ расходов.

Логист открывает блок «Расходы и экономия». Система отображает затраты на топливо, обслуживание и расчётную экономию. Пользователь выбирает период анализа. Сервер рассчитывает показатели и возвращает данные. Логист просматривает динамику расходов и формирует отчёт.

Сценарий 11. Просмотр рейсов в мобильном приложении.

Водитель открывает мобильную версию LogiTrack AI. Авторизуется в системе. На главном экране выбирает раздел «Мои рейсы». Система отображает назначенные перевозки. Водитель выбирает рейс №101 и открывает подробную информацию.

Сценарий 12. Использование навигации.

Водитель открывает экран «Навигация». Система отображает маршрут Москва → Тула. На карте показываются текущее положение и пункт назначения. Водитель просматривает оставшееся расстояние и время. При необходимости выбирает кнопку «Открыть в навигаторе». Запускается внешний навигационный сервис.

Сценарий 13. Изменение статуса рейса.

Водитель завершает погрузку. Открывает экран «Детали рейса». Выбирает переход к этапу «В пути». Система отправляет изменение на сервер. Сервер проверяет допустимость перехода и сохраняет событие. В веб-версии диспетчера статус рейса обновляется.

Сценарий 14. Отправка фото-документов.

Экспедитор завершает погрузку груза. Открывает экран «Фото-документы». Выбирает вкладку «Погрузка». Делает фотографии размещённого груза. Добавляет комментарий и нажимает «Отправить документы». Система загружает изображения на сервер. Фотографии становятся доступны диспетчеру в карточке рейса.

Сценарий 15. Общение с диспетчером.

Водитель открывает раздел «Сообщения». Отправляет диспетчеру информацию о задержке. Сообщение сохраняется на сервере. Диспетчер получает уведомление. После ответа диспетчера водитель видит новое сообщение в мобильном приложении.

Сценарий 16. Завершение рейса.

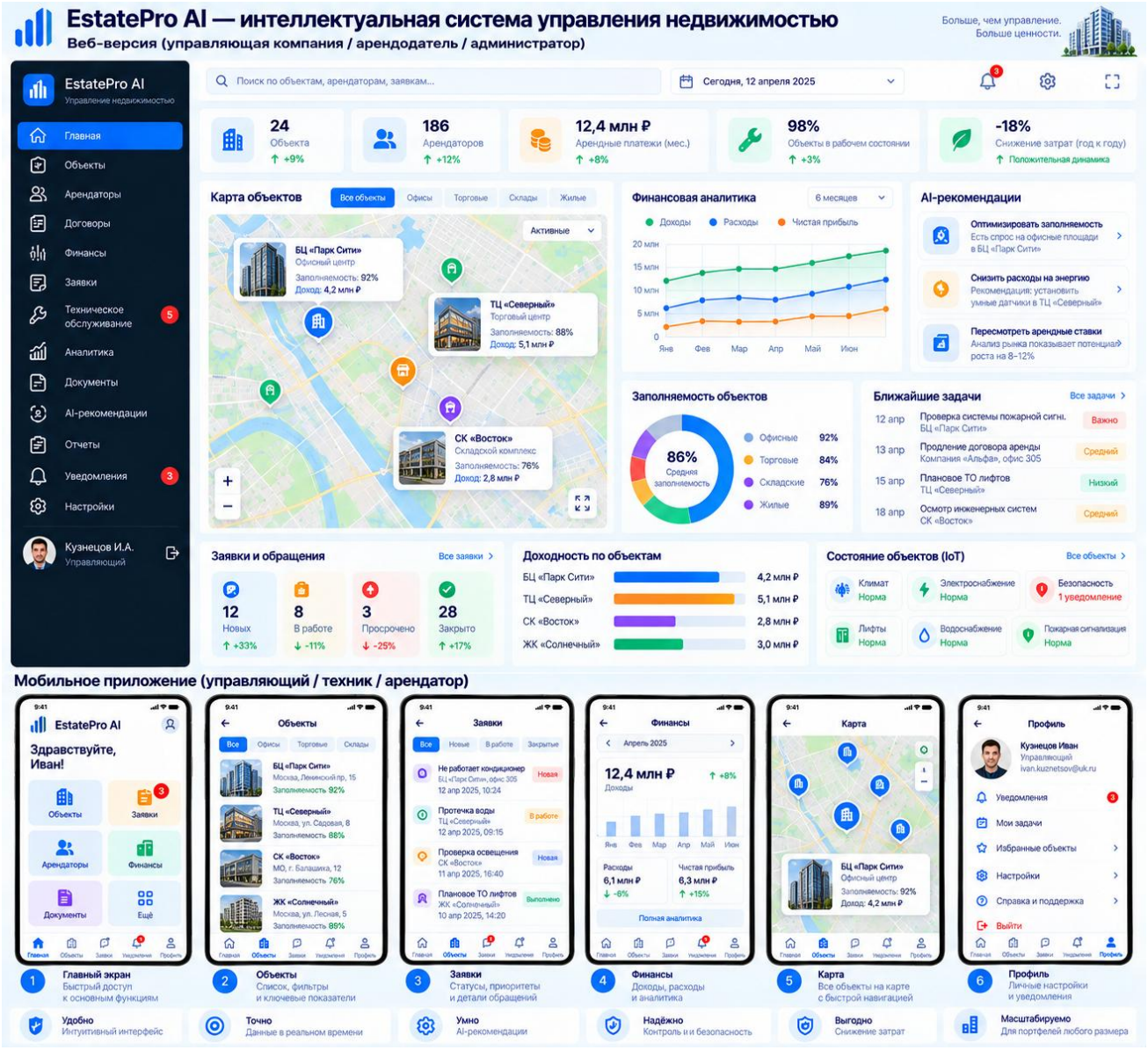
Водитель прибывает в пункт назначения. Открывает карточку рейса. Переводит перевозку на этап «Разгрузка». После завершения разгрузки загружает подтверждающие фотографии. Отмечает рейс как завершённый. Сервер сохраняет фактическое время доставки, обновляет статус рейса и связанных заявок. Показатели Dashboard и аналитики пересчитываются.

Сценарий 17. Формирование отчёта.

Диспетчер открывает раздел «Отчёты». Выбирает отчёт по выполненным рейсам. Указывает временной период. Система формирует таблицу перевозок с датами, направлениями, водителями, статусами и показателями своевременности. Пользователь скачивает отчёт в CSV или PDF.

Вариант 14. EstatePro AI — интеллектуальная система управления недвижимостью

Разработать Fullstack веб-приложение **EstatePro AI**, предназначенное для автоматизации управления коммерческой и жилой недвижимостью, мониторинга состояния объектов, учёта арендаторов, управления договорами аренды, контроля финансовых поступлений и расходов, обработки заявок на обслуживание и интеллектуального анализа эффективности использования недвижимости.



Система должна обеспечивать централизованное хранение и обработку информации об объектах недвижимости, помещениях, арендаторах, договорах, арендных платежах, эксплуатационных расходах, инженерных системах, техническом обслуживании, заявках и обращениях пользователей.

Приложение должно предоставлять управляющим компаниям инструменты для оперативного контроля портфеля недвижимости, отслеживания заполняемости помещений,

анализа доходности объектов, планирования технического обслуживания и формирования финансовой отчётности.

Арендаторы должны иметь возможность просматривать информацию об арендуемых помещениях, условиях договоров, начисленных платежах, создавать обращения и отслеживать ход их выполнения.

Технические специалисты должны получать назначенные заявки, просматривать информацию о неисправностях, изменять статусы работ и фиксировать результаты обслуживания. Администраторы должны обеспечивать управление пользователями, объектами недвижимости, справочниками, настройками системы и правами доступа.

В соответствии с представленным графическим концептом приложение должно включать интеллектуальный модуль **AI-рекомендаций**, который анализирует показатели заполняемости, финансовые данные, эксплуатационные расходы и состояние объектов, выявляет потенциальные проблемы и предлагает способы повышения доходности и эффективности управления недвижимостью.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для управляющей компании, арендодателя и администратора** — предназначена для управления объектами недвижимости, арендаторами, договорами, финансами, заявками, техническим обслуживанием, аналитикой и AI-рекомендациями.
- **Мобильная версия для управляющего, технического специалиста и арендатора** — предназначена для просмотра объектов, работы с заявками, контроля финансовых показателей, использования карты недвижимости, получения уведомлений и управления личным профилем.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт EstatePro AI, сохранив основные композиционные решения, структуру навигации, цветовую схему, расположение информационных блоков, способы визуализации финансовых показателей, интерактивную карту объектов и организацию мобильных экранов.

Для учебной реализации допускается использование заранее подготовленных объектов недвижимости, демонстрационных арендаторов, тестовых договоров, условных финансовых показателей, имитации состояния инженерных систем и алгоритмов формирования рекомендаций на основе заданных правил.

Интеграция с реальными системами управления зданиями (BMS), банковскими сервисами, государственными реестрами недвижимости и промышленными системами мониторинга инженерного оборудования не является обязательной.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью, верхней областью управления и информационными блоками, отражающими текущее состояние портфеля недвижимости. В боковом меню предусмотреть следующие разделы:

- Главная;
- Объекты;
- Арендаторы;
- Договоры;
- Финансы;
- Заявки;
- Техническое обслуживание;
- Аналитика;
- Документы;
- AI-рекомендации;
- Отчёты;
- Уведомления;
- Настройки.

В верхней панели разместить строку поиска по объектам, арендаторам и заявкам, элемент выбора даты, индикатор уведомлений, кнопку настроек и элемент управления режимом отображения. Основная область Dashboard должна содержать информационные карточки ключевых показателей:

1. **Объекты недвижимости** — общее количество объектов, находящихся под управлением.
2. **Арендаторы** — количество зарегистрированных арендаторов.
3. **Арендные платежи** — общая сумма арендных поступлений за выбранный период.
4. **Техническое состояние** — процент объектов, находящихся в исправном состоянии.
5. **Снижение затрат** — изменение эксплуатационных расходов относительно предыдущего периода.

В соответствии с графическим концептом для демонстрационных данных допускается использовать следующие значения:

- объектов недвижимости — 24;
- арендаторов — 186;
- арендные платежи за месяц — 12,4 млн ₽;
- объектов в рабочем состоянии — 98%;
- снижение затрат — 18%.

Под каждым показателем необходимо предусмотреть отображение динамики относительно предыдущего периода. Основная область Dashboard должна включать следующие блоки:

- «Карта объектов»;
- «Финансовая аналитика»;
- «AI-рекомендации»;
- «Заполняемость объектов»;
- «Ближайшие задачи»;
- «Заявки и обращения»;
- «Доходность по объектам»;
- «Состояние объектов (IoT)».

Информация должна загружаться с сервера и формироваться на основе данных об объектах недвижимости, арендаторах, договорах, платежах, заявках и техническом состоянии зданий. При изменении финансовых показателей, статусов заявок, состояния объектов или договоров соответствующие информационные блоки должны обновляться.

Разработать раздел «Объекты», предназначенный для регистрации, просмотра, редактирования и управления недвижимостью. Для каждого объекта предусмотреть хранение:

- уникального идентификатора;
- наименования;
- типа недвижимости;
- адреса;
- географических координат;
- общей площади;
- арендуемой площади;
- занятой площади;
- количества помещений;
- количества арендаторов;
- процента заполняемости;
- ежемесячного дохода;
- технического состояния;
- даты ввода в эксплуатацию;
- описания;
- фотографий.

Предусмотреть следующие типы объектов:

- офисные здания;
- торговые центры;
- складские комплексы;
- жилые комплексы;
- другие объекты недвижимости.

В соответствии с графическим концептом предусмотреть демонстрационные объекты:

1. **БЦ «Парк Сити»** — офисный центр, заполняемость 92%, доход 4,2 млн ₽.
2. **ТЦ «Северный»** — торговый центр, заполняемость 88%, доход 5,1 млн ₽.
3. **СК «Восток»** — складской комплекс, заполняемость 76%, доход 2,8 млн ₽.
4. **ЖК «Солнечный»** — жилой комплекс, заполняемость 89%, доход 3,0 млн ₽.

Предусмотреть возможность добавления нового объекта, изменения его характеристик и загрузки фотографий. Для каждого объекта реализовать подробную карточку с информацией о помещениях, арендаторах, договорах, финансовых показателях, заявках и техническом состоянии. Реализовать поиск, сортировку и фильтрацию объектов по типу, адресу, заполняемости и состоянию. Все изменения должны сохраняться в PostgreSQL.

Разработать раздел «Карта объектов», предназначенный для географического отображения недвижимости. Основную область должна занимать интерактивная карта с нанесёнными маркерами объектов. В соответствии с графическим концептом предусмотреть переключатели категорий:

- «Все объекты»;
- «Офисы»;
- «Торговые»;
- «Склады»;
- «Жилые».

При выборе категории на карте должны отображаться соответствующие объекты. Для каждого объекта предусмотреть маркер, цвет или значок которого соответствует типу недвижимости. При выборе маркера должна открываться информационная карточка, содержащая:

- фотографию объекта;
- наименование;
- тип недвижимости;
- адрес;
- процент заполняемости;
- ежемесячный доход.

В верхней части карты предусмотреть фильтр по статусу объекта. Реализовать возможность изменения масштаба, перемещения по карте и перехода к подробной карточке недвижимости. Для учебной реализации допускается использование Leaflet с OpenStreetMap либо другого доступного картографического решения. Координаты объектов могут быть заранее подготовлены и сохранены в PostgreSQL.

Разработать функциональность учёта отдельных помещений внутри объектов недвижимости. Для каждого помещения предусмотреть:

- уникальный идентификатор;
- связанный объект недвижимости;

- номер помещения;
- этаж;
- площадь;
- назначение;
- стоимость аренды;
- текущий статус;
- связанный договор аренды;
- арендатора при наличии.

Предусмотреть следующие статусы помещений:

- «Свободно»;
- «Занято»;
- «Забронировано»;
- «На ремонте»;
- «Недоступно».

Реализовать расчёт процента заполняемости объекта по формуле:

Заполняемость = (занятая арендуемая площадь / общая доступная для аренды площадь) × 100%.

На Dashboard разместить кольцевую диаграмму «Заполняемость объектов». В соответствии с графическим концептом предусмотреть следующие демонстрационные показатели:

- офисные помещения — 92%;
- торговые помещения — 84%;
- складские помещения — 76%;
- жилые помещения — 89%.

В центре диаграммы отображать среднюю заполняемость — 86%. Для расчёта среднего показателя необходимо использовать данные базы, а при необходимости — взвешенное среднее по площади. Предусмотреть возможность просмотра свободных помещений и фильтрации по типу, площади и стоимости аренды. При заключении нового договора статус соответствующего помещения должен обновляться.

Разработать раздел «Арендаторы», предназначенный для хранения и обработки информации о физических и юридических лицах, арендующих недвижимость. Для каждого арендатора предусмотреть:

- уникальный идентификатор;
- тип арендатора;
- наименование организации или ФИО;
- контактное лицо;
- телефон;

- адрес электронной почты;
- реквизиты при необходимости;
- арендуемые объекты и помещения;
- действующие договоры;
- статус;
- историю платежей;
- историю обращений.

Предусмотреть следующие статусы арендаторов:

- «Активный»;
- «Ожидает заключения договора»;
- «Задолженность»;
- «Неактивный».

Реализовать поиск и фильтрацию арендаторов по объекту, статусу и типу. При выборе арендатора должна открываться подробная карточка с информацией о договорах, платежах и заявках. Управляющий должен иметь возможность добавлять арендаторов, редактировать контактные данные и связывать их с помещениями.

Разработать раздел «Договоры», предназначенный для регистрации и контроля договорных отношений. Для каждого договора предусмотреть хранение:

- уникального идентификатора;
- номера договора;
- арендатора;
- объекта недвижимости;
- помещения;
- даты заключения;
- даты начала аренды;
- даты окончания аренды;
- ежемесячной арендной платы;
- размера обеспечительного платежа;
- условий оплаты;
- текущего статуса;
- прикреплённых документов.

Предусмотреть следующие статусы:

- «Черновик»;
- «Действует»;
- «Истекает»;
- «Завершён»;
- «Расторгнут».

Реализовать возможность создания, редактирования и просмотра договоров. Система должна проверять отсутствие конфликтующих договоров для одного помещения на пересекающиеся периоды. Предусмотреть автоматическое определение договоров, срок действия которых приближается к завершению. При наступлении заданного периода до окончания аренды система должна создавать уведомление. Реализовать фильтрацию договоров по арендатору, объекту, статусу и сроку действия.

Разработать раздел «Финансы», предназначенный для учёта доходов и расходов по объектам недвижимости. Предусмотреть регистрацию:

- арендных начислений;
- поступивших платежей;
- задолженностей;
- коммунальных расходов;
- затрат на техническое обслуживание;
- расходов на ремонт;
- административных расходов;
- прочих эксплуатационных затрат.

Для каждой финансовой операции предусмотреть:

- уникальный идентификатор;
- тип операции;
- категорию;
- объект недвижимости;
- договор или арендатора при наличии;
- сумму;
- дату;
- описание;
- статус оплаты.

На Dashboard разместить блок «Финансовая аналитика». В соответствии с графическим концептом использовать линейный график с тремя показателями:

- доходы — зелёная линия;
- расходы — синяя линия;
- чистая прибыль — оранжевая линия.

По горизонтальной оси отображать месяцы выбранного периода. В верхней части блока предусмотреть выбор периода, например «6 месяцев». Чистую прибыль для учебной модели рассчитывать как разницу между доходами и учитываемыми расходами. При изменении периода график должен обновляться. Предусмотреть отображение задолженностей арендаторов и возможность фильтрации финансовых операций по объекту. Для учебной реализации допускается использование демонстрационных начислений и условных платежей без интеграции с банковскими системами.

Разработать блок «Доходность по объектам», предназначенный для сравнения финансовой эффективности недвижимости. В соответствии с графическим концептом использовать горизонтальные диаграммы для следующих объектов:

- БЦ «Парк Сити» — 4,2 млн ₽;
- ТЦ «Северный» — 5,1 млн ₽;
- СК «Восток» — 2,8 млн ₽;
- ЖК «Солнечный» — 3,0 млн ₽.

Для каждого объекта предусмотреть отображение:

- наименования;
- суммы доходов;
- графического индикатора;
- периода анализа.

При выборе объекта должна открываться его подробная финансовая информация. Предусмотреть сортировку объектов по доходности. Дополнительно реализовать расчёт:

- общего дохода;
- эксплуатационных расходов;
- чистой прибыли;
- дохода на квадратный метр;
- процента заполняемости;
- уровня задолженности.

Показатели должны рассчитываться серверной частью на основе данных PostgreSQL.

Разработать раздел «Заявки», предназначенный для регистрации и обработки обращений арендаторов и сотрудников управляющей компании. Для каждой заявки предусмотреть:

- уникальный идентификатор;
- номер;
- объект недвижимости;
- помещение;
- заявителя;
- тип проблемы;
- описание;
- дату создания;
- приоритет;
- статус;
- назначенного исполнителя;
- плановую дату выполнения;
- фактическую дату завершения;

- комментарии;
- прикреплённые фотографии.

Предусмотреть следующие статусы:

- «Новая»;
- «В работе»;
- «Просрочена»;
- «Закрыта».

На Dashboard разместить блок «Заявки и обращения». В соответствии с графическим концептом использовать демонстрационные показатели:

- новых заявок — 12;
- в работе — 8;
- просроченных — 3;
- закрытых — 28.

Для каждого показателя отображать изменение относительно предыдущего периода. Реализовать создание заявки, назначение исполнителя, изменение приоритета и статуса. Предусмотреть фильтрацию по объекту, типу проблемы, статусу и дате. При изменении статуса заявки соответствующий пользователь должен получать уведомление.

Разработать раздел «Техническое обслуживание», предназначенный для планирования и контроля работ по содержанию недвижимости. Для каждого мероприятия предусмотреть:

- уникальный идентификатор;
- объект недвижимости;
- инженерную систему;
- вид обслуживания;
- описание;
- дату планирования;
- приоритет;
- назначенного исполнителя;
- статус;
- фактическую дату выполнения;
- результат работ.

Предусмотреть следующие виды работ:

- проверка пожарной безопасности;
- обслуживание климатических систем;
- обслуживание электроснабжения;
- проверка водоснабжения;

- обслуживание лифтов;
- проверка охранных систем;
- ремонтные работы;
- профилактические осмотры.

На Dashboard разместить блок «Ближайшие задачи». В соответствии с графическим концептом предусмотреть демонстрационные задачи:

1. 12 апреля — проверка системы пожарной сигнализации, БЦ «Парк Сити», высокий приоритет.
2. 13 апреля — продление договора аренды, компания «Альфа», офис 305, средний приоритет.
3. 15 апреля — плановое ТО лифтов, ТЦ «Северный», низкий приоритет.
4. 18 апреля — осмотр инженерных систем, СК «Восток», средний приоритет.

Для каждой задачи отображать дату, название, связанный объект и приоритет. Предусмотреть возможность создания задач, назначения исполнителей и отметки выполнения. Просроченные задачи должны выделяться визуально.

Разработать блок «Состояние объектов (IoT)», предназначенный для отображения текущего состояния инженерных систем недвижимости. В соответствии с графическим концептом предусмотреть следующие категории:

- климат;
- электроснабжение;
- безопасность;
- лифты;
- водоснабжение;
- пожарная сигнализация.

Для каждой инженерной системы предусмотреть хранение:

- уникального идентификатора;
- объекта недвижимости;
- типа системы;
- наименования;
- текущего состояния;
- даты последней проверки;
- описания;
- связанных событий.

Предусмотреть следующие состояния:

- «Норма»;
- «Предупреждение»;
- «Неисправность»;

- «Нет данных».

В соответствии с графическим концептом для демонстрационного объекта использовать:

- климат — норма;
- электроснабжение — норма;
- безопасность — одно уведомление;
- лифты — норма;
- водоснабжение — норма;
- пожарная сигнализация — норма.

Исправные системы выделять зелёным цветом. Предупреждения отображать оранжевым цветом. Неисправности выделять красным цветом. При выборе системы должна открываться подробная информация о состоянии и истории событий. Для учебной реализации допускается имитация IoT-показателей с периодическим обновлением данных через сервер. Подключение реальных датчиков не является обязательным.

Разработать раздел «AI-рекомендации», предназначенный для интеллектуального анализа эффективности управления недвижимостью. В соответствии с графическим концептом предусмотреть три основные категории рекомендаций.

Оптимизация заполняемости. Система анализирует свободные помещения, спрос и показатели аренды. При выявлении низкой заполняемости формируется рекомендация по привлечению арендаторов. Для демонстрационного примера: «Есть спрос на офисные площади в БЦ „Парк Сити“».

Снижение расходов на энергию. Система анализирует эксплуатационные расходы и показатели инженерных систем. При выявлении потенциальной экономии формируется рекомендация. Для демонстрационного примера: «Рекомендуем установить умные датчики в ТЦ „Северный“».

Пересмотр арендных ставок. Система анализирует текущие арендные ставки, заполняемость и демонстрационные рыночные показатели. При обнаружении возможности увеличения дохода формируется рекомендация. Для демонстрационного примера: «Анализ рынка показывает потенциал роста на 8–12%». Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- тип;
- связанный объект;
- описание;
- дату формирования;
- приоритет;
- предполагаемый эффект;
- статус.

Предусмотреть возможность просмотра подробной информации и изменения статуса рекомендации. Для учебной реализации допускается использование алгоритмов на основе заданных правил. Например:

- при снижении заполняемости ниже установленного порога система рекомендует пересмотреть условия аренды;
- при росте коммунальных расходов формируется рекомендация по энергосбережению;
- при высокой заполняемости и условном повышении спроса предлагается анализ арендных ставок;
- при увеличении числа неисправностей формируется рекомендация по профилактическому обслуживанию.

Интеграция с внешними AI-сервисами допускается, но не является обязательной.

Разработать раздел «Аналитика», предназначенный для комплексного анализа портфеля недвижимости. Предусмотреть следующие показатели:

- количество объектов;
- общая площадь;
- арендуемая площадь;
- процент заполняемости;
- количество арендаторов;
- доходы от аренды;
- эксплуатационные расходы;
- чистая прибыль;
- количество активных договоров;
- количество просроченных платежей;
- количество заявок;
- процент исправных инженерных систем.

Для визуализации использовать:

- линейные графики;
- столбчатые диаграммы;
- кольцевые диаграммы;
- горизонтальные индикаторы;
- информационные карточки.

Предусмотреть фильтрацию аналитики по объекту, типу недвижимости и временному периоду. Данные должны рассчитываться серверной частью на основе информации PostgreSQL.

Разработать раздел «Документы», предназначенный для хранения файлов, связанных с управлением недвижимостью. Предусмотреть следующие категории документов:

- договоры аренды;
- акты приёма-передачи;
- акты выполненных работ;
- техническая документация;
- финансовые документы;
- отчёты;
- прочие документы.

Для каждого документа предусмотреть:

- уникальный идентификатор;
- название;
- категорию;
- связанный объект;
- связанного арендатора или договор при наличии;
- дату загрузки;
- автора;
- тип файла;
- путь к файлу.

Реализовать загрузку, просмотр и скачивание документов. Предусмотреть поиск и фильтрацию по категории, объекту и дате. Доступ к документам должен ограничиваться в соответствии с ролью пользователя.

Разработать раздел «Отчёты», предназначенный для формирования сводной информации об управлении недвижимостью. Предусмотреть следующие виды отчётов:

- отчёт по объектам недвижимости;
- отчёт по арендаторам;
- отчёт по договорам аренды;
- отчёт по арендным платежам;
- отчёт по задолженностям;
- отчёт по доходам и расходам;
- отчёт по заполняемости;
- отчёт по заявкам;
- отчёт по техническому обслуживанию;
- отчёт по состоянию инженерных систем.

Пользователь должен иметь возможность выбрать тип отчёта, объект и временной период. После выбора параметров система должна сформировать таблицу с соответствующими данными. Предусмотреть экспорт отчётов в CSV или PDF. Данные должны формироваться серверной частью на основе информации PostgreSQL.

Разработать раздел «Уведомления», предназначенный для информирования пользователей о важных событиях. Предусмотреть следующие типы уведомлений:

- новая заявка;
- изменение статуса обращения;
- назначение технического обслуживания;
- просрочка выполнения задачи;
- приближение окончания договора;
- начисление арендной платы;
- просрочка платежа;
- изменение состояния инженерной системы;
- новая AI-рекомендация;
- новый документ.

Для каждого уведомления предусмотреть:

- идентификатор;
- тип;
- заголовок;
- описание;
- дату и время;
- связанный объект;
- статус прочтения.

В верхней панели отображать количество непрочитанных уведомлений. При выборе уведомления должен выполняться переход к соответствующей заявке, договору, объекту или задаче. Предусмотреть возможность отметки уведомлений как прочитанных.

Разработать систему регистрации, авторизации и разграничения прав доступа. Предусмотреть следующие роли:

- управляющий;
- арендодатель;
- арендатор;
- технический специалист;
- администратор.

Управляющий должен иметь возможность просматривать Dashboard, управлять объектами, арендаторами, договорами, финансами, заявками, техническим обслуживанием и отчётами.

Арендодатель должен иметь доступ к принадлежащим ему объектам, информации о заполняемости, договорах, доходах и финансовой аналитике.

Арендатор должен иметь доступ к арендуемым помещениям, своим договорам, начислениям, платежам, обращениям и документам.

Технический специалист должен иметь доступ к назначенным заявкам, техническим задачам и информации об инженерных системах соответствующих объектов.

Администратор должен управлять пользователями, ролями, справочниками, настройками и правами доступа.

Для каждого пользователя предусмотреть:

- уникальный идентификатор;
- фамилию и имя;
- адрес электронной почты;
- логин;
- защищённые учётные данные;
- роль;
- контактный телефон;
- изображение профиля;
- статус;
- дату регистрации.

Разграничение прав доступа должно выполняться на сервере.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу мобильного приложения EstatePro AI. В верхней части разместить логотип приложения, персональное приветствие и элемент перехода к профилю. В соответствии с графическим концептом предусмотреть приветствие: «Здравствуйте, Иван!»

Ниже разместить кнопки быстрого доступа:

- «Объекты»;
- «Заявки»;
- «Арендаторы»;
- «Финансы»;
- «Документы»;
- «Ещё».

Для раздела «Заявки» предусмотреть индикатор количества новых обращений. В нижней части разместить навигационную панель:

- «Главная»;
- «Объекты»;
- «Заявки»;
- «Уведомления»;
- «Профиль».

При выборе элемента должен выполняться переход к соответствующему экрану. Доступность разделов должна зависеть от роли авторизованного пользователя.

Разработать мобильный экран «Объекты». В верхней части предусмотреть переключатели:

- «Все»;
- «Офисы»;
- «Торговые»;
- «Склады».

Основная область должна содержать список объектов недвижимости. Для каждого объекта отображать:

- фотографию;
- наименование;
- адрес;
- процент заполняемости.

В соответствии с графическим концептом предусмотреть следующие демонстрационные объекты:

- БЦ «Парк Сити» — заполняемость 92%;
- ТЦ «Северный» — заполняемость 88%;
- СК «Восток» — заполняемость 76%;
- ЖК «Солнечный» — заполняемость 89%.

Реализовать поиск и фильтрацию объектов. При выборе объекта должна открываться подробная карточка с характеристиками, финансовыми показателями и состоянием инженерных систем.

Разработать мобильный экран «Заявки». В верхней части предусмотреть переключатели:

- «Все»;
- «Новые»;
- «В работе»;
- «Закрытые».

Основная область должна содержать список заявок и обращений. Для каждой заявки отображать:

- тип проблемы;
- объект недвижимости;
- помещение при наличии;
- дату создания;

- текущий статус.

В соответствии с графическим концептом предусмотреть демонстрационные заявки:

1. «Не работает кондиционер» — БЦ «Парк Сити», офис 305, новая.
2. «Протечка воды» — ТЦ «Северный», в работе.
3. «Проверка освещения» — СК «Восток», новая.
4. «Плановое ТО лифтов» — ЖК «Солнечный», выполнено.

При выборе заявки должна открываться подробная информация. Предусмотреть возможность создания обращения, добавления комментариев и фотографий. Технический специалист должен иметь возможность изменять статус назначенной заявки и фиксировать результат выполнения.

Разработать мобильный экран «Финансы». В верхней части предусмотреть переключение месяца. Основная область должна содержать карточку общего дохода за выбранный период. В соответствии с графическим концептом использовать демонстрационный показатель:

12,4 млн ₽ — доходы.

Ниже разместить столбчатую диаграмму динамики доходов по месяцам. Дополнительно предусмотреть карточки:

- расходы — 6,1 млн ₽;
- чистая прибыль — 6,3 млн ₽.

Для каждого показателя отображать изменение относительно предыдущего периода. В нижней части разместить кнопку «Полная аналитика». При выборе кнопки должен выполняться переход к подробным финансовым данным. Показатели должны загружаться с сервера и рассчитываться по выбранному периоду. Для арендатора этот раздел должен отображать только собственные начисления, платежи и задолженности.

Разработать мобильный экран «Карта». Основную область должна занимать интерактивная карта с маркерами объектов недвижимости. Предусмотреть:

- отображение объектов;
- изменение масштаба;
- перемещение по карте;
- выбор маркера;
- просмотр карточки объекта.

В соответствии с графическим концептом при выборе маркера должна отображаться карточка объекта, содержащая:

- фотографию;

- наименование;
- тип недвижимости;
- процент заполняемости;
- доход при наличии соответствующих прав.

Для демонстрационного примера использовать БЦ «Парк Сити» с заполняемостью 92% и доходом 4,2 млн ₽. Предусмотреть переход к подробной информации об объекте. Для учебной реализации допускается использование Leaflet и OpenStreetMap.

Разработать мобильный экран «Профиль». В верхней части разместить фотографию пользователя, фамилию и имя, роль и адрес электронной почты. В соответствии с графическим концептом предусмотреть демонстрационного пользователя:

- Кузнецов Иван;
- управляющий;
- ivan.kuznetsov@uk.ru.

Ниже разместить пункты:

- «Уведомления»;
- «Мои задачи»;
- «Избранные объекты»;
- «Настройки»;
- «Справка и поддержка»;
- «Выйти».

Для раздела «Уведомления» предусмотреть индикатор непрочитанных событий. При выборе пункта должен выполняться переход к соответствующему разделу. Предусмотреть возможность редактирования контактной информации и изображения профиля. Функция выхода должна завершать пользовательскую сессию.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение бизнес-логики, проверку входных данных, управление правами доступа и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Properties	Объекты недвижимости, адреса, характеристики
Units	Помещения, площадь, стоимость аренды, статусы
Tenants	Арендаторы и контактная информация
Ownerships	Связи собственников с объектами
LeaseContracts	Договоры аренды, сроки, условия
RentCharges	Начисления арендной платы
Payments	Платежи и статусы оплаты

Expenses	Эксплуатационные расходы
MaintenanceRequests	Заявки и обращения
MaintenanceTasks	Задачи технического обслуживания
EngineeringSystems	Инженерные системы объектов
IoTReadings	Показатели состояния инженерных систем
IoTEvents	События и предупреждения
AIRecommendations	Рекомендации по оптимизации
Documents	Документы и прикреплённые файлы
Notifications	Уведомления пользователей
AuditLogs	Журнал действий пользователей

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение и изменение профиля;
- управление объектами недвижимости;
- управление помещениями;
- управление арендаторами;
- регистрацию и изменение договоров;
- учёт арендных начислений;
- регистрацию платежей;
- учёт расходов;
- расчёт финансовых показателей;
- расчёт заполняемости;
- создание и обработку заявок;
- назначение технических специалистов;
- управление задачами обслуживания;
- получение состояния инженерных систем;
- регистрацию IoT-событий;
- формирование AI-рекомендаций;
- загрузку документов и фотографий;
- получение уведомлений;
- формирование аналитики и отчётов.

Предусмотреть разграничение доступа по ролям управляющего, арендодателя, арендатора, технического специалиста и администратора. Для обеспечения безопасности необходимо реализовать:

- хранение паролей в виде защищённых хешей;
- авторизацию на основе сессий или JWT;
- проверку корректности входных данных;
- ограничение доступа к персональным данным арендаторов;
- проверку прав на просмотр финансовой информации;
- проверку прав на изменение договоров и платежей;
- защиту загружаемых документов;

- безопасное хранение ключей внешних API;
- журналирование значимых операций.

Арендатор должен иметь доступ только к собственным договорам, начислениям, платежам и обращениям. Арендодатель должен получать доступ к объектам, принадлежащим ему или доступным по назначенным полномочиям. Технический специалист должен иметь доступ к назначенным задачам и необходимой информации об объектах. Все финансовые операции и изменения договоров должны сохраняться с указанием пользователя и времени выполнения.

Основные пользовательские сценарии

Сценарий 1. Авторизация и просмотр Dashboard.

Управляющий открывает веб-приложение EstatePro AI и авторизуется. Система загружает информацию об объектах недвижимости, арендаторах, договорах, финансах и техническом состоянии. На главной странице отображаются ключевые показатели, карта объектов, финансовая аналитика и ближайшие задачи. Управляющий просматривает общую ситуацию и выбирает объект, требующий внимания.

Сценарий 2. Добавление объекта недвижимости.

Управляющий открывает раздел «Объекты». Выбирает действие «Добавить объект». Указывает наименование, тип, адрес, площадь, количество помещений и другие характеристики. При необходимости загружает фотографию. Система проверяет корректность данных. После сохранения объект появляется в списке и на карте.

Сценарий 3. Просмотр объекта на карте.

Управляющий открывает раздел «Карта объектов». Система отображает маркеры недвижимости. Пользователь выбирает категорию «Офисы». На карте остаются офисные здания. Управляющий выбирает маркер БЦ «Парк Сити». Открывается карточка с фотографией, заполняемостью 92% и доходом 4,2 млн ₽.

Сценарий 4. Регистрация арендатора.

Управляющий открывает раздел «Арендаторы». Создаёт новую карточку арендатора. Указывает контактные данные и тип арендатора. После сохранения информация появляется в общем списке. Арендатора можно связать с помещением и договором аренды.

Сценарий 5. Создание договора аренды.

Управляющий открывает раздел «Договоры». Выбирает арендатора и свободное помещение. Указывает срок аренды, стоимость и условия оплаты. Система проверяет отсутствие пересекающихся договоров. После сохранения договор получает статус

«Действует». Помещение становится занятым, а показатель заполняемости пересчитывается.

Сценарий 6. Контроль окончания договора.

Система анализирует сроки действия договоров. Обнаруживается договор, срок которого приближается к завершению. Сервер создаёт уведомление. Управляющий открывает карточку договора. Просматривает условия и создаёт задачу по продлению аренды.

Сценарий 7. Учёт арендного платежа.

Управляющий открывает раздел «Финансы». Выбирает арендатора и договор. Регистрирует поступивший платёж. Система сохраняет финансовую операцию и обновляет состояние задолженности. Показатели доходов на Dashboard пересчитываются.

Сценарий 8. Анализ доходности недвижимости.

Управляющий открывает раздел «Аналитика». Выбирает период «6 месяцев». Система формирует график доходов, расходов и чистой прибыли. Пользователь просматривает доходность отдельных объектов. Выбирает ТЦ «Северный» и открывает подробные финансовые показатели.

Сценарий 9. Создание заявки арендатором.

Арендатор открывает мобильное приложение EstatePro AI. Переходит в раздел «Заявки». Выбирает действие «Создать заявку». Указывает проблему «Не работает кондиционер». Выбирает помещение и добавляет описание. При необходимости прикрепляет фотографию. После отправки заявка получает статус «Новая».

Сценарий 10. Назначение технического специалиста.

Управляющий открывает список новых заявок. Выбирает обращение о неисправности кондиционера. Назначает технического специалиста и устанавливает приоритет. Система переводит заявку в статус «В работе». Назначенный специалист получает уведомление.

Сценарий 11. Выполнение заявки.

Технический специалист открывает мобильное приложение. Переходит в раздел «Заявки». Выбирает назначенную задачу. Просматривает описание проблемы и адрес объекта. После выполнения работы добавляет комментарий и фотографию результата. Переводит заявку в статус «Закрыта». Управляющий и заявитель получают уведомление.

Сценарий 12. Мониторинг инженерных систем.

Управляющий открывает Dashboard. В блоке «Состояние объектов (IoT)» отображается предупреждение по системе безопасности. Пользователь выбирает соответствующий индикатор. Открывается информация об объекте, системе и зарегистрированном событии. Управляющий создаёт задачу на проверку оборудования.

Сценарий 13. Формирование AI-рекомендации.

Система анализирует показатели заполняемости и финансовые данные. Обнаруживается объект с потенциальной возможностью повышения доходности. Формируется рекомендация по пересмотру арендных ставок. Управляющий открывает раздел «AI-рекомендации». Просматривает описание, обоснование и предполагаемый эффект. При необходимости создаёт задачу для дальнейшего рассмотрения.

Сценарий 14. Оптимизация эксплуатационных расходов.

Система анализирует расходы на электроэнергию. На основе демонстрационных данных обнаруживается возможность снижения затрат. Формируется рекомендация по установке энергосберегающего оборудования. Управляющий просматривает рекомендацию и оценивает предполагаемую экономию. После подтверждения создаёт задачу техническому специалисту.

Сценарий 15. Просмотр объектов в мобильном приложении.

Управляющий открывает мобильную версию EstatePro AI. На главном экране выбирает раздел «Объекты». Система отображает список недвижимости с фотографиями и показателями заполняемости. Пользователь выбирает БЦ «Парк Сити». Открывается подробная информация об объекте.

Сценарий 16. Просмотр финансов в мобильном приложении.

Управляющий открывает раздел «Финансы». Система отображает доходы за выбранный месяц. Пользователь просматривает диаграмму динамики и показатели расходов. Выбирает кнопку «Полная аналитика». Открывается подробная финансовая информация.

Сценарий 17. Использование мобильной карты.

Управляющий открывает раздел «Карта». Система отображает объекты недвижимости на карте. Пользователь выбирает маркер объекта. Открывается карточка с фотографией, заполняемостью и доходностью. При выборе карточки выполняется переход к подробной информации.

Сценарий 18. Получение уведомления.

Управляющему поступает уведомление о новой заявке. На главном экране появляется индикатор непрочитанных событий. Пользователь открывает раздел «Уведомления».

Выбирает сообщение. Система открывает связанную заявку. После просмотра уведомление отмечается как прочитанное.

Сценарий 19. Формирование отчёта.

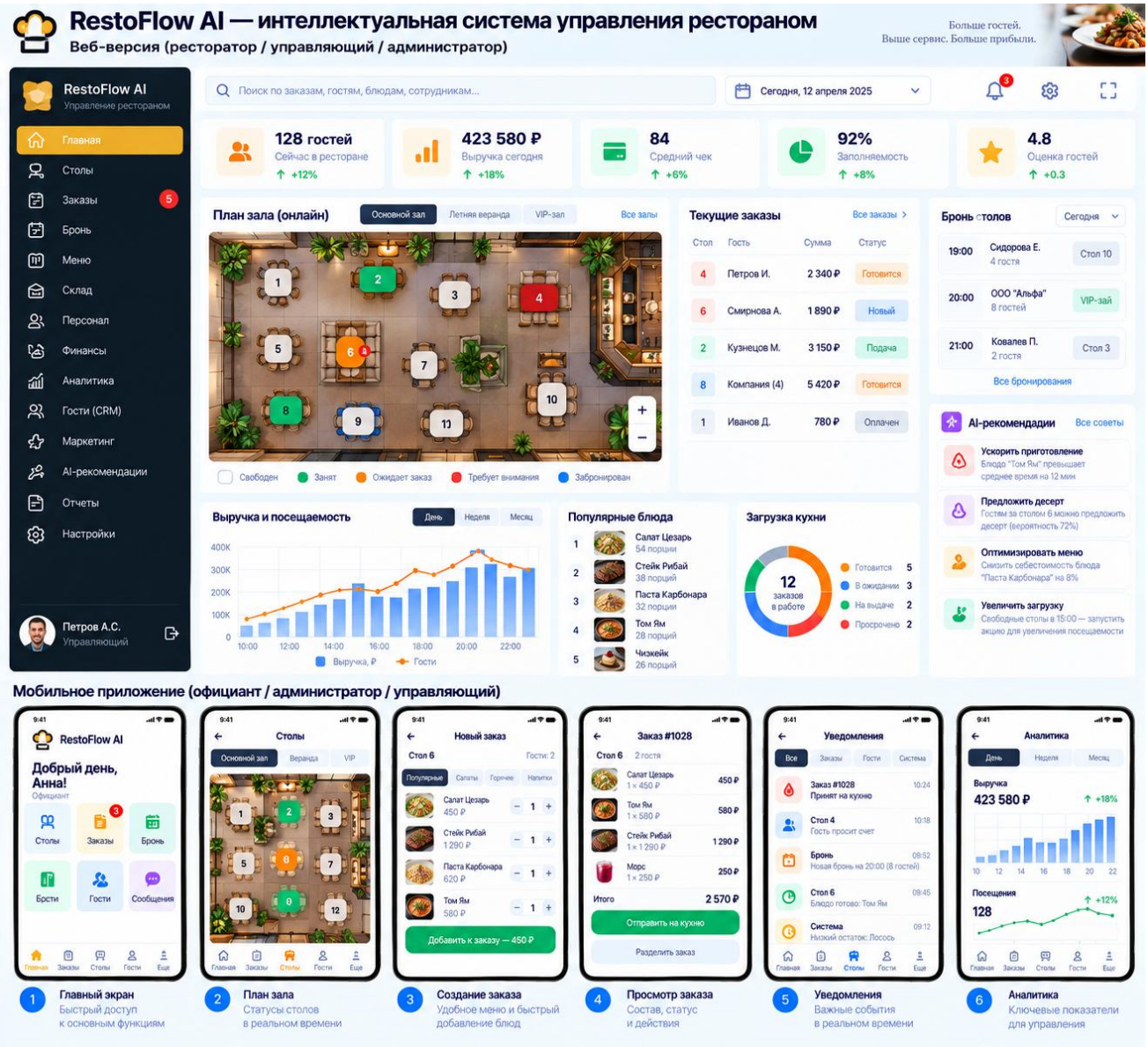
Управляющий открывает раздел «Отчёты». Выбирает отчёт по доходам и расходам. Указывает временной период и объект недвижимости. Система формирует таблицу финансовых операций и итоговых показателей. Пользователь скачивает отчёт в CSV или PDF.

Сценарий 20. Управление профилем.

Пользователь открывает мобильный экран «Профиль». Просматривает персональную информацию и роль. Переходит в раздел «Мои задачи». После просмотра возвращается в профиль. При необходимости изменяет контактные данные или завершает пользовательскую сессию.

Вариант 15. RestoFlow AI — интеллектуальная система управления рестораном

Разработать Fullstack веб-приложение **RestoFlow AI**, предназначенное для автоматизации управления рестораном, контроля занятости столов, обработки заказов, бронирования мест, управления меню, учёта продуктов, координации работы персонала, анализа финансовых показателей и интеллектуальной оптимизации ресторанных процессов.



Система должна обеспечивать централизованное хранение и обработку информации о ресторанных залах, столах, гостях, бронированиях, заказах, блюдах, ингредиентах, складских остатках, сотрудниках, финансовых операциях и показателях эффективности работы ресторана.

Приложение должно предоставлять управляющим и администраторам инструменты для оперативного контроля работы ресторана, отслеживания текущей загрузки зала, анализа выручки, управления персоналом, контроля выполнения заказов и формирования аналитической отчётности.

Официанты должны иметь возможность просматривать план зала, определять доступность столов, создавать заказы, добавлять блюда, отправлять заказы на кухню, отслеживать готовность блюд и фиксировать завершение обслуживания.

Администраторы зала должны управлять бронированиями, распределять гостей по столам, контролировать текущую заполняемость ресторана и координировать работу обслуживающего персонала. Сотрудники кухни должны получать новые заказы, просматривать состав блюд, изменять статусы приготовления и передавать информацию о готовности заказов официантам.

В соответствии с представленным графическим концептом приложение должно включать интеллектуальный модуль **AI-рекомендаций**, который анализирует заказы, посещаемость, популярность блюд, загрузку кухни, финансовые показатели и предпочтения гостей, выявляет потенциальные проблемы и предлагает способы повышения качества обслуживания и прибыльности ресторана.

Приложение должно включать две взаимосвязанные пользовательские части:

- **Веб-версия для ресторатора, управляющего и администратора** — предназначена для управления рестораном, залами, столами, бронированиями, заказами, меню, складом, персоналом, финансами, аналитикой и AI-рекомендациями.
- **Мобильная версия для официанта, администратора и управляющего** — предназначена для просмотра плана зала, создания и обработки заказов, управления бронированиями, получения уведомлений и просмотра ключевых показателей ресторана.

Обе версии должны использовать единый серверный API и общую базу данных PostgreSQL. В качестве основы оформления необходимо использовать представленный графический концепт RestoFlow AI, сохранив основные композиционные решения, структуру навигации, цветовую схему, расположение информационных блоков, интерактивный план зала, визуализацию заказов, финансовые графики и организацию мобильных экранов.

Для учебной реализации допускается использование заранее подготовленного плана ресторана, демонстрационных столов, тестовых гостей, условных заказов, блюд, складских остатков, финансовых операций и алгоритмов формирования рекомендаций на основе заданных правил.

Интеграция с реальными кассовыми системами, платёжными терминалами, сервисами доставки, программами лояльности и промышленными системами автоматизации ресторанов не является обязательной.

Функциональные требования к веб-версии

Разработать главную страницу системы с боковой навигационной панелью, верхней областью управления и информационными блоками, отражающими текущее состояние ресторана. В боковом меню предусмотреть следующие разделы:

- Главная;
- Столы;

- Заказы;
- Бронь;
- Меню;
- Склад;
- Персонал;
- Финансы;
- Аналитика;
- Гости (CRM);
- Маркетинг;
- AI-рекомендации;
- Отчёты;
- Настройки.

В верхней панели разместить строку поиска по заказам, гостям, блюдам и сотрудникам, элемент выбора даты, индикатор уведомлений, кнопку настроек и элемент управления режимом отображения. Основная область Dashboard должна содержать информационные карточки ключевых показателей:

1. **Гости в ресторане** — количество гостей, находящихся в ресторане в текущий момент.
2. **Выручка сегодня** — общая сумма оплаченных заказов за текущий день.
3. **Средний чек** — средняя стоимость оплаченного заказа.
4. **Заполняемость** — процент занятых посадочных мест или столов.
5. **Оценка гостей** — средняя оценка качества обслуживания.

В соответствии с графическим концептом для демонстрационных данных допускается использовать следующие значения:

- гостей в ресторане — 128;
- выручка сегодня — 423 580 ₽;
- средний чек — 84 условные единицы;
- заполняемость — 92%;
- оценка гостей — 4,8 из 5.

Для практической реализации средний чек необходимо рассчитывать по выбранной единой валюте на основании суммы оплаченных заказов и их количества.

Под каждым показателем необходимо предусмотреть отображение динамики относительно предыдущего периода.

Основная область Dashboard должна включать следующие блоки:

- «План зала (онлайн)»;
- «Текущие заказы»;
- «Бронь столов»;
- «Выручка и посещаемость»;
- «Популярные блюда»;
- «Загрузка кухни»;
- «AI-рекомендации».

Информация должна загружаться с сервера и формироваться на основе данных о столах, бронированиях, заказах, меню, гостях и финансовых операциях. При изменении статусов столов, создании заказов, подтверждении бронирований или завершении обслуживания соответствующие информационные блоки должны обновляться.

Разработать функциональность управления залами ресторана. Для каждого зала предусмотреть хранение:

- уникального идентификатора;
- наименования;
- описания;
- количества столов;
- общей вместимости;
- текущего статуса;
- схемы расположения столов.

В соответствии с графическим концептом предусмотреть следующие демонстрационные залы:

1. Основной зал.
2. Летняя веранда.
3. VIP-зал.

На Dashboard в блоке «План зала (онлайн)» разместить переключатели между залами. При выборе зала должна отображаться соответствующая схема расположения столов. Управляющий должен иметь возможность добавлять новые залы, редактировать их характеристики и изменять расположение столов. Для учебной реализации допускается использование заранее подготовленной двумерной схемы ресторана. План зала должен быть интерактивным и обеспечивать выбор отдельных столов.

Разработать раздел «Столы», предназначенный для регистрации, просмотра и управления ресторанными столами. Для каждого стола предусмотреть хранение:

- уникального идентификатора;
- номера;
- связанного зала;
- количества посадочных мест;
- координат расположения на схеме;
- текущего статуса;
- активного заказа;
- текущего бронирования при наличии;
- назначенного официанта.

Предусмотреть следующие статусы столов:

- «Свободен»;
- «Занят»;
- «Ожидает заказ»;
- «Требует внимания»;
- «Забронирован».

В соответствии с графическим концептом использовать цветовую индикацию:

- белый — свободный стол;
- зелёный — занятый стол;
- оранжевый — ожидает заказ;
- красный — требует внимания;
- синий — забронирован.

На плане основного зала предусмотреть демонстрационные столы с номерами от 1 до 12. При выборе стола должна открываться информационная карточка, содержащая:

- номер стола;
- количество посадочных мест;
- текущий статус;
- количество гостей;
- назначенного официанта;
- активный заказ;
- время начала обслуживания.

Реализовать возможность изменения статуса стола, назначения официанта и перехода к созданию заказа. После завершения обслуживания стол должен переходить в свободное состояние после подтверждения его готовности к следующей посадке. Все изменения должны сохраняться в PostgreSQL.

Разработать интерактивный визуальный план ресторана. Основную область должен занимать двумерный план выбранного зала с расположением столов и элементами интерьера. Для каждого стола отображать:

- номер;
- цветовой индикатор статуса;
- информацию о наличии активного заказа;
- индикатор необходимости внимания.

В соответствии с графическим концептом предусмотреть возможность выбора между основным залом, летней верандой и VIP-залом. Реализовать следующие действия:

- просмотр состояния стола;
- выбор стола;
- создание заказа;
- просмотр текущего заказа;
- изменение статуса;
- назначение официанта;
- переход к информации о бронировании.

Предусмотреть возможность увеличения и уменьшения масштаба плана. При изменении состояния стола визуальный индикатор должен обновляться. Для учебной реализации допускается использование HTML, CSS, SVG или Canvas для отображения схемы зала.

Разработать раздел «Заказы», предназначенный для создания, обработки и контроля заказов гостей. Для каждого заказа предусмотреть хранение:

- уникального идентификатора;
- номера заказа;
- номера стола;
- связанного зала;
- количества гостей;
- официанта;
- даты и времени создания;
- списка блюд;
- количества порций;
- стоимости каждой позиции;
- общей суммы;
- комментария;
- текущего статуса;
- статуса оплаты.

Предусмотреть следующие статусы заказа:

- «Новый»;
- «Готовится»;
- «Готов к подаче»;
- «Подача»;
- «Завершён»;
- «Отменён».

На Dashboard разместить блок «Текущие заказы». Для каждого заказа отображать:

- номер стола;
- имя гостя или условное обозначение;
- общую сумму;
- текущий статус.

В соответствии с графическим концептом предусмотреть демонстрационные заказы:

1. Стол 4 — Петров И., 2 340 ₽, готовится.
2. Стол 6 — Смирнова А., 1 890 ₽, новый.
3. Стол 2 — Кузнецов М., 3 150 ₽, подача.
4. Стол 8 — компания из четырёх гостей, 5 420 ₽, готовится.
5. Стол 1 — Иванов Д., 780 ₽, оплачен.

Предусмотреть возможность создания нового заказа, добавления и удаления позиций, изменения количества блюд и просмотра итоговой суммы. При отправке заказа на кухню его статус должен изменяться. При завершении обслуживания и подтверждении оплаты заказ должен переходить в завершённое состояние.

Разработать интерфейс создания заказа, позволяющий официанту выбирать блюда из электронного меню. В верхней части разместить информацию о выбранном столе и количестве гостей. Ниже предусмотреть категории меню:

- популярные блюда;
- салаты;
- горячие блюда;

- напитки;
- десерты.

Для каждого блюда отображать:

- фотографию;
- название;
- цену;
- доступность;
- кнопку добавления.

В соответствии с графическим концептом предусмотреть следующие демонстрационные блюда:

- салат «Цезарь» — 450 ₽;
- стейк «Рибай» — 1 290 ₽;
- паста «Карбонара» — 620 ₽;
- суп «Том Ям» — 580 ₽;
- морс — 250 ₽.

При добавлении блюда система должна увеличивать количество соответствующей позиции в заказе. Предусмотреть кнопки увеличения и уменьшения количества. Общая стоимость должна пересчитываться автоматически. Для выбранных позиций необходимо сохранять цену на момент оформления заказа, чтобы последующее изменение меню не влияло на стоимость ранее созданных заказов. После подтверждения заказ должен сохраняться в PostgreSQL.

Разработать раздел «Меню», предназначенный для регистрации и редактирования блюд ресторана. Для каждого блюда предусмотреть хранение:

- уникального идентификатора;
- названия;
- категории;
- описания;
- фотографии;
- цены;
- себестоимости;
- времени приготовления;
- доступности;
- состава ингредиентов;
- статуса.

Предусмотреть следующие категории:

- салаты;
- супы;
- горячие блюда;
- гарниры;
- десерты;
- напитки;
- другие категории.

Реализовать добавление, редактирование и удаление блюд. Предусмотреть возможность временно скрывать блюдо из доступного меню без удаления его из базы данных. Для каждого блюда должна отображаться информация о популярности и количестве заказанных порций. При изменении доступности блюда информация должна обновляться в интерфейсе создания заказа.

Разработать раздел «Бронь», предназначенный для регистрации и контроля бронирований. Для каждого бронирования предусмотреть:

- уникальный идентификатор;
- имя гостя;
- контактный телефон;
- дату;
- время начала;
- предполагаемую продолжительность;
- количество гостей;
- номер стола;
- зал;
- комментарий;
- текущий статус.

Предусмотреть следующие статусы:

- «Ожидает подтверждения»;
- «Подтверждено»;
- «Гости прибыли»;
- «Завершено»;
- «Отменено»;
- «Неявка».

На Dashboard разместить блок «Бронь столов». В соответствии с графическим концептом использовать демонстрационные бронирования:

1. 19:00 — Сидорова Е., 4 гостя, стол 10.
2. 20:00 — ООО «Альфа», 8 гостей, VIP-зал.
3. 21:00 — Ковалев П., 2 гостя, стол 3.

Предусмотреть возможность создания, редактирования и отмены бронирования. При выборе даты система должна отображать доступные столы. Необходимо реализовать проверку пересечения бронирований по времени. Система не должна допускать подтверждения нескольких конфликтующих бронирований одного стола. После прибытия гостей администратор должен иметь возможность изменить статус бронирования и начать обслуживание.

Разработать функциональность управления приготовлением заказов. Предусмотреть отдельный интерфейс для сотрудников кухни. Для каждого заказа на кухне отображать:

- номер заказа;
- номер стола;
- время поступления;
- список блюд;

- количество порций;
- комментарии;
- текущий статус;
- время ожидания.

Предусмотреть следующие состояния кухонного заказа:

- «В ожидании»;
- «Готовится»;
- «Готов к подаче»;
- «Просрочено».

На Dashboard разместить блок «Загрузка кухни». В соответствии с графическим концептом использовать кольцевую диаграмму с демонстрационными показателями:

- готовится — 5 заказов;
- в ожидании — 3 заказа;
- на выдаче — 2 заказа;
- просрочено — 2 заказа.

В центре диаграммы отображать общее количество заказов в работе — 12. При поступлении нового заказа он должен появляться в очереди кухни. Сотрудник кухни должен иметь возможность изменить статус приготовления. При переходе заказа в состояние «Готов к подаче» официанту должно поступать уведомление. Если время приготовления превышает установленный норматив, заказ должен выделяться как просроченный.

Разработать раздел «Склад», предназначенный для контроля продуктов и ингредиентов. Для каждого ингредиента предусмотреть хранение:

- уникального идентификатора;
- наименования;
- категории;
- единицы измерения;
- текущего количества;
- минимального остатка;
- закупочной стоимости;
- срока годности при необходимости;
- поставщика;
- даты последнего поступления.

Предусмотреть следующие операции:

- добавление ингредиента;
- регистрация поступления;
- списание;
- корректировка остатка;
- просмотр истории движения;
- поиск и фильтрация.

Реализовать связь между блюдами и используемыми ингредиентами. Для каждого блюда предусмотреть технологическую карту с перечнем ингредиентов и

необходимым количеством. После подтверждения приготовления блюда система должна выполнять расчёт расхода ингредиентов и корректировать складские остатки. Необходимо исключить повторное списание ингредиентов при повторном изменении статуса заказа. При снижении количества ингредиента ниже минимального значения система должна формировать уведомление. Для учебной реализации допускается использование демонстрационных складских операций.

Разработать раздел «Персонал», предназначенный для регистрации сотрудников ресторана и распределения обязанностей. Для каждого сотрудника предусмотреть:

- уникальный идентификатор;
- ФИО;
- должность;
- контактный телефон;
- адрес электронной почты;
- график работы;
- назначенный зал;
- текущий статус;
- список задач.

Предусмотреть следующие должности:

- управляющий;
- администратор зала;
- официант;
- повар;
- сотрудник кухни;
- кладовщик.

Реализовать возможность добавления сотрудников, редактирования данных и назначения рабочих смен. Предусмотреть отображение текущего состава смены. Управляющий должен иметь возможность просматривать количество обслуженных заказов по официантам и количество выполненных кухонных заказов. Для учебной реализации допускается использование упрощённого графика смен.

Разработать раздел «Финансы», предназначенный для учёта выручки и расходов ресторана. Предусмотреть регистрацию:

- оплаченных заказов;
- возвратов;
- закупок продуктов;
- расходов на персонал;
- коммунальных расходов;
- эксплуатационных затрат;
- прочих расходов.

Для каждой финансовой операции предусмотреть:

- уникальный идентификатор;
- тип;
- категорию;

- сумму;
- дату;
- описание;
- связанный заказ при наличии;
- ответственного пользователя.

Реализовать расчёт:

- общей выручки;
- количества оплаченных заказов;
- среднего чека;
- расходов;
- прибыли;
- динамики финансовых показателей.

Выручку необходимо рассчитывать по подтверждённым оплатам с учётом возвратов. Предусмотреть фильтрацию по дате и категории операции. Для учебной реализации допускается имитация оплаты без подключения реальных платёжных систем.

Разработать блок «Выручка и посещаемость». В соответствии с графическим концептом использовать комбинированную диаграмму:

- синие столбцы — выручка;
- оранжевая линия — количество гостей.

По горизонтальной оси отображать временные интервалы. Предусмотреть переключатели:

- «День»;
- «Неделя»;
- «Месяц».

При выборе периода график должен обновляться. Для каждого временного интервала рассчитывать:

- сумму выручки;
- количество гостей;
- количество заказов;
- средний чек.

Дополнительно предусмотреть сравнение с предыдущим аналогичным периодом. Данные должны рассчитываться серверной частью на основе информации PostgreSQL.

Разработать блок «Популярные блюда», предназначенный для отображения наиболее востребованных позиций меню. Для каждого блюда отображать:

- место в рейтинге;
- фотографию;
- название;
- количество проданных порций.

В соответствии с графическим концептом предусмотреть демонстрационный рейтинг:

1. Салат «Цезарь» — 54 порции.
2. Стейк «Рибай» — 38 порций.
3. Паста «Карбонара» — 32 порции.
4. Суп «Том Ям» — 28 порций.
5. Чизкейк — 26 порций.

Предусмотреть возможность выбора периода анализа. При выборе блюда должна открываться подробная информация о количестве продаж, выручке и себестоимости. Реализовать сортировку по популярности, выручке и количеству проданных порций.

Разработать раздел «Гости (CRM)», предназначенный для хранения информации о посетителях ресторана. Для каждого зарегистрированного гостя предусмотреть:

- уникальный идентификатор;
- ФИО;
- контактный телефон;
- адрес электронной почты при наличии;
- дату первого посещения;
- количество посещений;
- историю бронирований;
- историю заказов при наличии;
- предпочтения;
- среднюю сумму заказа;
- дату последнего посещения.

Предусмотреть возможность поиска гостей по имени и телефону. При выборе гостя должна открываться карточка с историей взаимодействия с рестораном. Реализовать возможность добавления заметок о предпочтениях гостя. Персональные данные должны использоваться только в рамках предоставленных прав и необходимых согласий. Для учебной реализации допускается использование тестовых гостей и демонстрационной истории посещений.

Разработать раздел «Маркетинг», предназначенный для управления акциями и специальными предложениями. Для каждой акции предусмотреть:

- уникальный идентификатор;
- название;
- описание;
- период действия;
- тип скидки;
- размер скидки;
- применимые блюда или категории;
- условия участия;
- текущий статус.

Предусмотреть возможность создания, редактирования и завершения акции. Реализовать отображение активных маркетинговых предложений. Для каждой акции предусмотреть анализ:

- количества заказов с применением акции;
- общей выручки;
- суммы предоставленных скидок;
- изменения продаж выбранных блюд.

Для учебной реализации допускается использование демонстрационных акций без подключения внешних рекламных сервисов.

Разработать раздел «AI-рекомендации», предназначенный для интеллектуального анализа работы ресторана. В соответствии с графическим концептом предусмотреть четыре основные категории рекомендаций.

Система анализирует время приготовления заказов и количество просроченных позиций. При выявлении перегрузки формируется рекомендация. Демонстрационный пример: «Ускорить приготовление. Блюдо „Том Ям“ превышает среднее время на 12 минут».

Персональные предложения гостям. Система анализирует историю заказов и предпочтения зарегистрированных гостей. На основе демонстрационных данных формируется рекомендация: «Предложить десерт. Гостям за столом 6 можно предложить десерт (вероятность 72%)». Указанная вероятность может быть демонстрационным значением, если в проекте не реализована обученная модель прогнозирования.

Оптимизация меню. Система анализирует популярность блюд, себестоимость и прибыльность. Демонстрационный пример: «Снизить себестоимость блюда „Паста Карбонара“ на 8%».

Увеличение загрузки ресторана. Система анализирует посещаемость и загруженность столов по времени. Демонстрационный пример: «Свободные столы в 15:00 — запустить акцию для увеличения посещаемости». Для каждой рекомендации предусмотреть:

- уникальный идентификатор;
- тип;
- заголовок;
- описание;
- дату формирования;
- приоритет;
- предполагаемый эффект;
- связанный объект;
- статус.

Предусмотреть возможность просмотра подробной информации и изменения статуса рекомендации. Для учебной реализации допускается использование алгоритмов на основе заданных правил. Например:

- при превышении нормативного времени приготовления система рекомендует проверить загрузку кухни;
- при снижении посещаемости в определённые часы предлагается маркетинговая акция;
- при высокой себестоимости блюда предлагается анализ состава ингредиентов;

- при росте спроса на определённые блюда рекомендуется увеличить запас соответствующих продуктов;
- при низкой заполняемости зала предлагается специальное предложение.

Интеграция с внешними AI-сервисами допускается, но не является обязательной.

Разработать раздел «Отчёты», предназначенный для формирования сводной информации о работе ресторана. Предусмотреть следующие виды отчётов:

- отчёт по заказам;
- отчёт по выручке;
- отчёт по среднему чеку;
- отчёт по посещаемости;
- отчёт по популярности блюд;
- отчёт по загрузке кухни;
- отчёт по бронированиям;
- отчёт по складским остаткам;
- отчёт по расходам;
- отчёт по работе персонала;
- отчёт по эффективности маркетинговых акций.

Пользователь должен иметь возможность выбрать тип отчёта и временной период. После выбора параметров система должна сформировать таблицу с соответствующими данными. Предусмотреть экспорт отчётов в CSV или PDF. Данные должны формироваться серверной частью на основе информации PostgreSQL.

Разработать систему уведомлений о важных событиях ресторана. Предусмотреть следующие типы уведомлений:

- новый заказ;
- заказ принят кухней;
- блюдо готово к подаче;
- задержка приготовления;
- новый запрос на бронирование;
- прибытие гостей;
- стол требует внимания;
- низкий складской остаток;
- новая AI-рекомендация;
- изменение рабочей смены;
- системное сообщение.

Для каждого уведомления предусмотреть:

- уникальный идентификатор;
- тип;
- заголовок;
- описание;
- дату и время;
- связанный заказ, стол или бронирование;
- получателя;
- статус прочтения.

В верхней панели отображать количество непрочитанных уведомлений. При выборе уведомления должен выполняться переход к соответствующему заказу, столу или бронированию. Предусмотреть возможность отметки уведомлений как прочитанных.

Разработать систему регистрации, авторизации и разграничения прав доступа. Предусмотреть следующие роли:

- ресторатор;
- управляющий;
- администратор;
- официант;
- сотрудник кухни;
- кладовщик.

Ресторатор должен иметь доступ к финансовой аналитике, отчётам, показателям эффективности, управлению меню и настройкам ресторана.

Управляющий должен иметь возможность контролировать залы, столы, заказы, бронирования, персонал, склад, финансы и AI-рекомендации.

Администратор должен управлять бронированиями, размещением гостей, статусами столов и текущей работой зала.

Официант должен иметь доступ к плану зала, назначенным столам, созданию и обработке заказов.

Сотрудник кухни должен иметь доступ к поступившим заказам и изменению статусов приготовления.

Кладовщик должен иметь доступ к складским остаткам, поступлениям, списаниям и учёту ингредиентов.

Для каждого пользователя предусмотреть:

- уникальный идентификатор;
- ФИО;
- логин;
- адрес электронной почты;
- защищённые учётные данные;
- роль;
- контактный телефон;
- изображение профиля;
- статус;
- дату регистрации.

Разграничение прав доступа должно выполняться на сервере.

Функциональные требования к мобильной версии

Мобильный интерфейс должен соответствовать шести основным экранам, представленным на графическом концепте.

Разработать стартовую страницу мобильного приложения RestoFlow AI. В верхней части разместить логотип приложения, персональное приветствие и информацию о роли пользователя. В соответствии с графическим концептом предусмотреть приветствие: «Добрый день, Анна!»

Ниже разместить кнопки быстрого доступа:

- «Столы»;
- «Заказы»;
- «Бронь»;
- «Блюда»;
- «Гости»;
- «Сообщения».

Для раздела «Заказы» предусмотреть индикатор количества новых событий. В нижней части разместить навигационную панель:

- «Главная»;
- «Заказы»;
- «Столы»;
- «Гости»;
- «Ещё».

При выборе элемента должен выполняться переход к соответствующему экрану. Доступность разделов должна зависеть от роли авторизованного пользователя.

Разработать мобильный экран «Столы». В верхней части предусмотреть переключатели:

- «Основной зал»;
- «Веранда»;
- «VIP».

Основная область должна содержать интерактивный план выбранного зала. Для каждого стола отображать номер и цветовой индикатор состояния. Предусмотреть возможность выбора стола. При выборе стола должна открываться карточка с информацией о количестве гостей, текущем заказе и статусе обслуживания. Официант должен иметь возможность перейти к созданию нового заказа или просмотру существующего. Состояние столов должно обновляться при изменениях в системе.

Разработать мобильный экран «Новый заказ». В верхней части разместить номер выбранного стола и количество гостей. Ниже предусмотреть категории блюд:

- популярные;
- салаты;
- горячие;
- напитки.

Основная область должна содержать список блюд с фотографиями, названиями и ценами. Для каждого блюда предусмотреть кнопки увеличения и уменьшения количества.

При выборе блюда оно должно добавляться в текущий заказ. В нижней части разместить кнопку добавления выбранных позиций. Например:

«Добавить к заказу — 450 ₽».

После подтверждения позиции должны сохраняться в базе данных. Если заказ уже существует, новые блюда должны добавляться к нему без удаления ранее выбранных позиций.

Разработать мобильный экран подробной информации о заказе. В верхней части отображать:

- номер заказа;
- номер стола;
- количество гостей.

Ниже разместить список заказанных блюд. Для каждой позиции отображать:

- фотографию;
- название;
- количество;
- цену;
- итоговую стоимость позиции.

В нижней части отображать общую сумму заказа. В соответствии с графическим концептом предусмотреть демонстрационный заказ №1028 для стола 6. В состав заказа входят:

- салат «Цезарь» — 450 ₽;
- суп «Том Ям» — 580 ₽;
- стейк «Рибай» — 1 290 ₽;
- морс — 250 ₽.

Общая сумма — 2 570 ₽.

Предусмотреть следующие действия:

- «Отправить на кухню»;
- «Разделить заказ».

При отправке заказа на кухню его статус должен изменяться, а соответствующие позиции должны появляться в очереди приготовления. Функция разделения заказа должна позволять распределить позиции между отдельными счетами гостей.

Разработать мобильный экран «Уведомления». В верхней части предусмотреть фильтры:

- «Все»;
- «Заказы»;
- «Гости»;
- «Система».

Основная область должна содержать список уведомлений. В соответствии с графическим концептом предусмотреть демонстрационные события:

1. «Заказ №1028 принят на кухню».
2. «Стол 4 просит счёт».
3. «Новая бронь на 20:00 (8 гостей)».
4. «Блюдо готово: Том Ям».
5. «Низкий остаток: Лосось».

Для каждого уведомления отображать значок, заголовок, описание и время. При выборе уведомления должен выполняться переход к соответствующему разделу. Непрочитанные уведомления должны визуально выделяться. Предусмотреть возможность отметки уведомлений как прочитанных.

Разработать мобильный экран «Аналитика». В верхней части предусмотреть переключатели:

- «День»;
- «Неделя»;
- «Месяц».

Основная область должна содержать показатели выручки и посещаемости. В соответствии с графическим концептом использовать демонстрационные значения:

- выручка — 423 580 ₽;
- рост выручки — 18%;
- посещения — 128;
- рост посещаемости — 12%.

Для визуализации выручки использовать столбчатую диаграмму. Для отображения посещаемости использовать линейный график. При выборе другого периода графики должны обновляться. Данные должны загружаться с сервера. Доступ к финансовой аналитике должен предоставляться только пользователям с соответствующими правами.

Требования к серверной части и базе данных

Разработать серверное приложение на Node.js с Express либо Python Flask. Сервер должен обеспечивать обработку HTTP-запросов, выполнение бизнес-логики, проверку входных данных, управление правами доступа и взаимодействие с PostgreSQL. Минимальная структура базы данных должна включать следующие сущности:

Сущность	**Основные данные**
Users	Пользователи, роли, защищённые учётные данные
Employees	Сотрудники, должности, рабочие смены
RestaurantHalls	Ресторанные залы, характеристики
RestaurantTables	Столы, вместимость, координаты, статусы
Guests	Гости, контакты, история посещений
Reservations	Бронирования, даты, время, статусы
MenuCategories	Категории меню
MenuItems	Блюда, цены, фотографии, доступность
Ingredients	Ингредиенты и складские остатки
Recipes	Технологические карты блюд

RecipeIngredients	Состав блюд и нормы расхода ингредиентов
StockMovements	Поступления, списания, корректировки
Orders	Заказы, столы, суммы, статусы
OrderItems	Позиции заказов, количество, цены
KitchenOrders	Очередь приготовления
Payments	Оплаты заказов и возвраты
Expenses	Финансовые расходы
MarketingCampaigns	Акции и специальные предложения
AIRecommendations	Рекомендации по оптимизации
Notifications	Уведомления пользователей
AuditLogs	Журнал действий пользователей

Необходимо определить первичные и внешние ключи, обеспечить связи между таблицами и реализовать основные CRUD-операции. Для взаимодействия клиентской и серверной частей разработать REST API, обеспечивающее:

- регистрацию и авторизацию пользователей;
- получение и изменение профиля;
- управление ресторанными залами;
- управление столами;
- получение интерактивного плана зала;
- управление гостями;
- создание и изменение бронирований;
- управление категориями меню;
- управление блюдами;
- создание и обработку заказов;
- добавление позиций заказа;
- расчёт общей стоимости;
- управление кухонной очередью;
- изменение статусов приготовления;
- учёт складских остатков;
- регистрацию складских операций;
- учёт оплат;
- регистрацию расходов;
- управление сотрудниками;
- управление маркетинговыми акциями;
- формирование AI-рекомендаций;
- получение уведомлений;
- расчёт аналитических показателей;
- формирование отчётов.

Для обновления состояния столов, заказов и кухонной очереди в реальном времени рекомендуется использовать WebSocket или Server-Sent Events. Для учебной реализации допускается периодический опрос серверного API. Необходимо обеспечить согласованность данных при одновременной работе нескольких пользователей. Например, если два официанта пытаются изменить один заказ, система должна предотвращать потерю изменений. При создании бронирований необходимо проверять отсутствие конфликтов по столу и времени. При регистрации оплаты необходимо предотвращать повторное проведение одной операции. При списании ингредиентов необходимо использовать транзакции базы данных. Для обеспечения безопасности необходимо реализовать:

- хранение паролей в виде защищённых хешей;

- авторизацию на основе сессий или JWT;
- проверку корректности входных данных;
- разграничение прав доступа;
- защиту персональных данных гостей;
- контроль доступа к финансовой информации;
- безопасную обработку загружаемых изображений;
- журналирование значимых операций.

Все основные операции должны выполняться через серверную часть с сохранением результатов в PostgreSQL.

Основные пользовательские сценарии

Сценарий 1. Авторизация и просмотр Dashboard.

Управляющий открывает веб-приложение RestoFlow AI и авторизуется. Система загружает информацию о столах, заказах, бронированиях, выручке и загрузке кухни. На главной странице отображаются ключевые показатели, интерактивный план зала и текущие заказы. Управляющий просматривает состояние ресторана и определяет зоны, требующие внимания.

Сценарий 2. Просмотр плана зала.

Администратор открывает раздел «Столы». Система отображает интерактивный план основного зала. На схеме показываются свободные, занятые и забронированные столы. Администратор выбирает стол 6. Открывается карточка с текущим статусом и информацией об обслуживании.

Сценарий 3. Размещение гостей.

В ресторан прибывает группа гостей. Администратор открывает план зала. Выбирает свободный стол с достаточным количеством мест. Указывает количество гостей и назначает официанта. После подтверждения стол получает статус «Занят». Информация обновляется на веб- и мобильных устройствах.

Сценарий 4. Создание заказа официантом.

Официант открывает мобильное приложение RestoFlow AI. Переходит в раздел «Столы». Выбирает стол 6. Открывает экран создания заказа. Добавляет салат «Цезарь», суп «Том Ям» и стейк «Рибай». Система автоматически рассчитывает общую сумму. После подтверждения заказ сохраняется в PostgreSQL.

Сценарий 5. Отправка заказа на кухню.

Официант открывает карточку заказа №1028. Проверяет список блюд и количество порций. Нажимает кнопку «Отправить на кухню». Система изменяет статус заказа и передаёт информацию в кухонную очередь. Сотрудник кухни получает новый заказ.

Сценарий 6. Приготовление заказа.

Повар открывает интерфейс кухни. Выбирает поступивший заказ. Просматривает список блюд. Переводит заказ в статус «Готовится». После завершения приготовления отмечает блюда как готовые к подаче. Официанту поступает уведомление.

Сценарий 7. Контроль задержки приготовления.

Система анализирует время приготовления заказов. Обнаруживается заказ, превышающий установленный норматив. Заказ выделяется красным цветом. На Dashboard увеличивается количество просроченных заказов. Управляющий получает уведомление.

Сценарий 8. Завершение обслуживания.

Гость просит счёт. Официант открывает заказ. Проверяет итоговую сумму. После подтверждения оплаты система регистрирует финансовую операцию. Заказ получает статус «Завершён». После подготовки стола к следующей посадке его статус меняется на «Свободен».

Сценарий 9. Создание бронирования.

Администратор открывает раздел «Бронь». Выбирает дату и время. Указывает имя гостя, количество посетителей и контактный телефон. Система отображает доступные столы. Администратор выбирает стол и подтверждает бронирование. Бронирование сохраняется в PostgreSQL.

Сценарий 10. Прибытие гостей по бронированию.

Гости прибывают в ресторан. Администратор открывает список бронирований. Находит соответствующую запись. Изменяет статус на «Гости прибыли». Система связывает бронирование с выбранным столом. После размещения гостей стол получает статус «Занят».

Сценарий 11. Добавление блюда в меню.

Управляющий открывает раздел «Меню». Выбирает действие «Добавить блюдо». Указывает название, категорию, описание, цену и время приготовления. Загружает фотографию. После сохранения блюдо появляется в электронном меню. Официанты могут добавлять его в заказы.

Сценарий 12. Учёт складских остатков.

Кладовщик открывает раздел «Склад». Выбирает ингредиент. Регистрирует поступление продуктов. Система увеличивает складской остаток. При использовании ингредиентов в приготовленных блюдах количество корректируется. Если остаток становится ниже минимального значения, создаётся уведомление.

Сценарий 13. Просмотр популярных блюд.

Управляющий открывает раздел «Аналитика». Выбирает период анализа. Система формирует рейтинг наиболее популярных блюд. Пользователь просматривает количество проданных порций. Выбирает салат «Цезарь». Открывается подробная информация о продажах и выручке.

Сценарий 14. Анализ выручки.

Управляющий открывает раздел «Финансы». Выбирает текущий день. Система отображает общую выручку и количество заказов. На графике показывается динамика продаж по часам. Пользователь сравнивает показатели с предыдущим днём.

Сценарий 15. Работа с гостями (CRM).

Администратор открывает раздел «Гости». Находит карточку постоянного посетителя. Просматривает историю бронирований и заказов. Добавляет заметку о предпочтениях гостя. Информация сохраняется и становится доступной уполномоченным сотрудникам.

Сценарий 16. Формирование AI-рекомендации.

Система анализирует время приготовления блюд. Обнаруживает увеличение времени приготовления супа «Том Ям». Формирует рекомендацию по оптимизации работы кухни. Управляющий открывает раздел «AI-рекомендации». Просматривает проблему и предполагаемые действия.

Сценарий 17. Оптимизация посещаемости.

Система анализирует загруженность столов по часам. Обнаруживает низкую посещаемость в 15:00. Формирует рекомендацию по запуску специального предложения. Управляющий открывает раздел «Маркетинг». Создаёт акцию на выбранный временной интервал.

Сценарий 18. Получение уведомления.

Официанту поступает уведомление о готовности блюда. На мобильном устройстве появляется индикатор нового события. Официант открывает раздел «Уведомления». Выбирает сообщение «Блюдо готово: Том Ям». Система открывает соответствующий заказ.

Сценарий 19. Просмотр мобильной аналитики.

Управляющий открывает мобильное приложение. Переходит в раздел «Аналитика». Система отображает выручку, посещаемость и динамику показателей. Пользователь переключает период с «День» на «Неделя». Графики обновляются.

Сценарий 20. Формирование отчёта.

Управляющий открывает раздел «Отчёты». Выбирает отчёт по продажам. Указывает временной период. Система формирует таблицу заказов, выручки и среднего чека. Пользователь скачивает отчёт в CSV или PDF.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Хоган Б. HTML5 и CSS3. Веб-разработка по стандартам нового поколения. 2-е изд. – СПб.: Питер, 2014. – 320 с.
2. Дакетт Д. HTML и CSS. Разработка и дизайн веб-сайтов / Джон Дакетт; [пер. с англ. М.А. Райтмана]. – М.: Эксмо, 2013. – 480 с.
3. Дронов В.А. PHP, MySQL, HTML5 и CSS3. Разработка современных динамических Web-сайтов. – СПб.: БХВ-Петербург, 2016. – 688 с.
4. Дронов В.А. HTML5, CSS3 и Web 2.0. Разработка современных Web-сайтов. – СПб.: БХВ-Петербург, 2011. – 416 с.
5. Полуэктова Н.Р. Разработка веб-приложений: учебное пособие для вузов / Н.Р. Полуэктова. – 2-е изд. – Москва: Издательство Юрайт, 2024. – 204 с.
6. Янцев В.В. Разработка web-страниц на HTML, CSS и JavaScript: учебное пособие для вузов / В.В. Янцев. – Санкт-Петербург: Лань, 2024. – 148 с.
7. Пауэрс Ш. Изучаем Node. Переходим на сторону сервера. 2-е изд. Изд. Питер, 2017 – 304с.
8. Давид Хэррон. Node.js Разработка серверных веб-приложений на JavaScript. –М.: ДМК Пресс, 2012 – 144с.
9. Сухов.К.К. Node.js. Путеводитель по технологии. –М.: ДМК Пресс, 2015. - 416 с.
10. Кантелон М., Хартер, М. Головайчук TJ, Рейлих Н. Node.js в действии. – СПб.: Питер, 2015. – 448 с.
11. Аквино К., Ганди Т. Front-end. Клиентская разработка для профессионалов. Node.js, ES6, REST. Изд. Питер, 2017. – 512с.